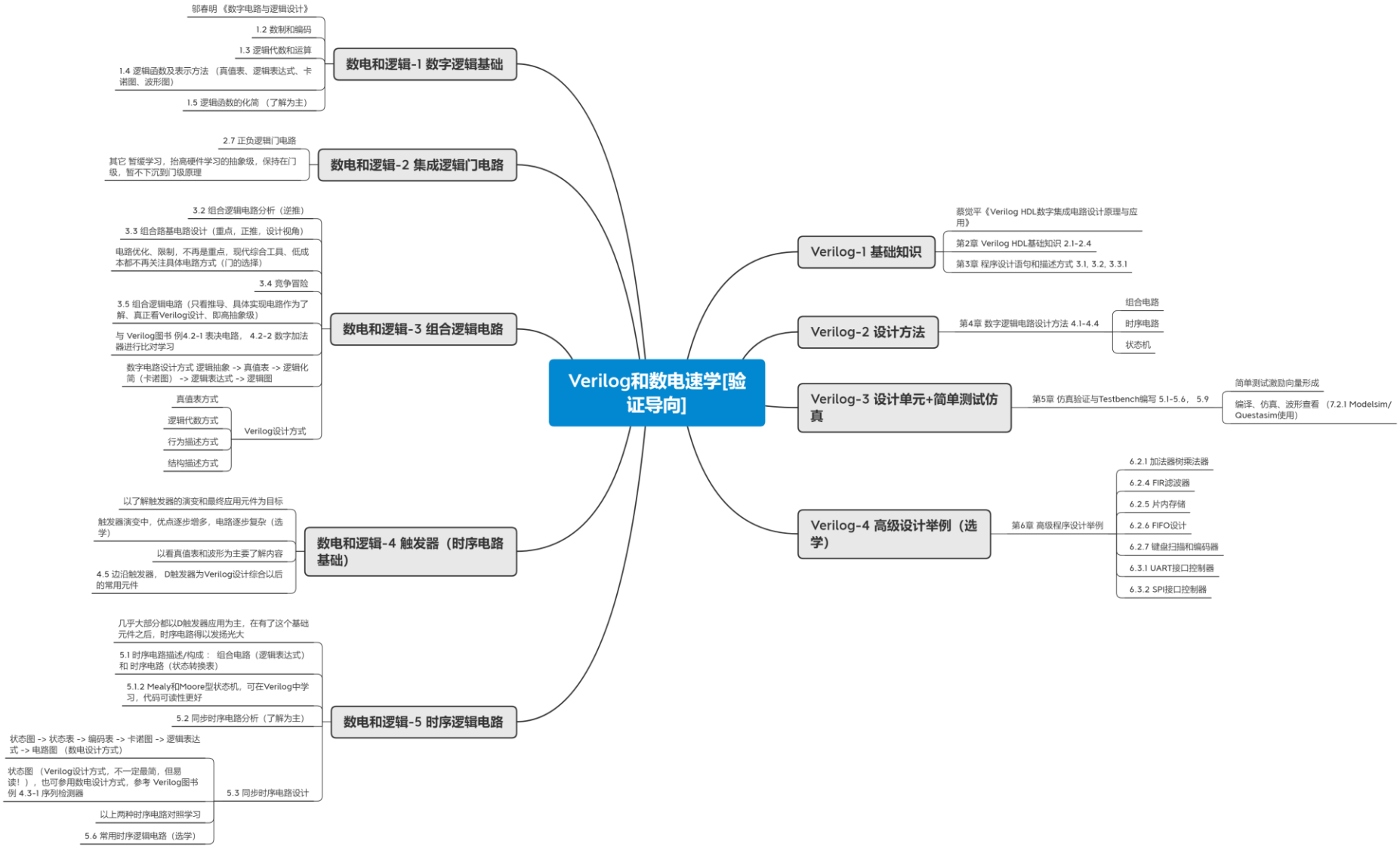
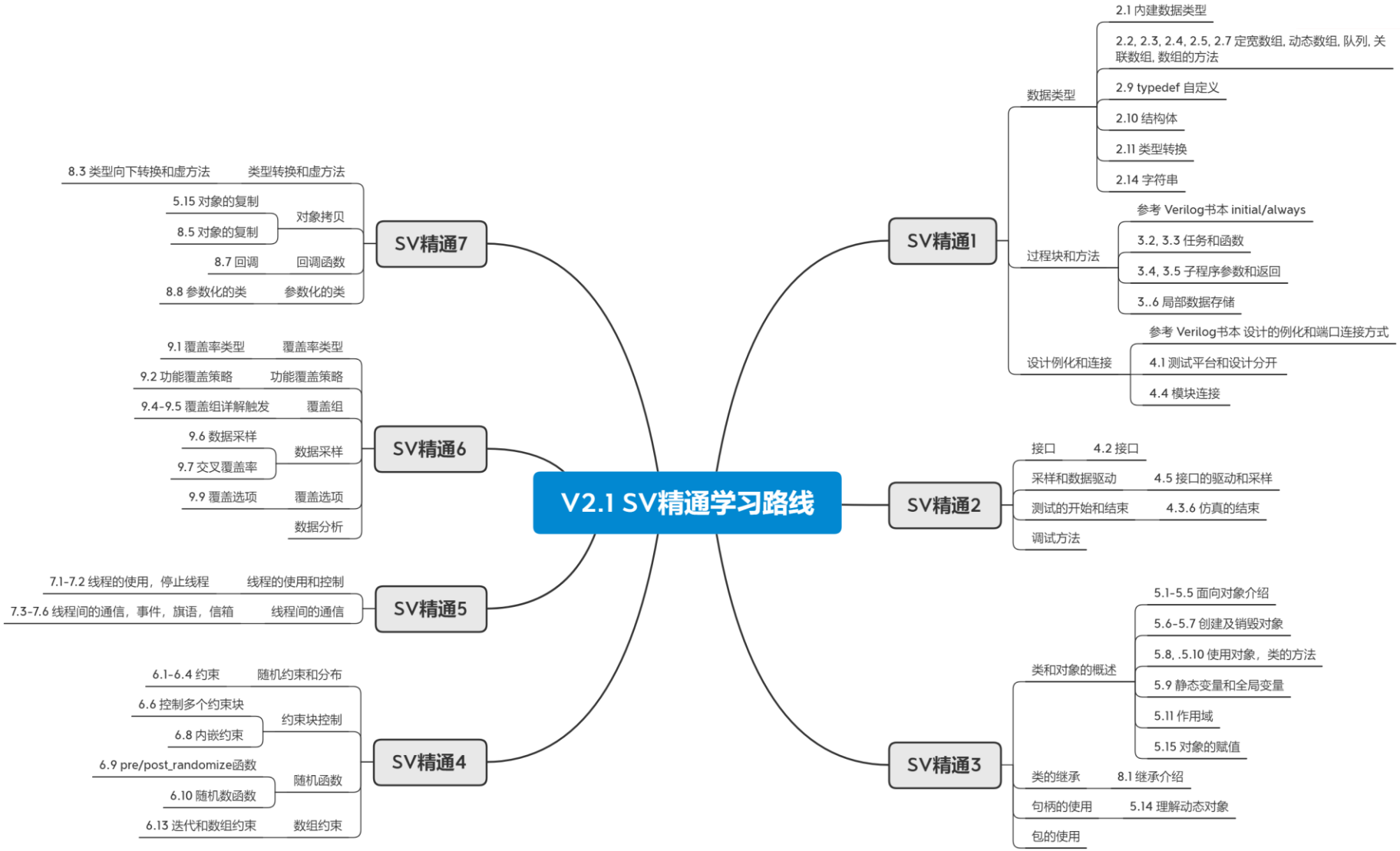
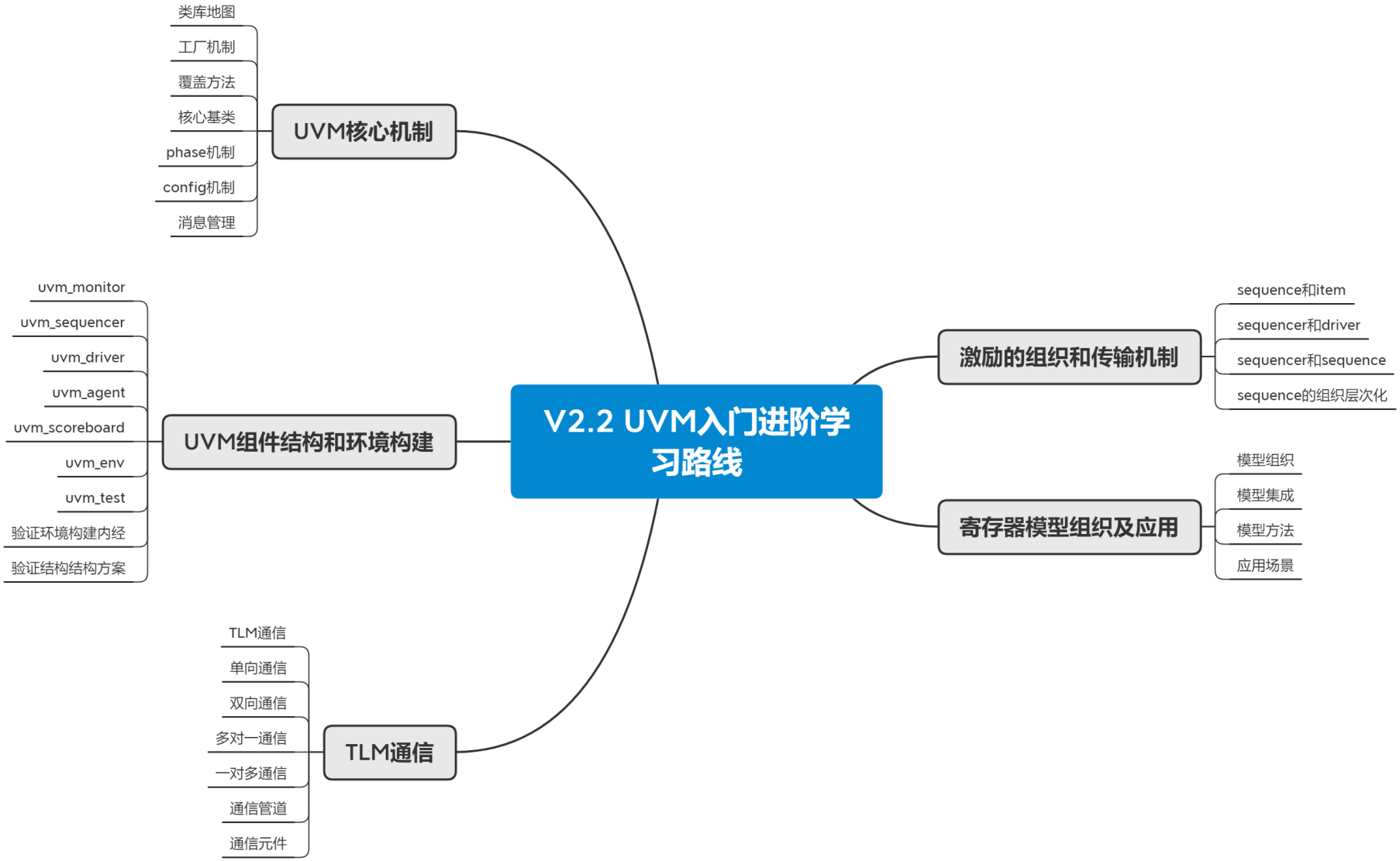
RKV
路科验证

芯片验证V1 Pro课程
SV + UVM 轻量化入门
全新项目PPA实践

路桑
bin.liu@rockeric.com


Verilog和数电速学[验证导向]
数电和逻辑-1 数字逻辑基础
邬春明《数字电路与逻辑设计》
1.2 数制和编码
1.3 逻辑代数和运算
1.4 逻辑函数及表示方法 （真值表、逻辑表达式、卡诺图、波形图）
1.5 逻辑函数的化简 （了解为主）
数电和逻辑-2 集成逻辑门电路
2.7 正负逻辑门电路
其它 暂缓学习，抬高硬件学习的抽象级，保持在门级，暂不下沉到门级原理
数电和逻辑-3 组合逻辑电路
3.2 组合逻辑电路分析（逆推）
3.3 组合路基电路设计（重点，正推，设计视角）
电路优化、限制，不再是重点，现代综合工具、低成本都不再关注具体电路方式（门的选择）
3.4 竞争冒险
3.5 组合逻辑电路（只看推导、具体实现电路作为了解、真正看Verilog设计、即高抽象级）
与 Verilog图书 例4.2-1 表决电路， 4.2-2 数字加法器进行比对学习
数字电路设计方式 逻辑抽象 -> 真值表 -> 逻辑化简（卡诺图） -> 逻辑表达式 -> 逻辑图
Verilog设计方式
真值表方式
逻辑代数方式
行为描述方式
结构描述方式
数电和逻辑-4 触发器（时序电路基础）
以了解触发器的演变和最终应用元件为目标
触发器演变中，优点逐步增多，电路逐步复杂（选学）
以看真值表和波形为主要了解内容
4.5 边沿触发器， D触发器为Verilog设计综合以后的常用元件
数电和逻辑-5 时序逻辑电路
几乎大部分都以D触发器应用为主，在有了这个基础元件之后，时序电路得以发扬光大
5.1 时序电路描述/构成： 组合电路（逻辑表达式）和 时序电路（状态转换表）
5.1.2 Mealy和Moore型状态机，可在Verilog中学习，代码可读性更好
5.2 同步时序电路分析（了解为主）
5.3 同步时序电路设计
状态图 -> 状态表 -> 编码表 -> 卡诺图 -> 逻辑表达式 -> 电路图 （数电设计方式）
状态图 （Verilog设计方式，不一定最简，但易读！），也可参用数电设计方式，参考 Verilog图书 例 4.3-1 序列检测器
以上两种时序电路对照学习
5.6 常用时序逻辑电路（选学）
Verilog-1 基础知识
蔡觉平《Verilog HDL数字集成电路设计原理与应用》
第2章 Verilog HDL基础知识 2.1-2.4
第3章 程序设计语句和描述方式 3.1, 3.2, 3.3.1
Verilog-2 设计方法
第4章 数字逻辑电路设计方法 4.1-4.4
组合电路
时序电路
状态机
Verilog-3 设计单元+简单测试仿真
第5章 仿真验证与Testbench编写 5.1-5.6， 5.9
简单测试激励向量形成
编译、仿真、波形查看 （7.2.1 Modelsim/Questasim使用）
Verilog-4 高级设计举例（选学）
第6章 高级程序设计举例
6.2.1 加法器树乘法器
6.2.4 FIR滤波器
6.2.5 片内存储
6.2.6 FIFO设计
6.2.7 键盘扫描和编码器
6.3.1 UART接口控制器
6.3.2 SPI接口控制器


V2.1 SV精通学习路线
SV精通1
数据类型
2.1 内建数据类型
2.2, 2.3, 2.4, 2.5, 2.7 定宽数组, 动态数组, 队列, 关联数组, 数组的方法
2.9 typedef 自定义
2.10 结构体
2.11 类型转换
2.14 字符串
过程块和方法
参考 Verilog书本 initial/always
3.2, 3.3 任务和函数
3.4, 3.5 子程序参数和返回
3..6 局部数据存储
设计例化和连接
参考 Verilog书本 设计的例化和端口连接方式
4.1 测试平台和设计分开
4.4 模块连接
SV精通2
接口
4.2 接口
采样和数据驱动
4.5 接口的驱动和采样
测试的开始和结束
4.3.6 仿真的结束
调试方法
SV精通3
类和对象的概述
5.1-5.5 面向对象介绍
5.6-5.7 创建及销毁对象
5.8, .5.10 使用对象，类的方法
5.9 静态变量和全局变量
5.11 作用域
5.15 对象的赋值
类的继承
8.1 继承介绍
句柄的使用
5.14 理解动态对象
包的使用
SV精通4
随机约束和分布
6.1-6.4 约束
约束块控制
6.6 控制多个约束块
6.8 内嵌约束
随机函数
6.9 pre/post_randomize函数
6.10 随机数函数
数组约束
6.13 迭代和数组约束
SV精通5
线程的使用和控制
7.1-7.2 线程的使用，停止线程
线程间的通信
7.3-7.6 线程间的通信，事件，旗语，信箱
SV精通6
覆盖率类型
9.1 覆盖率类型
功能覆盖策略
9.2 功能覆盖策略
覆盖组
9.4-9.5 覆盖组详解触发
数据采样
9.6 数据采样
9.7 交叉覆盖率
覆盖选项
9.9 覆盖选项
数据分析
SV精通7
类型转换和虚方法
8.3 类型向下转换和虚方法
对象拷贝
5.15 对象的复制
8.5 对象的复制
回调函数
8.7 回调
参数化的类
8.8 参数化的类


V2.2 UVM入门进阶学习路线
UVM核心机制
类库地图
工厂机制
覆盖方法
核心基类
phase机制
config机制
消息管理
UVM组件结构和环境构建
uvm_monitor
uvm_sequencer
uvm_driver
uvm_agent
uvm_scoreboard
uvm_env
uvm_test
验证环境构建内经
验证结构结构方案
TLM通信
TLM通信
单向通信
双向通信
多对一通信
一对多通信
通信管道
通信元件
激励的组织和传输机制
sequence和item
sequencer和driver
sequencer和sequence
sequence的组织层次化
寄存器模型组织及应用
模型组织
模型集成
模型方法
应用场景

Agenda

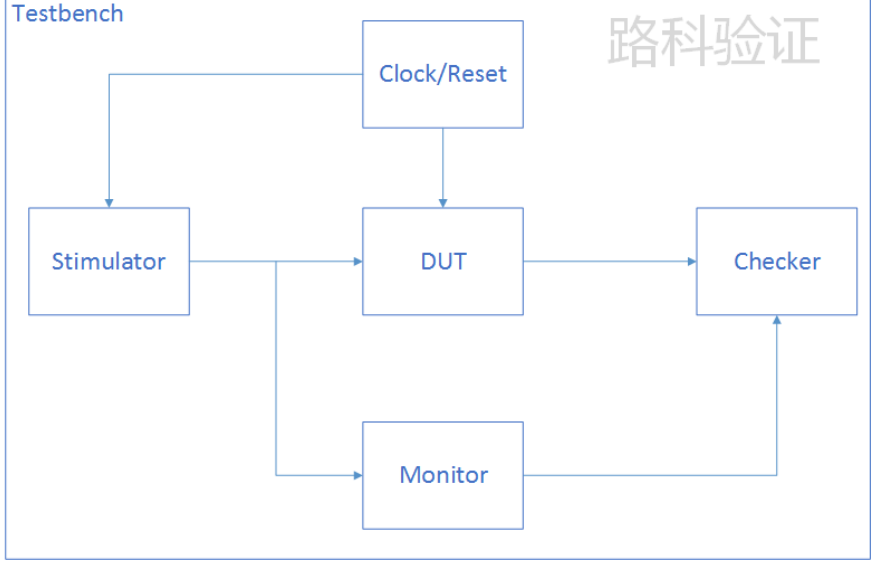
1. **芯片验证概述**
2. SystemVerilog概述
3. 数据类型
4. 自定义类型
5. 字符串类型
6. 设计特性
7. 接口

芯片开发概述
什么是验证?



- 用来证明设计功能正确，符合设计功能描述的流程。
- 确认产品功能、服务或系统是否符合规则、要求、规范和强制条件，**通常是还没有面向客户的内部过程，看有没有"把事情做对"。**
- 验证要占据前端设计的70%左右的工作量。
- 当你买了一件衬衫，怎么样来看衬衫好不好呢？首先我们会看这件衬衣质量合格吗，是不是有两个相同长度的袖子，尺寸合适吗，颜色是我要的吗，纽扣有没有少一颗，这些都可以理解为你在去验证（Verify）这件衬衣。

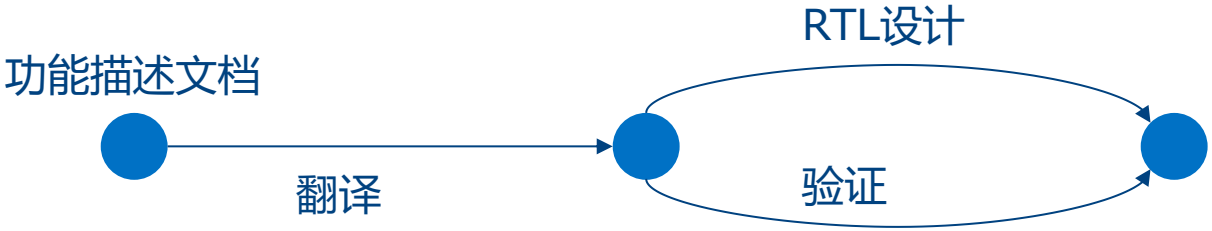
芯片开发概述
什么是测试平台?



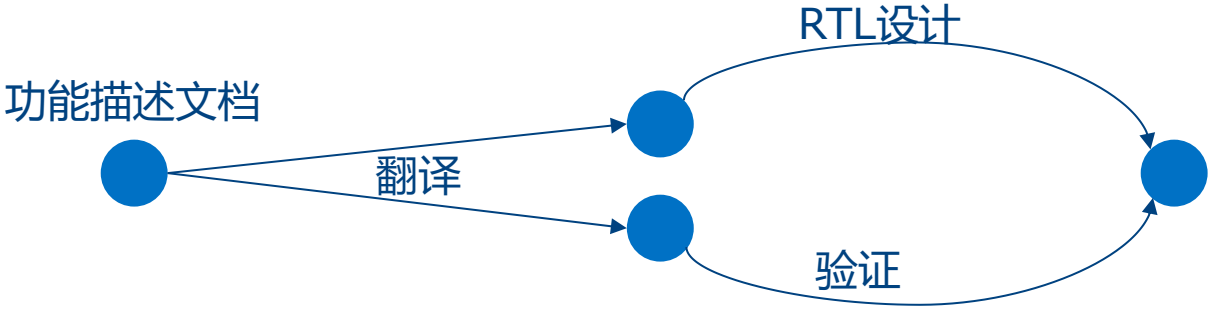
- 对DUT创建测试序列
- 观察DUT的输入输出
- 对DUT的输出数据与预期数据进行比对
- 报告检查结果

芯片开发概述
设计与验证要不要独立工作?

- 如果是一个人同时做RTL设计和验证的话



- 如果是不同的人做RTL设计和验证的话



芯片开发概述
芯片体积有多大?

超大规模集成电路芯片目前的入门量级是以十亿门为单位的
蹭热点：GPU大厂 2017年5月 一款芯片 采用12nm FFN工艺，
核心面积为惊人的815平方毫米，共包含211亿个晶体管。

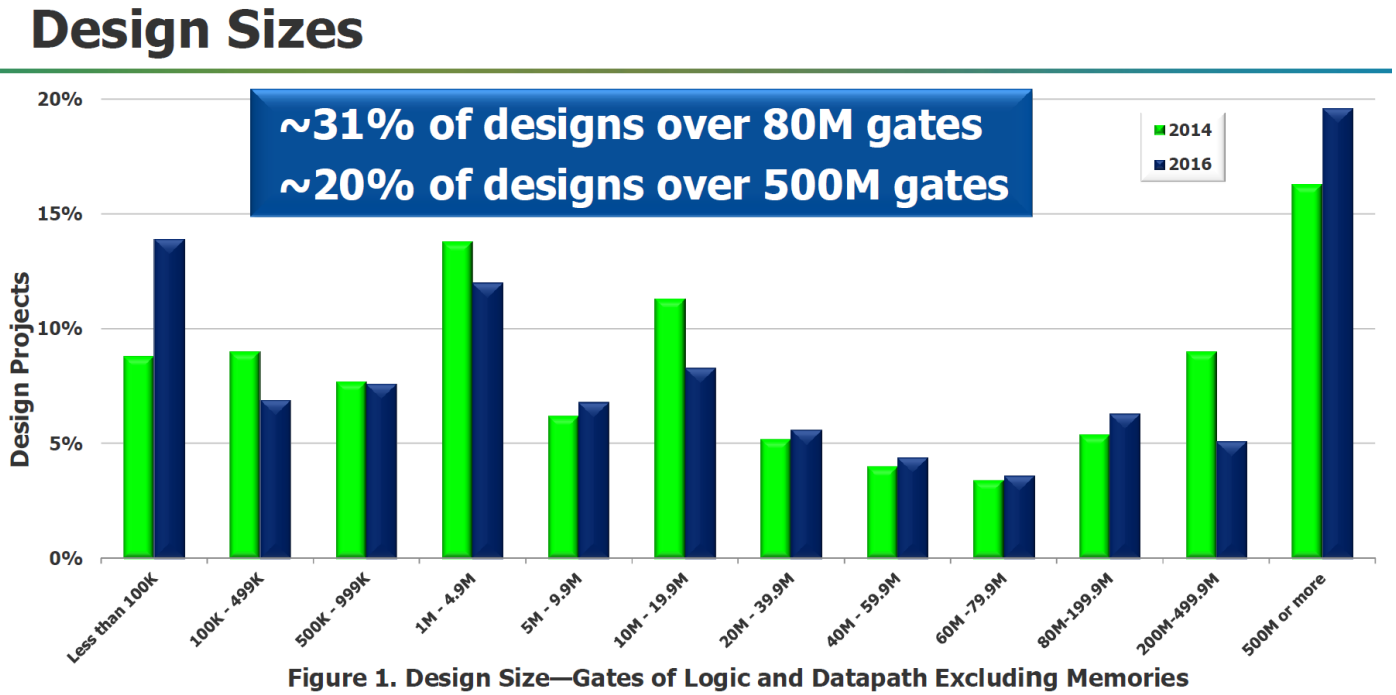
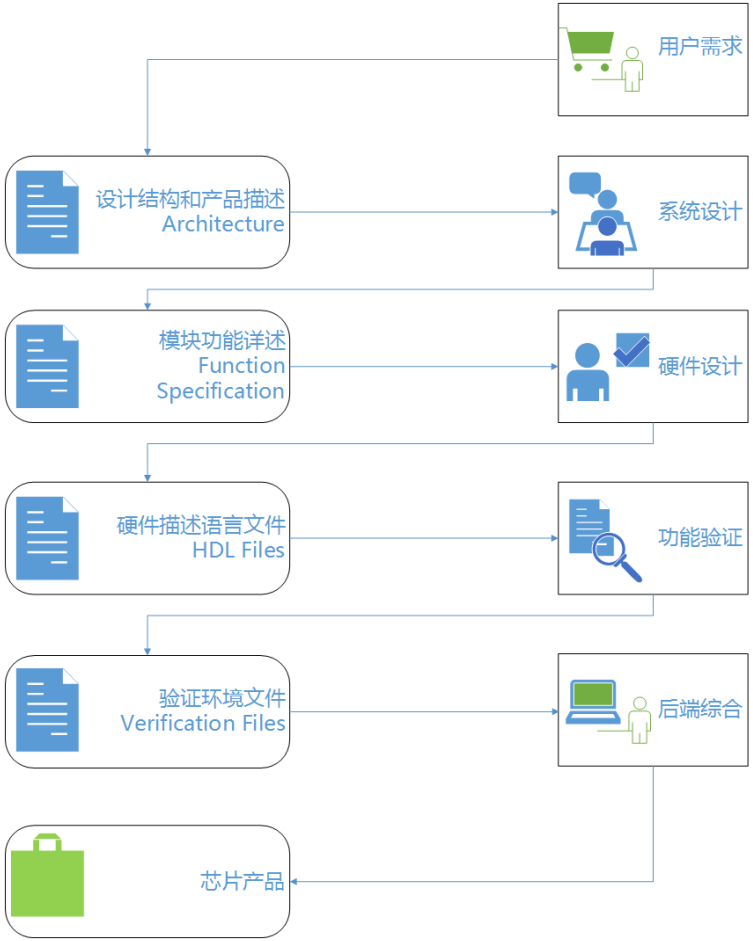


Figure 1. Design Size—Gates of Logic and Datapath Excluding Memories

芯片开发概述
芯片开发流程

1. 从市场人员与客户沟通开始。
2. 系统设计人员按照功能划分为各个子系统。
3. 子系统被进一步划分为功能模块，并由设计团队实现。
4. 验证人员对设计功能展开验证，发现设计缺陷，交由设计人员修正。
5. 验证没有出现漏洞后，交由后端人员进行综合、布局、布线。
6. 后端人员将核心数据交由FAB进行流片。

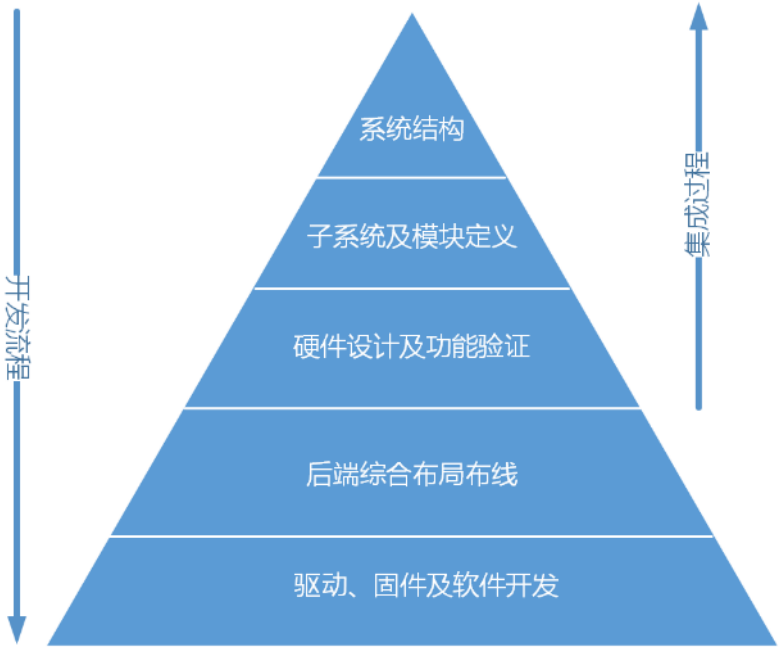


芯片开发概述
验证和设计的紧密关系

- 设计如果没有经过验证，是没有任何信心去流片的。
- 设计如果没有经过充分验证，是不够信心去流片的。
- 设计如果没有经过充分量化验证，是缺一点信心去流片的。
- 验证如果不懂设计，发现了漏洞是没法跟设计好好沟通的。
- 设计如果不懂验证，是没法体会验证已经在慢慢转向软件化的。
- 设计需要验证尽早尽快尽量地去测试设计发现漏洞。
- 验证人员需要行业的尊重，使其认识到验证为公司带来的价值。

芯片开发概述
验证和设计的紧密关系（续）

- 设计和验证都需要围绕功能描述文档
- 设计初步实现之后需要验证入场
- 验证发现结果不符合预期时，如果漏洞明显可交由设计修正，待返回再测试；如果功能实现与设计存在分歧，则需共同回顾功能描述，决定哪一方理解正确，统一对功能的理解。
- 在系统由低层向高层集成过程中，验证与设计需要在每一个层次展开各自工作，确保验证在每一个阶段的充分性。



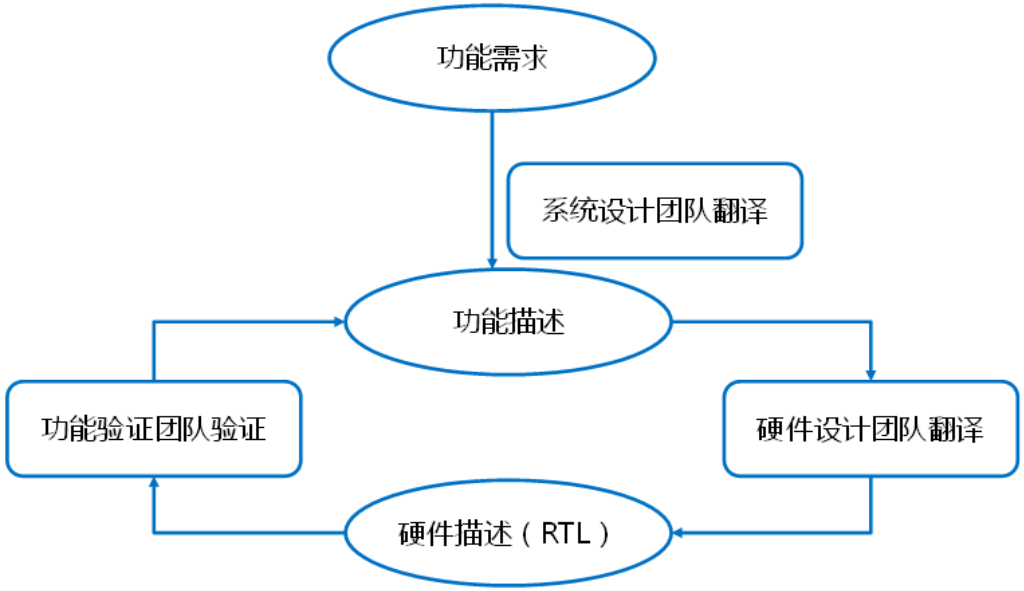
芯片开发概述
验证人员做了哪些工作?

在设计人员根据设计功能描述，实现各个模式RTL代码之后，开始构建验证环境，做几项工作来检查设计：

- 设计文件是否正确地按照功能描述文档去实施了?
- 硬件设计人员是否有遗漏掉的边界情况（corner case）？
- 硬件设计是否足够稳定来处理一些错误情况（error response）？

芯片开发概述
验证和设计的协作

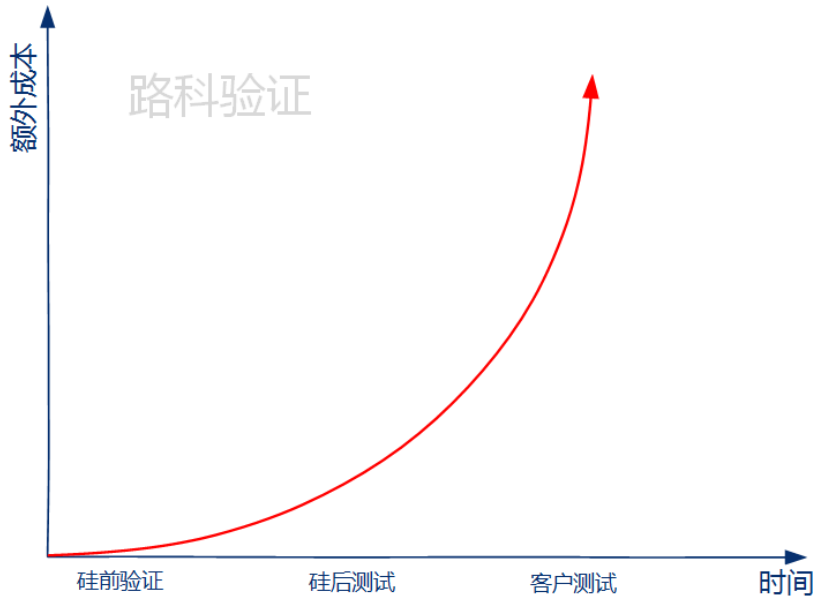
- 验证和设计都需要认真阅读功能描述文档
- 设计会将其翻译为RTL模型
- 验证会按照其功能发送激励和比较结果

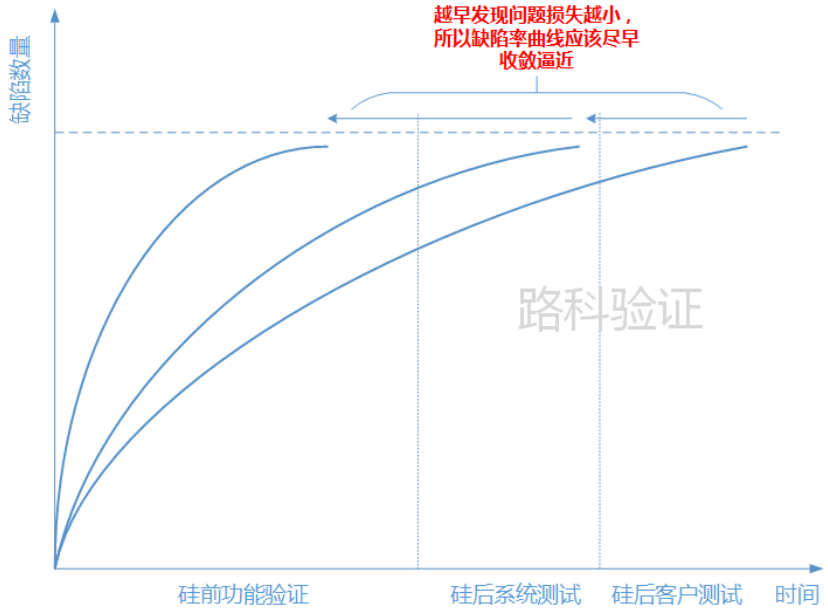


验证的概述

验证的重要性

1. 重大缺陷造成额外的成本损失是越靠后越巨大
2. 缺陷率增长曲线的曲率又是逐渐减小的
3. 快而全地提供缺陷率增长是理想目标





验证任务和目标
验证的目标

- 对于一名验证师而言，他的工作就是完成分配给他的任务，这个任务可能是模块级（module level）、子系统级（subsystem level）或者系统级（chip level）的。
- 准确来讲验证的目标，就是“按时保质低耗”完成目标硬件设计的验证工作，这句话实际也包含了要完成验证目标需要考虑到的三个方面。

验证任务和目标
按时

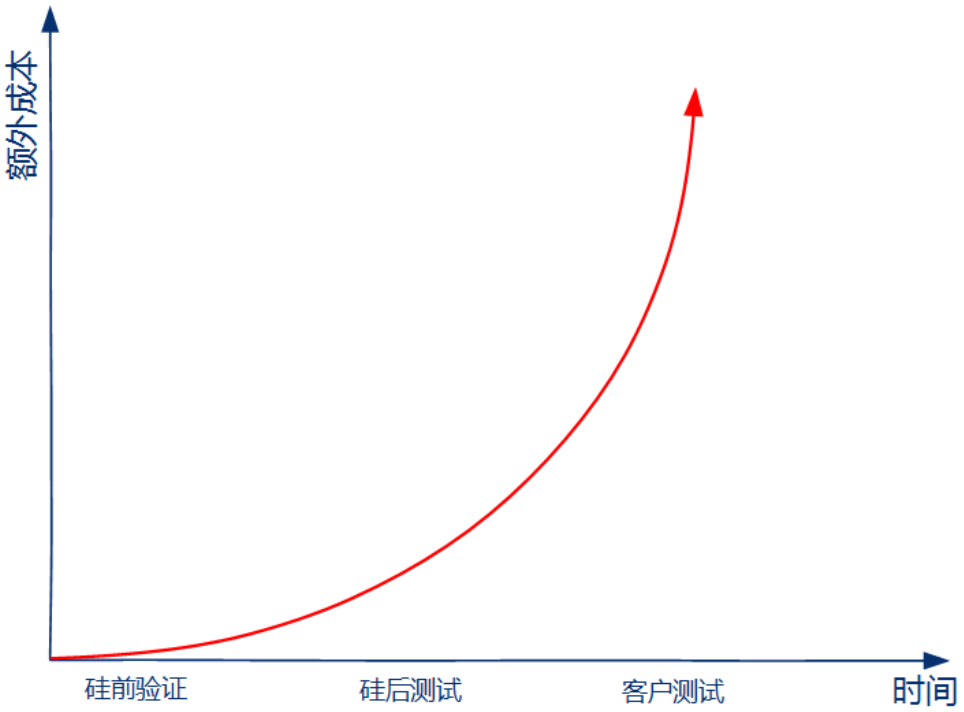
- 验证师需要按照项目预先的进度来考虑验证的节点（milestone），在项目一开始的时候就将节点记挂在心上。
- 要知道“一个都不能少”在芯片流程中的重要性，没有一款芯片可以因为其中一个模块的验证延迟而有信心去流片。
- 所以整支验证队伍自上到下，覆盖到各个模块的验证师都应该有这种意识，即无论何时，时间总是第一位的，时间就意味着客户的耐心和市场的窗口。

验证任务和目标
保质

- 保质指的是尽可能少地将缺陷暴露在流片以后，至少要尽可能少地暴露在客户和市场面前。
- 因为从成本的角度来看，由于缺陷的暴露在不同阶段造成的损失是指数递增的。而且，如果芯片交付给客户以后才被反馈出一些大的缺陷，那么芯片设计方就会背负很大的压力，除了要同客户一起进行高密度的对话、联调 ，整个产品链都要为这个缺陷付出更大的人力物力成本。
- 如果芯片是在客户一侧通过测试却被市场发现自身性能不如预期的话，那么会对芯片设计公司和客户双方都造成消极的影响，无论是从市场反馈，还是用户对品牌的认知度上面都是如此。

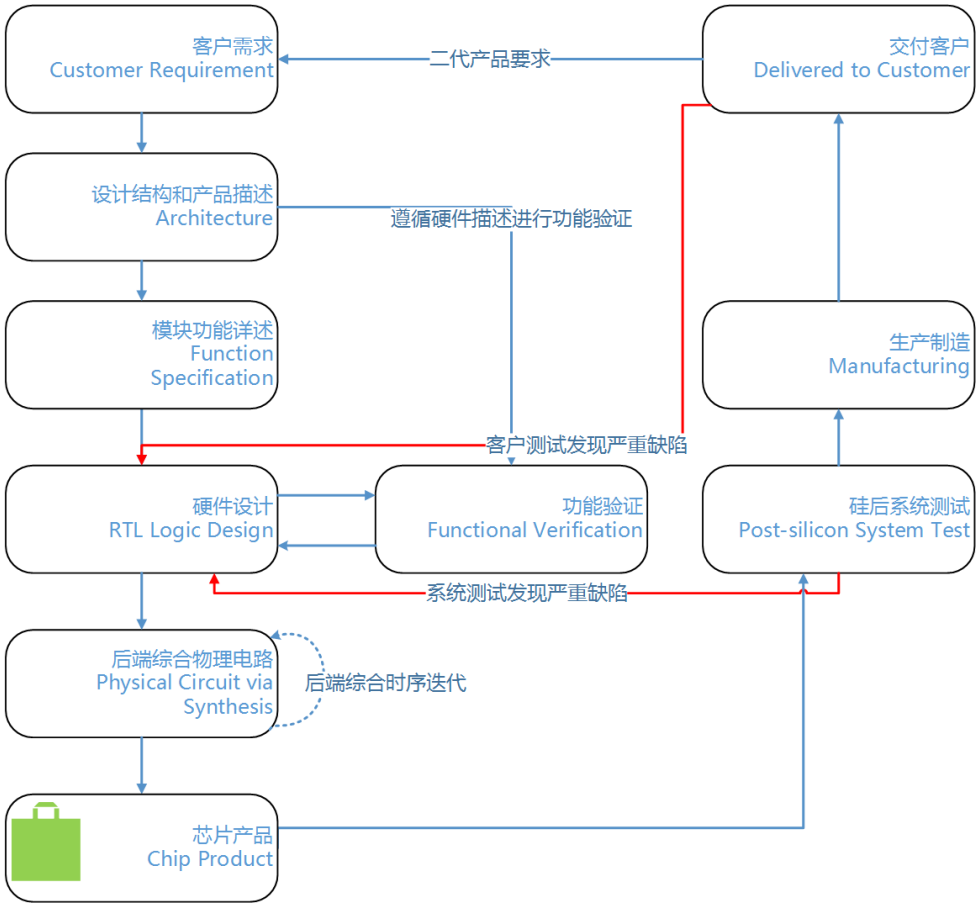
验证任务和目标
低耗

- 低消耗有两方面，用更短的时间、更少的人力来完成芯片设计任务，这对于芯片设计公司而言，是一笔前期看得见的可以预期控制的成本。
- 同时，也有一些成本是突发的，其中一个就是缺陷的暴露问题。从下图可以看出暴露在不同研发阶段的缺陷，其造成的额外成本是会影响芯片项目成败的。



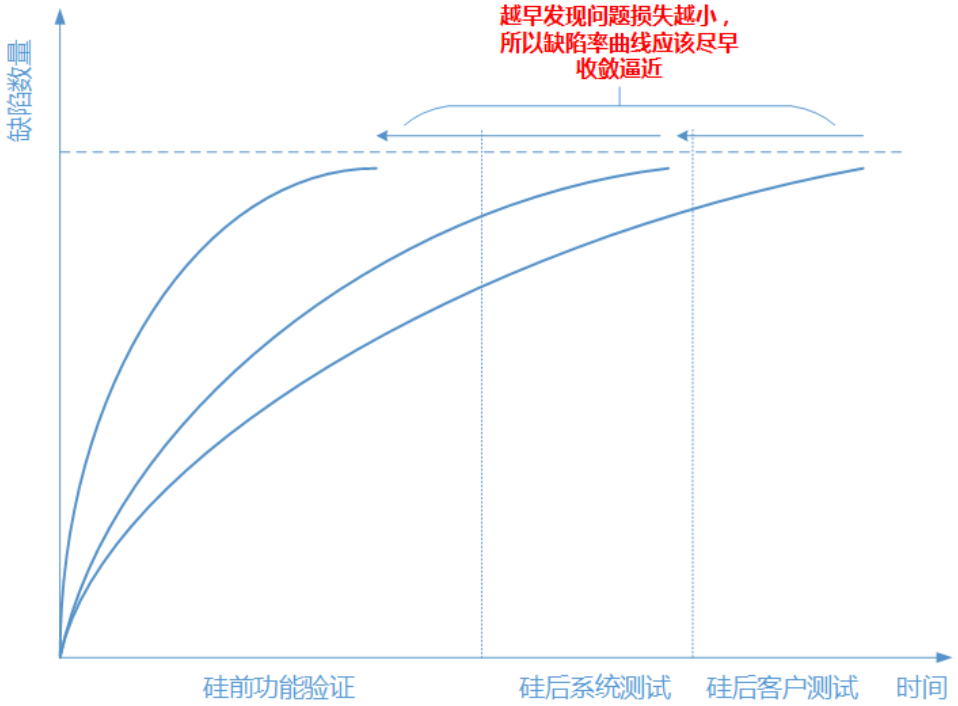
验证任务和目标
芯片研发与客户反馈

- 一旦芯片在出片以后被检测出的严重缺陷会直接导致芯片的二次流片，这对于成本控制是一种额外的损失，同时也会将时间和人力资源消耗在本可以避免的二次流片上面。
- 所以功能验证是唯一可以用低成本在硅前流程将上述目标“按时、保质、低耗”达成的方法。
- 也正因为如此，对于功能验证而言，验证经理会通过量化的方式来衡量验证的产出的进度。用来衡量的两个标准，一个是时间，一个是发现的缺陷数量。



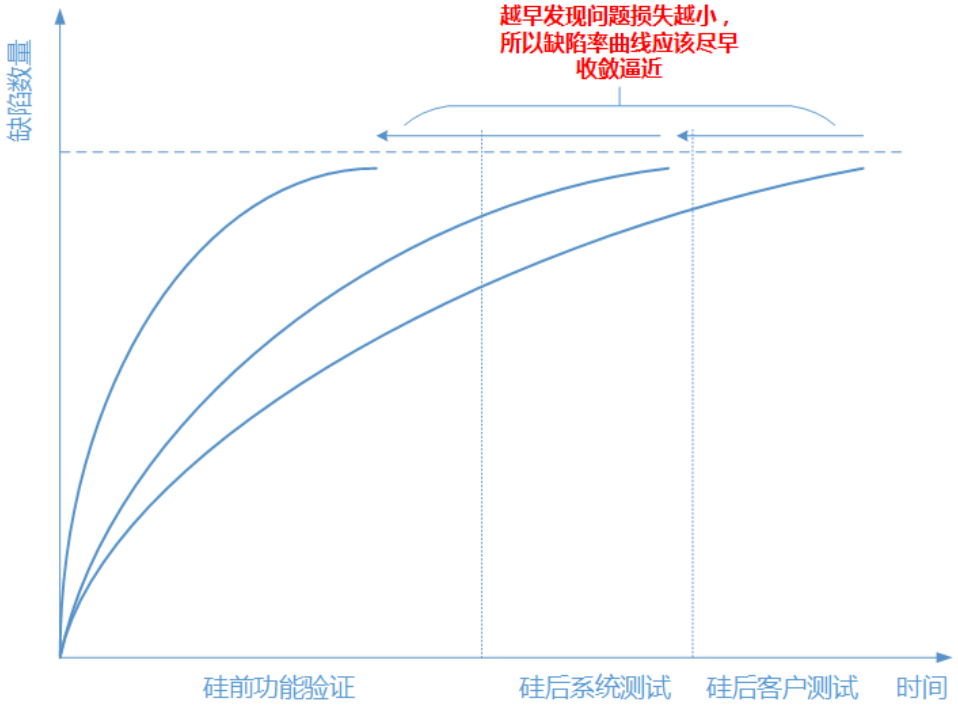
验证任务和目标
缺陷增长曲线

- 通过缺陷数量在时间线上的记录，我们可以绘制出缺陷数量的增长曲线。
- 一般来讲，缺陷数量的增长曲线是逐渐逼近趋于缓慢的。
- 功能验证需要保证的就是将缺陷数量的增值（至少是致命缺陷数量）保证在硅前阶段，不应该让其发生在硅后测试阶段。
- 针对缺陷的类型，我们一般会遵循先易后难的验证方法。



验证任务和目标
缺陷增长曲线（续）

- 我们给出的激励向量应该是先易后难，我们发现出的缺陷也应该是先基本后高级。
- 缺陷率的曲线是否在收敛，或者说斜率是否在变小，这一定程度上可以说明验证的状态是否在收敛和趋于完备。
- 需要注意验证过程中的发现的缺陷种类，应该是从基本缺陷再到高级缺陷。假如到了后期缺陷率尽管在收敛，然而却发现了基本缺陷，那么这时就应该对整个验证质量打一个问号了。



验证的环节

验证周期


模块功能详述
创建验证计划
验证计划回顾
开发验证环境
调试环境和HDL文件
验证代码检查
递归测试
流片前验证完备性检查
硅后系统测试
展开逃逸分析
吸取教训
路科验证
验证周期

Agenda

1. 芯片验证概述
2. **SystemVerilog概述**
3. 数据类型
4. 自定义类型
5. 字符串类型
6. 设计特性
7. 接口

SystemVerilog概述

发展历史

- SystemVerilog是IEEE 1364-2005 Verilog标准的扩展。
- 此扩展既包含了用来实现可综合设计的设计语言特性，也包含了用来对大型设计做验证的验证语言特性。
- 由此可以见得，SystemVerilog是用来建立一种新的硬件描述和验证语言（HDVL， Hardware Description and Verification Language）。
- SV旨于通过一种统一的语言来帮助工程师对大型复杂硬件系统进行建模，并且对其功能进行验证。

SystemVerilog概述

SV标准的诞生

- SV标准由Accellera标准组织牵头发展。
- Accellera是一家非盈利性的旨在于支持EDA（Electronic Design Automation）语言发展的组织。
- SystemVerilog 3.0于2002年6月由Accellera发布，该版本目的在于扩展Verilog的可综合语言特性，以及如何在更高层次上构建硬件模型。
- SystemVerilog 3.0的版本是在Verilog-1995和Verilog-2001基础上的第三个版本。
- SystemVerilog 3.1版本扩展了若干有关验证的特性，并且于2003年5月发布。
- 2004年5月，SystemVerilog 3.1a的版本作为IEEE标准的基础版本，并且在2005年由IEEE正式纳入标准，即IEEE 1800-2005。

SystemVerilog概述

SV的语言继承历史

- SUPERLOG的扩展综合语言子集（Co-Design Automation公司）。
- OpenVERA验证语言（Synopsys公司）。
- PSL断言（IBM公司）。
- OpenVERA断言（OVA，Synopsys公司）。
- DirectC和覆盖率应用编程接口（API，Synopsys公司）。
- 独立编译和$readmem扩展（Mentor公司）。
- 高层次语言特性（BlueSpec公司）。
- SV语言对于Verilog语言是完全向后兼容的。

SystemVerilog概述

核心语言特性

- 接口（interface）-V1
- 面向对象特性（class & package） -V1
- 约束随机（constrained random） -V1
- 线程控制和通信（process control and communication） -V1
- 功能覆盖率（function coverage） -V1
- 外部语言编程接口（Direct Programming Interface） -V2
- 断言（assertion） -V2

Agenda

1. 芯片验证概述
2. SystemVerilog概述
3. **数据类型**
4. 自定义类型
5. 字符串类型
6. 设计特性
7. 接口

数据类型

Verilog的数据类型

- Verilog语言提供两种基本的数据类型，即变量类型（variables）和线网类型（nets），这两种类型都是四值逻辑，即可表示0、1、X和Z值。
- 例如reg，integer和time等变量类型可以用来存储组合逻辑或者时序逻辑的值。
- 例如wire，wor，wand和其它线网类型用来连接硬件模块，以及提供特殊的线网逻辑解决方案。

数据类型

Verilog 线网类型

- 线网用于建模结构化描述中的连接线路和总线线网的值由线网的驱动器决定。
- 驱动器可以是门或模块的实例或者连续赋值的输出。

数据类型

Verilog 变量类型

- 变量类型保存在initial、always、task 和function 内赋的值。
- 变量只能用过程赋值来赋值。
- 类型是integer 或time 的变量一般像有相同数量位数的reg 一样运转。
- 在表达式中integer 的值作为有符号数处理，而reg 或time的值作为无符号数处理。
- 用reg 描述逻辑，integer 描述循环变量和计算，real 在系统模型中使用，time 和realtime 保存测试装置的仿真时间。

数据类型

SystemVerilog新添加的数据类型

- Verilog并没有严格区分信号的类型，变量和线网类型均是四值逻辑。
- SV添加了许多新的数据类型来帮助设计和验证工程师。
- 同时，SV将硬件信号区分为“类型”和“数据类型”。
- 类型即表示该信号为变量或者线网类型。
- 数据类型则表示该数据是四值逻辑（logic）还是二值逻辑（bit）。

数据类型

四值逻辑数据类型

- 在Verilog中，reg经常用来表示存储组合逻辑或者时序逻辑的变量，不过初学者容易混淆其综合后的硬件单元。实际上，reg并不一定会被综合为register，而它只是用原来与线网类型相对，表示存储数据的变量。
- 在SV中，我们可以直接使用logic（数据类型）来达到通用的存储硬件数据的目的：

```
logic resetN; // a 1-bit wide 4-state variable
logic [63:0] data; // a 64-bit wide variable
logic [0:7] array [0:255]; // an array of 8-bit variables
```

- logic虽然只是表示数据类型，而在声明时，它默认会表示变量类型（variable），用户也可以显示声明其类型：

```
var logic [63:0] addr; // a 64-bit wide variable
wire logic [63:0] data; // a 64-bit wide net
```

数据类型

二值逻辑数据类型

- SV添加了二值逻辑数据类型，用来对在RTL更高抽象级的模型建模，例如系统级或者事务级模型，这些二值逻辑数据类型包括：

```
bit : a 1-bit 2-state integer
byte : an 8-bit 2-state integer, similar to a C char
shortint : a 16-bit 2-state integer, similar to a C short
int : a 32-bit 2-state integer, similar to a C int
longint : a 64-bit 2-state integer, similar to a C longlong
```

- 在RTL级别，X值用来捕捉设计错误，例如寄存器未初始化，Z值用来表示未连接或者三态的设计逻辑,但是在系统级或者事务级，Z和X很少被使用。
- 类似于logic类型默认为变量类型，bit类型默认也为变量类型。

数据类型

有符号类型和无符号类型

- logic或者bit构成的向量（vector）是无符号类型。
- integer、byte、shortint、int、longint为有符号类型。
- 可以在有符号类型后添加unsigned来表示无符号类型，例如：
  byte有符号类型，表示的数值范围是[-128, 127]。
  byte unsigned表示无符号类型，等同于bit[7:0]，表示的数值范围是[0, 255]。
- 在构建验证环境总线功能模型（BFM，Bus-Functional Model）时，无需关注硬件底层逻辑（X或者Z值），所以可使用二值逻辑实现。
- 同样，SV在与C发生语言交互时，也可以使用二值逻辑来使得两种语言的边界数据传输更为简单。

数据类型

仿真行为

4-state Value	Converts To
0	0
1	1
Z	0
X	0

- 四值逻辑变量例如reg、logic或者integer等，在仿真开始时的初值为X。
- 二值逻辑变量例如bit等，在仿真开始时的初值为0。
- 如果四值逻辑与二值逻辑的数据类型之间发生的默认转换，那么Z和X值将转换为0。
- 二值逻辑也可以用来实现可综合电路，只是二值逻辑由于没有X和Z值，因此可能会出现仿真行为同综合电路结果不一致的情况。

数据类型

其它类型

- SV添加void类型来表示空类型，经常用在函数定义时表示不会返回数值，同C语言的void使用方法。
- SV添加shortreal表示32位单精度浮点类型，同C语言的float；而Verilog的real类型表示双精度浮点类型，同C语言的double。

Agenda

1. 芯片验证概述
2. SystemVerilog概述
3. 数据类型
4. **自定义类型**
5. 字符串类型
6. 设计特性
7. 接口

自定义类型

概述

- 通过用户自定义类型，以往Verilog的代码将可以通过更少的代码来表示更多的功能。
- 用户自定义类型使得代码自身的可读性更好。
- 通过typedef来创建用户自定义类型。
- 通过enum来创建枚举类型。
- 通过struct来创建结构体类型。

自定义类型

自定义类型

- Verilog语言无法提供方便的特性来使得用户扩展变量和线网类型。
- SV则提供了特性使得用户可以构建更高抽象层的数据类型。
- 就同C语言一样，用户可以利用已有的数据类型来定义新的数据类型，一旦定义了新的数据类型，则此类型的变量则可以声明。

```
typedef int unsigned uint;
...
uint a, b; // two variables of type uint
```

- 为了使代码更易读和维护，通常我们对于自定义的类型，都习惯添加“_t”的后缀用来表示它是一个自定义类型（type）。

自定义类型

枚举类型

- 枚举类型（enum）提供方法来描述抽象变量的合法值范围，其每一个值都需要提供一个用户自定义的名字。
- 下面的枚举类型RGB可以拥有red、green和blue的三个数值：

```
enum {red,green,blue} RGB;
```

- Verilog语言自身不提供枚举类型，因此为了提供类似于枚举类型可实现的便利，我们不得不采用parameter常量来表示可取的值范围，或者使用`define来定义各个合法值对应的宏名称。

自定义类型

枚举类型（续）

```
`define FETCH 3'h0
`define WRITE 3'h1
`define ADD   3'h2
`define SUB   3'h3
`define MULT  3'h4
`define DIV   3'h5
`define SHIFT 3'h6
`define NOP   3'h7

module controller (output reg        read, write,
                   input  wire [2:0] instruction,
                   input  wire       clock, resetN);

  parameter WAITE = 0,
            LOAD  = 1,
            STORE = 2;

  reg [1:0] State, NextState;
  always @(posedge clock, negedge resetN)
    if (!resetN) State <= WAITE;
    else         State <= NextState;

  always @(State) begin
    case (State)
      WAITE:  NextState = LOAD;
      LOAD:   NextState = STORE;
      STORE:  NextState = WAITE;
    endcase
  end

  always @(State, instruction) begin
    read = 0; write = 0;
    if (State == LOAD && instruction == `FETCH)
      read = 1;
    else if (State == STORE && instruction == `WRITE)
      write = 1;
  end
endmodule
```

自定义类型

枚举类型（续）

```
package chip_types;
  typedef enum {FETCH, WRITE, ADD, SUB,
                MULT, DIV, SHIFT, NOP  } instr_t;
endpackage

import chip_types::*;  // import package definitions into $unit

module controller (output logic   read, write,
                   input  instr_t instruction,
                   input  wire    clock, resetN);

  enum {WAITE, LOAD, STORE} State, NextState;

  always_ff @(posedge clock, negedge resetN)
    if (!resetN) State <= WAITE;
    else         State <= NextState;

  always_comb begin
    case (State)
      WAITE: NextState = LOAD;
      LOAD:  NextState = STORE;
      STORE: NextState = WAITE;
    endcase
  end

  always_comb begin
    read = 0; write = 0;
    if (State == LOAD && instruction == FETCH)
      read = 1;
    else if (State == STORE && instruction == WRITE)
      write = 1;
  end
endmodule
```

自定义类型

定义枚举值

- 默认的枚举类型是int，即32位的二值逻辑数据类型。
- 为了能够更准确地描述硬件，SV允许指明枚举类型的数据类型，例如：

```
enum bit {TRUE, FALSE} Boolean;
enum logic [1:0] {WAITE, LOAD, READY} state;
```

- 如果一个枚举类型数值被赋值，那么所赋的值必须符合其数据类型。

```
enum logic [2:0] {WAITE = 3'b001,
                  LOAD = 3'b010,
                  READY = 3'b100} state;
```

- 如果枚举类型是四值逻辑数据类型，那么对枚举值赋为X或者Z也是合法的。

```
enum logic {ON=1'b1, OFF=1'bz} out;
```

自定义类型

自定义枚举类型

- 枚举类型也可以声明为自定义类型，这就使得可以用同一个枚举类型来声明多个变量或者线网。

```
typedef enum {WAITE, LOAD, READY} states_t;
states_t state, next_state;
```

- 如果枚举类型并没有伴随着typedef，那么该枚举类型指的则是一个匿名枚举类型（anonymous enumerated type）。
- 多数的Verilog或者SV数据类型之间可以通过不同类型的赋值完成“隐式”的数据转换，所以Verilog/SV的数据类型转换是宽松的。

自定义类型

枚举类型赋值

- 枚举类型赋值时则相对严格，例如下面的例子中，赋值操作符“=”的左右两侧应该尽量为相同的枚举类型。

```
typedef enum {WAITE, LOAD, READY} states_t;
states_t state, next_state;
int foo;
state = next_state; // legal operation
foo = state + 1; // legal operation
state = foo + 1; // ERROR: illegal assignment
state = state + 1; // illegal operation
state++; // illegal operation
next_state += state; // illegal operation
```

自定义类型

结构体类型

- 设计或者验证的数据经常会有逻辑相关的数据信号组，例如一个总线协议的所有控制信号，或者在一个状态控制器中用到的所有的信号。
- Verilog语言没有方便的特性可以将相关的信号收集整理到一个信号组中。
- SV添加了同C一样的结构体struct，而结构体的成员可以是任何变量类型，包括自定义类型或者其它常量类型。

```
struct {
int a, b; // 32-bit variables
opcode_t opcode; // user-defined type
logic [23:0] address; // 24-bit variable
bit error; // 1-bit 2-state var.
} Instruction_Word;
```

自定义类型

结构体类型

- 正是由于结构体是变量的合集，因此结构体类型的变量也可以用来索引到其内部的变量，索引方式同C语言一致：

```
<structure_name>.<variable_name>

Instruction_Word.address = 32'hF000001E;
```

- 结构体类型默认也是变量类型，用户也可以显示声明其为var或者wire类型。

自定义类型

结构体类型

- 类似于枚举类型，结构体类型也可以伴随着typedef来实现自定义结构体类型。

```
typedef struct { // structure definition
  logic [31:0] a, b;
  logic [ 7:0] opcode;
  logic [23:0] address;
} instruction_word_t;
instruction_word_t IW; // structure allocation
```

自定义类型

结构体类型

- 结构体变量可以通过索引其各个成员做依次的成员赋值:

```
always @(posedge clock, negedge resetN)
  if (!resetN) begin
    IW.a = 100; // reference structure member
    IW.b = 5;
    IW.opcode = 8'hFF;
    IW.address = 0;
  end
  else begin
    ...
  end
```

- 也可以通过分号'和花括号{ }来实现整体赋初值:

```
IW = '{100, 3, 8'hFF, 0};
IW = '{address:0, opcode:8'hFF, a:100, b:5};
```

Agenda

1. 芯片验证概述
2. SystemVerilog概述
3. 数据类型
4. 自定义类型
5. **字符串类型**
6. 设计特性
7. 接口

字符串类型

概述

- Verilog语言对于字符串的处理手段非常有限。
- SV引入了string类型用来容纳可变长度的字符串。
- 字符串类型变量的存储单元为byte类型。
- 字符串类型变量长度为N时，其字符成员索引值为从0到N-1.
- 不同于C函数，字符串结尾没有“空字符”即null字符“\0”。
- 字符串的内存是动态分配的，用户无需担心内存空间管理。

字符串类型

常见使用方式

```
typedef logic [15:0] r_t;
r_t r;
integer i = 1;
string b = "";
string a = {"Hi", b};
r = r_t'(a); // OK
b = string'(r); // OK
b = "Hi"; // OK
b = {5{"Hi"}}; // OK
a = {i{"Hi"}}; // OK (non-constant replication)
a = {i{b}}; // OK
a = {a,b}; // OK
a = {"Hi",b}; // OK
b = {"H",""}; // yields "H". "" is the empty string
a[0] = "h"; // operation on single character
```

字符串类型

字符串内建方法

- str.len()：返回字符串的长度。
- str.putc(i, c)：将第i个字符替换为字符c，等同于str[i]=c。
- str.getc(i)：返回第i个字符。
- str.substr(i, j)：将从第i个字符到第j个字符的字符串返回。
- str.{atoi(), atohex(), atooct, atobin}：将字符串转变为十进制、十六进制、八进制或者二进制数据。

```
str = "123";
int i = str.atoi(); // assigns 123 to i.
```

Agenda

1. 芯片验证概述
2. SystemVerilog概述
3. 数据类型
4. 自定义类型
5. 字符串类型
6. **设计特性**
7. 接口

设计特性

面向可综合设计的主要优化

- 添加接口（interface）从而将通信和协议检查进一步封装。
- 添加类似C语言的数据类型，例如int、byte。
- 添加用户自定义类型，枚举类型，结构体类型。
- 添加类型转换（$cast(T, S)或者’()）。
- 添加包（package）从而使得多个设计之间可以共享公共类型和方法。
- 添加方便的赋值操作符和运算操作符，例如++、+=、===。
- 添加priority和unique case语句。
- 添加always_comb、always_latch和always_ff等过程语句块。

设计特性

过程语句块新特性

- SV添加了新的面向硬件的过程语句块，从而使得该语句块可以更清楚地表达设计者的意图。
- always语句块被细分为了：
  - 组合逻辑语句块 always_comb
  - 锁存逻辑语句块 always_latch
  - 时序逻辑语句块 always_ff
- 通过以上细分的方式，降低了Verilog always语句块容易混淆的方式，也使得仿真工具、综合工具、形式检查工具、语义检查工具和其它EDA软件工具可以更准确地检查设计的实现意图。

设计特性

过程语句块 always_comb

- always_comb可以自动嵌入敏感列表。
- always_comb可以禁止共享变量，即赋值左侧的变量无法被另一个过程块所赋值。
- 软件工具会检查该过程块，如果其所表示的不是组合逻辑，那么会发出警告。

```
always @(a, en) if (en) y = a; // Verilog
=>
 always_comb if (en) y = a; // SystemVerilog
```

- always_comb在仿真0时刻会自动触发一次，无论在0时刻是否有敏感信号列表中的信号发生变化。

设计特性

过程语句块 always_comb

- Verilog @*的敏感列表声明方式不同于always_comb：
  - @*不要求可综合的建模要求，但always_comb则会限制其他过程块对同一变量进行赋值。
  - @*的敏感列表可能不完全，例如如果一个过程块调用一个函数，那么@*则只会将该函数的形式参数自动声明到敏感列表，而不会将该函数展开。
  - always_comb则将被调用函数中可能参与运算的其它信号也声明到敏感列表中。

设计特性

过程语句块 always_comb

- Verilog @*的敏感列表声明方式不同于always_comb：

```
always @* begin
  a1 = data << 1;
  b1 = decode();
  ...
end
always_comb begin
  a2 = data << 1;
  b2 = decode();
  ...
end

function decode;  // function with no inputs
  begin
    case (sel)
      2'b01:   decode = d | e;
      2'b10:   decode = d & e;
      default: decode = c;
    endcase
  end
endfunction
```

Infers @(data)

Infers
@(data, sel, c, d, e)

设计特性

过程语句块 always_latch

- always_latch表示锁存逻辑，且自动插入敏感列表。

```
always_latch
if (enable) q <= d;
```

- EDA 工具会检查always_latch过程块是否真正实现了锁存逻辑。

```
module register_reader (input clk, ready, resetN,
                        output logic [4:0] read_pointer);

  logic enable;    // internal enable signal for the counter
  logic overflow;  // internal counter overflow flag

  always_latch begin  // latch the ready input
    if (!resetN)
      enable <= 0;
    else if (ready)
      enable <= 1;
    else if (overflow)
      enable <= 0;
  end

  always @(posedge clk, negedge resetN) begin // 5-bit counter
    if (!resetN)
      {overflow,read_pointer} <= 0;
    else if (enable)
      {overflow,read_pointer} <= read_pointer + 1;
  end
endmodule
```

设计特性

过程语句块 always_ff

- always_ff用来表示时序逻辑。

```
always_ff @(posedge clock, negedge resetN)
if (!resetN) q <= 0;
else q <= d;
```

- 敏感列表必须指明posedge或者negedge（综合要求），从而使得EDA工具实现同步或者异步的复位逻辑。
- EDA工具也会验明always_ff过程块语句是否实现了时序逻辑。

设计特性

赋值操作符

Operator	Description
+=	add right-hand side to left-hand side and assign
-=	subtract right-hand side from left-hand side and assign
*=	multiply left-hand side by right-hand side and assign
/=	divide left-hand side by right-hand side and assign
%=	divide left-hand side by right-hand side and assign the remainder
&=	bitwise AND right-hand side with left-hand side and assign
|=	bitwise OR right-hand side with left-hand side and assign
^=	bitwise exclusive OR right-hand side with left-hand side and assign
<<=	bitwise left-shift the left-hand side by the number of times indicated by the right-hand side and assign
>>=	bitwise right-shift the left-hand side by the number of times indicated by the right-hand side and assign
<<<=	arithmetic left-shift the left-hand side by the number of times indicated by the right-hand side and assign
>>>=	arithmetic right-shift the left-hand side by the number of times indicated by the right-hand side and assign

设计特性

赋值操作符

- Verilog没有简单的方法可以对向量填充1。

```
parameter N = 64;
reg [N-1:0] data_bus;
data_bus = 64'hFFFFFFFFFFFFFFFF; //set all bits of data_bus to 1
```

- SV可以通过'0，'1，'z和'x来分别填充0，1，z和x。
- 通过这种方法，代码会根据向量的宽度自动填充，这提高了代码的便捷性和复用性。

设计特性

赋值操作符

- SV在比较数据时，可以通过==?来进行通配比较。
- 在比较操作符的右侧操作数，如果在某些位置有X或者Z，那么它表示的是在该位置上会与左侧操作数的相同位置的任何值相匹配。

```
logic [7:0] opcode;
if (opcode ==? 8'b11011???) // mask out low bits
  ...
```

a	b	a == b	a === b	a ==? b	a != b	a !== b	a !=? b
0000	0000	true	true	true	false	false	false
0000	0101	false	false	false	true	true	true
010Z	0101	unknown	false	unknown	unknown	false	unknown
010Z	010Z	unknown	true	true	unknown	false	false
010X	010Z	unknown	false	true	unknown	true	false
010X	010X	unknown	true	true	unknown	false	false

设计特性

赋值操作符

- SV添加了inside操作符，用来检查数值是否在一系列值的集合当中。

```
logic [2:0] a;
if ( a inside {3'b001, 3'b010, 3'b100} )
  ...
```

- 下面的if语句中的多个条件判断可以变得更为简化：

```
logic [2:0] a;
if ( (a==3'b001) || (a==3'b010) || (a==3'b100) )...
=>
if ( a inside {3'b001, 3'b010, 3'b100} )...
```

设计特性

增强的case语句

- 仿真和综合可以会将case语句做不同的翻译。Verilog定义case语句在执行时按照优先级，而综合编译器则会优化case语句中多余的逻辑。
- 为了保持仿真与综合的一致性，SV提供了unique和priority的声明，结合case，casex和casez来进一步实现case对应的硬件电路。

```
unique case (<case_expression>)
  ... // case items
endcase
priority case (<case_expression>)
  ... // case items
endcase
```

设计特性

增强的case语句

- unique case要求每次case选择必须只能满足一条case选项。
- unique case不同有重叠的选项，即多个满足条件的选项。

```
logic [2:0] request;
always_comb // compilation error below with unique case
  unique casez (request) // should only make one grant a time
    3'b1??: slave1_grant = 1;
    3'b?1?: slave2_grant = 1;
    3'b??1: slave3_grant = 1;
  endcase
```

设计特性

增强的case语句

- unique case可以并行执行，并且case选项必须完备。

```
logic [2:0] opcode; // 3-bit wide vector
always_comb // run-time warning due to incomplete case items
  unique case (opcode)
    3'b000: y = a + b;
    3'b001: y = a - b;
    3'b010: y = a * b;
    3'b100: y = a / b;
  endcase
```

设计特性

增强的case语句

- priority case则表示必须至少有一个case选项满足要求。
- 如果有多个case选项满足时，第一个满足的分支将会被执行。
- priority case的逻辑同if..else的逻辑一致。
- unique和priority的声明也可以结合if..else条件语句使用。
- unique和priority使得仿真行为同设计者希望实现的综合电路保持一致。

Agenda

1. 芯片验证概述
2. SystemVerilog概述
3. 数据类型
4. 自定义类型
5. 字符串类型
6. 设计特性
7. **接口**

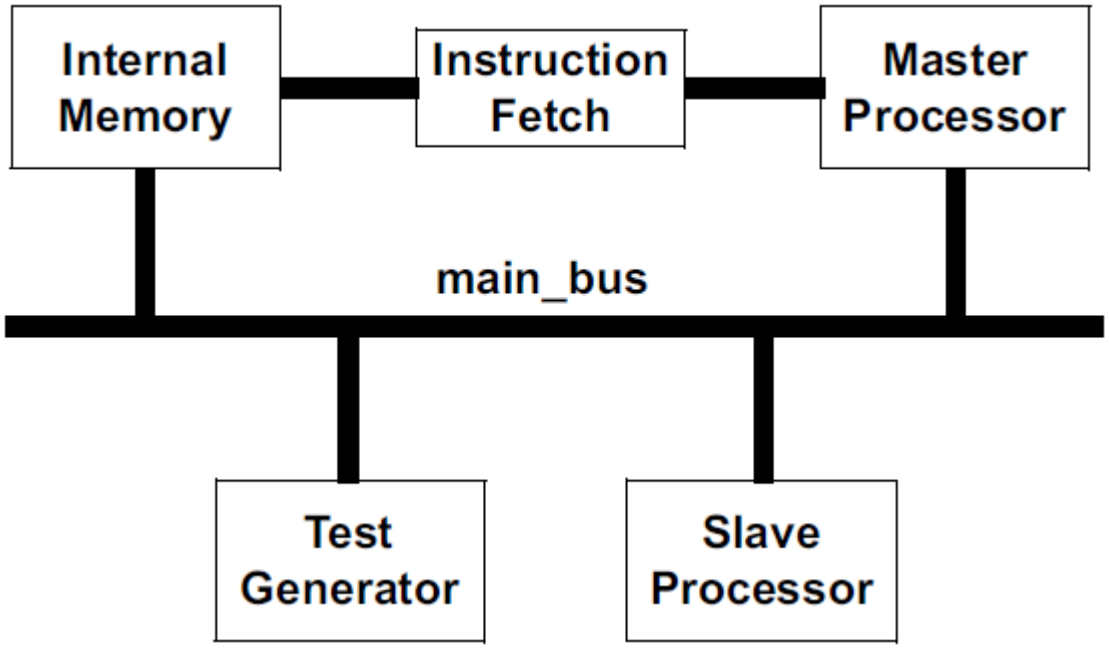
接口

概述

- SV在Verilog语言基础上扩展了接口（interface）。
- 接口提供了一种新型的对抽象级建模的方式。
- 接口的使用可以简化建模和验证大型复杂的设计。
- Verilog是通过模块之间进行端口连接来完成模块间通信的。
- 对于大型的设计，通过端口进行连接将会让硬件集成变得非常乏味和容易出错。

接口

概述



- 对于以上的连接，按照Verilog的方式，我们将按照以下步骤进行
  - 对每一个子模块进行端口声明。
  - 在上层环境，需要声明非常多的网线。
  - 将上层环境的网线在各个模块之间进行连接。
- 这种方式使得一些常用总线端口也不得不在多个模块重复声明。
- 一些通信协议也不得不在多个模块中重复定义。
- 在不同模块之间的连接可能会出现不匹配的信号声明和连接。
- 一个设计发生了变化，可能会影响多个模块的端口声明和连接。

接口
概述

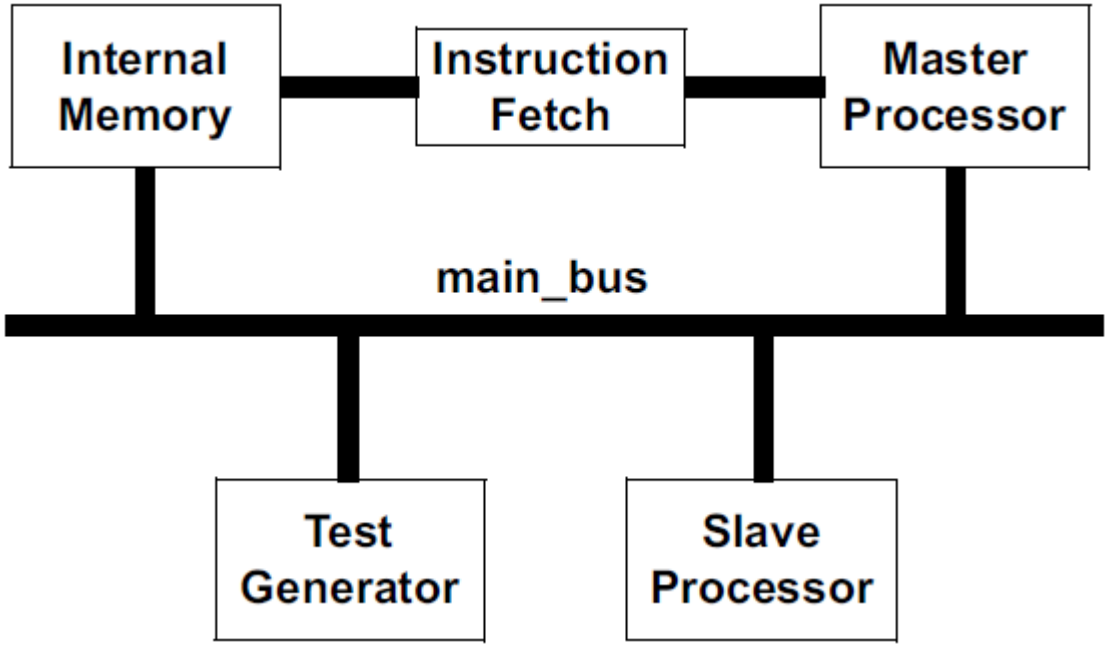
```
/********************** Top-level Netlist ********************/
module top (input wire clock, resetN, test_mode);
  wire [15:0] data, address, program_address, jump_address;
  wire [ 7:0] instruction, next_instruction;
  wire [ 3:0] slave_instruction;
  wire        slave_request, slave_ready;
  wire        bus_request, bus_grant;
  wire        mem_read, mem_write;
  wire        data_ready;

  processor proc1 (
    // main_bus ports
    .data(data),
    .address(address),
    .slave_instruction(slave_instruction),
    .slave_request(slave_request),
    .bus_grant(bus_grant),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .bus_request(bus_request),
    .slave_ready(slave_ready),
    // other ports
    .jump_address(jump_address),
    .instruction(instruction),
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );
```

signals for main_bus must be individually connected to each module instance

接口

接口的优势



- SV添加了新的抽象端口类型interface。
- interface允许多个信号被整合到一起用来表示一个单一的抽象端口。
- 多个模块因此可以使用同一个interface，继而避免分散的多个端口信号连接。

接口

接口的优势

```
/******************* Interface Definitions *******************/
interface main_bus;
  wire  [15:0] data;
  wire  [15:0] address;
  logic [ 7:0] slave_instruction;
  logic        slave_request;
  logic        bus_grant;
  logic        bus_request;
  logic        slave_ready;
  logic        data_ready;
  logic        mem_read;
  logic        mem_write;
endinterface
/********************* Top-level Netlist *******************/
module top (input logic clock, resetN, test_mode);
  logic [15:0] program_address, jump_address;
  logic [ 7:0] instruction, next_instruction;

  main_bus bus ( );  // instance of an interface
                     // (instance name is bus)

  processor proc1 (
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
    .jump_address(jump_address),
    .instruction(instruction),
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );
```

signals for main_bus are defined in just one place

each module instance has a single connection for main_bus

接口

接口的内容

- 接口不单单可以包含变量或者线网，它还可以封装模块之间通信的协议。
- 此外接口中还可以嵌入与协议有关的断言检查、功能覆盖率收集等模块。
- 接口不同于模块（module）的地方在于，接口不允许包含设计层次，即接口无法例化module，但是接口可以例化接口。
- 接口中可以进一步声明modport来约束不同模块连接时的信号方向。

接口

接口的声明

- 接口的定义同模块定义类似。
- 接口也可以有端口，例如外部接入的时钟或者复位信号。
- 接口内部可以声明所有的变量或者线网类型。

```
/******************** Interface Definitions ********************/
interface main_bus (input logic clock, resetN, test_mode);
  wire  [15:0] data;
  wire  [15:0] address;
  logic [ 7:0] slave_instruction;
  logic        slave_request;
  logic        bus_grant;
  logic        bus_request;
  logic        slave_ready;
  logic        data_ready;
  logic        mem_read;
  logic        mem_write;
endinterface

/********************** Top-level Netlist ********************/
module top (input logic clock, resetN, test_mode);
  logic [15:0] program_address, jump_address;
  logic [ 7:0] instruction, next_instruction;

  main_bus bus (    // instance of an interface
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );
```

discrete signals are inputs to the interface

discrete signals are connected to the interface instance

接口

接口的例化

- 接口的例化方式同模块例化一致。
- 模块的端口如果声明为input、output或者inout，那么在例化时可以不连接。
- 模块的端口如果声明为interface，那么在例化时则必须连接到一个接口实例，或者另外一个接口端口。
- 如果一个模块拥有一个接口类型端口，那么要索引该接口中的信号，需要通过以下方式进行：

```
<port_name>.<internal_interface_signal_name>
always @(posedge bus.clock, negedge bus.resetN)
   ...
```

接口

接口的例化

```
module slave (
    // main_bus interface port
    main_bus bus
    // other ports
  );
  // internal signals
  logic [15:0] slave_data, slave_address;
  logic [15:0] operand_A, operand_B;
  logic        mem_select, read, write;

  assign bus.address = mem_select? slave_address: 'z;
  assign bus.data = bus.slave_ready? slave_data: 'z;

  enum logic [4:0] {RESET    = 5'b00001,
                    START    = 5'b00010,
                    REQ_DATA = 5'b00100,
                    EXECUTE  = 5'b01000,
                    DONE     = 5'b10000} State, NextState;

  always_ff @(posedge bus.clock, negedge bus.resetN) begin: FSM
    if (!bus.resetN) State <= START;
    else             State <= NextState;
  end

  always_comb begin : FSM_decode
    unique case (State)
      START:    if (!bus.slave_request) begin
                  bus.bus_request = 0;
                  NextState = State;
                end
                else begin
                  operand_A       = bus.data;
                  slave_address   = bus.address;
                  bus.bus_request = 1;
                  NextState       = REQ_DATA;
                end
      ...  // decode other states
    endcase
  end: FSM_decode
endmodule
```

接口

modport

- 接口中的变量或者线网信号，对于连接到该接口的不同模块则可能具备这不同的连接方向。
- 接口引入了modport来作为module port的缩写，表示不同的模块看到同一组信号时的视角（连接方向）。
- 在接口中声明modport，需要指明modport中各个信号的方向。

接口
modport

- 接口中的变量或者线网信号，对于连接到该接口的不同模块则可能具备着不同的连接方向。
- 接口引入了modport来作为module port的缩写，表示不同的模块看到同一组信号时的视角（连接方向）。
- 在接口中声明modport，需要指明modport中各个信号的方向。

```
interface chip_bus (input logic clock, resetN);
  logic interrupt_request, grant, ready;
  logic [31:0] address;
  wire  [63:0] data;

  modport master (input  interrupt_request,
                  input  address,
                  output grant, ready,
                  inout  data,
                  input  clock, resetN);
  modport slave  (output interrupt_request,
                  output address,
                  input  grant, ready,
                  inout  data,
                  input  clock, resetN);
endinterface
```

接口
modport

```
interface chip_bus (input logic clock, resetN);
  modport master (...);
  modport slave  (...);
endinterface

module primary   (interface  pins);  // generic interface port
  ...
endmodule

module secondary (chip_bus pins);    // specific interface port
  ...
endmodule

module chip (input logic clock, resetN);

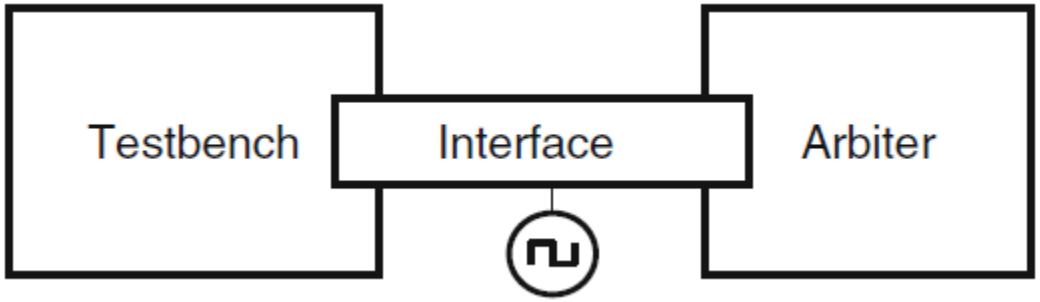
  chip_bus  bus (clock, resetN);  // instance of an interface
  primary   i1 (bus.master);  // use the master modport view
  secondary i2 (bus.slave);   // use the slave modport view

endmodule
```

- 当一个模块在例化时，可以选择连接到interface端口中具体的某一个modport。
- 这种方式可以降低方向连接错误的可能，进而避免信号多驱动的情况。

接口

验证中的应用



- 利用接口，也可以将测试平台同DUT连接在一起。

```
module top;
  bit  clk;
  always #50 clk = ~clk;

  arb_if arbif(clk);
  arb_with_port a1 (.grant   (arbif.grant), // .port (ifc.signal)
                    .request (arbif.request),
                    .rst     (arbif.rst),
                    .clk     (arbif.clk));
  test_with_ifc t1(arbif);    // From Sample 4-6
endmodule : top
```

接口
总结

- 接口对于设计复用非常有利。
- 接口减少了模块之间错误连接的可能性。
- 如果要添加新的信号，只需要在接口中声明，而不必在模块中声明。
- 由于接口将有关信号都集合在一起，因此在使用这些信号时需要多添加一个层次（接口实例名）。
- 接口往往会将有关的信号集合在一起，这意味着对于拥有多组不相关信号的设计而言，它可能需要有多个接口实例才能完成与其它模块的连接。

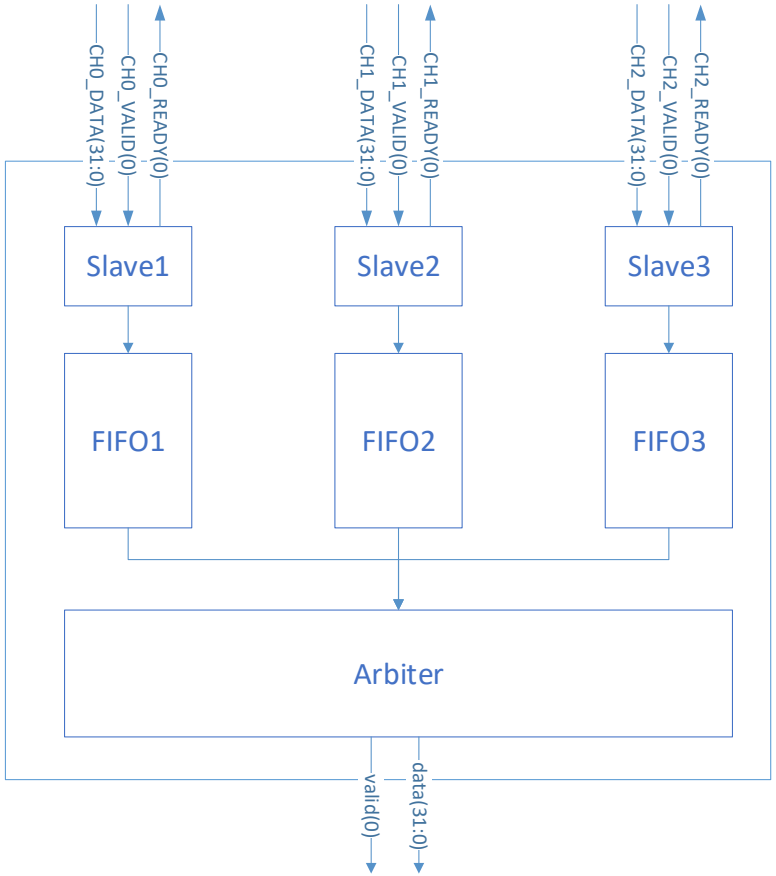
Agenda

1. MCDT设计介绍

2. Questasim基本使用

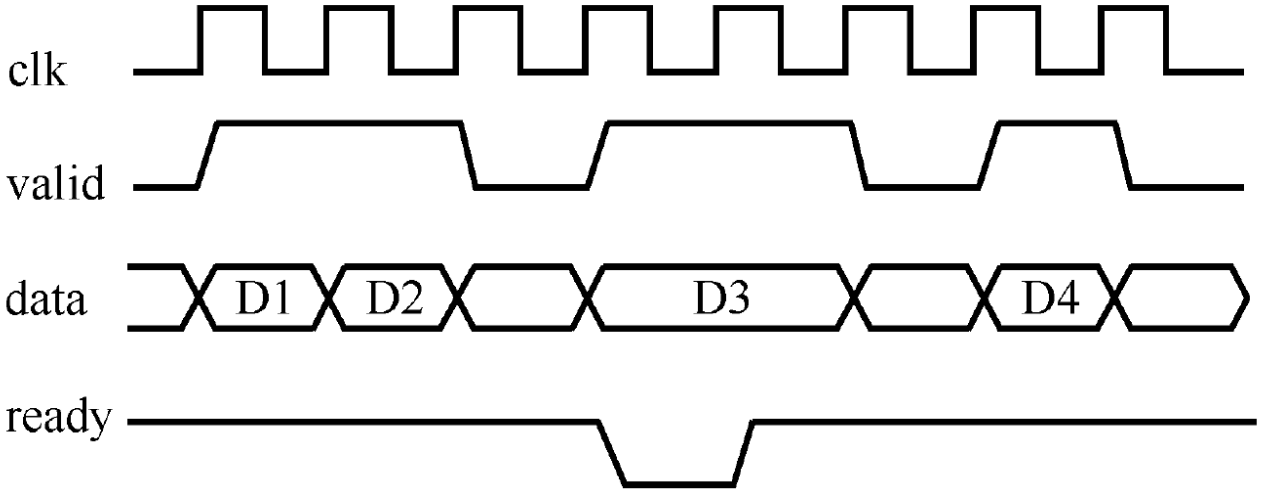
MCDT设计介绍

结构

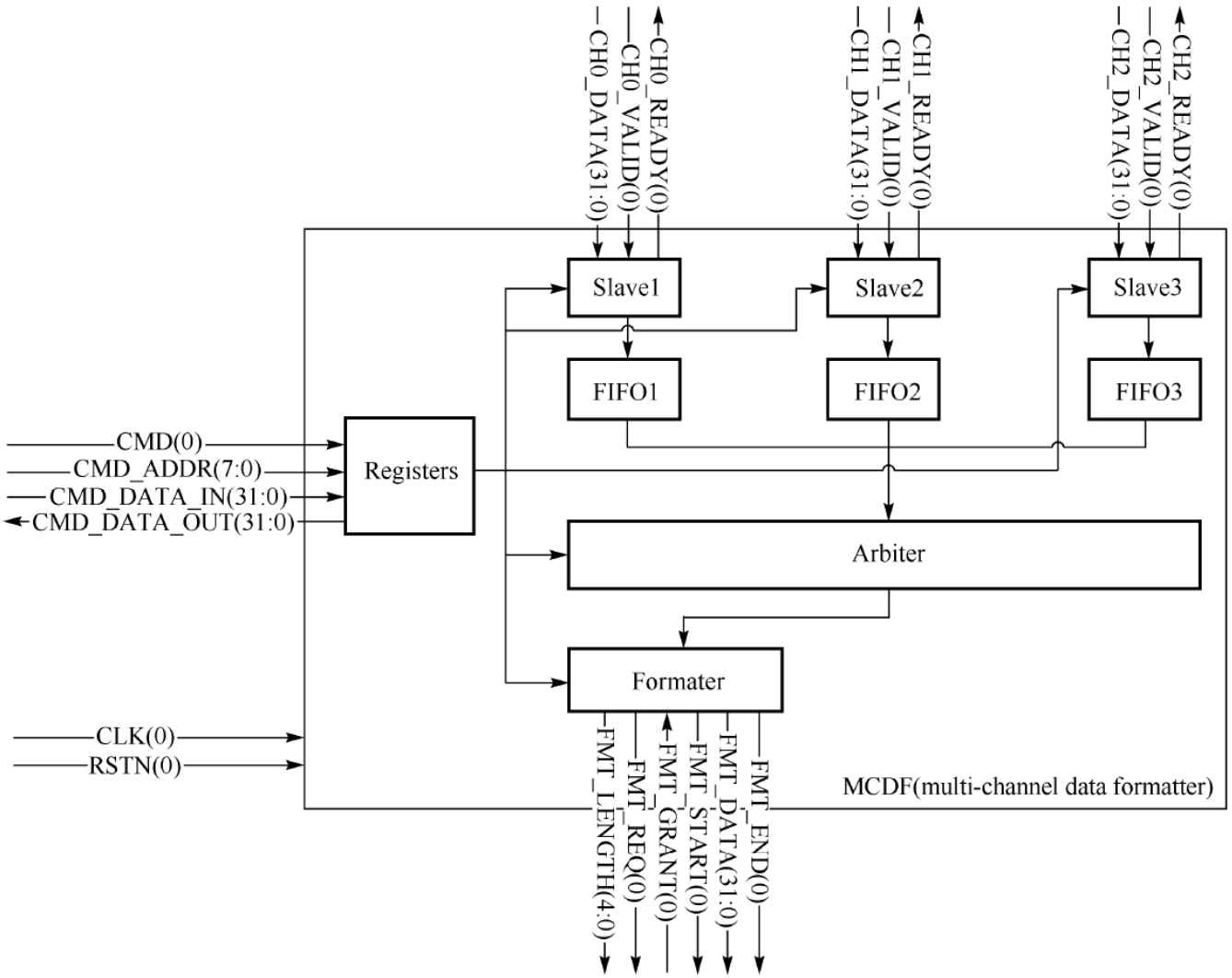

CH0_DATA(31:0)
CH0_VALID(0)
CH0_READY(0)
CH1_DATA(31:0)
CH1_VALID(0)
CH1_READY(0)
CH2_DATA(31:0)
CH2_VALID(0)
CH2_READY(0)
Slave1
Slave2
Slave3
FIFO1
FIFO2
FIFO3
Arbiter
valid(0)
data(31:0)

MCDT设计介绍
接口及时序

- **通道从端接口**
- CHx_DATA(31:0)：通道数据输入。
- CHx_VALID(0)：通道数据有效标志信号，高位有效。
- CHx_READY(0)：通道数据接收信号，高位表示接收成功。



MCDF设计结构（V2模块）



Agenda

1. MCDT设计介绍

2. Questasim基本使用

Agenda

1. **验证环境结构**
2. 硬件设计描述
3. 验证环境组件
4. 数组
5. 类的封装
6. 类的继承
7. 包的使用

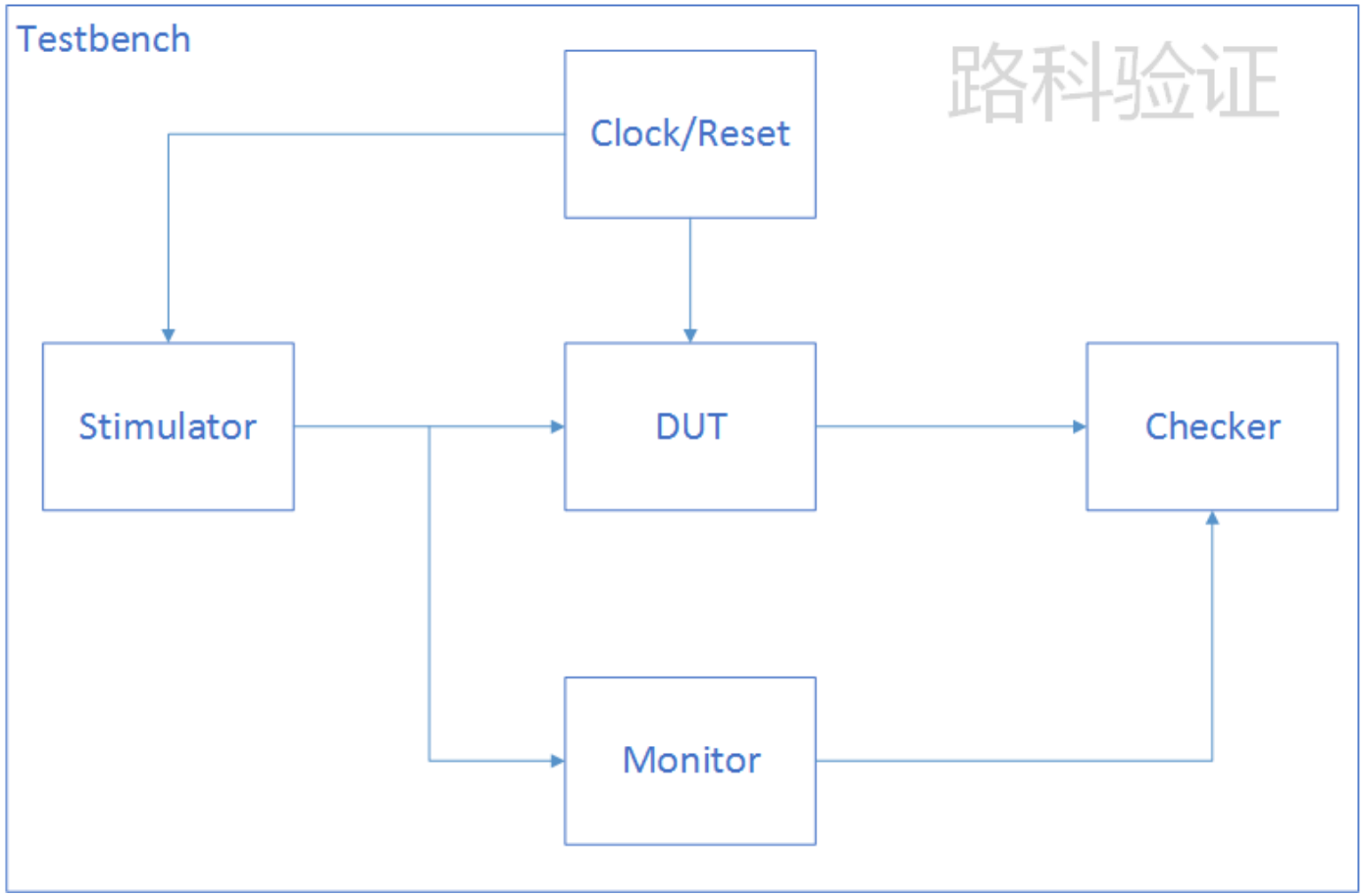
验证环境结构

概述

- **测试平台（testbench）**是整个验证系统的总称。
- 它包括验证结构中的**各个组件、组件之间的连接关系、测试平台的配置和控制。**
- 从更系统的意义来讲，它还包括**编译仿真**的流程、**结果分析**报告和**覆盖率**检查等。
- 从狭义上讲，我们主要关注验证平台的**结构和组件**部分，他们可以**产生**设计所需要的各种**输入**，也会在此基础上进行**设计功能的检查。**

验证环境结构

测试平台结构图



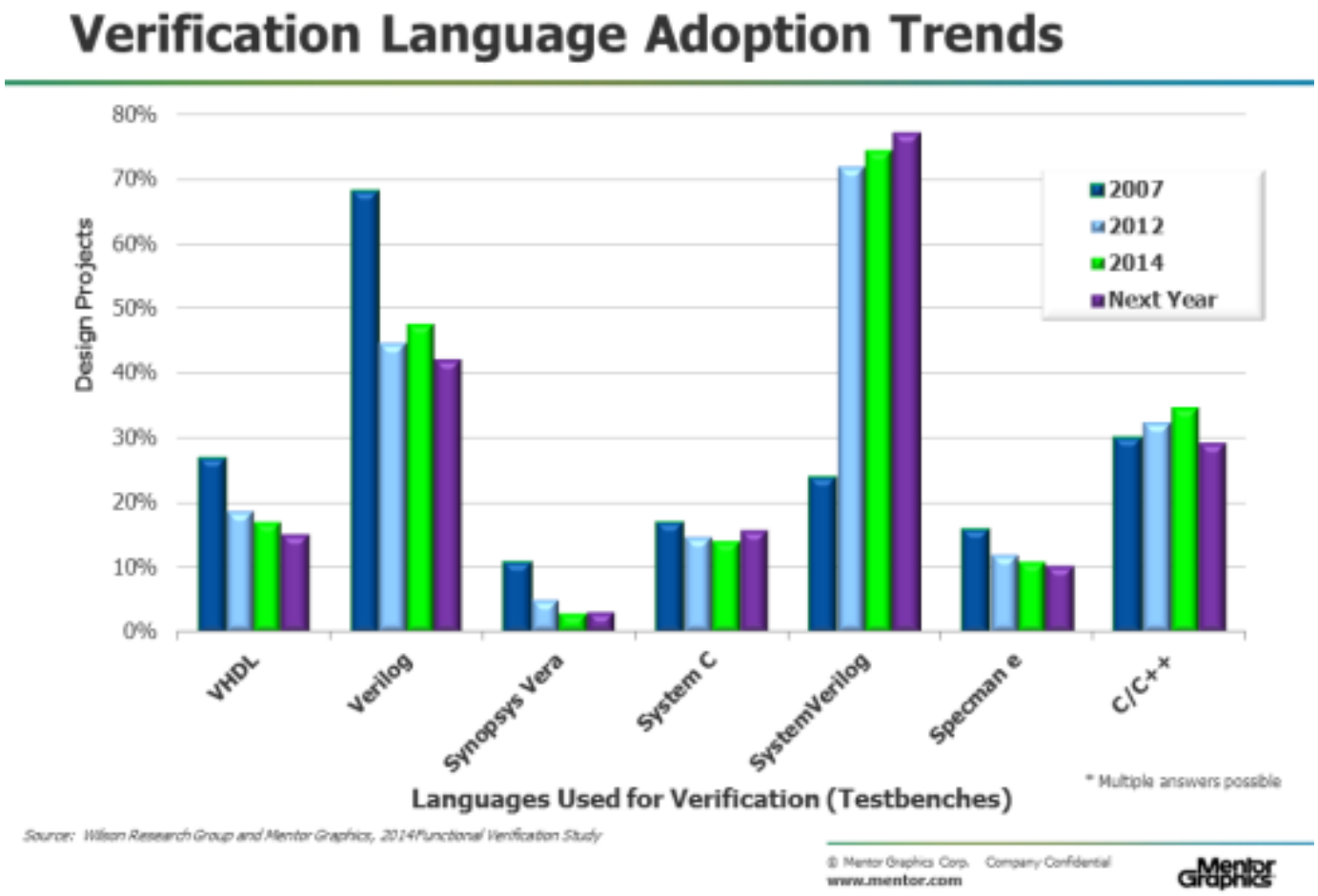
验证环境结构

测试平台结构图（续）

- 各个组件之间是相互**独立**的
- 验证组件与设计之间需要**连接**
- 验证组件之间也需要进行**通信**
- 验证环境也需要时钟和复位信号的**驱动**

验证环境结构

验证语言应用趋势



验证环境结构

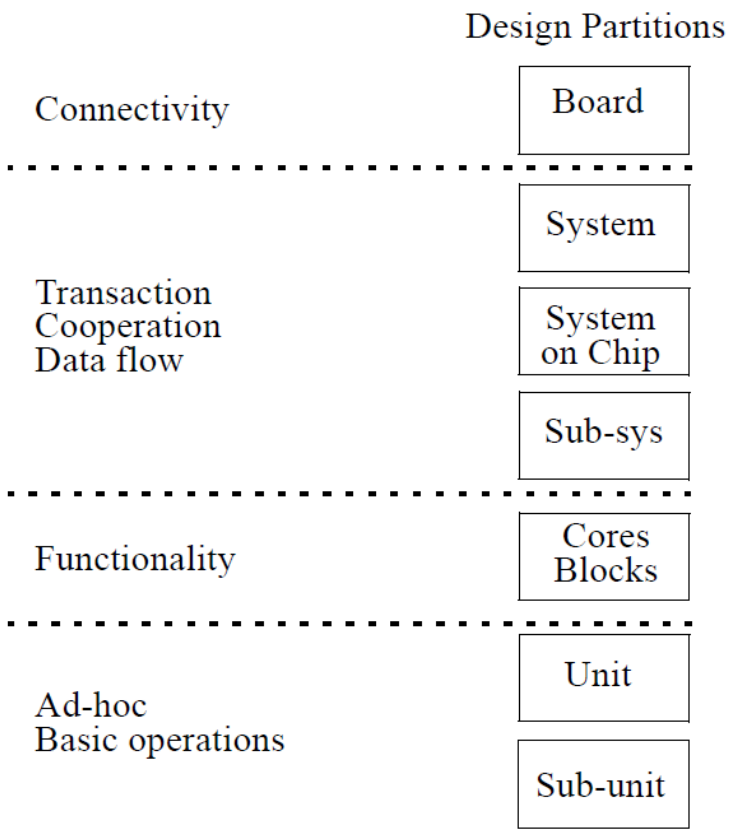
验证语言应用趋势（续）

- 近年来**SystemVerilog**的使用比例明显占据主导地位。
- **SystemC**和**C/C++**在验证部分也有其应用空间。
- 借助于语言的统一趋势，方法学（**UVM**）的统一也已经完成。
- 验证初学者赶上了一个好时代，一个不再纷乱繁杂的**统一时代**。

验证环境结构

验证结构的层次

- 设计是由多个层次构成的，无论是物理分区例如FPGA/ASIC，还是逻辑分区例如合成单元/核心子系统。
- 验证也可以按照不同的级别来安排目标。
- 每个验证级别都有最合适的验证目标。



验证环境结构

验证结构的层次

- 较小的模块更容易验证，因为它们提供更大的可控性和可观察性。
- 对于它们，很容易设置条件和状态组合，并观察其反应是否符合预期。
- 由小模块组成的子系统，则需要以较低的可控性和可观察性为代价去验证。
- 也因此，不同级别的验证有着不同的验证重心。

验证环境结构

验证结构的层次

- 任何层次的待验设计，都应该具备相对稳定的接口和预期的功能。
- 理想情况下，每个子系统或者模块，都应该有自己的硬件描述文档。
- 如果接口或者功能不断变化，那么测试平台也将一同发生变化，这毫无疑问将严重影响验证的进度。
- 对设计稳定性的要求，一般是先期待其接口稳定下来，再使得功能可以稳定下来。

Agenda

1. 验证环境结构
2. **硬件设计描述**
3. 验证环境组件
4. 数组
5. 类的封装
6. 类的继承
7. 包的使用

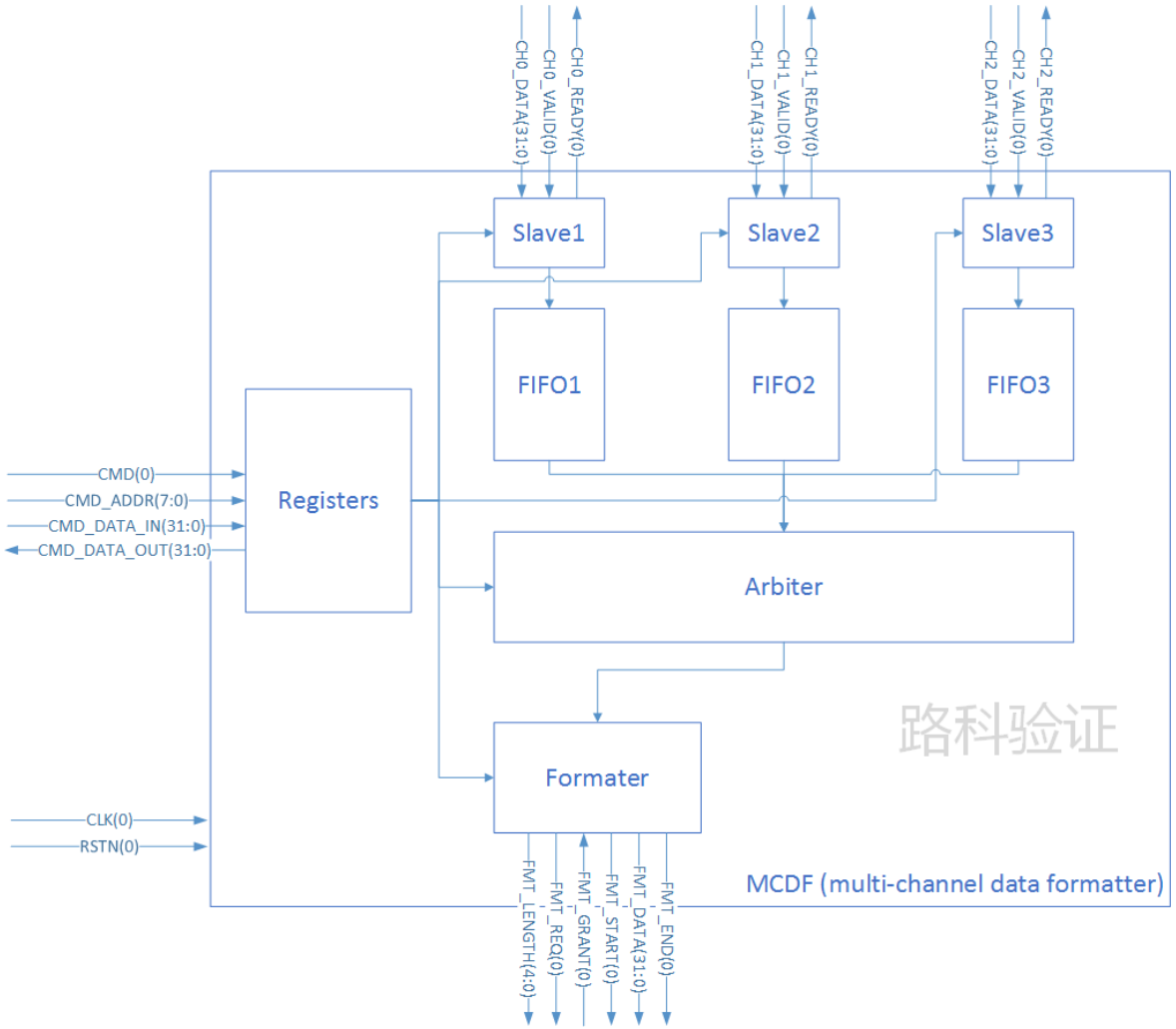
硬件设计描述

MCDF简介

- 遵循硬件设计描述的方式，介绍它的**结构、功能、寄存器和时序。**
- **熟悉硬件描述**的方式，也是进入验证领域的一项基本技能。
- 多通道数据整形器（MCDF，multi-channel data formatter），它可以**将上行（uplink）多个通道数据经过内部的FIFO，最终以数据包（data packet）的形式送出。**
- **上行**数据和**下行**数据的接口协议不同。
- 多通道数据整形器也有**寄存器的读写接口**，可以支持更多的控制功能。

硬件设计描述

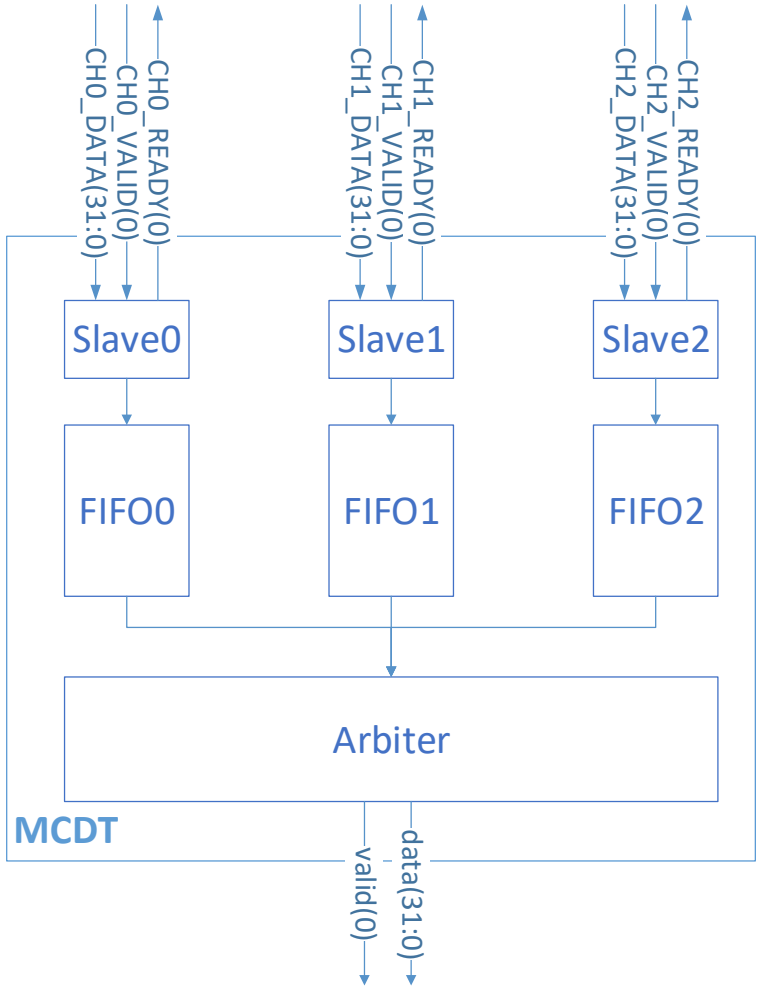
MCDF设计结构



硬件设计描述

MCDT结构（MCDF mini版）

- 只保留了slave channel和arbiter。
- 结构更为简单，目的在于大家能够着眼于更为简单的验证环境搭建。



硬件设计描述

MCDT接口描述

系统信号接口

- CLK(0)：时钟信号。
- RSTN(0)：复位信号，低位有效。

通道从端接口

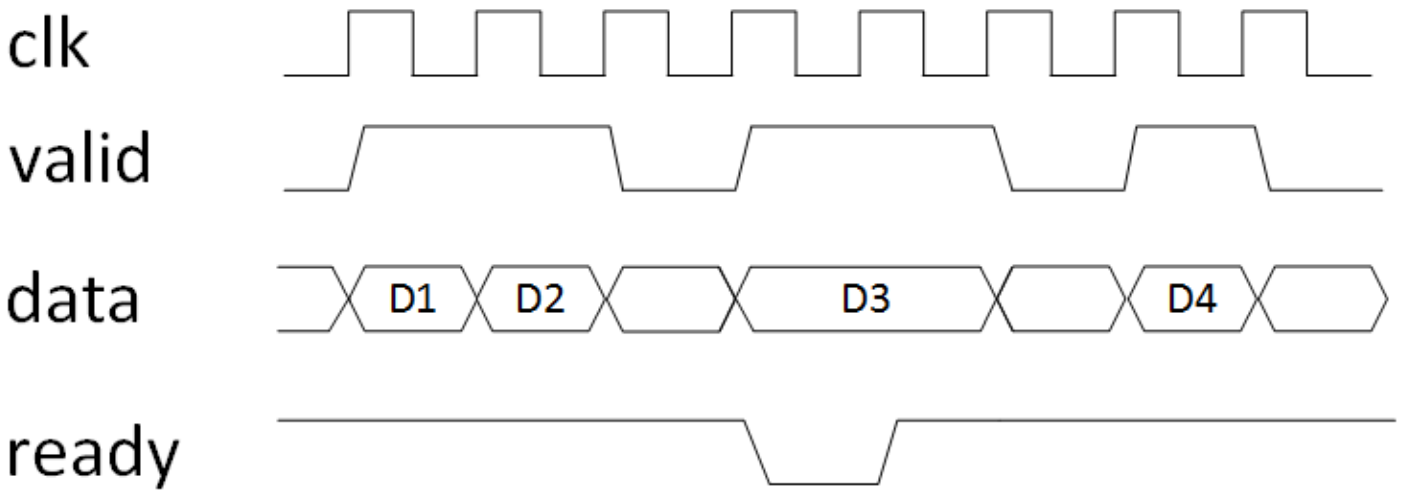
- CHx_DATA(31:0)：通道数据输入。
- CHx_VALID(0)：通道数据有效标志信号，高位有效。
- CHx_READY(0)：通道数据接收信号，高位表示接收成功。

硬件设计描述

MCDT接口时序

通道从端接口时序

路科验证



Agenda

1. 验证环境结构
2. 硬件设计描述
3. **验证环境组件**
4. 数组
5. 类的封装
6. 类的继承
7. 包的使用

验证环境组件

检查响应的策略

- 如何发送激励要相对容易一些，因为你完全可以控制激励的时序和内容。
- 然而要检查设计的响应则变得困难，因为你需要计划如何去获取这些响应以及如何去验证它们。
- 我们应当尽可能地去实现一些可以自检（self-checking）的测试平台。
- 然而有时响应无法被立即监测或者检查，这也增加了检查的困难。

验证环境组件

激励发生器

- **Stimulator（激励发生器）**是验证环境的重要部件，在一些场合中，它也被称为**driver**（驱动器）、**BFM**（bus function model，总线功能模型），**behavioral**（行为模型）或者**generator**（发生器）。
- Stimulator的主要职责是**模拟与DUT相邻设计的接口协议**，只需要关注于如何模拟接口信号，使其能够以真实的接口协议来发送激励给DUT。
- Stimulator**不应该违反协议**，但**不拘束于真实的硬件行为**，还可以给出更多丰富的只要协议允许的激励场景。
- **比真实硬件行为更丰富的激励**，会使得在模块级的验证更加充分，因为它不但验证过了硬件普通的接口协议情景，还模拟出更多复杂的、在更高系统级别无法产生出来的场景。

验证环境组件

激励发生器

- Stimulator的**接口主要是同DUT之间连接**，此外，也应该有时钟和复位的输入，确保生成的数据同DUT的接口一侧是同步的关系。
- 较精细的stimulator还可以有**其它的配置接口用来控制接口的数据生成**。
- Stimulator也可以有**存储接口数据生成历史**的功能，这可以用来在仿真运行时或者结束后查看接口数据，方便统计或者调试。

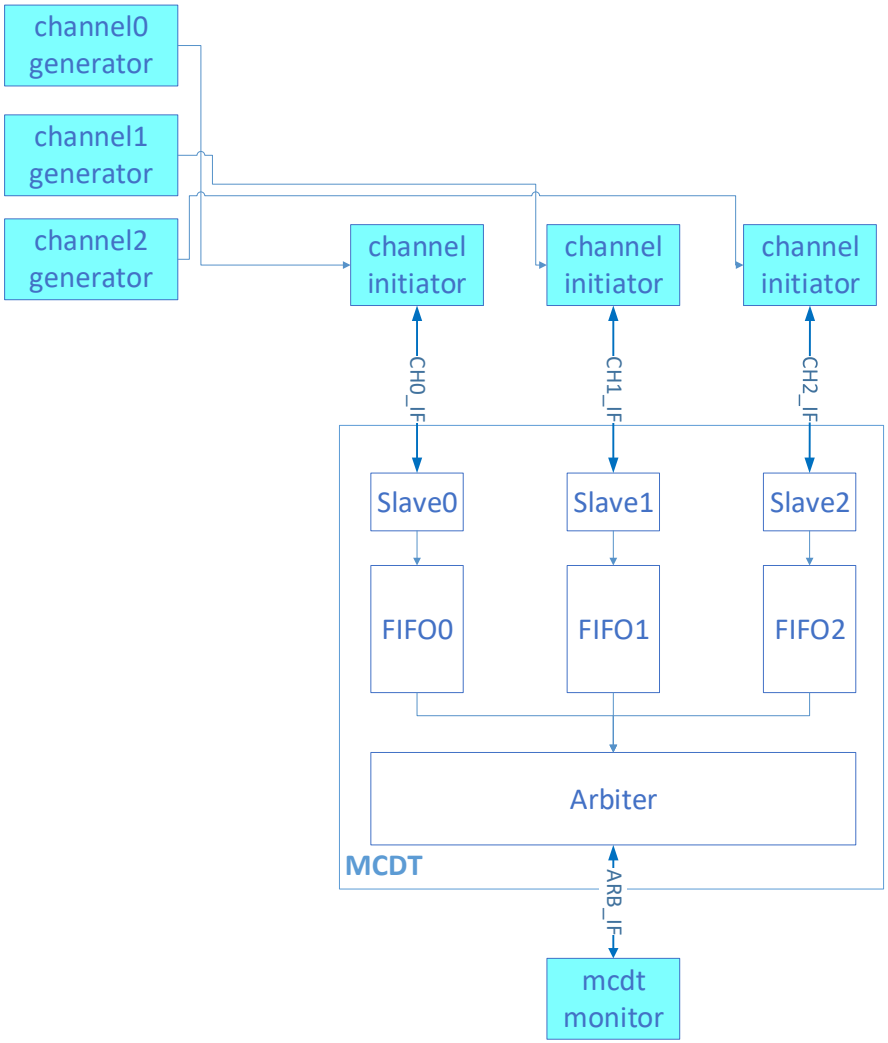
验证环境组件

激励发生器

- 从stimulator同DUT的连接关系来看，我们可以将其进一步分为两种：**initiator（发起器）和responder（响应器）。**
- 就我们要验证的MCDF来看，与下行通道从端（channel slave）的连接或寄存器接口的连接，这两部分的stimulator都属于**initiator**，它们的功能是**主动发起接口数据传输。**
- 与MCDF formatter接口的连接，该stimulator则属于**responder**，它的职责是**对接口的数据发送请求做出响应，而它本身不会主动发送数据。**

验证环境组件

激励器组件结构框图



验证环境组件

激励器实现考虑因素

Channel initiator

- 由于channel从端接口协议上有握手信号，我们需要遵照接口时序，确保chx_ready为低时，保证chx_data和chx_valid保持不变。
- 相邻数据之间没有数据包的限制，所以相邻数据之间的关系较弱，但也应该考虑数据之间是否有空闲周期，以及整体数据的传输速率设定。
- 由于每一个数据从端都有对应的FIFO缓存数据，所以也需要考虑如何使得FIFO的状态可遍历。例如，典型的FIFO状态可以分为empty、full以及中间状态即有数据存储但未写满。要使得FIFO可以触发这些状态，我们就应该控制channel initiator的传输速率。

验证环境组件

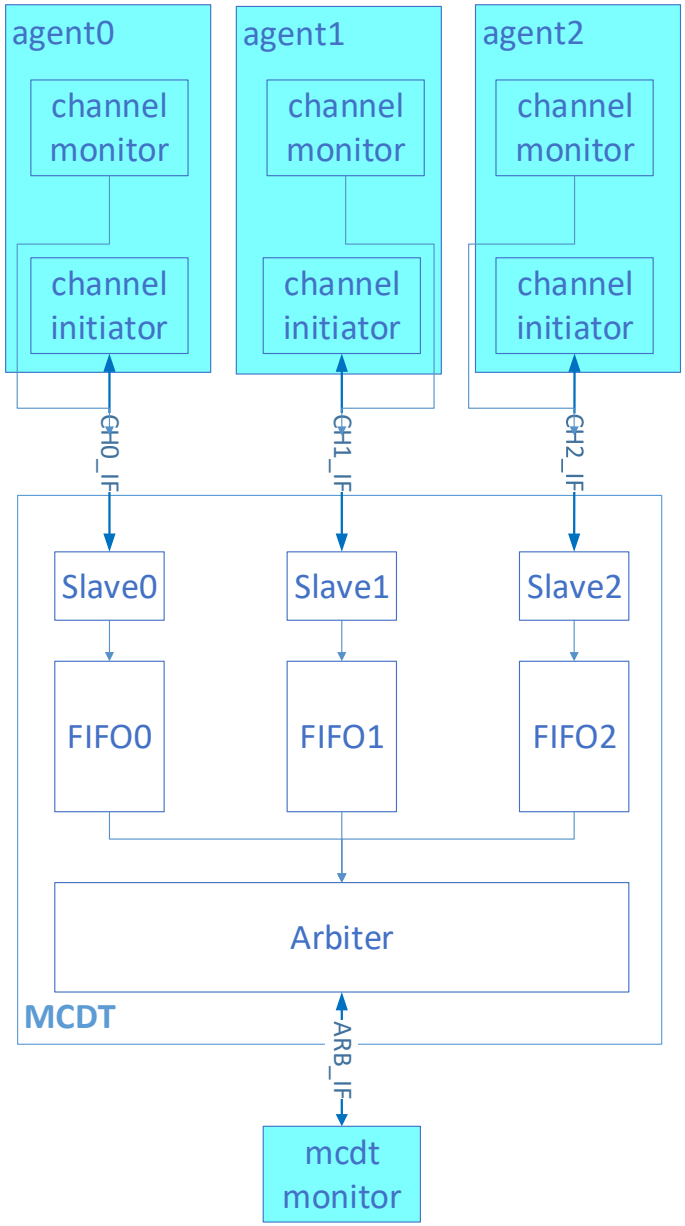
监测器

- Monitor（监测器）的主要功能是用来**观察DUT的边界或者内部信号，并且经过打包整理传送给其它验证平台的组件**，例如checker（比较器）。
- 从监测信号的层次来划分monitor的功能，它们可以分为观察DUT边界信号和观察DUT内部信号。
- **观察DUT边界信号。**对于系统信号如时钟，可以监测其频率变化；对于总线信号，可以监测总线的传输类型和数据内容，以及检查总线时序是否符合协议。
- **观察DUT内部信号。**从灰盒验证的手段来看，往往需要探视DUT内部信号，用来指导stimulator的激励发送，或者完成覆盖率收集，又或者完成内部功能的检查。

验证环境组件

监测器组件结构框图

- 分布式的monitor结构，monitor一般会对应一个stimulator。
- 对于MCDF验证结构，我们需要3个channel monitor来监测对应的channel接口，以及1个arbiter monitor来监测arbiter接口。



验证环境组件

监测器

- 如果没有特殊的需要，我们应采取**灰盒验证**的策略（而非白盒）。
- 观察的**内部信号应当尽量少**，且应当是表示状态的信号。不建议采集中间变量信号的原因在于，这些信号的时序、逻辑甚至留存性都不稳定，这种不稳定对于验证环境的收敛是有害的。
- 可以通过接口信息计算的，就尽量少去监测内部信号，因为这种方式也**有悖于假定设计有缺陷的验证思想**。我们观测到的内部信号在被环境采纳之前也有必要确认它们的逻辑正确性，这一要求可以通过动态检查或者断言触发的方式来实现。

验证环境组件

比较器

- 无论是从实现难度，还是从维护人力上来讲，checker（比较器）都应当是最需要时间投入的验证组件了。
- checker肩负了模拟设计行为和功能检查的任务。
- **缓存从各个monitor收集到的数据。**
- 将DUT输入接口侧的数据汇集给内置的**reference model（参考模型）**。Reference model在这里扮演了模拟硬件功能的角色，也是需要较多精力维护的部分，因为验证者需要在熟悉硬件功能的情况下实现该模型，而又不应参考真实硬件的逻辑。

验证环境组件

比较器

- 通过**数据比较**的方法，检查实际收集到的DUT输出端接口数据是否同reference model产生的期望数据一致。
- 对于设计内部的关键功能模块，也有相**对应的线程进行独立的检查**。
- 在检查过程中，可以将检查成功的信息统一纳入到**检查报告中**，便于仿真后的追溯。如果检查失败，也可以采取暂停仿真同时报告错误信息的方式，进行在线调试。

验证环境组件

比较器

- **线上比较（online check）**：在仿真时收集数据和在线比较，并且实时报告。
- **线下比较（offline check）**：将仿真时收集到的数据记录在文件中，在仿真结束后，通过脚本或者其它手段，进行数据比较。
- 在硬件设计发展初期，由于DUT的功能较为简单，采取定向测试（directed test）和线下比较的方式就不足为奇了，甚至验证者没有数据处理脚本或者参考模型，进行人为比较（manual check）的古老方式也是存在的。

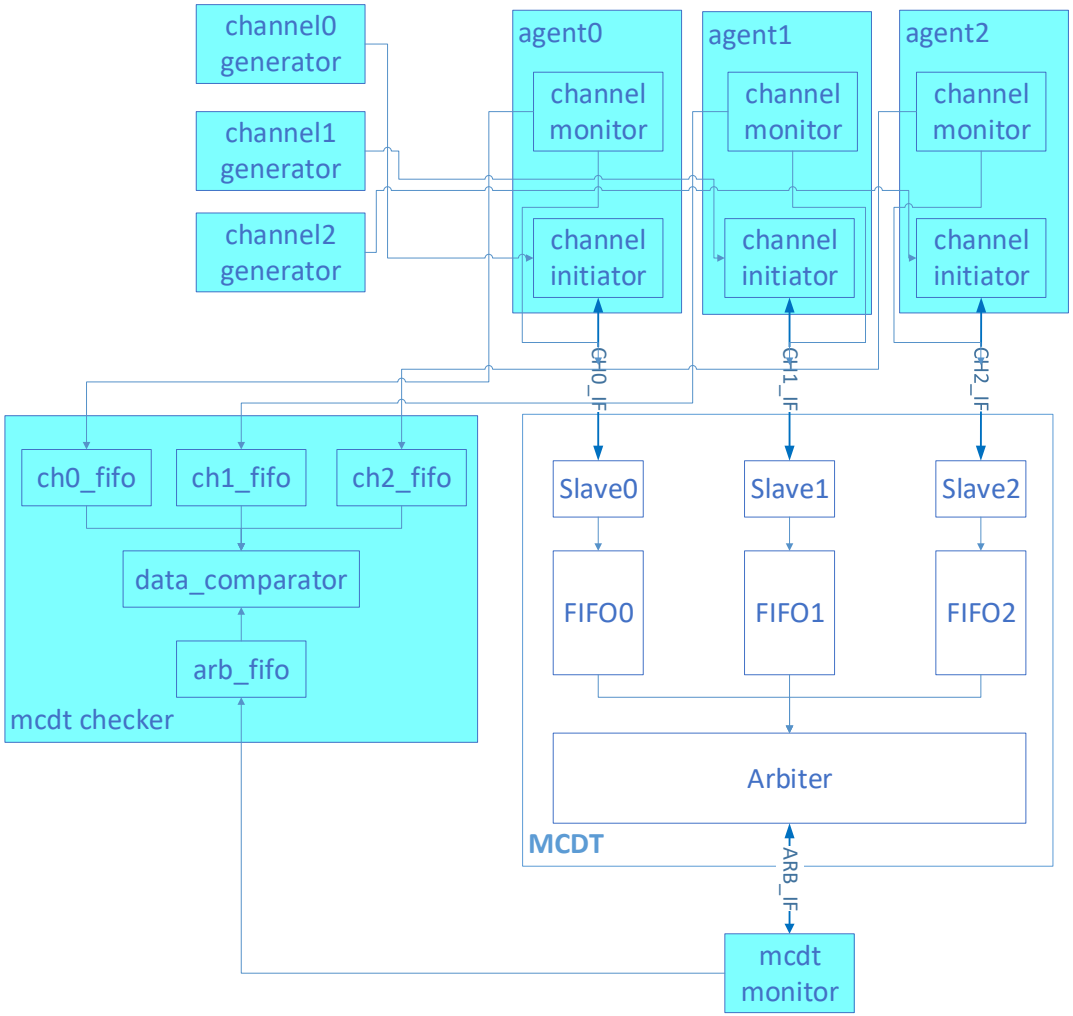
验证环境组件

比较器

- 而伴随着设计的功能愈加复杂，靠验证者每次进行繁琐检查的方式也缺少了可靠性。
- 于是我们将checker添加到验证环境中，需要它分析DUT的边界激励，理解数据的输入，并且按照硬件功能来预测输出的数据内容。
- 这种**预测（prediction）**的过程，发生在reference model中，有时我们也将其称为predictor。
- reference model也会**内置一些缓存**，分别存放**从DUT输入端观察到的数据**，以及经过功能转换的数据，同时checker也有其它缓存来存放**从输出端采集到的数据**。

验证环境组件

比较器组件结构框图



验证环境组件
比较器实现建议

- 对于复杂的系统验证，我们倾向于集中管理stimulator和checker，因为它们两者都需要主动给出激励或者判断结果，也需要较多的协调处理。
- 而monitor则相对更独立，因为它只是作为监测方，将其兢兢业业观察到的数据一字不落地交给checker即可，至于checker怎么使用，monitor并不需要关心。
- 因此，stimulator和monitor是一一对应，我们通常将它们进一步封装在agent单元组件中，而checker则最终集群搁置在中心化的位置。

Agenda

1. 验证环境结构
2. 硬件设计描述
3. 验证环境组件
4. **数组**
5. 类的封装
6. 类的继承
7. 包的使用

数组

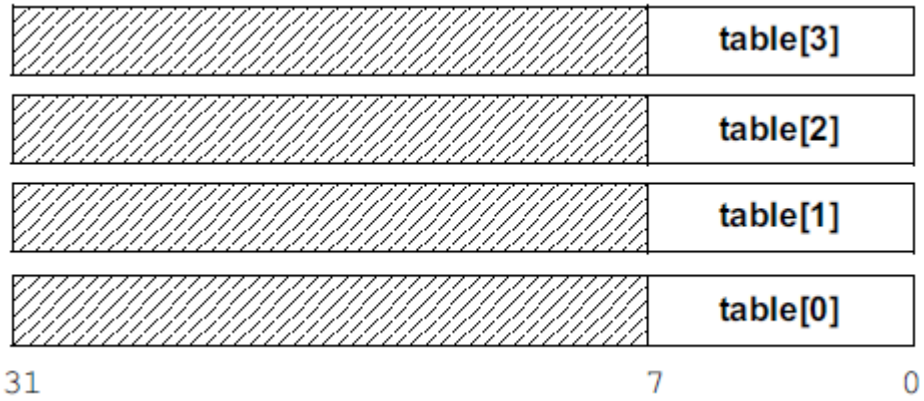
非组合型(unpacked)

- 对于Verilog，数组经常会被用来做数据存储，例如

```
reg [15:0] RAM [0:4095]; // memory array
```

- SV将Verilog这种声明数组的方式称之为非组合型声明，即数组中的成员之间存储数据都是互相独立的。
- Verilog也不会指定软件去如何存储数组中的成员。

```
wire [7:0] table [3:0];
```



数组
非组合型(unpacked)

- SV保留了非组合型的数组声明方式，并且扩展了允许的类型，包括event，logic，bit，byte，int，longint，shortreal和real类型。
- SV也保留了Verilog索引非组合型数组或者数组片段的能力，这种方式为数组以及数组片段的拷贝带来了方便。

```
int a1 [7:0][1023:0]; // unpacked array
int a2 [1:8][1:1024]; // unpacked array
a2 = a1; // copy an entire array
a2[3] = a1[0]; // copy a slice of an array
```

- 声明数组的方式，以下两种皆可

```
logic [31:0] data [1024];
logic [31:0] data [0:1023];
```

数组
组合型(packed)

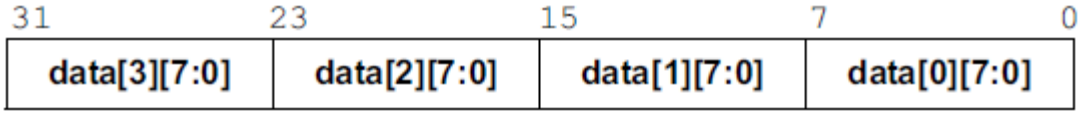
- SV将Verilog的向量作为组合型数组声明方式

```
wire [3:0] select; // 4-bit "packed array"
reg [63:0] data; // 64-bit "packed array"
```

- SV也进一步允许多维组合型数组的声明

```
logic [3:0][7:0] data; // 2-D packed array
```

- 组合型数组会进一步规范数据的存储方式，而不需要关心编译器或者操作系统的区别。



数组

组合型(packed)

- 组合型（packed）除了可以运用的数组声明，也可以用来定义结构体的存储方式：

```
typedef struct packed {
logic [ 7:0] crc;
logic [63:0] data;
} data_word;
data_word [7:0] darray; // 1-D packed array of
                        // packed structures
```

- 组合型数组和其数组片段也可以灵活选择，用来拷贝和赋值等

```
logic [3:0][7:0] data; // 2-D packed array
wire [31:0] out = data; // whole array
wire sign = data[3][7]; // bit-select
wire [3:0] nib = data [0][3:0]; // part-select
byte high_byte;
assign high_byte = data[3]; // 8-bit slice
logic [15:0] word;
assign word = data[1:0]; // 2 slices
```

数组
初始化

- 组合型（packed）数组初始化时，同向量初始化一致：

```
logic [3:0][7:0] a = 32'h0; // vector assignment
logic [3:0][7:0] b = {16'hz,16'h0}; // concatenate operator
logic [3:0][7:0] c = {16{2'b01}}; // replicate operator
```

- 非组合型（unpacked）数组初始化时，则需要通过'{}来对数组的每一个维度进行赋值。

```
int d [0:1][0:3] = '{ '{7,3,0,5}, '{2,0,1,6} };
// d[0][0] = 7
// d[0][1] = 3
// d[0][2] = 0
// d[0][3] = 5
// d[1][0] = 2
// d[1][1] = 0
// d[1][2] = 1
// d[1][3] = 6
```

数组
赋值

- 非组合型数组在初始化时，也可以类似结构体初始化，通过'{}和default关键词即可以完成

```
int a1 [0:7][0:1023] = '{default:8'h55};
```

- 非组合型数组的数据成员或者数组本身均可以为其赋值

```
byte a [0:3][0:3];
a[1][0] = 8'h5; // assign to one element
a[3] = '{'hF, 'hA, 'hC, 'hE};
// assign list of values to slice of the array
```

- 以下是组合型数组的赋值方法

```
logic [1:0][1:0][7:0] a; // 3-D packed array
a[1][1][0] = 1'b0; // assign to one bit
a = 32'hF1A3C5E7; // assign to full array
a[1][0][3:0] = 4'hF; // assign to a part select
a[0] = 16'hFACE; // assign to a slice
a = {16'bz, 16'b0}; // assign concatenation
```

数组
拷贝

- 对于组合型数组，由于数组会被视为向量，因此当赋值左右两侧操作数的大小和维度不相同时，也可以做赋值。
- 如果当尺寸不相同时，则会通过截取或者扩展右侧操作数的方式来对左侧操作数赋值。

```
bit [1:0][15:0] a; // 32 bit 2-state vector
logic [3:0][ 7:0] b; // 32 bit 4-state vector
logic [15:0] c; // 16 bit 4-state vector
logic [39:0] d; // 40 bit 4-state vector
b = a; // assign 32-bit array to 32-bit array
c = a; // upper 16 bits will be truncated
d = a; // upper 8 bits will be zero filled
```

数组
拷贝

- 对于非组合型数组，在发生书组间拷贝时，则要求做左右两侧操作数的维度和大小必须严格一致。

```
logic [31:0] a [2:0][9:0];
logic [0:31] b [1:3][1:10];
a = b; // assign unpacked array to unpacked array
```

- 非组合型数组无法直接赋值给组合型数组，同样地，组合型数组也无法直接赋值给非组合型数组。

数组

foreach循环结构

- SV添加foreach循环来对一维或者多维数组进行循环索引，而不需要指定该数组的维度大小。

```
int sum [1:8] [1:3];
foreach ( sum[i,j] )
sum[i][j] = i + j; // initialize array
```

- foreach循环结构中的变量无需声明。
- foreach循环结构中的变量是只读的，其作用域只在此循环结构中。

数组

系统函数

- $dimensions(array_name)用来返回数组的维度。
- $left(array_name, dimension)返回指定维度的最左索引值（msb）。

```
logic [1:2][7:0] word [0:3][4:1];
$left(word,1) will return 0
$left(word,2) will return 4
$left(word,3) will return 1
$left(word,4) will return 7
```

- 与$left()类似的，还有${right, low, high}(array_name, dimension)。
- $size(array_name, dimension)可以返回指定维度的尺寸大小。
- $increment(array_name, dimension)，如果指定维度的最左索引值大于或等于最右索引值，那么返回1，否则返回-1。
- $bits(expression)可以用来返回数组存储的比特数目。

```
wire [3:0][7:0] a [0:15]; // $bits(a) 返回512
struct packed {byte tag; logic [31:0] addr;} b; $bits(b) 返回40
```

数组

动态数组

- 与之前的定长数组相比，SV还提供了可以重新确定大小的动态数组。
- 动态数组在声明时需要使用[]，这表示不会在编译时为其制定尺寸，而是在仿真运行时来确定。
- 动态数组一开始为空，而需要使用new[]来为其分配空间。

```
int dyn[], d2[];                    // Declare dynamic arrays

initial begin
  dyn = new[5];                     // A: Allocate 5 elements
  foreach (dyn[j]) dyn[j] = j;      // B: Initialize the elements
  d2 = dyn;                         // C: Copy a dynamic array
  d2[0] = 5;                        // D: Modify the copy
  $display(dyn[0],d2[0]);           // E: See both values (0 & 5)
  dyn = new[20](dyn);               // F: Allocate 20 ints & copy
  dyn = new[100];                   // G: Allocate 100 new ints
                                    //    Old values are lost
  dyn.delete();                     // H: Delete all elements
end
```

数组
动态数组

- 内建方法size()可以返回动态数组的大小。
- 内建方法delete()可以清空动态数组，使其尺寸变为0。
- 动态数组在声明时也可以完成其初始化

```
bit [7:0] mask[] = '{8'b0000_0000, 8'b0000_0001,
                     8'b0000_0011, 8'b0000_0111,
                     8'b0000_1111, 8'b0001_1111,
                     8'b0011_1111, 8'b0111_1111,
                     8'b1111_1111};
```

数组
队列

- SV引入了队列类型，它结合了数组和链表。
- 可以在队列的任何位置添加或者删除数据成员。
- 可以通过索引来访问队列的任何一个成员。
- 通过[$]来声明队列，队列的索引值从0到$。
- 可以通过内建方法push_back(val)、push_front(val)、pop_back()和pop_front()来顺序添加或者移除并且获得数据成员。
- 可以通过insert(val, pos)来在指定位置插入数据成员。
- 可以通过delete()来删除所有数据成员。

数组
队列

```
int j = 1,
    q2[$] = {3,4},          // Queue literals do not use '
    q[$] = {0,2,3};         // {0,2,3}

initial begin
  q.insert(1, j);           // {0,1,2,3}  Insert j before ele #1
  q.delete(1);              // {0,2,3}    Delete element #1

  // These operations are fast
  q.push_front(6);          // {6,0,2,3}  Insert at front
  j = q.pop_back;           // {6,0,2}    j = 3
  q.push_back(8);           // {6,0,2,8}  Insert at back
  j = q.pop_front;          // {0,2,8}    j = 6
  foreach (q[i])
    $display(q[i]);         //            Print entire queue
  q.delete();               // {}         Delete queue
end
```

数组
队列

```
int j = 1,
    q2[$] = {3,4},          // Queue literals do not use '
    q[$] = {0,2,5};         // {0,2,5}

initial begin               // Result
  q = {q[0], j, q[1:$]};    // {0,1,2,5}     Insert 1 before 2
  q = {q[0:2], q2, q[3:$]}; // {0,1,2,3,4,5} Insert queue in q
  q = {q[0], q[2:$]};       // {0,2,3,4,5}   Delete elem. #1

  // These operations are fast
  q = {6, q};               // {6,0,2,3,4,5} Insert at front

  j = q[$];                 // j = 5         pop_back
  q = q[0:$-1];             // {6,0,2,3,4}      equivalent

  q = {q, 8};               // {6,0,2,3,4,8} Insert at back

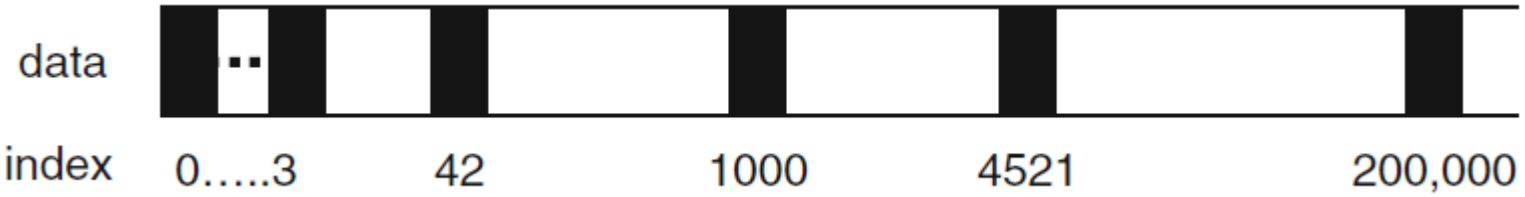
  j = q[0];                 // j = 6         pop_front
  q = q[1:$];               // {0,2,3,4,8}      equivalent

  q = {};                   // {}            Delete contents
end
```

数组

关联数组

- 如果想要在仿真时创建一个大的数组，也许动态数组是一个选择，不过有的时候，我们并不需要那么大的一个数组。
- 由于处理器在访问存储时的访问的随机或者散乱的，这意味在一个测试中，处理器也许只会访问几百个存储地址，而剩下大多数的地址都将被初始化为0并且浪费了仿真时的存储空间。
- SV引入了关联数组，可以用来存放散列的数据成员。这一点同其它脚本语言例如Perl或者Python类似，散列的索引类型除了为整形以外还可以为字符串或者其它类型，而散列存储的数据成员也可以为任意类型。



数组

关联数组

```
byte assoc[byte], idx = 1;
initial begin
  // Initialize widely scattered values
  do begin
    assoc[idx] = idx;
    idx = idx << 1;
  end while (idx != 0);

  // Step through all index values with foreach
  foreach (assoc[i])
    $display("assoc[%h] = %h", i, assoc[i]);

  // Step through all index values with functions
  if (assoc.first(idx))   // Get first index
    do
      $display("assoc[%h]=%h", idx, assoc[idx]);
    while (assoc.next(idx));  // Get next index

  // Find and delete the first element
  void'(assoc.first(idx));
  void'(assoc.delete(idx));
  $display("The array now has %0d elements", assoc.num());
end
```

数组

关联数组

```
/* Input file contains:
   42   min_address
   1492 max_address
*/

int switch[string], min_address, max_address, i, file;
initial begin
  string s;
  file = $fopen("switch.txt", "r");
  while (! $feof(file)) begin
    $fscanf(file, "%d %s", i, s);
    switch[s] = i;
  end
  $fclose(file);

  // Get the min address
  // If string not found, use default value of 0 for int array
  min_address = switch["min_address"];

  // Get the max address.
  // Use 1000 if max_address does not exist
  if (switch.exists("max_address"))
    max_address = switch["max_address"];
  else
    max_address = 1000;

  // Print all switches
  foreach (switch[s])
    $display("switch['%s']=%0d", s, switch[s]);
end
```

数组

缩减方法

- 基本的数组缩减方法是把一个数组缩减成一个值。
- 最常用的缩减方法是sum，它对数组中的所有元素求和。

```
byte b[$] = {2, 3, 4, 5};
int w;
w = b.sum();      //  14 = 2 + 3 + 4 + 5
w = b.product(); // 120 = 2 * 3 * 4 * 5
w = b.and();      // 0000_0000 = 2 & 3 & 4 & 5
```

- 其它的数组缩减方法还有product(积)，and(与)，or(或)和xor(异或)。

数组
定位方法

- 对于非合并数组，可以使用数组定位方法，其返回值将是一个队列而非一个数据成员。

```
int f[6] = '{1,6,2,6,8,6};    // Fixed-size array
int d[]  = '{2,4,6,8,10};     // Dynamic array
int q[$] = {1,3,5,7},         // Queue
    tq[$];                    // Temporary queue for result

tq = q.min();                 // {1}
tq = d.max();                 // {10}
tq = f.unique();              // {1,6,2,8}
```

数组
定位方法

- 使用foreach也可以实现数组的搜索，不过使用find...with则在查找满足条件的数据成员时，更为方便。

```
int d[] = '{9,1,8,3,4,4}, tq[$];

// Find all elements greater than 3
tq = d.find with (item > 3);                // {9,8,4,4}
// Equivalent code
tq.delete();
foreach (d[i])
  if (d[i] > 3)
    tq.push_back(d[i]);

tq = d.find_index with (item > 3);          // {0,2,4,5}
tq = d.find_first with (item > 99);         // {} - none found
tq = d.find_first_index with (item==8);     // {2} d[2]=8
tq = d.find_last with (item==4);            // {4}
tq = d.find_last_index with (item==4);      // {5} d[5]=4
```

数组

排序方法

- 可以通过排序方法改变数组中元素的顺序，可以对它们进行正向、逆向或者乱序的排列。

```
int d[] =          '{9,1,8,3,4,4};
d.reverse();   // '{4,4,3,8,1,9}
d.sort();      // '{1,3,4,4,8,9}
d.rsort();     // '{9,8,4,4,3,1}
d.shuffle();   // '{9,4,3,8,1,4}
```

Agenda

1. 验证环境结构
2. 硬件设计描述
3. 验证环境组件
4. 数组
5. **类的封装**
6. 类的继承
7. 包的使用

类的封装

类的概述

- 类是一种可以包含数据和方法（function，task）的类型。
- 例如一个数据包，可能被定义为一个类，类中可以包含指令、地址、队列ID、时间戳和数据等成员。
- packet这个类可以在其中对这些数据做初始化，设置指令，读取该类的状态以及检查队列ID。
- 每一个packet类例化的具体对象其数据成员都可能不相同，然而packet类作为描述这些数据的抽象类型，将其对应的数据成员和操作这些数据成员的方法都定义在其中。

类的封装

类的概述

```
class Packet ;
   //data or class properties
   bit [3:0] command;
   bit [40:0] address;
   bit [4:0] master_id;
   integer time_requested;
   integer time_issued;
   integer status;
   typedef enum { ERR_OVERFLOW= 10, ERR_UNDERFLOW = 1123} PCKT_TYPE;
   const integer buffer_size = 100;
   const integer header_size;

   // initialization
   function new();
      command = 4'd0;
      address = 41'b0;
      master_id = 5'bx;
      header_size = 10;
   endfunction
   // methods
   // public access entry points
   task clean();
      command = 0; address = 0; master_id = 5'bx;
   endtask
   task issue_request( int delay );
      // send request to bus
   endtask
   function integer current status();
      current_status = status;
   endfunction
endclass
```

类的封装

类的概述

- 面向对象编程（OOP，Object-Oriented Programming）使用户能够创建复杂的数据类型，并且将它们跟使用这些数据类型的程序紧密地结合在一起。
- 用户可以在更加抽象的层次建立测试平台和系统级模型，通过调用函数来执行一个动作而不是简单地改变信号的电平。
- 在验证环境中，包括stimulator、monitor、checker以及其它验证组件接下来都将按照OOP的方式来构建。

类的封装

OOP术语

- 类（class）：包含成员变量和成员方法。
- 对象（object）：类在例化后的实例。
- 句柄（handle）：指向对象的指针。
- 原型（prototype）：程序的声明部分，包含程序名、返回类型和参数列表。

类的封装

构建函数

- SV并不像C++语言一样要求复杂的存储空间开辟和销毁的手段，而是采用了像Java一样空间自动开辟和回收的手段。
- 因此SV的类在定义时，只需要定义构建函数（constructor），而不需要定义析构函数（destructor）。
- 类在定义时，需要定义构建函数，如果未定义，则系统会自动帮助定义一个空的构建函数（没有形式参数，函数体亦为空）。
- 对象在创建时，需要先声明再例化，同时进行亦可。

```
class Packet;
  integer command;
  function new();
    command = IDLE;
  endfunction
endclass
Packet p = new;
```

类的封装

静态成员（变量/方法）

- 类的成员（变量/方法）默认都是动态（automatic）生命周期，即每一个对象的变量和方法都会为其开辟新的空间。
- 如果多个对象为了共享一个成员（变量/方法），那么可以为其添加关键字static。
- 多个对象因此可以共享同一个成员变量或者方法。
- 访问该成员时，无需进行对象的例化。

```
class Packet ;
  static integer fileID = $fopen( "data", "r" );
  ...
endclass
Packet p;
c = $fgetc( p.fileID ); // OR Packet::fileID
```

类的封装

静态成员（变量/方法）

- 成员方法也可以声明为静态。
- 静态方法无法访问非静态成员（变量/方法），否则会发生编译错误。

```
class id;
  static int current = 0;
  static function int next_id();
    next_id = ++current; // OK to access static variable
  endfunction
endclass
```

类的封装

this

- this是用来明确索引当前所在对象的成员（变量/参数/方法）。
- this只可以用来在类的非静态成员、约束和覆盖组中使用。
- this的使用可以明确所指向变量的作用域。

```
class Demo ;
  integer x;
  function new (integer x);
    this.x = x;
  endfunction
endclass
```

类的封装
赋值和拷贝

- 声明变量和创建对象是两个过程，也可以一步完成。

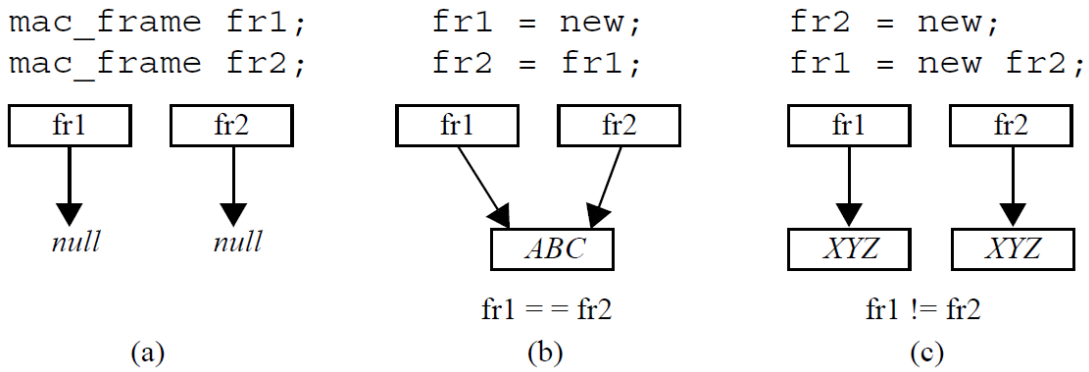
```
Packet p1;
p1 = new;
```

- 如果将p1赋值给另外一个变量p2，那么依然只有一个对象，只是指向这个对象的句柄有p1和p2.
- 以下这种方式表示p1和p2代表两个不同的对象。在创建p2对象时，将从p1拷贝其成员变量例如integer、string和句柄等，该种拷贝方式称为浅拷贝（shallow copy）。

```
Packet p1;
Packet p2;
p1 = new;
p2 = new p1;
```

类的封装

类的引用和实例



- 当你声明一个类的变量时， 你只是在声明将来要指向某个该类的实例的句柄（指针），而默认情况下它们并不指向任何实例。
- 如果想要它们指向实例，你可以通过`new()`函数来为其创建一个实例；或者你可以将某个句柄赋值给另外一个句柄。
- 如果两个句柄指向同一个对象，其中一个句柄在修改了该对象的成员以后，那么另外一个句柄也将受到影响，因为它俩指向的是同一个对象。
- 如果想拷贝一个对象，那么可以使用`new + handle`的形式。

类的封装

类的引用和实例

```
atm_cell cell = new;
atm_cell cells[$];

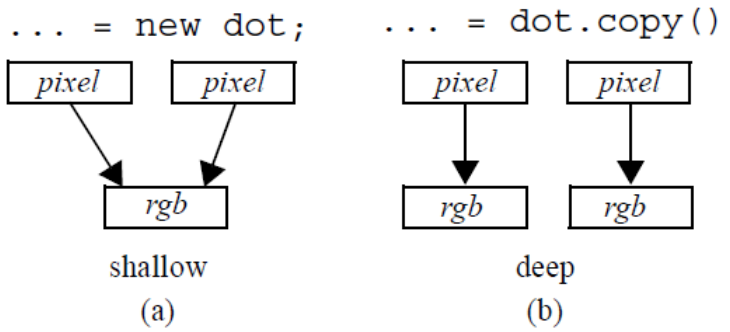
while (cells.size() < 10) begin
   cell.randomize();
   cells.push(cell);
end
```

- 以上这段代码是经常容易出错的类的拷贝方式。
- 大家来看，在这个队列中，一共有句柄?
- 这些句柄指向了几个对象呢?
- 明白它错在哪里了吗?
- 如果要你修改，你准备怎么办?

类的封装

类的引用和实例

```
class rgb;
   byte red;
   byte green;
   byte blue;
endclass: rgb
class pixel;
   int x;
   int y;
   rgb color;
endclass: pixel
pixel dot = new;
```



- 在SV中，对于句柄的比较或者拷贝，只能针对对象中的成员变量。
- 对于拷贝而言，如果对象中还有句柄，例如上图的pixel类包含color句柄，那么在利用new函数拷贝对象时，只能对color句柄做拷贝，而不会对其指向的对象再做拷贝，这种方式称之为浅拷贝（shallow copy）。
- 如果你想要对对象中的句柄指向的对象也做拷贝，即做递归的拷贝，那么这种方式称之为深拷贝（deep copy）。
- SV利用new函数只能做浅拷贝，用户需要自己定义深拷贝的函数才能实现该需求。

类的封装

数据的隐藏和封装

- 类的成员（变量/方法）默认情况下，即是公共属性的。这表示对于类自身和外部均可以访问该成员。
- 对于商业开发，类的提供方会限制一些类成员的外部访问权限，继而隐藏类成员的更多细节。
- 这种方式也使得类的外部访问接口更为精简，减轻了类的维护工作量，也使得类在修改时便于与旧版本保持兼容。
- 数据隐藏的方式使得类的测试和维护都变得更为简单。

类的封装

数据的隐藏和封装

- 对于成员的限定，如果使用local，则只有该类可以访问此成员，而子类或者外部均无法访问。
- 对于成员的限定，如果使用protected，则表示该类和其子类均可以访问此成员，而外部无法访问。

```
class Packet;
  local integer i;
  function integer compare (Packet other);
    compare = (this.i == other.i);
  endfunction
endclass
```

Agenda

1. 验证环境结构
2. 硬件设计描述
3. 验证环境组件
4. 数组
5. 类的封装
6. **类的继承**
7. 包的使用

类的继承

继承和子类

- 之前定义过的类Packet，可以进一步扩展构成一个新的类LinkedPacket。
- 通过extends，LinkedPacket继承于其父类Packet，包括继承其所有的成员（变量/方法）。

```
class LinkedPacket extends Packet;
  LinkedPacket next;
  function LinkedPacket get_next();
    get_next = next;
  endfunction
endclass
```

类的继承

成员覆盖

- 子类可以继承父类的成员，所以LinkedPacket对象也是一个合法的Packet对象。
- 由此，父类句柄可以指向子类的对象。

```
LinkedPacket lp = new;
Packet p = lp;
```

- 如果子类中声明了与父类同名的成员（变量/方法），那么子类中对其同名成员的访问都将指向子类，而父类的成员将被隐藏。

类的继承

成员覆盖

```
class Packet;
  integer i = 1;
  function integer get();
    get = i;
  endfunction
endclass
class LinkedPacket extends Packet;
  integer i = 2;
  function integer get();
    get = -i;
  endfunction
endclass
LinkedPacket lp = new;
Packet p = lp;
j = p.i; // j = 1, not 2
j = p.get(); // j = 1, not -1 or -2
```

类的继承

super

- super是用来访问当前对象其父类的成员。
- 尤其当子类的成员如果与父类的成员同名，那么需要使用super来指定访问其父类成员，而非默认的子类成员。

```
class Packet; // base class
  integer value;
  function integer delay();
    delay = value * value;
  endfunction
endclass
class LinkedPacket extends Packet; // derived class
  integer value;
  function integer delay();
    delay = super.delay()+ value * super.value;
  endfunction
endclass
```

Agenda

1. 验证环境结构
2. 硬件设计描述
3. 验证环境组件
4. 数组
5. 类的封装
6. 类的继承
7. **包的使用**

包的使用

包的定义

- 在大型的验证项目中，很容易出现模块重名的情况。
- 对于重名的硬件模块我们可以将它们置入到不同编译的库中。
- 对于重名的软件类、方法等，我们可以将它们置入到不同的包中。
- 我们需要承认，作为verifier，我们可能接受来自不同提供商的验证IP，因此我们也无法预测这些软件类是否可能重名。
- 因此，通过包（package）可以将关联的类和方法并入到同一个逻辑联系组当中。

包的使用

包的定义

- 为了使得可以在多个模块（硬件）或者类（软件）之间共享用户定义的类型，SV添加了包（package）。
- 包的概念参考于VHDL，用户自定义的类型譬如类、方法、变量、结构体、枚举类等都可以在package...endpackage中定义。

```
package definitions;
  parameter VERSION = "1.1";
  typedef enum {ADD, SUB, MUL} opcodes_t;
  typedef struct {
    logic [31:0] a, b;
    opcodes_t opcode;
  } instruction_t;
  function automatic [31:0] multiplier(input[31:0] a, b);
    return a * b; // abstract multiplier
  endfunction
endpackage
```

包的使用
导出包的内容

- module、interface、class等可以使用包中定义或者声明的内容。
- 可以通过域的索引符::号直接引用。

```
definitions::parameter
definitions::instruction_t inst
```

- 可以指定索引一些需要的包中定义的类型到指定的容器中（module/class/...）。

```
module M;
  import definitions::instruction_t;
  instruction_t inst;
endmodule
```

- 通过通配符*来将包中所有的类别导入到指定容器中。

```
module M;
  import definitions::*;
  instruction_t inst;
endmodule
```

包的使用
导出包的内容

- 可以通过域的索引符::号直接引用。

```
module ALU(input definitions::instruction_t IW,
           input logic clock,
           output logic [31:0] result
          );
  always_ff @(posedge clock) begin
    case (IW.opcode)
      definitions::ADD : result = IW.a + IW.b;
      definitions::SUB : result = IW.a - IW.b;
      definitions::MUL : result =
                         definitions::multiplier(IW.a, IW.b);
    endcase
  end
endmodule
```

包的使用
导出包的内容

- 可以指定索引一些需要的包中定义的类型到指定的容器中（module/class/...）。

```
module ALU(...);
  import definitions::ADD;
  import definitions::SUB;
  import definitions::MUL;
  import definitions::multiplier;
  always_comb begin
    case (IW.opcode)
      ADD : result = IW.a + IW.b;
      SUB : result = IW.a - IW.b;
      MUL : result = multiplier(IW.a, IW.b);
    endcase
  end
endmodule
```

包的使用
导出包的内容

- 通过通配符*来将包中所有的类别导入到指定容器中。

```
module ALU(...);
  import definitions::*; // wildcard import
  always_comb begin
    case (IW.opcode)
      ADD : result = IW.a + IW.b;
      SUB : result = IW.a - IW.b;
      MUL : result = multiplier(IW.a, IW.b);
    endcase
  end
endmodule
```

Agenda

1. **随机约束**
2. 任务和函数
3. 线程控制
4. 进程间同步和通信
5. 虚方法

随机约束

概述

- 随着设计变得越来越大，要产生一个完整的激励来测试设计的功能也变得越来越困难。
- 定向激励的测试方法早已经无法满足检查功能完整性的要求。
- SoC的集成度提高带来的模块之间交互的复杂度也是指数攀升，这就使得verifier无法预测接下来用户使用过程中会发生什么样的状况。
- 随机-约束，两个词组合在一起就构成了目前动态仿真验证的主流方法。
- 随机约束测试（CRT，Constrained-Random Test）即能够产生你感兴趣的、你想不到的的测试向量，通过回归测试、替换随机种子的方式来提高单位测试用例的覆盖率收集效率。

随机约束

概述

- 随机测试带来的额外负担是环境的复杂度会提高，因为从环境组件上考虑不再只需要发送激励的组件，而且还包括监测器、比较器等。
- 随机测试带来的环境复杂度还包括由于对环境复用和测试复用带来的组件封装要求，这会使得代码量加大。
- 这种额外的代码量、环境集成要求也随之带来了更加灵活、更易集成复用的验证环境。
- 我们接下来的试验中，将会有基于MCDT的SV验证试验以及UVM验证试验，主要目的在于帮助同学建立验证环境结构的概念以及理解测试激励的生成及发送过程。

随机约束

如何简单地产生一个随机数?

- 在我们学习如何定义一个"随机"属性类之前，先了解一下如何利用系统函数产生一个随机数吧。

```
module stim;
  bit [15:0] addr;
  bit [31:0] data;
  function bit gen_stim();
    bit success, rd_wr;
    // call std::randomize
    success = randomize( addr, data, rd_wr );
    return rd_wr ;
  endfunction
  ...
endmodule
```

- 通过系统函数std::randomize()对一些变量即可完成随机化，或者理解为产生随机数并赋予这些变量。

随机约束

如何简单地产生一个随机数?

- 我们还可以通过其它一些与随机数生成有关的系统函数来产生。
- $urandom()，可以生成一个32位的无符号随机数。
- $urandom_range(maxval, minval=0)，可以生成间与maxval与minval之间的数。

随机约束

什么是系统而有机地组织随机变量？

- 比起以上我们独立地生成一些随机数，在面向DUT的随机激励发生过程中，为了符合协议、满足测试需求，我们还需要添加一些“约束”。
- 这些“约束”会使得变量朝着希望他们变化的方向去随机。
- 不但如此，这些约束也会对变量与变量之间的关系生效。
- 因此，我们需要一个“载体”去容纳这些变量以及它们之间的约束。
- 这个“载体”即是类，而类的成员变量均可声明为“随机”属性，用rand或者randc来表示。

随机约束

随机变量

- 任何类中的整形（bit/byte/int）变量都可以声明为rand/randc。
- 定长数组、动态数组、关联数组和队列都可以声明为rand/randc，可以对动态数组和队列的长度加以约束。

```
rand bit [7:0] len;
rand integer data[];
constraint db { data.size == len; }
```

- 指向对象的句柄成员，也可以声明为rand（不能被声明为randc），随机时该句柄指向对象中的随机变量也会一并被随机。

随机约束

随机变量

- 非组合型结构体可以声明为rand，非组合型的成员可以声明为rand/randc。

```
typedef struct {
  randc int addr = 1 + constant;
  int crc;
  rand byte data [] = {1,2,3,4};
} header;
rand header h1;
```

随机约束

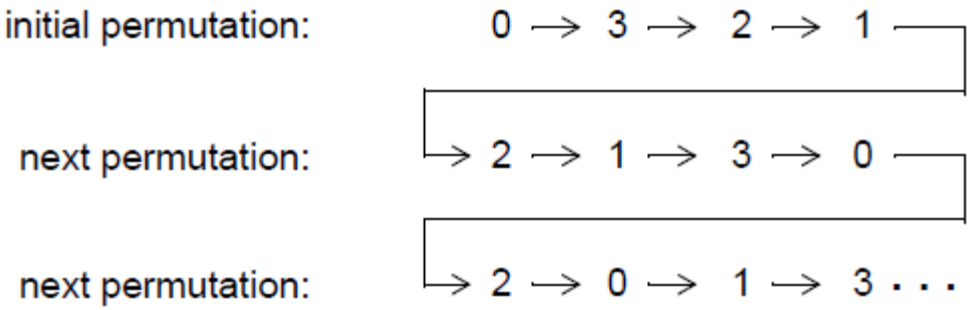
修饰符rand和randc

- 对于rand修饰符，表示在可生成的范围内，每个值的可能性是相同的。

```
rand bit [7:0] y;
```

- 对于randc修饰符，它的值将会随机并且遍历其可取值范围。

```
randc bit [1:0] y;
```



随机约束

带有随机变量的简单类

```
class Packet;
  // The random variables
  rand  bit [31:0] src, dst, data[8];
  randc bit [ 7:0] kind;
  // Limit the values for src
  constraint c {src > 10;
                src < 15;}
endclass

Packet p;
initial begin
  p = new();// Create a packet
  if (!p.randomize())
    $finish;
  transmit(p);
end
```

随机约束

约束块

- 有用的激励不仅仅是随机值，变量之间也有着相互关系。
- 没有约束的随机变量会包含许多无效的和非法的值，这会使得有效激励的产生变得低效。
- 需要用包含一个或多个约束表达式的约束块定义这些相互关系。

随机约束

约束块（成员集合）

- 约束块支持整形通过set操作符来设置它们的可取值范围。

```
rand integer x, y, z;
constraint c1 {x inside {3, 5, [9:15], [24:32], [y:2*y], z};}

rand integer a, b, c;
constraint c2 {a inside {b, c};}

integer fives[4] = '{ 5, 10, 15, 20 };
rand integer v;
constraint c3 { v inside {fives}; }
```

随机约束

约束块（权重分布）

- 除了成员集合设置，约束块也支持设置可取值的同时也为其设置随机时的权重。
- 对于:=操作符，它们表示每一个值的权重是相同的。
- 对于:/操作符，它们表示权重会平均分配到每一个值。

x在100,101,102,200和300的权重是1-1-1-2-5。

```
x dist { [100:102] := 1, 200 := 2, 300 := 5}
```

x在100,101,102,200和300的权重是1/3-1/3-1/3-2-5。

```
x dist { [100:102] :/ 1, 200 := 2, 300 := 5}
```

随机约束

约束块（唯一标识）

- **unique**可以用来约束一组变量，使得其在随机后变量之间不会有相同的数值。
- a[2],a[3],b和excluded在随机化之后将包含不相同的数值。

```
rand byte a[5];
rand byte b;
rand byte excluded;
constraint u { unique {b, a[2:3], excluded}; }
constraint exclusion { excluded == 5; }
```

随机约束

约束块（条件约束）

- 可以使用if-else或者->操作符来表示条件约束。

```
mode == little -> len < 10;
mode == big -> len > 100;

bit [3:0] a, b;
constraint c { (a == 0) -> (b == 1); }

if (mode == little)
  len < 10;
else if (mode == big)
  len > 100;
```

随机约束

约束块（迭代约束）

- **foreach**可以用来迭代约束数组中的元素，这些数组可以是定长数组、动态数组、关联数组或者队列。

```
class C;
  rand byte A[] ;
  constraint C1 { foreach (A[i]) A[i] inside
                    {2,4,8,16};}
  constraint C2 { foreach (A[j]) A[j] > 2 * j; }
endclass
```

随机约束

约束块（迭代约束）

- 也可以使用数组的缩减方法做迭代约束。

```
class C;
  rand bit [7:0] A[] ;
  constraint c1 { A.size() == 5; }
  constraint c2 { A.sum() < 1000; }
endclass

A[0] + A[1] + A[2] + A[3] + A[4] < 1000
```

随机约束

约束块（函数调用）

- 有时候在一些表达式中无法简单地来表述约束，例如要计算一个合并数组中的'1'：

```
function int count_ones ( bit [9:0] w );
  for( count_ones = 0; w != 0; w = w >> 1 )
  count_ones += w & 1'b1;
endfunction
```

- 可以在约束块中调用该函数来描述约束：

```
constraint C1 { length == count_ones( v ) ; }
```

随机约束

约束块（软约束）

- 在没有**soft**描述时的约束，我们称之为硬约束，而带有soft描述的则是软约束。
- 软约束用来指定变量的默认值和权重。
- 如果用户在使用时，指定了外部约束对同一个变量做二次约束，或者用户定义了子类，也对同一个变量做二次约束时，那么硬约束可以"覆盖"软约束，并且不会导致随机数产生的失败。

```
class Packet;
  rand int length;
  constraint deflt {soft length inside {32,1024}; }
endclass
Packet p = new();
p.randomize() with { length == 1512; }
```

随机约束

随机方法

- 在类中声明的随机变量，需要伴随着类句柄调用方法randomize()，这是SV类的内建方法：

```
virtual function int randomize();
```

- 如果随机化成功则会返回1，如果失败则会返回0。

```
class SimpleSum;
  rand bit [7:0] x, y, z;
  constraint c {z == x + y;}
endclass
SimpleSum p = new;
int success = p.randomize();
if (success == 1 ) ...
```

随机约束
内嵌约束

- 在调用类方法randomize()时可以伴随着with来添加额外的约束。

```
class SimpleSum;
  rand bit [7:0] x, y, z;
  constraint c {z == x + y;}
endclass
task InlineConstraintDemo(SimpleSum p);
  int success;
  success = p.randomize
endtask
```

随机约束

内嵌约束（指向模糊）

```
class C1;
  rand integer x;
endclass
class C2;
  integer x;
  integer y;
  task doit(C1 f, integer x, integer z);
    int result;
    result = f.randomize() with {x < y + z;};
  endtask
endclass
```

随机约束

内嵌约束（指向明确）

```
class C;
  rand integer x;
endclass
function int F(C obj, integer y);
  F = obj.randomize() with (x) { x < y; };
endfunction
```

随机约束
local域指向

- 之前在使用内嵌约束随机时，对于同名的变量处在不同的域中，可能会出现指向模糊的情况，之前我们已经给出了一种方法。
- 接下来我们可以通过local::的域索引方式来明确随机变量的指向，即local::指向的变量会在包含randomize()方法的对象中。

```
class C;
  rand integer x;
endclass
function int F(C obj, integer x);
  F = obj.randomize() with { x < local::x; };
endfunction
```

随机约束
随机控制

- rand_mode可以用来使能或者禁止随机变量。
- 当随机数被禁止时，它会同为声明为随机变量的普通变量一样，不会参与到随机化过程当中。
- 从以下两个函数声明可以看到，可以就单个随机变量调用其rand_mode，或者对整个对象调用rand_mode来控制其中所有的随机变量。

```
task object[.random_variable]::rand_mode( bit on_off );
function int object.random_variable::rand_mode();
```

随机约束

随机控制

```
class Packet;
  rand integer source_value, dest_value;
  ... other declarations
endclass

int ret;
Packet packet_a = new;
// Turn off all variables in object
packet_a.rand_mode(0);

// ... other code
// Enable source_value
packet_a.source_value.rand_mode(1);
ret = packet_a.dest_value.rand_mode();
```

随机约束

约束控制

- 类似于随机控制，一些约束块或者某个类的约束块集合都可以单个控制或者集体控制。

```
task object[.constraint_identifier]::constraint_mode(bit on_off );
function int object.constraint_identifier::constraint_mode();
```

- 可以通过约束控制函数来使能或者关闭某些约束块。

随机约束

约束控制

```
class Packet;
  rand integer source_value;
  constraint filter1 { source_value > 2 * m; }
endclass

function integer toggle_rand( Packet p );
  if ( p.filter1.constraint_mode() )
    p.filter1.constraint_mode(0);
  else
    p.filter1.constraint_mode(1);

  toggle_rand = p.randomize();
endfunction
```

随机约束

内嵌变量控制

- 在使用类的随机化函数randomize()时，如果伴有参数，那么只会随机化这些变量，而其余变量无论是否之前被声明为rand/randc，都将不会参与到随机化当中。

```
class CA;
  rand byte x, y;
  byte v, w;
  constraint c1 { x < v && y > w;  };
endclass
CA a = new;
a.randomize(); // random variables: x, y
a.randomize( x ); // random variables: x
a.randomize( v, w ); // random variables: v, w
a.randomize( w, x ); // random variables: w, x
```

Agenda

1. 随机约束
2. **任务和函数**
3. 线程控制
4. 进程间同步和通信
5. 虚方法

任务和函数

概述

- 类似于C语言，函数（function）和任务（task）可以提高代码的复用性和整洁度。
- 它们的目的都在于将大型的过程块切分为更细小的片段，而便于阅读和代码维护。
- 相比于大家更为熟悉的函数，SV引入了任务的概念。function与task之间有相同点和不同点。

任务和函数
区别

- function不会消耗仿真时间，而task则可能会消耗仿真时间。
- function无法调用task，而task可以调用function。
- 一个可以返回数据的function只能返回一个单一数值，而任务或者void function不会返回数值。
- 一个可以返回数据的function可以作为一个表达式中的操作数，而该操作数的值即function的返回值。

任务和函数

任务task

- 任务的定义可以指定参数，input、output、inout及ref皆可。
- 任务可以消耗仿真时间。
- 任务可以调用其它任务或者函数。

```
task mytask1 (output int x, input logic y);
   ...
endtask
task mytask2;
  output x;
  input y;
  int x;
  logic y;
  ...
endtask
```

任务和函数

任务task

- 虽然task不会返回值，但是我们依然可以利用return来使task结束。

```
logic red;
parameter red_tics = 100;
always begin // sequence to control the lights
  red = on; // turn red light on
  light(red, red_tics); // and wait.
end
// task to wait for 'tics' positive edge clocks
// before turning 'color' light off
task light (output color, input [31:0] tics);
  repeat (tics) @ (posedge clock);
  color = off; // turn light off.
endtask: light
```

任务和函数

函数function

- 任务的首要目的在于为运算表达式提供返回值，这样既便于简化原有的代码，也便于大型代码的维护。
- void函数不会返回数值。
- 类似于task，函数的参数列表方向也可以声明为input、output、inout和ref。

```
function logic [15:0] myfunc1(int x, int y);
   ...
endfunction
function logic [15:0] myfunc2;
   input int x;
   input int y;
   ...
endfunction
```

任务和函数

函数function

- 对于函数返回的方式，也同C一致，有两种方法，既可以使用return直接返回值，也可以将值赋给与函数同名的变量。
- 只是return会立即返回，而赋值给函数同名变量后将继续执行后续代码。

```
function [15:0] myfunc1 (input [7:0] x,y);
  myfunc1 = x * y - 1;
endfunction
function [15:0] myfunc2 (input [7:0] x,y);
  return x * y - 1;
endfunction
```

任务和函数

函数function

- 如果调用具有返回值的函数，但是又不使用该返回值时，我们建议为其添加void’()进行转换。

```
void'(some_function());
```

任务和函数

参数传递

- input参数方向在方法调用时，属于值传递。即传递的过程中，外部变量的值会经过拷贝，赋值于形式参数。
- 同样地，output、inout也会在传入或者传出的时候发生值传递的过程。
- 值传递的过程只发生在方法的调用时和返回时。
- ref参数在传递时不会发生值拷贝，而是将变量“指针”传递到方法中，在方法内部对该参数的操作将会同时影响外部变量。
- 如果为了避免外部传入的ref参数会被方法修改，则可以添加const修饰符，来表示变量是只读变量。

任务和函数

参数默认值

- SV允许方法声明参数的默认值，参数的方向可以是input，inout，output和ref。

```
subroutine( [ direction ] [ type ] argument =
default_expression);
```

- 带有参数默认值的方法被调用时，如果这些参数没有被传递值，那么编译器将会为这些参数传入对应的默认值。

```
task read(int j = 0, int k, int data = 1 );
   ...
endtask
read( , 5 ); // is equivalent to read( 0, 5, 1 );
read( 2, 5 ); // is equivalent to read( 2, 5, 1 );
read( , 5, ); // is equivalent to read( 0, 5, 1 );
read( , 5, 7 ); // is equivalent to read( 0, 5, 7 );
read( 1, 5, 2 ); // is equivalent to read( 1, 5, 2 );
read( ); // error; k has no default value
read( 1, , 7 ); // error; k has no default value
```

任务和函数

参数传递

- SV允许类似于模块例化一样，可由参数位置顺序在调用方法时传递参数，也可以由参数名字调用方式时绑定参数。

```
function int fun( int j = 1, string s = "no" );
  ...
endfunction
fun( .j(2), .s("yes") ); // fun( 2, "yes" );
fun( .s("yes") ); // fun( 1, "yes" );
fun( , "yes" ); // fun( 1, "yes" );
fun( .j(2) ); // fun( 2, "no" );
fun( .s("yes"), .j(2) ); // fun( 2, "yes" );
fun( .s(), .j() ); // fun( 1, "no" );
fun( 2 ); // fun( 2, "no" );
fun( ); // fun
```

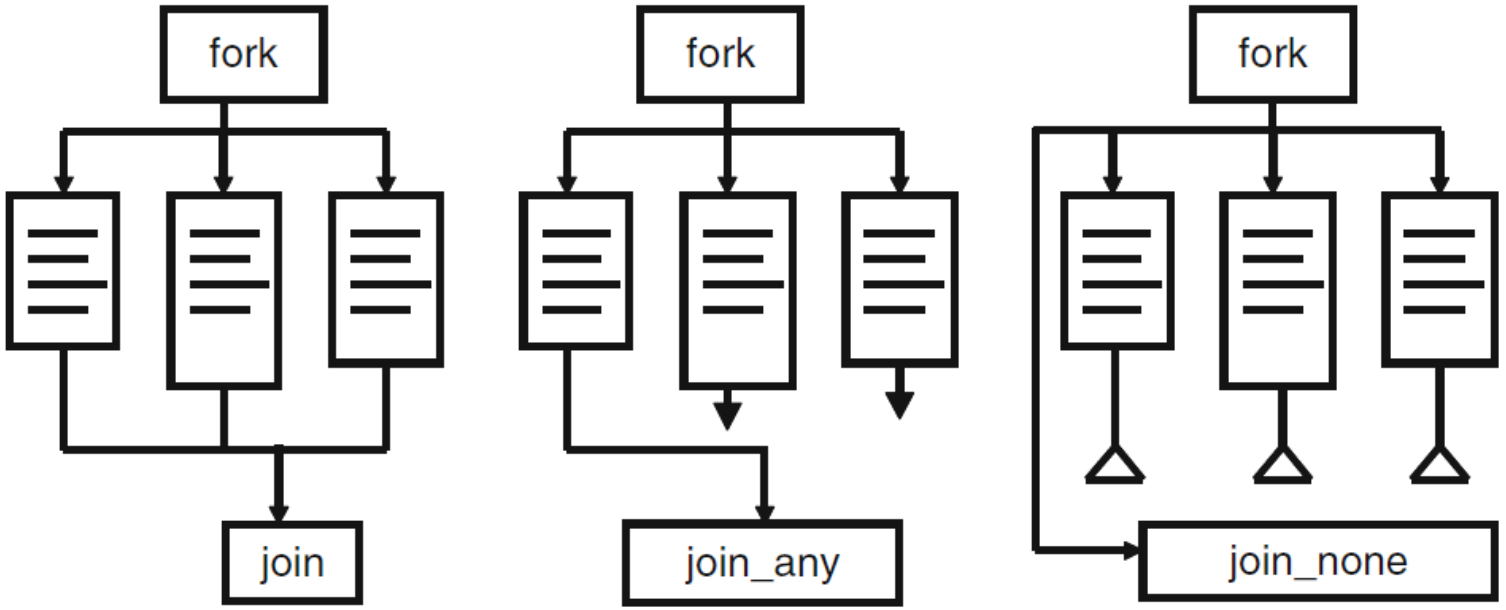
Agenda

1. 随机约束
2. 任务和函数
3. **线程控制**
4. 进程间同步和通信
5. 虚方法

线程控制

并行线程

- Verilog中与顺序线程begin...end相对的是并行线程fork...join。
- SV引入了两种新新的创建线程的方法，fork...join_none和fork...join_any。



线程控制

并行线程

- fork...join需要所有并行的线程都结束以后才会继续执行。
- fork...join_any则会等到任何一个线程结束以后就继续执行。
- fork...join_none则不会等待其子线程而继续执行。

```
fork
  begin
    $display( "First Block\n" );
    # 20ns;
  end
  begin
    $display( "Second Block\n" );
    @eventA;
  end
join
```

线程控制

并行线程

- 需要注意的是，fork...join_any和fork...join_none继续执行后，其一些未完成的子程序仍将在后台运行。
- 如果要等待这些子程序全部完成，或者停止这些子程序，可以使用wait fork或者disable fork。

```
task do_test;
  fork
    exec1();
    exec2();
  join_any
  fork
    exec3();
    exec4();
  join_none
  wait fork; // block until exec1 ... exec4 complete
endtask
```

线程控制

并行线程

```
task get_first( output int adr );
  fork
    wait_device( 1, adr );
    wait_device( 7, adr );
    wait_device( 13, adr );
  join_any
  disable fork;
endtask
```

线程控制
时序控制

- SV可以通过延迟控制或者事件等待来对过程块完成时序控制。
- 延迟控制即通过#来完成。

```
#10 rega = regb;
```

- 事件（event）控制即通过@来完成。

```
@r rega = regb;
@(posedge clock) rega = regb;
```

- wait语句也可以与事件或者表达式结合来完成。

```
real AOR[];
initial wait(AOR.size() > 0) ....;
```

Agenda

1. 随机约束
2. 任务和函数
3. 线程控制
4. **进程间同步和通信**
5. 虚方法

进程间同步和通信

概述

- 测试平台中的所有线程都需要同步并交换数据。
- 一个线程等待另外一个，例如验证环境需要等待所有激励结束、比较结束才可以结束仿真。
- 比如监测器需要将监测到的数据发送至比较器，比较器又需要从不同的缓存获取数据进行比较。

进程间同步和通信

事件event

- 可以通过event来声明一个命名event变量，并且去触发它。
- 这个命名event可以用来控制进程的执行。
- 可以通过->来触发事件。
- 其它等待该事件的进程可以通过@操作符或者wait()来检查event触发状态来完成。

进程间同步和通信

事件event

```
event done, blast; // declare two new events
event done_too = done; // done_too as alias to done
task trigger( event ev );
  -> ev;
endtask
...

fork
  @ done_too; // wait for done through done_too
  #1 trigger( done ); // trigger done
join

fork
  -> blast;
  wait ( blast.triggered );
join
```

进程间同步和通信

wait_order()

- wait_order可以使得进程保持等待，直到在参数列表中的事件event按照顺序从左到右依次完成。
- 如果参数列表中的事件被触发但是没有按照要求的顺序，那么会使得等待操作失败。

```
wait_order( a, b, c);

wait_order( a, b, c ) else $display( "Error: events
out of order" );

bit success;
wait_order( a, b, c ) success = 1; else success = 0;
```

进程间同步和通信

旗语（semaphore）

- 旗语从概念上讲，是一个容器。
- 在创建旗语的时候，会为其分配固定的钥匙数量。
- 使用旗语的进程必须先获得其钥匙，才可以继续执行。
- 旗语的钥匙数量可以有多个，等待旗语钥匙的进程也可同时有多个。
- 旗语通常用于互斥，对共享资源的访问控制，以及基本的同步。

进程间同步和通信

旗语（semaphore）

- 创建旗语，并为其分配钥匙的方式如下：

```
semaphore sm;
sm = new();
```

- 创建一个具有固定钥匙数量的旗语：`new (N = 0)`
- 从旗语那里获取一个或多个钥匙（阻塞型）：`get (N = 1)`
- 将一个或多个钥匙返回到旗语中：`put (N = 1)`
- 尝试获取一个或多个钥匙而不会阻塞（非阻塞型）：`try_get (N = 1)`

进程间同步和通信

semaphore::new()

- new () 的原型如下:
  ```
  function new (int keyCount = 0) ;
  ```
- keyCount指定最初分配给旗语的钥匙数目。
- 当更多钥匙放入旗语时，钥匙数目可以超出初始时的keyCount数量，而不是删除。
- keyCount的默认值为0。
- new () 函数返回旗语的句柄。

进程间同步和通信

semaphore::put()

- put（）方法用于将钥匙数量返回给旗语。
- put（）的原型如下：

```
function void put (int keyCount = 1) ;
```

- keyCount指定返回到旗语的钥匙数量。 默认值为1。
- 调用semaphore.put（）函数时，指定数量的钥匙将返回到旗语。
- 如果其它进程已经在等待旗语，则该进程应在有足够数量钥匙的情况下返回。

进程间同步和通信

semaphore::get()

- `get()`方法用于从旗语中获取指定数量的钥匙。
- `get()`的原型如下：

```
task get(int keyCount = 1);
```

- `keyCount`指定从旗语获取所需的钥匙数，默认值为`1`。
- 如果指定数量的钥匙可用，则该方法返回并继续执行。
- 如果指定数量的钥匙不足，进程将阻塞，直到钥匙数目充足。
- 旗语的等待队列是先进先出（`FIFO`），即先排队等待旗语的将优先得到钥匙。

进程间同步和通信

semaphore::try_get()

- try_get（）方法用于从信号量中获取指定数量的钥匙，但不会被阻塞。
- try_get（）的原型如下：

```
function int try_get (int keyCount = 1) ;
```

- keyCount指定从旗语处获取所需的钥匙数目，默认值为1。
- 如果指定数量的钥匙可用，则该方法返回正数并继续执行。
- 如果指定数量的钥匙不足，则该方法返回0。

进程间同步和通信

信箱的内建方法

- 创建信箱：new()
- 将信息写入信箱：put()
- 试着写入信箱但不会阻塞：try_put()
- 获取信息：get()同时会取出数据，peek()不会取出数据
- 试着从信箱取出数据但不会阻塞：try_get()/try_peek()
- 获取信箱信息的数目：num()

进程间同步和通信

mailbox::new()

- 可以在创建信箱的时候限定或者不限定其大小。

```
function new(int bound = 0);
```

- 默认情况下，如果不传入参数，bound默认值为0，表示不限定信箱大小，如果传入的数值大于0，那么表示信箱的最大容量。
- bound应为正数，如果未负数的话，系统会提示警告和出现无法预期的行为。

进程间同步和通信

mailbox::num()

- num()会返回信箱目前的消息数目。
- 可以结合num()与get()或者put()，防止get()/put()方法在信箱为空或者为满的时候被阻塞。

进程间同步和通信

mailbox::{put(), try_put()}

```
task put( singular message);
function int try_put( singular message);
```

- put()会将信息按照FIFO的顺序写入到信箱中，如果信箱此时已满，则put()任务会挂起，直到信箱有新的空间可以容纳消息。
- try_put()也会按照FIFO顺序写入信箱，不会发生阻塞。如果信箱已满，则写入失败，返回0；如果信箱未满，则写入成功，返回1。

进程间同步和通信

mailbox::{get(), try_get()}

```
task get( ref singular message );
function int try_get( ref singular message );
```

- get()会将信息从信箱中取出，如果信箱此时为空，则get()任务会挂起，直到信箱中有消息可以读取，任务才会返回。该方法会将读取到的消息从信箱中移除。
- try_get()也会将信息从信箱中取出，只是该函数不会发生阻塞。如果信箱为空，则读取失败，返回0；如果信箱不为空，则读取成功，返回1。该方法也会将读取到的消息从信箱中移除。

进程间同步和通信

mailbox::{peek(), try_peek()}

```
task peek( ref singular message );
function int try_peek( ref singular message );
```

- peek()会将信息从信箱中拷贝，如果信箱此时为空，则peek()任务会挂起，直到信箱中有消息可以拷贝，任务才会返回。该方法不会将读取到的消息从信箱中移除。
- try_peek()也会将信息从信箱中拷贝，只是该函数不会发生阻塞。如果信箱为空，则拷贝失败，返回0；如果信箱不为空，则拷贝成功，返回1。该方法也不会将拷贝到的消息从信箱中移除。

进程间同步和通信

参数化信箱

- 默认的信箱，在没有指定存储类型的情况下，可以存储任何类型的数据，这是一个强大的功能（程序也可能会为此受伤）。
- 为了避免运行时错误和类型不匹配，我们建议在声明信箱的时候，为其指定存储的类型。
- 这种参数化信箱的方式可以使得在编译时就能够检查出类型不匹配的情况。

```
typedef mailbox #(string) s_mbox;
s_mbox sm = new;
string s;
sm.put( "hello" );
...
sm.get( s ); // s <- "hello"
```

Agenda

1. 随机约束
2. 任务和函数
3. 线程控制
4. 进程间同步和通信
5. **虚方法**

虚方法

概述

- 类的成员方法可加以修饰词virtual（虚方法）。
- 虚方法是一种基本的多态（polymorphic）结构。
- 一个虚方法可以覆盖基类的同名方法。
- 在父类和子类中声明的虚方法，其方法名、参数名、参数方向等都应该保持一致。
- 在调用虚方法时，它将调用句柄指向对象的方法，而不受句柄类型的影响。

虚方法

示例

```
class BasePacket;
  int A = 1;
  int B = 2;
  function void printA;
    $display("BasePacket::A is %d", A);
  endfunction : printA
  virtual function void printB;
    $display("BasePacket::B is %d", B);
  endfunction : printB
endclass : BasePacket

class My_Packet extends BasePacket;
  int A = 3;
  int B = 4;
  function void printA;
    $display("My_Packet::A is %d", A);
  endfunction: printA
  virtual function void printB;
    $display("My_Packet::B is %d", B);
  endfunction : printB
endclass : My_Packet
```

虚方法
示例

```
BasePacket P1 = new;
My_Packet P2 = new;

initial begin
  P1.printA; // displays 'BasePacket::A is 1'
  P1.printB; // displays 'BasePacket::B is 2'
  P1 = P2; // P1 has a handle to a My_packet object
  P1.printA; // displays 'BasePacket::A is 1'
  P1.printB; // displays 'My_Packet::B is 4‘ latest derived method
  P2.printA; // displays 'My_Packet::A is 3'
  P2.printB; // displays 'My_Packet::B is 4'
end
```

类型转换

静态转换

- 静态转换操作符不对转换值进行检查。
- 转换时指定目标类型，并在要转换的表达式前加上单引号即可。
- Verilog对整数和实数类型，或者不同位宽的向量之间进行隐式转换。

```
int i;
real r;
i = int '(10.0 - 0.1); // cast is optional
r = real'(42);         // cast is optional
```

类型转换
动态转换

- 我们总是可以将子类的句柄赋值给父类的句柄。
- 但是在我们将父类的句柄赋值给子类的句柄时，编译将会报错。
- $cast()系统函数可以将父类句柄转换为子类句柄，只要该父类句柄指向的是一个子类的对象。

```
function int $cast( singular dest_var, singular
source_exp );
```

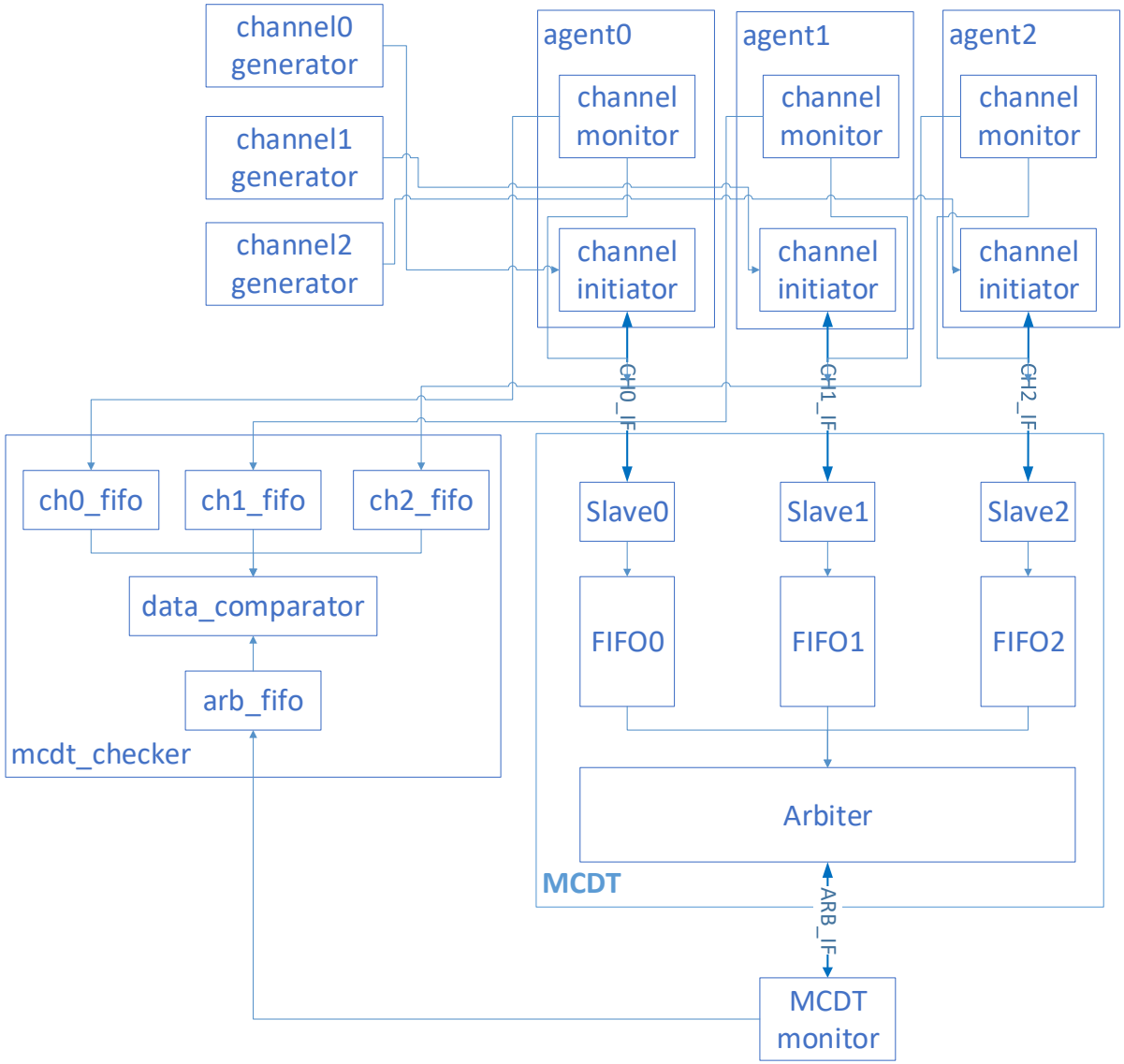
或者

```
task $cast( singular dest_var, singular source_exp );
```

Agenda

1. **MCDT测试平台结构**
2. 实现channel stimulator
3. 实现channel monitor
4. 实现mcdt monitor
5. 实现mcdt checker
6. 实现顶层环境
7. 实现测试用例

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

MCDT测试平台结构

- channel initiator与channel monitor一一对应
- initiator与monitor被封装在了agent中
- 由于arbiter不需要握手需要，因此只外置了mcdt monitor而不需要mcdt responder
- 所有的monitor都将监测到的数据送至mcdf checker
- mcdf checker内置若干FIFO，接收从MCDT的输入端和输出端监测到的数据
- 由于checker输入和输出端FIFO中存储的数据类型一致，这为数据比较带来了方便

Agenda

1. MCDT测试平台结构
2. **实现channel stimulator**
3. 实现channel monitor
4. 实现mcdt monitor
5. 实现mcdt checker
6. 实现顶层环境
7. 实现测试用例

实现channel stimulator

```
class chnl_trans;
  rand bit[31:0] data[];
  rand int ch_id;
  rand int pkt_id;
  rand int data_nidles;
  rand int pkt_nidles;
  bit rsp;
  local static int obj_id = 0;
  constraint cstr{
    soft data.size inside {[4:8]};
    foreach(data[i])
      data[i] == 'hC000_0000 + (this.ch_id<<24) + (this.pkt_id<<8) + i;
    soft ch_id == 0;
    soft pkt_id == 0;
    data_nidles inside {[0:2]};
    pkt_nidles inside {[1:10]};
  };
```

实现channel stimulator

```
  function new();
    this.obj_id++;
  endfunction

  function chnl_trans clone();
    chnl_trans c = new();
    c.data = this.data;
    c.ch_id = this.ch_id;
    c.pkt_id = this.pkt_id;
    c.data_nidles = this.data_nidles;
    c.pkt_nidles = this.pkt_nidles;
    c.rsp = this.rsp;
    return c;
  endfunction
endclass: chnl_trans
```

实现channel stimulator

```
class chnl_initiator;
  local string name;
  local virtual chnl_intf intf;
  mailbox #(chnl_trans) req_mb;
  mailbox #(chnl_trans) rsp_mb;

  function new(string name = "chnl_initiator");
    this.name = name;
  endfunction

  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

实现channel stimulator

```
task run();
  this.drive();
endtask

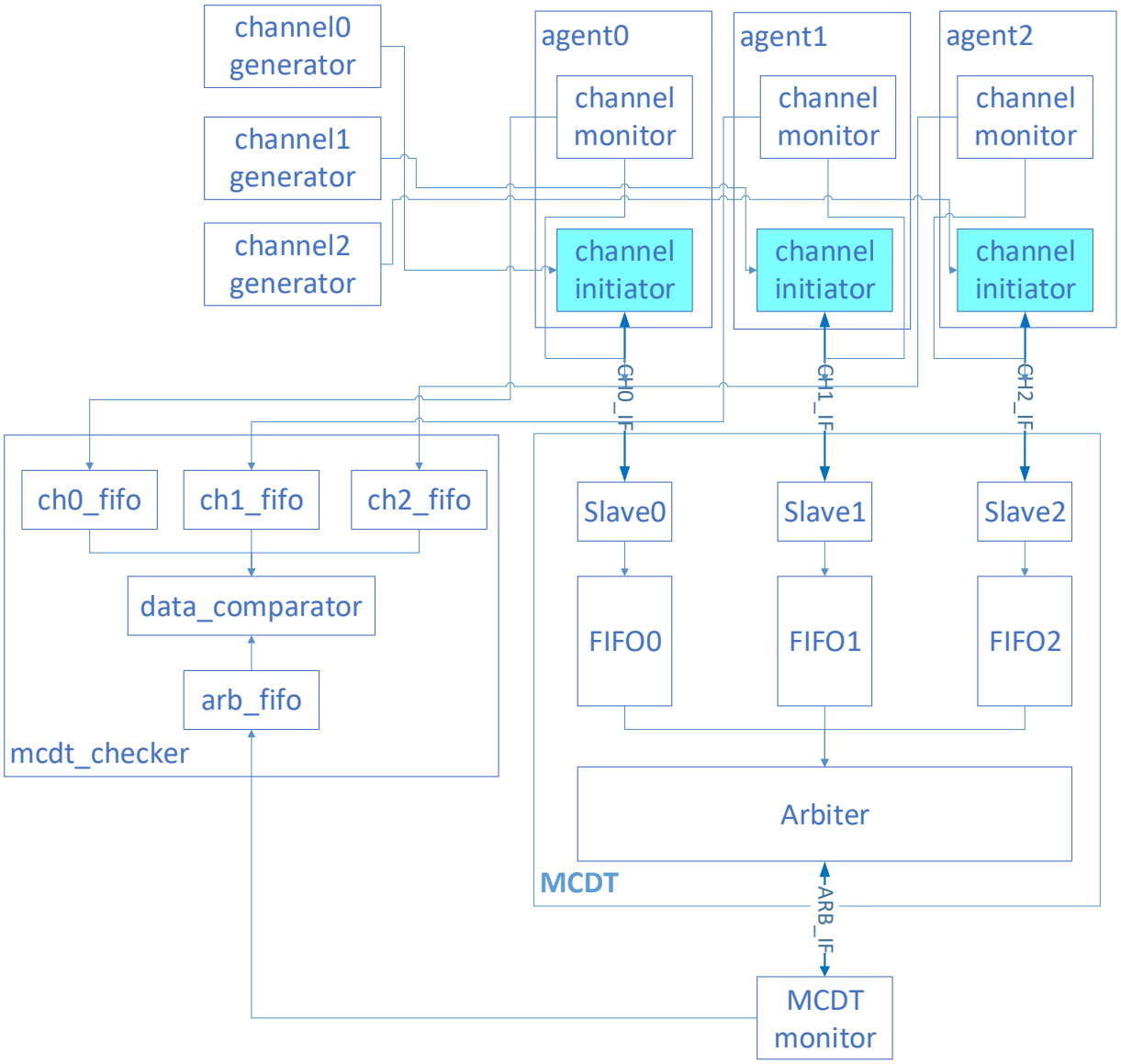
task drive();
  chnl_trans req, rsp;
  @(posedge intf.rstn);
  forever begin
    this.req_mb.get(req);
    this.chnl_write(req);
    rsp = req.clone();
    rsp.rsp = 1;
    this.rsp_mb.put(rsp);
  end
endtask
```

实现channel stimulator

```
  task chnl_write(input chnl_trans t);
    foreach(t.data[i]) begin
      @(posedge intf.clk);
      intf.ch_valid <= 1;
      intf.ch_data <= t.data[i];
      @(negedge intf.clk);
      wait(intf.ch_ready === 'b1);
      $display("%0t channel initiator [%s] sent data %x",
               $time, name, t.data[i]);
      repeat(t.data_nidles) chnl_idle();
    end
    repeat(t.pkt_nidles) chnl_idle();
  endtask

  task chnl_idle();
    @(posedge intf.clk);
    intf.ch_valid <= 0;
    intf.ch_data <= 0;
  endtask
endclass: chnl_initiator
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

实现channel stimulator

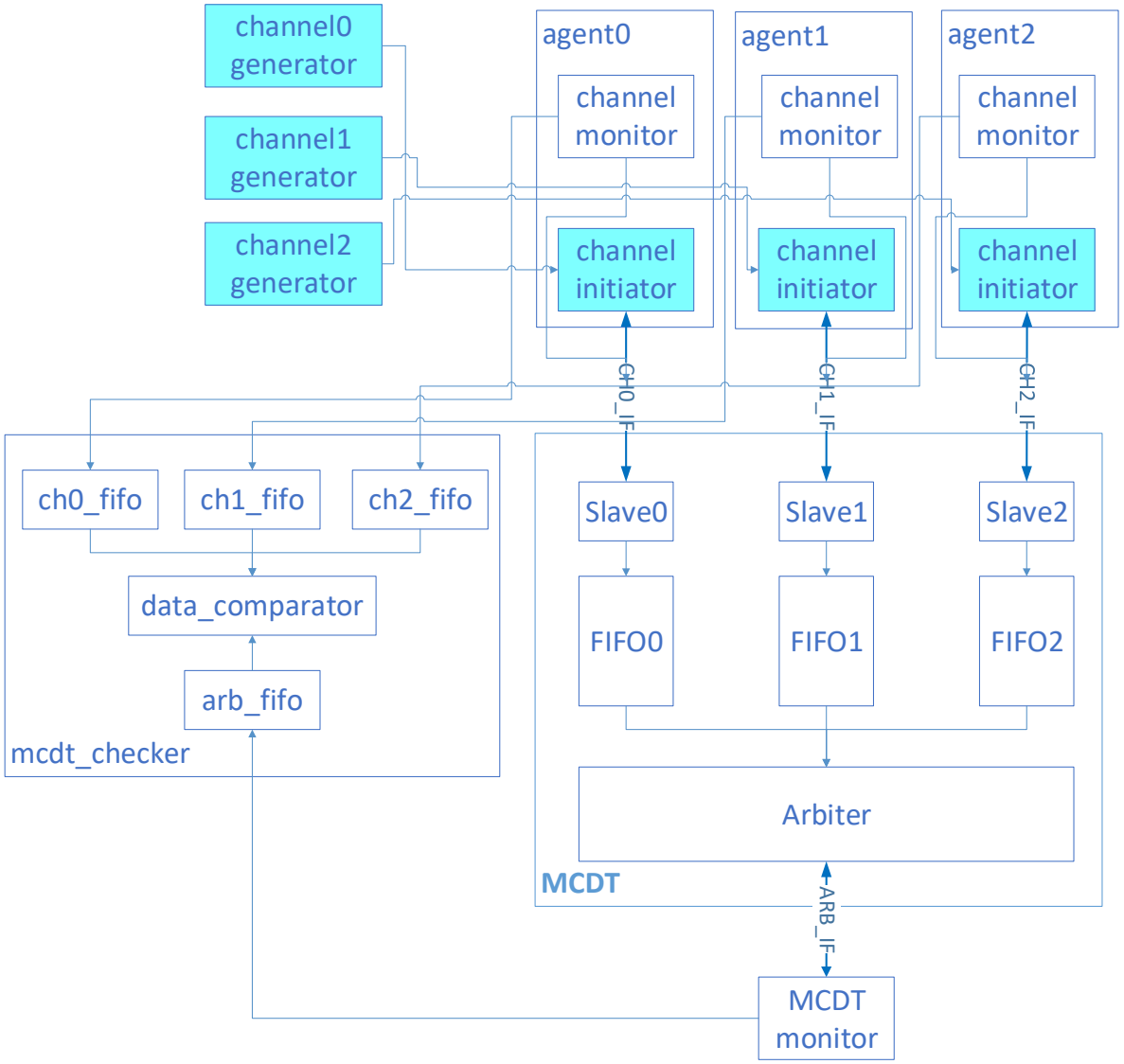
```
class chnl_generator;
   rand int pkt_id = -1;
   rand int ch_id = -1;
   rand int data_nidles = -1;
   rand int pkt_nidles = -1;
   rand int data_size = -1;
   rand int ntrans = 10;
   mailbox #(chnl_trans) req_mb;
   mailbox #(chnl_trans) rsp_mb;
   constraint cstr{
     soft ch_id == -1;
     soft pkt_id == -1;
     soft data_size == -1;
     soft data_nidles == -1;
     soft pkt_nidles == -1;
     soft ntrans == 10;
   }
   function new();
     this.req_mb = new();
     this.rsp_mb = new();
   endfunction
```

实现channel stimulator

```
  task run();
    repeat(ntrans) send_trans();
    run_stop_flags.put();
  endtask

  // generate transaction and put into local mailbox
  task send_trans();
    chnl_trans req, rsp;
    req = new();
    assert(req.randomize with {local::ch_id >= 0 -> ch_id == local::ch_id;
                               local::pkt_id >= 0 -> pkt_id == local::pkt_id;
                               local::data_nidles >= 0 -> data_nidles == local::data_nidles;
                               local::pkt_nidles >= 0 -> pkt_nidles == local::pkt_nidles;
                               local::data_size >0 -> data.size() == local::data_size;
                              })
      else $fatal("[RNDFAIL] channel packet randomization failure!");
    this.pkt_id++;
    this.req_mb.put(req);
    this.rsp_mb.get(rsp);
    assert(rsp.rsp)
      else $error("[RSPERR] %0t error response received!", $time);
  endtask
endclass: chnl_generator
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

Agenda

1. MCDT测试平台结构
2. 实现channel stimulator
3. **实现channel monitor**
4. 实现mcdt monitor
5. 实现mcdt checker
6. 实现顶层环境
7. 实现测试用例

实现channel monitor

```
typedef struct packed {
  bit[31:0] data;
  bit[1:0] id;
} mon_data_t;

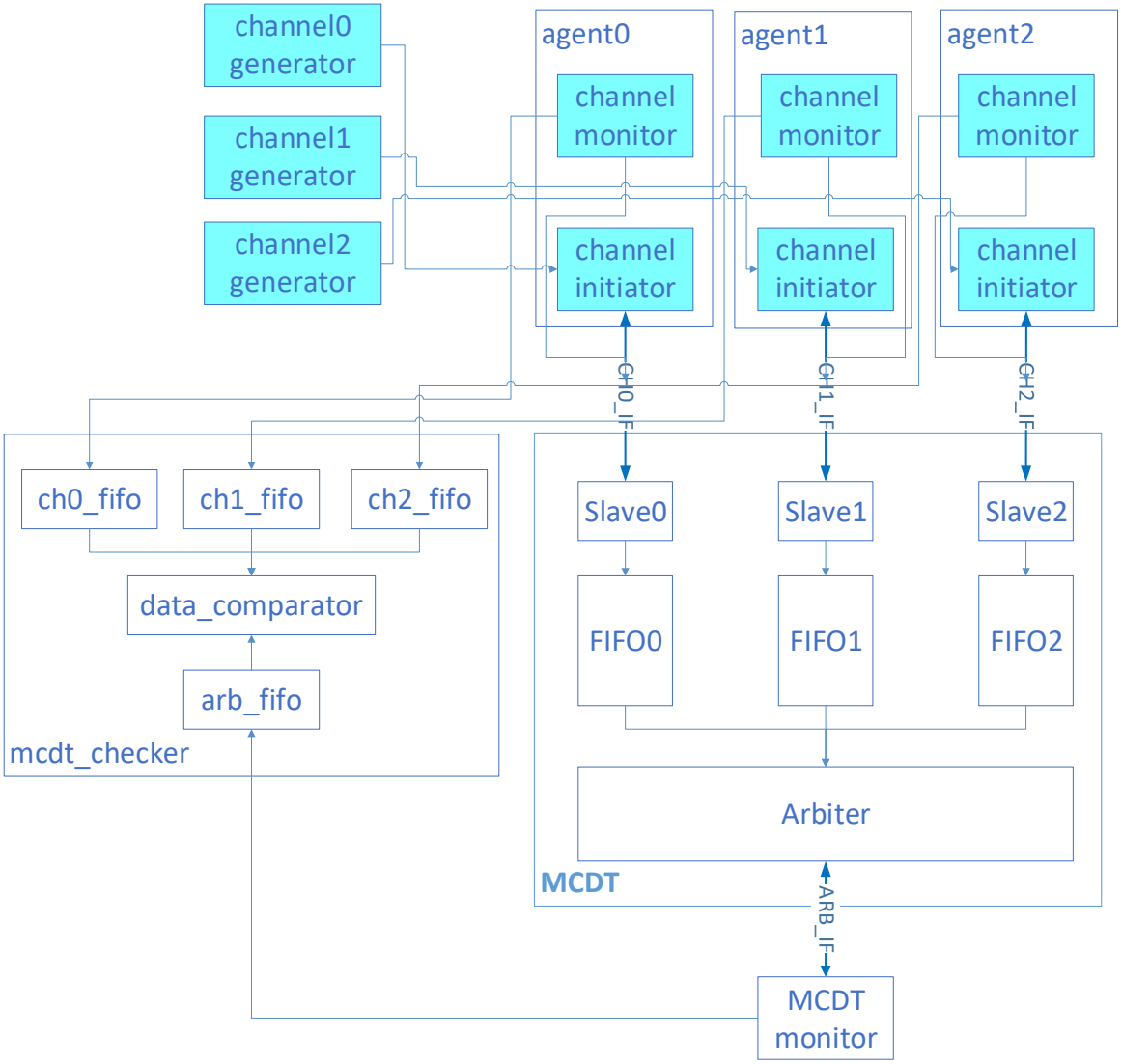
class chnl_monitor;
  local string name;
  local virtual chnl_intf intf;
  mailbox #(mon_data_t) mon_mb;
  function new(string name="chnl_monitor");
    this.name = name;
  endfunction
  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

实现channel monitor

```
  task run();
    this.mon_trans();
  endtask

  task mon_trans();
    mon_data_t m;
    forever begin
      @(posedge intf.clk iff (intf.ch_valid===1'b1 && intf.ch_ready===1'b1));
      m.data = intf.ch_data;
      mon_mb.put(m);
      $display("%0t %s monitored channle data %8x",
               $time, this.name, m.data);
    end
  endtask
endclass: chnl_monitor
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

Agenda

1. MCDT测试平台结构
2. 实现channel stimulator
3. 实现channel monitor
4. **实现mcdt monitor**
5. 实现mcdt checker
6. 实现顶层环境
7. 实现测试用例

实现mcdt monitor

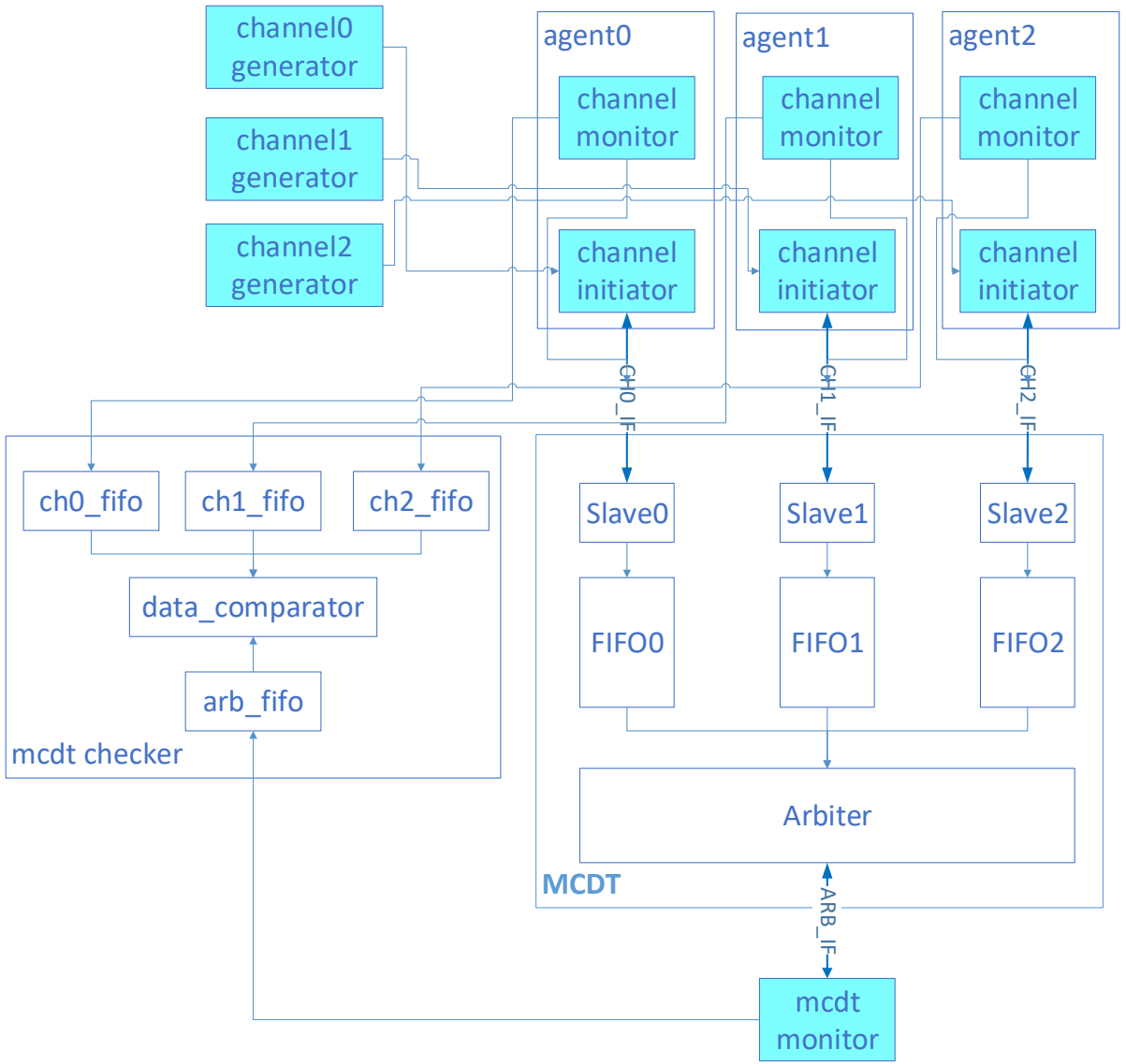
```
class mcdt_monitor;
  local string name;
  local virtual mcdt_intf intf;
  mailbox #(mon_data_t) mon_mb;
  function new(string name="mcdt_monitor");
    this.name = name;
  endfunction
  task run();
    this.mon_trans();
  endtask

  function void set_interface(virtual mcdt_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

实现mcdt monitor

```
  task mon_trans();
     mon_data_t m;
     forever begin
       @(posedge intf.clk iff intf.mcdt_val===1'b1);
       m.data = intf.mcdt_data;
       m.id = intf.mcdt_id;
       mon_mb.put(m);
       $display("%0t %s monitored mcdt data %8x and id %0d",
                $time, this.name, m.data, m.id);
     end
   endtask
endclass: mcdt_monitor
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

Agenda

1. MCDT测试平台结构
2. 实现channel stimulator
3. 实现channel monitor
4. 实现mcdt monitor
5. **实现mcdt checker**
6. 实现顶层环境
7. 实现测试用例

实现mcdt checker

```
class mcdt_checker;
  local string name;
  local int error_count;
  local int cmp_count;
  mailbox #(mon_data_t) in_mbs[3];
  mailbox #(mon_data_t) out_mb;

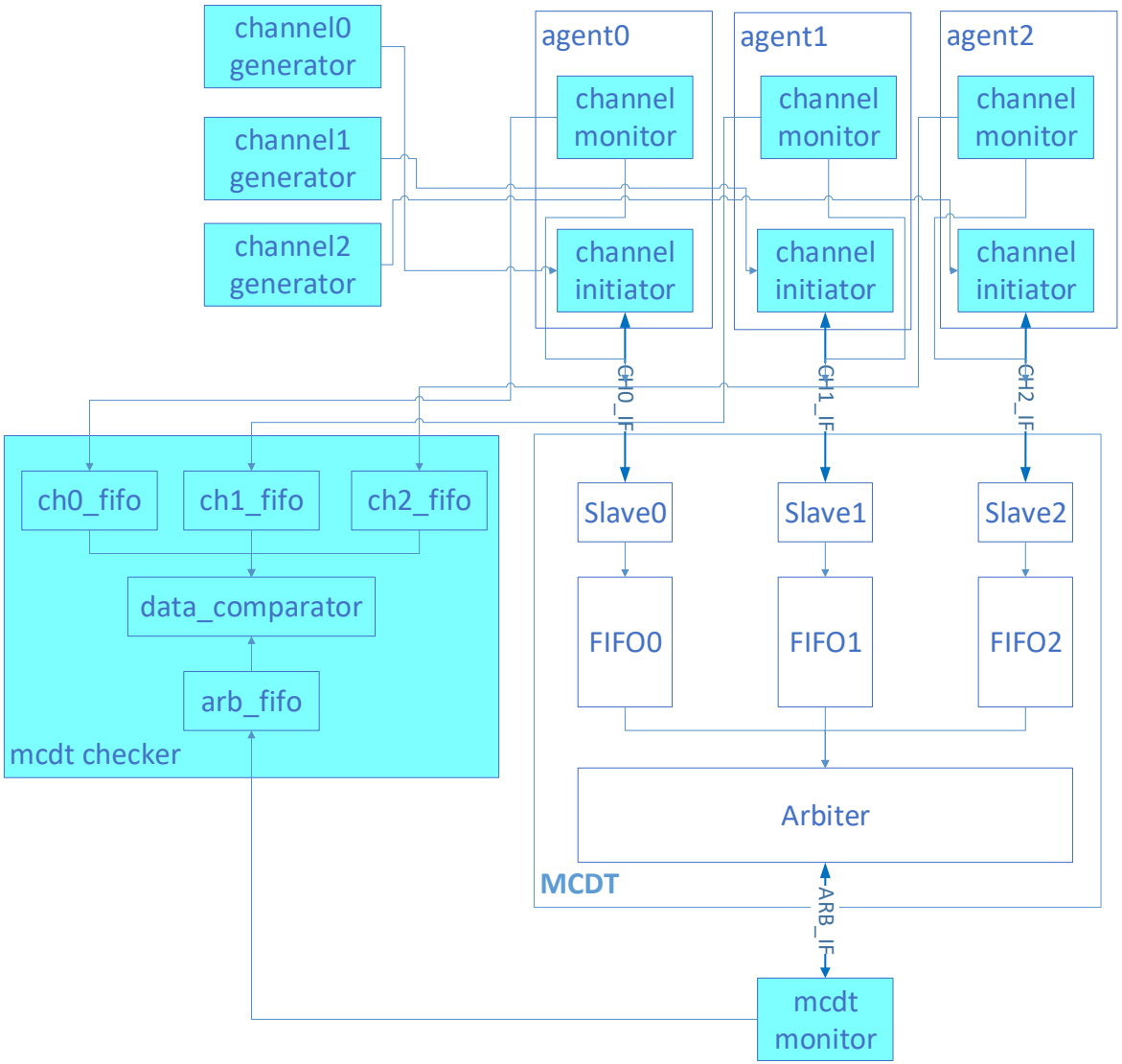
  function new(string name="mcdt_checker");
    this.name = name;
    foreach(this.in_mbs[i]) this.in_mbs[i] = new();
    this.out_mb = new();
    this.error_count = 0;
    this.cmp_count = 0;
  endfunction

  task run();
    this.do_compare();
  endtask
```

实现mcdt checker

```
  task do_compare();
    mon_data_t im, om;
    forever begin
      out_mb.get(om);
      case(om.id)
        0: in_mbs[0].get(im);
        1: in_mbs[1].get(im);
        2: in_mbs[2].get(im);
        default: $fatal("id %0d is not available", om.id);
      endcase
      if(om.data != im.data) begin
        this.error_count++;
        $error("[CMPFAIL] Compared failed! mcdt out data %8x ch_id %0d is
               not equal with channel in data %8x",
               om.data, om.id, im.data);
      end
      else begin
        $display("[CMPSUCD] Compared succeeded! mcdt out data %8x
                  ch_id %0d is equal with channel in data %8x",
                  om.data, om.id, im.data);
      end
      this.cmp_count++;
    end
  endtask
endclass: mcdt_checker
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

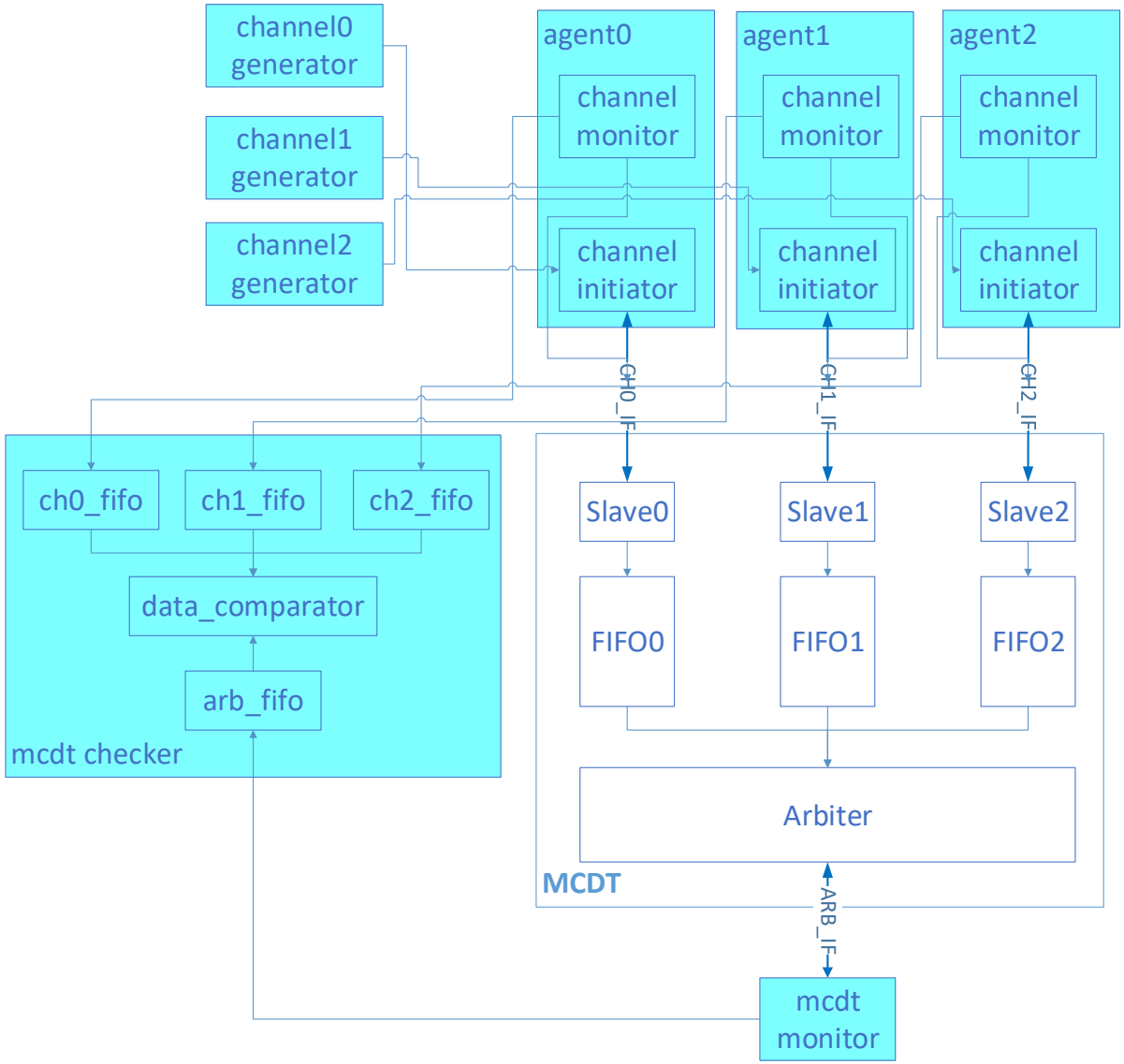
Agenda

1. MCDT测试平台结构
2. 实现channel stimulator
3. 实现channel monitor
4. 实现mcdt monitor
5. 实现mcdt checker
6. **实现顶层环境**
7. 实现测试用例

实现顶层环境

```
class chnl_agent;
  local string name;
  chnl_initiator init;
  chnl_monitor mon;
  virtual chnl_intf vif;
  function new(string name = "chnl_agent");
    this.name = name;
    this.init = new({name, ".init"});
    this.mon = new({name, ".mon"});
  endfunction
  function void set_interface(virtual chnl_intf vif);
    this.vif = vif;
    init.set_interface(vif);
    mon.set_interface(vif);
  endfunction
  task run();
    fork
      init.run();
      mon.run();
    join
  endtask
endclass: chnl_agent
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
channel monitor
channel initiator
agent1
channel monitor
channel initiator
agent2
channel monitor
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

实现顶层环境

```
class mcdt_root_test;
  chnl_generator gen[3];
  chnl_agent agents[3];
  mcdt_monitor mcdt_mon;
  mcdt_checker chker;
  protected string name;
  event gen_stop_e;
  function new(string name = "mcdt_root_test");
    this.name = name;
    this.chker = new();
    foreach(agents[i]) begin
      this.agents[i] = new($sformatf("agents[%0d]",i));
      this.gen[i] = new();
      this.agents[i].init.req_mb = this.gen[i].req_mb;
      this.agents[i].init.rsp_mb = this.gen[i].rsp_mb;
      this.agents[i].mon.mon_mb = this.chker.in_mbs[i];
    end
    this.mcdt_mon = new();
    this.mcdt_mon.mon_mb = this.chker.out_mb;
    $display("%s instantiated and connected objects", this.name);
  endfunction
```

实现顶层环境

```
virtual task gen_stop_callback();
  // empty
endtask

virtual task run_stop_callback();
  $display("run_stop_callback enterred");
  // by default, run would be finished once generators raised 'finish'
  $display("%s: wait for all generators have generated
            and tranferred transcations", this.name);
  run_stop_flags.get(3);
  $finish();
endtask
```

实现顶层环境

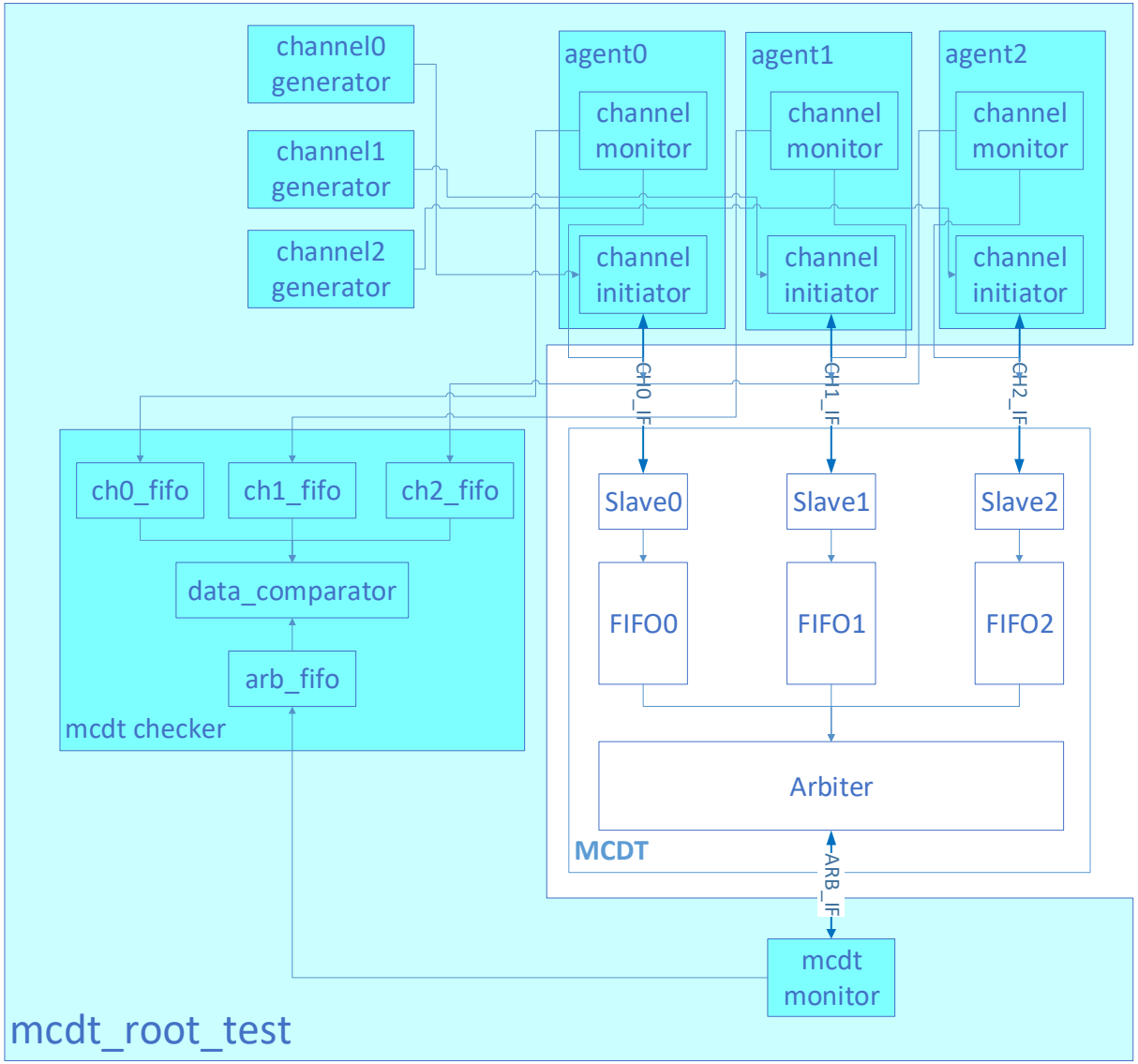
```
virtual task run();
  this.do_config();
  fork
    agents[0].run();
    agents[1].run();
    agents[2].run();
    mcdt_mon.run();
    chker.run();
  join_none
  // run first the callback thread to conditionally disable gen_threads
  fork
    this.gen_stop_callback();
    @(this.gen_stop_e) disable gen_threads;
  join_none
  fork : gen_threads
    gen[0].run();
    gen[1].run();
    gen[2].run();
  join
  run_stop_callback(); // wait until run stop control task finished
endtask
```

实现顶层环境

```
  virtual function void set_interface(virtual chnl_intf ch0_vif
                                      ,virtual chnl_intf ch1_vif
                                      ,virtual chnl_intf ch2_vif
                                      ,virtual mcdt_intf mcdt_vif
                                     );
    agents[0].set_interface(ch0_vif);
    agents[1].set_interface(ch1_vif);
    agents[2].set_interface(ch2_vif);
    mcdt_mon.set_interface(mcdt_vif);
  endfunction

  virtual function void do_config();
  endfunction
endclass: mcdt_root_test
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor
mcdt_root_test

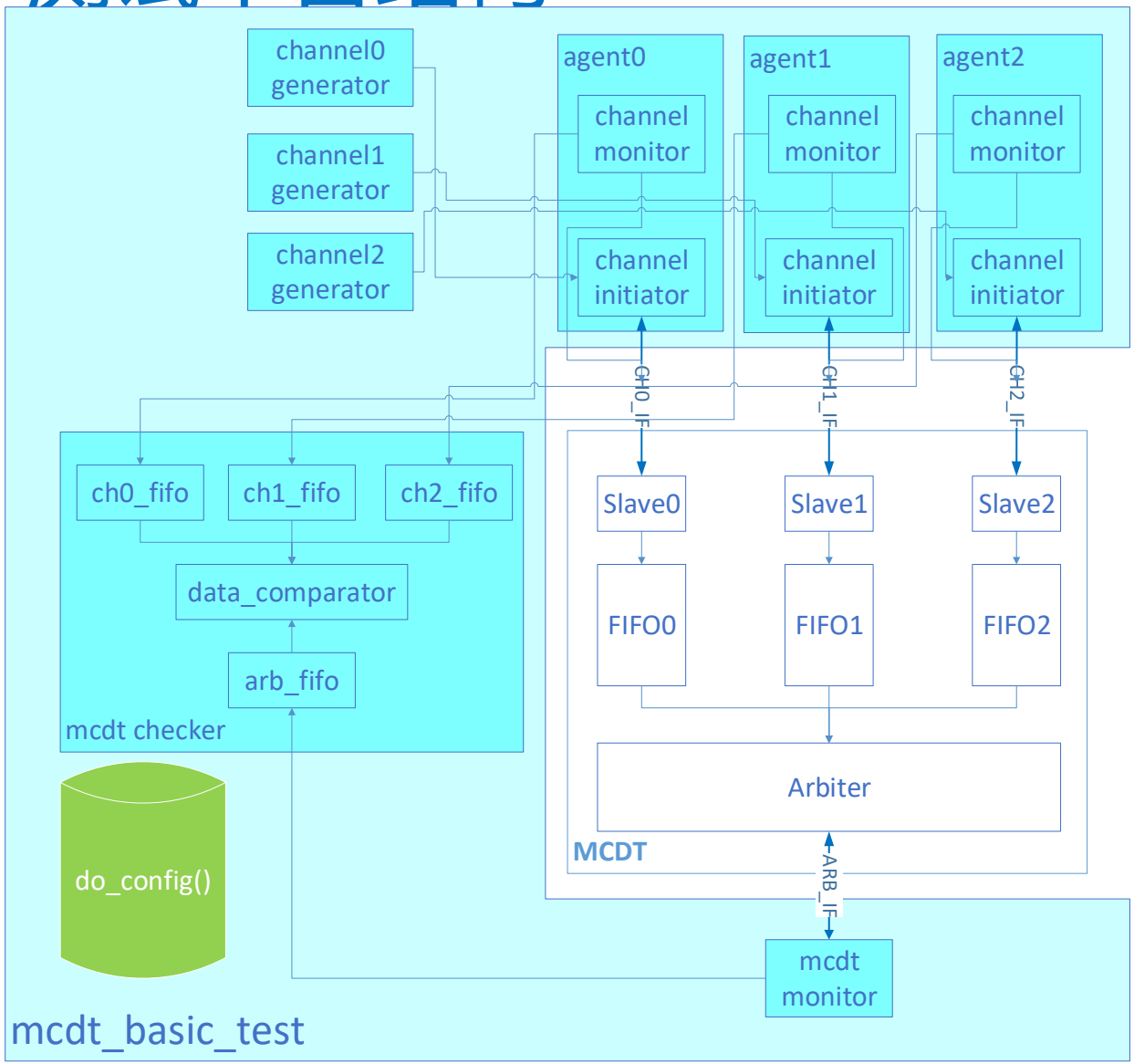
Agenda

1. MCDT测试平台结构
2. 实现channel stimulator
3. 实现channel monitor
4. 实现mcdt monitor
5. 实现mcdt checker
6. 实现顶层环境
7. **实现测试用例**

实现测试用例

```
class mcdt_basic_test extends mcdt_root_test;
  function new(string name = "mcdt_basic_test");
    super.new(name);
  endfunction
  virtual function void do_config();
    super.do_config();
    assert(gen[0].randomize() with
           {ntrans==100; data_nidles==0; pkt_nidles==1; data_size==8;})
      else $fatal("[RNDFAIL] gen[0] randomization failure!");
    assert(gen[1].randomize() with
          {ntrans==50; data_nidles inside {[1:2]};
           pkt_nidles inside {[3:5]}; data_size==6;})
      else $fatal("[RNDFAIL] gen[1] randomization failure!");
    assert(gen[2].randomize() with
           {ntrans==80; data_nidles inside {[0:1]};
            pkt_nidles inside {[1:2]}; data_size==32;})
      else $fatal("[RNDFAIL] gen[2] randomization failure!");
  endfunction
endclass: mcdt_basic_test
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
do_config()
mcdt monitor
mcdt_basic_test

Agenda

1. **概述**
2. 覆盖率的种类
3. 代码覆盖率
4. 功能覆盖率
5. 从功能描述到覆盖率

概述

- “验证如果没有量化，那么就意味着没有尽头。”
- 伴随着复杂SoC系统的验证难度系数成倍增加，无论是定向测试还是随机测试，我们在验证的过程中终究需要回答两个问题：
  - 是否所有设计的功能在验证计划中都已经验证？
  - 代码中的某些部分是否从未执行过？
- 覆盖率就是用来帮助我们在仿真中回答以上问题的指标
- 覆盖率已经被广泛采用，作为衡量验证过程中的重要数据

概述

- 一旦通过覆盖率来量化验证，我们可以在更复杂的情况下捕捉一些功能特性是否被覆盖：
  - 当我们在测试X特性的时候，Y特性是否也在同一时刻被使能和测试？
  - 是否可以精简我们已有的测试来加速仿真，并且取得同样的覆盖率？
  - 覆盖率在达到一定的数值的时候，是否停滞，不再继续上升？
- 简单而言，覆盖率就是用来衡量验证精度和完备性的数据指标
- 覆盖率可以告诉我们在仿真时设计的哪些结构被触发，当然，它也可以告诉我们设计在仿真时的哪些结构从未被触发过

概述

- 只有满足以下三个条件，才可以在仿真中实现高质量的验证：
  - 测试平台必须产生合适的激励来触发一个设计错误
  - 测试平台仍然需要产生合适的激励使得被触发的错误可以进一步传导到输出端口
  - 测试平台需要包含一个监测器（monitor）用来监测被激活的设计错误，以及在它传播的某个节点（内部或者外部）可以捕捉到它

Agenda

1. 概述
2. **覆盖率的种类**
3. 代码覆盖率
4. 功能覆盖率
5. 从功能描述到覆盖率

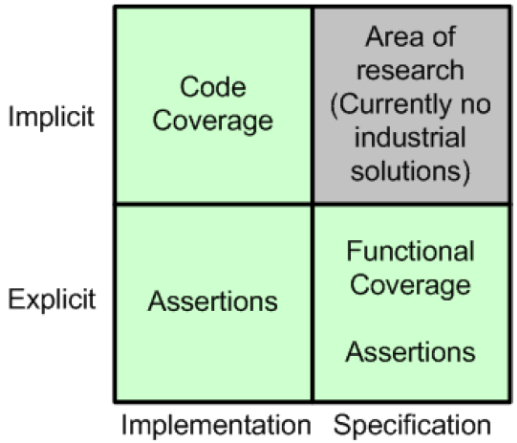
覆盖率的种类

概述

- 没有任何一种单一的覆盖率可以完备地去衡量验证过程
- 即使我们可以达到100%的代码覆盖率，但这并不意味着100%的功能覆盖率。原因在于代码覆盖率并不是用来衡量设计内部的功能运转，或者模块之间的互动，或者功能时序的触发等。
- 类似地，我们即便达到了100%功能覆盖率，也可能只达到了90%的代码覆盖率。原因可能在于我们疏漏了去测试某些功能，或者一些实现的功能并没有被描述。
- 从上述关于代码覆盖率和功能覆盖率简单的论述就可以证明，如果想要得到全面的验证精度，我们就需要多个覆盖率种类的指标。

覆盖率的种类

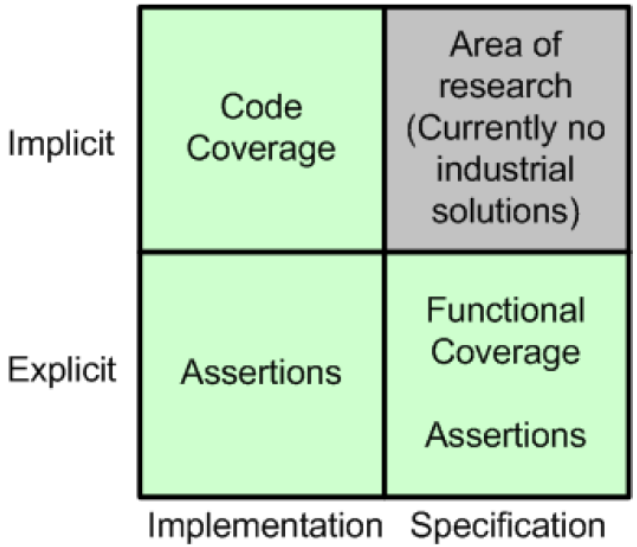
分类



- 最常见的划分覆盖率的两种方法
  - 按照覆盖率生成的方法，即隐性生成还是显性生成。
  - 按照覆盖率溯源，即它们从功能描述而来还是从设计实现而来。
- 例如功能覆盖率是显性的需要人为定义的覆盖率，而代码覆盖率则是隐性覆盖率这是因为仿真工具可以自动从RTL代码来生成。

覆盖率的种类

分类



- 如果将上述两个分类的方式进行组合，那么可以将代码覆盖率、断言覆盖率以及功能覆盖率分别置入到不同的象限。
- 但是需要注意，目前有一个象限仍然处于研究阶段，没有隐性的可以从功能描述生成某种覆盖率的方法，这也是为什么功能覆盖率依然需要人为定义的原因。
- 接下来我们将认识主要的两种覆盖率
  - 代码覆盖率（隐性覆盖率）
  - 功能覆盖率（显性覆盖率）

Agenda

1. 概述
2. 覆盖率的种类
3. **代码覆盖率**
4. 功能覆盖率
5. 从功能描述到覆盖率

代码覆盖率

概述

- 代码覆盖是一种技术，可以识别在验证设计中已执行的代码。
- 包含未知错误的设计的问题在于它看起来就像一个非常好的设计。
- 我们绝对不可能知道被验证的设计在功能上是完全正确的。
- 即便所有测试平台都成功仿真，但是否有部分RTL代码未运行，因此未触发可能的功能错误？ 这是代码覆盖可以帮助回答的问题。

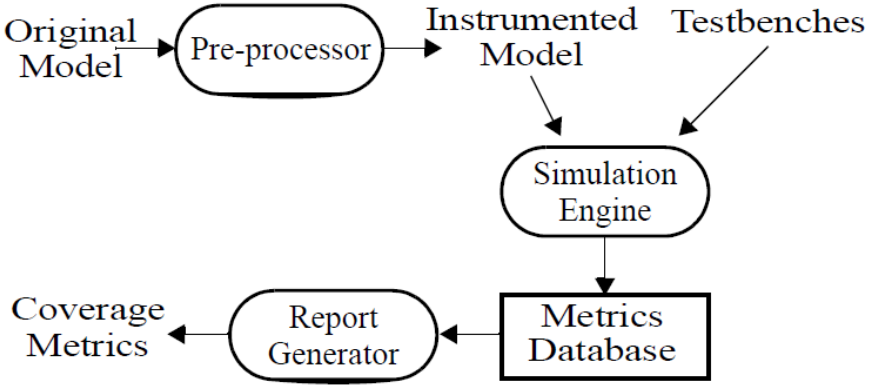
代码覆盖率

概述

- 代码覆盖率并不是SV独有的，这项技术已经在软件工程中使用了相当长的一段时间。
- 代码覆盖率的一个优势在于它可以由仿真工具自动收集，继而用来指出在测试程序中设计源代码哪些被激活触发，而哪些则一直处于非激活的状态。
- 由于代码覆盖率自动的特性，在仿真过程中使能它变得很简单，它不需要修改设计或者验证环境。

代码覆盖率

设计代码 or 测试代码?



- 源代码首先进行检测， 即在源代码的各个位置添加检查点，以在仿真中记录是否执行了某些特定轨迹。
- 代码覆盖的目的是确定我们是否忘记在设计中执行某些代码，无需跟踪测试平台的代码并确认它已执行。
- 如果测试平台中有相当数量的代码没有执行，这也将意味着设计中与该测试意图对应的代码也并未执行。
- 只有在检测到错误时才执行大部分测试平台代码，因此我们对测试平台代码的代码覆盖率指标不太感兴趣。

代码覆盖率

局限

- 代码覆盖率100%并不意味着足够的功能覆盖率
- 研究发现，在回归测试实现了90%的代码覆盖率时，平均只有54%的代码会被监测，这意味着即便代码覆盖率的完备性满足，但依然可能在这些代码中存在漏洞
- 上述的推断是因为不是所有被覆盖的代码都会得到足够的监测，也由于没有得到足够的监测，因此一些即便被触发的漏洞也在传播的过程中没有到达监测点上——漏洞便从眼皮底下"逃逸"了
- 另外，代码覆盖率的数据无法直接映射到哪些设计功能被测试了，所以从这一点来看，代码覆盖率和功能覆盖率之间又是互相独立的

代码覆盖率

行覆盖率（statement/line）

- 用来衡量源代码哪些代码行被执行过，以此来指出哪些代码行没有被执行过
- 从每一行执行的次数，如果设置最小的执行次数，也可以用来做代码测试的一项指标
- 代码覆盖率可以指出在一些缺乏输入激励的情况下，某些赋值的代码行没有被执行的情况，它也可以指出在一些漏洞影响或者无用代码的影响下，一些代码行也无法被执行的情况
- 对于那些无用的代码，也就是永远不会被执行的代码，在代码分析时，我们可以将它们从覆盖率数据中过滤掉

代码覆盖率

行覆盖率（statement/line）

- 行覆盖率也可以称为块覆盖率，其中块是在执行单个语句时执行的语句序列。
- 只要if语句中的表达式求值为true，就会完全执行名为acked的块。 因此，计算该块的执行等同于计算该块中的四个单独语句的执行。

```
if (dtack == 1'b1) begin: acked
    as      <= 1'b0;
    data    <= 16'hZZZZ;
    bus_rq  <= 1'b0;
    state   <= IDLE;
end: acked
```

代码覆盖率

行覆盖率（statement/line）

- 语句块可能未必明确分隔。
- 如下两个语句块：一个在wait语句之前，一个在之后。 等待语句可能从未完成，执行正在等待。 后续的顺序语句可能尚未执行。 因此，它们形成一个单独的语句块。

```
address <= 16'hFFED;
ale     <= 1'b1;
rw      <= 1'b1;
wait (dtack == 1'b1);
read_data = data;
ale     <= 1'b0;
```

代码覆盖率

行覆盖率（statement/line）

```
✓ if (parity == ODD || parity == EVEN) begin
    tx <= compute_parity(data, parity);
    #(tx_time);
  end
✓ tx <= 1'b0;
✓ #(tx_time);
✓ if (stop_bits == 2) begin
✓   tx<=1'b0;
✓   #(tx_time);
  end
```

当parity != ODD &&
parity != EVEN &&
stop_bits == 2

- 行覆盖率 = 6/8 = 75%

代码覆盖率

行覆盖率（statement/line）

- 语句、行或块覆盖率可以衡量验证执行的总代码行数。
- 图形用户界面通常允许用户浏览源代码并快速识别未执行的语句。
- 如下图以图形方式显示了一小部分代码的语句覆盖率报告，来自不同代码覆盖工具或源代码浏览器的报告的实际形式可能会有所不同。

```
if (parity == ODD || parity == EVEN) begin
    tx <= compute_parity(data, parity);
    #(tx_time);
end
tx <= 1'b0;
#(tx_time);
if (stop_bits == 2) begin
    tx <= 1'b0;
    #(tx_time);
end
```

代码覆盖率

行覆盖率（statement/line）

- 要使语句覆盖率度量达到100%，有必要了解导致执行未覆盖语句所需的条件。 在这种情况下，奇偶校验必须设置为ODD或EVEN。 一旦确定了条件，就必须了解它们从未发生过的原因。
- 这是一个永远不会发生的情况吗？ 它是否应该由现有测试环境来验证？ 或者这是一个被遗忘的情况？
- 如果导致未覆盖的语句执行的条件应该被测试，，则表明测试平台在功能上不正确或不完整。 如果条件完全被遗忘，则需要添加到测试平台，创建一个全新的测试用例或使用不同的种子进行额外的运行。

```
if (parity == ODD || parity == EVEN) begin
    tx <= compute_parity(data, parity);
    #(tx_time);
end
tx <= 1'b0;
#(tx_time);
if (stop_bits == 2) begin
    tx <= 1'b0;
    #(tx_time);
end
```

代码覆盖率

分支覆盖率（branch）

- 分支覆盖率是用来对条件语句（if/else， case，?:），指出其执行的分支轨迹。
- 例如判断下列分支的布尔表达式为true或者false

```
if (parity == ODD || parity == EVEN) begin
```

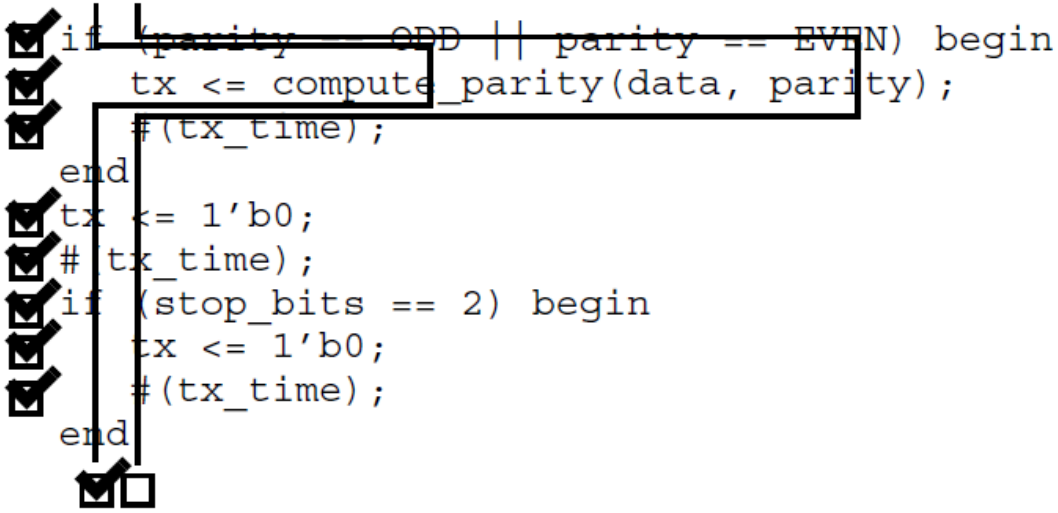
代码覆盖率

条件覆盖率（condition/expression）

- 条件覆盖率是用来衡量一些布尔表达式中各个条件真伪判断的执行轨迹。
- 例如下列if条件语句中的两个条件是否各自衡量为true/false

```
if (parity == ODD || parity == EVEN) begin
```

  - **parity == ODD 或者 parity != ODD**
  - **paraity == EVEN 或者 parity != EVEN**



代码覆盖率

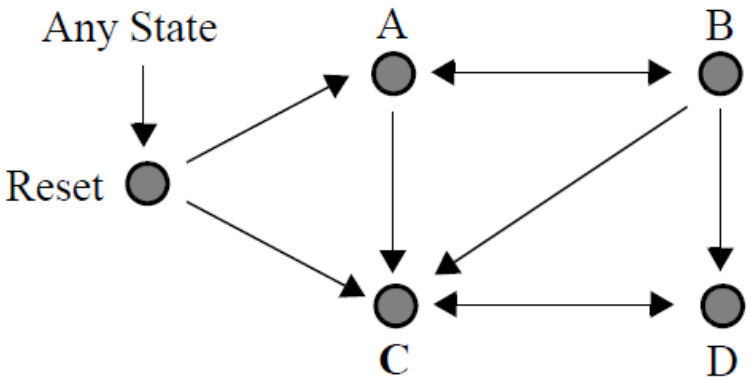
状态机覆盖率（FSM）

- 仿真工具由于可以自动识别状态机，因此在收集覆盖率时，也可以将覆盖率状态的执行情况监测到。
- 每个状态的进入次数，状态之间的跳转次数，以及多个状态的跳转顺序都可以由仿真工具记录下来。
- 因为FSM中的每个状态通常使用case语句中的选项进行编码，所以任何未访问的状态都可以通过未覆盖的语句清楚地识别。

代码覆盖率

状态机覆盖率（FSM）

- 虽然它只有五种状态，但它有更多可能的转换：相邻状态之间存在14种可能的转换。
- 通过序列Reset，A，B，D，然后C，可以很容易地达到100%的状态覆盖率。但是，这将只产生36%的过渡覆盖率。 为了完全验证此FSM的实现，有必要确保设计根据所有转换的期望执行。



代码覆盖率

状态机覆盖率（FSM）

- 覆盖率工具将自动从FSM的实现中提取状态的转换。
- 覆盖工具无法确定转换是否是意图的一部分，或者是否缺少预期的转换。
- 检查提取的状态转换以确保仅有的和所有预期的转换发生。
- FSM仅显示五种指定状态。 一旦合成到硬件中，就需要一个3位状态寄存器。 这留下了三个未指定的可能状态值，如果收到外界干扰状态值进入未指定状态，设计的正确性会受到影响吗?
- 在未知状态下，除非应用复位，否则无法让设计恢复到指定的行为。
- 设计安全性和可靠性问题因此也在验证工程师工作的范围。

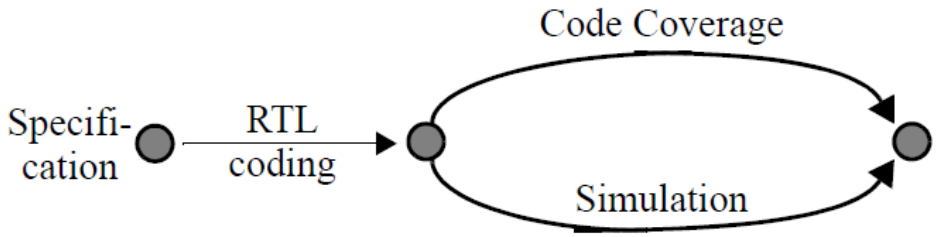
代码覆盖率

跳转覆盖率（toggle）

- 用来衡量寄存器跳转的次数（0->1，1->0）
- 一般项目会要求模块的端口实现至少1次0到1，以及1次1到0的跳转来保证模块的集成和模块之间的互动
- 也有的项目会要求所有的寄存器都应该同端口一样满足跳转的最少次数
- 端口跳转覆盖率经常用来测试IP模块之间的基本连接性，例如检查一些输入端口是否没有连接，或者已经连接的两个端口的比特位数是否不匹配，又或者一些已经连接的输入是否被给定的固定值等，都可以通过跳转覆盖率来发现问题

代码覆盖率

代码覆盖率100%?



- 简而言之，它表示整个设计已经执行。
- 代码覆盖率表示整个验证套件运行源代码的彻底程度，但它并未以任何方式提供有关验证套件正确性或完整性的指示。
- 代码覆盖率无助于验证设计意图，只是完全执行了RTL代码，无论是否正确。
- 代码覆盖率的结果应该用于帮助识别验证未执行的边界情况或依赖于设计实现的特性。
- 对于没有覆盖到的测试场景，我们还应考虑其是否相关，或者是处于覆盖率工具的盲区所导致的。

Agenda

1. 概述
2. 覆盖率的种类
3. 代码覆盖率
4. **功能覆盖率**
5. 从功能描述到覆盖率

功能覆盖率

概述

- 功能验证的目标在于确定设计有关的功能描述是否被全部实现了?
- 这一检查中可能会存在一些不期望的情况:
  - 一些功能没有被实现
  - 一些功能被错误地实现了
  - 一些没有被要求的功能也被实现了
- 我们无法通过代码覆盖率得知要求的功能是否被实现了，而需要显性地通过功能覆盖率与设计功能描述做映射，继而量化功能验证的进程。

功能覆盖率

概述

- 在约束随机测试流行于验证时，由于随机测试在仿真时会自动产生上千条测试激励，但我们却无法知道随机产生的激励究竟测试了什么功能。
- 功能覆盖率是最好的可以协助在回归测试时自动监测哪些功能被激活的方法。
- 创建功能覆盖率模型需要完成以下两个步骤：
  - 从功能描述文档提取拆分需要测试的功能点
  - 将功能点量化为与设计实现对应的测试场景和功能覆盖率代码

功能覆盖率

覆盖组（covergroup）

- 覆盖组与类相似，在一次定义以后便可以多次进行例化。
- 覆盖组含有覆盖点（coverpoint）、选项（option）、形式参数（argument）和可选触发（trigger event）。
- 一个覆盖组包含了一个或者多个数据点，全都在同一时间采集。
- 覆盖组可以定义在类里，也可以定义在模块或者程序（program）中。

功能覆盖率

覆盖组（covergroup）

- 覆盖组可以采集任何可见的变量，比如程序或模块变量、接口信号或者设计中的任何信号
- 在类中的覆盖组也可以采集类的成员变量
- 覆盖组应该定义在适当的抽象层次上
- 对任何事务的采样都必须等到数据被待测设计接收到以后
- 一个类也可以包含多个覆盖组，每个覆盖组可以根据需要将它们使能或者禁止

功能覆盖率

覆盖组（covergroup）

```
enum { red, green, blue } color;
bit [3:0] pixel_adr, pixel_offset, pixel_hue;
covergroup g2 @(posedge clk);
  Hue: coverpoint pixel_hue;
  Offset: coverpoint pixel_offset;
  AxC: cross color, pixel_adr; // cross 2 variables
  all: cross color, Hue, Offset; // cross 1 VARs and 2 CPs
endgroup
g2 cg_inst = new();
```

- covergroup...endgroup来定义覆盖组
- 内部可以定义多个coverpoint
- 如果不在covergroup声明时指定采样事件，那么默认该覆盖组只能依赖于其另外一个手动采样函数sample()

功能覆盖率

覆盖组（covergroup）

- 我们更多地将covergroup定义在类中，从而可以去覆盖类的成员变量
- 在类中声明的covergroup的方式被称为嵌入式覆盖组声明，以下的方式将会声明一个匿名的覆盖组类型和一个覆盖组变量cov1

```
class xyz;
  bit [3:0] m_x;
  int m_y;
  bit m_z;
  covergroup cov1 @m_z; // embedded covergroup
    coverpoint m_x;
    coverpoint m_y;
  endgroup
  function new();
    cov1 = new;
  endfunction
endclass
```

功能覆盖率

覆盖组（covergroup）

- 一个类中可以声明多个covergroup

```
class MC;
  logic [3:0] m_x;
  local logic m_z;
  bit m_e;
  covergroup cv1 @(posedge clk);
    coverpoint m_x;
  endgroup
  covergroup cv2 @m_e ;
    coverpoint m_z;
  endgroup
endclass
```

功能覆盖率

覆盖组（covergroup）

- covergroup可以与其它对象同步（@m_obj.m_ev）

```
class Helper;
  int m_ev;
endclass
class MyClass;
  Helper m_obj;
  int m_a;
  covergroup Cov @(m_obj.m_ev);
    coverpoint m_a;
  endgroup
  function new();
    m_obj = new;
    // Create embedded covergroup after creating m_obj
    Cov = new;
  endfunction
endclass
```

功能覆盖率

覆盖点（coverpoint）

- 一个covergroup可以包含一个或者多个coverpoint，一个coverpoint可以用来采样数据或者数据的变化
- 一个coverpoint可以对应多个bin（仓）
- 这些仓可以显性指定，也可以隐性指定
- coverpoint对数据的采样发生在covergroup采样的时候

功能覆盖率

覆盖点（coverpoint）

- 在定义coverpoint时可以不给名字或者给名字，我们建议给不同的coverpoint以不同的名字
- 这些有名字的coverpoint可以用来做更进一步的处理，例如在交叉覆盖率中使用某个coverpoint
- 可以通过iff在一些情况下禁止coverpoint的采集

```
covergroup g4;
   coverpoint s0 iff(!reset);
endgroup
```

功能覆盖率

覆盖点（coverpoint）

```
covergroup cg ( ref int x , ref int y, input int c);
  coverpoint x; // 创建CP x
  b: coverpoint y; // 创建CP b
  cx: coverpoint x; // 创建CP cx
  option.weight = c;
  bit [7:0] d: coverpoint y[31:24]; // 创建CP d
  e: coverpoint x {
    option.weight = 2;
  }
  cross x, y { // 创建crossCP xXy
    option.weight = c;
  }
endgroup
```

功能覆盖率

仓（bin）

- 关键词bins可以用来将每个感兴趣的数值均对一个独立的bin，或者将所有值对应到一个共同的bin
- iff语句也可以用在bin的定义，它表示条件为false，那么在采集该bin的时候，该bin的采样数目不会增长

```
bit [9:0] v_a;
  covergroup cg @(posedge clk);
  coverpoint v_a {
    bins a = { [0:63],65 };
    bins b[] = { [127:150],[148:191] };
    bins c[] = { 200,201,202 };
    bins d = { [1000:$] };
    bins others[] = default;
  }
endgroup
```

功能覆盖率

仓（bin）：数值覆盖

- covergroup的参数也可以被传递到bin的定义中

```
covergroup cg (ref int ra, input int low, int high )
@(posedge clk);
  coverpoint r {
    bins good = { [low : high] };
    bins bad[] = default;
  }
  endgroup
  ...
  int va, vb;
  cg c1 = new( va, 0, 50 );
  cg c2 = new( vb, 120, 600 );
```

功能覆盖率
仓（bin）

- 在定义bin时，可以使用with来进一步限定其关心的数值，with可以用表达式或者函数来衡量

```
a: coverpoint x {
  bins mod3[] = {[0:255]} with (item % 3 == 0);
}

coverpoint b {
  bins func[] = b with (myfunc(item));
}
```

功能覆盖率

仓（bin）：值变化覆盖

- 除了可以覆盖数值，还可以覆盖数值的变化

```
value1 => value2
value1 => value3 => value4 => value5

range_list1 => range_list2
1,5 => 6, 7

trans_item [* repeat_range ]

3 [* 5]
表示 3=>3=>3=>3=>3

3 [* 3:5]
表示( 3=>3=>3 ), ( 3=>3=>3=>3 )或( 3=>3=>3=>3=>3 )
```

功能覆盖率

仓（bin）：值变化覆盖

- 除了使用=>来表示相邻采样点的变化，也可以使用->来表示非相邻采样点的数值变化，在=>序列后的下一个时序**必须紧跟**=>序列的最后一个事件

```
3 [-> 3]
```
表示 `...=>3...=>3...=>3`

```
1 => 3 [ -> 3] => 5
```
表示 `1...=>3...=>3...=>3 =>5`

功能覆盖率

仓（bin）：值变化覆盖

- 与[-> repeat_range]类似的有[= repeat_range]也表示非连续的循环，只是与->有区别的在于，跟随->序列的下一次值变化可以发生在->结束后的**任何时刻。**

```
3 [= 2]
表示 ...=>3...=>3

1 => 3 [=2] => 6
表示 1...=>3...=>3...=>6
```

功能覆盖率

仓（bin）：值变化覆盖

```
bit [4:1] v_a;
covergroup cg @(posedge clk);
  coverpoint v_a {
    bins sa = (4 => 5 => 6), ([7:9],10=>11,12);
    bins sb[] = (4=> 5 => 6), ([7:9],10=>11,12);
    bins sc = (12 => 3 [-> 1]);
  }
endgroup
```

功能覆盖率

仓（bin）：自动生成

- 如果coverpoint没有指定任何bin，那么SV将会为其自动生成bin，遵循的原则是：
  - 如果变量是枚举类型，那么bin的数量是枚举类型的基数（所有枚举数值的合）
  - 如果变量是整形（M位宽），那么bin的类型将是2^M和auto_bin_max选项的较小值

功能覆盖率

仓（bin）：通配应用

- 默认情况下，数值的变化可以针对四值类型变量进行覆盖，例如bin如果包含x或者z，则表示只有该变量对应位也为x或者z的时候，bin也才可以被采样到。
- wildcard修饰符可以使得bin中包含x，z和?的数值都将用来表示0或者1，也就是通配符的意思

```
wildcard bins g12_15 = { 4'b11?? };
```

g12_15可以用来表示12到15之间的值，即
1100, 1101, 1110, 1111

功能覆盖率

仓（bin）：忽略类型和非法类型

- ignore_bins用来将其排除在有效统计的bin集合之外

```
covergroup cg23;
  coverpoint a {
    ignore_bins ignore_vals = {7,8};
    ignore_bins ignore_trans = (1=>3=>5);
  }
endgroup
```

- illegal_bins用来指出采样到的数值为非法制，如果illegal_bins被采样到，那么仿真将报错

```
covergroup cg3;
  coverpoint b {
    illegal_bins bad_vals = {1,2,3};
    illegal_bins bad_trans = (4=>5=>6);
  }
endgroup
```

功能覆盖率

交叉覆盖率（cross）

- covergroup可以在两个或者更多的coverpoint或者变量之间定义交叉覆盖率（cross coverage）
- 在对a和b产生交叉覆盖率之前，系统会先为它们隐性产生对应的coverpoint和bin，每个coverpoint都有16个自动产生的bin
- 两个coverpoint交叉以后将生成256个交叉的bin

```
bit [3:0] a, b;
covergroup cov @(posedge clk);
  aXb : cross a, b;
endgroup
```

功能覆盖率

交叉覆盖率（cross）

- 下面的CP A有10个bin，b_var对应的CP将有16个bin，因此CC将会有10x16=160个bin
- 被声明为default/ignore/illegal的bin将不会参与cross coverage的运算
- 交叉覆盖率只允许在同一个covergroup中定义的CP参与运算

```
bit [31:0] a_var;
bit [3:0] b_var;
covergroup cov3 @(posedge clk);
  A: coverpoint a_var { bins yy[] = { [0:9] }; }
  CC: cross b_var, A;
endgroup
```

功能覆盖率

交叉覆盖率：定义

- 除了系统会自动为交叉覆盖率生成bin以外，用户还可以自己定义交叉覆盖率的bin
- binsof()的参数可以是coverpoint或者变量，表示对应的bin总和，可以利用binsof()对其结果做进一步的过滤

```
int i,j;
covergroup ct;
  coverpoint i { bins i[] = { [0:1] }; }
  coverpoint j { bins j[] = { [0:1] }; }
  x1: cross i,j;
  x2: cross i,j {
    bins i_zero = binsof(i) intersect { 0 };
  }
endgroup
```

功能覆盖率

交叉覆盖率：定义

- 交叉X1有以下自动分配的bin

```
<i[0],j[0]>
<i[1],j[0]>
<i[0],j[1]>
<i[1],j[1]>
```

- 交叉X2有以下自定义的bin

```
i_zero // 自定义bin for <i[0],j[0]> and <i[0],j[1]>
<i[1],j[0]> // 保留该自动生成bin
<i[1],j[1]> // 保留该自动生成bin
```

- 需要初学者注意的是，**即便自己为cross定义了bin，但是系统仍然会自动生成bin**，自动生成bin的原则是，只要该bin不在自定义bin的范围，那么就仍然会保留

功能覆盖率

交叉覆盖率（cross）

```
bit [7:0] v_a, v_b;
covergroup cg @(posedge clk);
  a: coverpoint v_a {
    bins a1 = { [0:63] };
    bins a2 = { [64:127] };
    bins a3 = { [128:191] };
    bins a4 = { [192:255] };
  }
  b: coverpoint v_b {
    bins b1 = {0};
    bins b2 = { [1:84] };
    bins b3 = { [85:169] };
    bins b4 = { [170:255] };
  }
  c : cross a, b {
    bins c1 = ! binsof(a) intersect {[100:200]};// 4 products
    bins c2 = binsof(a.a2) || binsof(b.b2);// 7 products
    bins c3 = binsof(a.a1) && binsof(b.b4);// 1 product
  }
endgroup
```

功能覆盖率

交叉覆盖率（cross）

- c1中，需要对a排除a2/a3/a4，保留a1，因此与b的交叉结果是<a1,b1>, <a1,b2>, <a1,b3>, 和 <a1,b4>
- c2中，包括a2与{b1,b2,b3,b4}的交叉以及b2与{a1,a2,a3,a4}的交叉，最终交叉结果是<a2,b1>, <a2,b2>,<a2,b3>, <a2,b4>, <a1,b2>, <a3,b2>和<a4,b2>
- c3中只包含<a1,b4>
- 由于上述的3个cross中有重叠的数值组合，最终共有10个有效组合
- a交叉b的理论结果是16个数值组合，排除用户指定的10个，另外剩余的6个将做为保留的自动数值组合结果<a3,b1>, <a4,b1>, <a3,b3>, <a4,b3>,<a3,b4>和<a4,b4>

功能覆盖率

指定覆盖率选项

- 覆盖率选项可以在covergroup、coverpoint或者cross中分别设置
- 常见的参数，譬如weight可以在covergroup设置来表示它实例的比重，也可以在coverpoint/cross中设置来表示它们在covergroup中的比重，默认值是1
- per_instance如果设置为1，将会记录每个covergroup实例的覆盖率数据，默认值是0
- comment则是用来对covergroup/coverpoint/cross做进一步的说明，方便在覆盖率数据分析时阅读理解

功能覆盖率

指定覆盖率选项

- 覆盖率选项按照在covergroup、coverpoint和cross不同的域中是否具备，下表做了归纳：

Option name	Allowed in syntactic level		
	covergroup	coverpoint	cross
name	Yes	No	No
weight	Yes	Yes	Yes
goal	Yes	Yes	Yes
comment	Yes	Yes	Yes
at_least	Yes (default for coverpoints & crosses)	Yes	Yes
detect_overlap	Yes (default for coverpoints)	Yes	No
auto_bin_max	Yes (default for coverpoints)	Yes	No
cross_num_print_missing	Yes (default for crosses)	No	Yes
per_instance	Yes	No	No

功能覆盖率

预定义覆盖率方法

Method (function)	Can be called on			Description
	covergroup	coverpoint	cross	
void sample()	Yes	No	No	Triggers sampling of the covergroup
real get_coverage() **real** get_coverage(**ref int**, **ref int**)	Yes	Yes	Yes	Calculates type coverage number (0...100)
real get_inst_coverage() **real** get_inst_coverage(**ref int**, **ref int**)	Yes	Yes	Yes	Calculates the coverage number (0...100)
void set_inst_name(**string**)	Yes	No	No	Sets the instance name to the given string
void start()	Yes	Yes	Yes	Starts collecting coverage information
void stop()	Yes	Yes	Yes	Stops collecting coverage information

Agenda

1. 概述
2. 覆盖率的种类
3. 代码覆盖率
4. 功能覆盖率
5. **从功能描述到覆盖率**

从功能描述到覆盖率

概述

- 要实现功能覆盖率的收敛，就需要按照以下步骤考虑：
  - 哪些功能需要测试
  - 明白在什么条件下需要测试对应的功能
  - 为了测试这些功能，需要提供什么样的测试平台组件以便提供激励和监测
  - 测试平台如何检查这些功能正常工作
- 由于功能覆盖率不是自动的过程，因此它需要将功能描述同设计实现对应起来

从功能描述到覆盖率

从功能描述到待测功能点

- 从功能描述文档中，可以罗列出所有要验证的功能点。
- 系统工程师和设计工程师也可以帮助添加要验证的其它功能点。这些附加的功能点可能是由于一些验证工程师对于设计本身不够熟悉的情况下，或者不能直接从功能描述中提出出来。
- 提取功能点一般遵循从外部接口到内部功能再到边界情况的方法。

从功能描述到覆盖率

提取接口功能点

- 对于要验证的设计的各个接口，可通过以下问题来获得接口功能点：
  - 必须应用哪些传输?
  - 什么样的取值范围?
  - 什么样的传输顺序?
  - 相关的传输带宽是多少?
  - 设计能够承受哪些协议违规?
  - 此接口与其他接口或内部设计结构之间的相关交互是什么?
  - 接口上的事务是否需要与另一个接口的事务同步?

从功能描述到覆盖率

提取内部功能点

遵循设计的主要数据路径，列举必须验证的转换和决策，可通过以下问题来获得内部功能点：

- 所有相关配置是什么？
- 可以对数据执行哪些可能的转换？
- 可能的转换顺序是什么？
- 触发转换的敏感数据值是什么？
- 影响每次转换的敏感值是什么？
- 转换后的数据应该在哪里结束？
- 数据序列如何受到影响？
- 存在哪些错误检测机制以及它们是如何触发的？
- 错误机制如何报告错误？
- 错误数据会导致什么？

从功能描述到覆盖率

提取结构功能点

最后，基于对设计架构的详细了解，确定将设计推向极限的条件， 可询问以下问题来获得基于体系结构的功能点：

- 我可以溢出或下溢缓冲区吗？ 如果是这样，会发生什么？
- 资源瓶颈在哪里？
- 可以同时发生对这些资源的多个请求吗？
- 数据转换路径是否可以影响、阻止或阻止另一个？

从功能描述到覆盖率

标记功能点

- 功能点应被标记并有简短说明， 根据需要验证的条件和预期结果来描述该功能，而不是如何实施。
- 每个功能点都应该与功能描述文档中描述它的部分交叉引用。
- 指定验证该功能的验证层次。
- 发现违反时，应在错误消息中使用功能标签。
- 在错误消息中包含功能标签将有助于识别假设出错的内容以及评估行为是否确实不正确。

从功能描述到覆盖率

验证分层

- 在例举功能时，请注意为它们指定适当的验证级别。
- 某些功能在块级别可以得到更好的验证，而其他功能必须在（子）系统级别进行验证。
- 通常会有大量功能点与设计中的某些关键功能或单元有关。
- 如果实现该功能、或者包含该单元的设计没有被单独验证，那么，可能是时候重新考虑你的验证计划了，因为这可能表明该单元需要独立验证以达到必要的验证完备性水平。

从功能描述到覆盖率

优先级划分

- 并非所有功能都是同等优先级的。
- 对于一些设计必须具备（must-have）的功能，才能正常运行来满足市场需求，完成对这些功能的验证可以成功完成项目，而验证这些功能的测试平台通常也位于关键路径上。
- 对于这些“必备”功能，我们需要对所有可能的配置和使用选项进行全面验证。
- 对于设计的商业成功而言，应有（should-have）的功能是次要的。他们可能只是提供扩展能力帮助在竞争中脱颖而出。 对于这些应有功能，主要验证其操作的基本功能。 如果时间和资源允许，可以完成更详细的验证。 如果有更高优先级的功能，则需重新分配资源，减少或者取消对这些功能的验证。

从功能描述到覆盖率

优先级划分（续）

- 一些功能仅仅是建议的（nice-to-have）。
- 它们仅在时间允许的情况下被验证，不过从目前的项目压力来看，这些功能很少在第一次流片前被"照顾"到。
- 在项目进度很紧张的情况下，可以需要在验证过程中对验证计划和功能点做出调整。
- 有的时候，需要删除一些必备功能的验证来追赶项目节点，那么这就需要重新评估项目产品的目标，而使得对项目计划的修改是合理的。

从功能描述到覆盖率

从随机测试到功能覆盖率

- 随机测试所产生的激励数量和功能组合场景要远远多于定向测试。
- 定向测试与待测试功能点联系紧密，而随机测试则不仅与待测功能点，还可能同时测试其它功能。
- 因此衡量随机测试的进程不应该以测试是否通过为准，而是应该通过功能覆盖率模型，在项目初期到结束一直收集随机激励的类型。
- 也只有通过覆盖率模型，才能让随机约束的历史产生轨迹变得有迹可循。

从功能描述到覆盖率
过程指南

- 功能覆盖率应该基于对DUT端口或者内部信号的监测
- 功能覆盖率的有效性应该首先建立在测试通过的情况下，这还需要测试平台有自动检查（scoreboard/reference model）来保证对应的功能得到检验
- 在创建覆盖率模型时，同创建property（assertion）一样，不要贪大求全，而应该对每一个covergroup、coverpoint、bin和cross都做到简明扼要，尽量将逻辑和时序都拆分开，便于后期的覆盖率分析
- 选取一个适当的范围进行覆盖率收集，例如一个32位的整形变量，在收集数值时可以结合其与设计功能的特点，找到边界值、分段范围，再在合适的抽象层次上进行收集
- 采用统一的覆盖率编码风格，例如covergroup的名字，采样的方式，是否填写comment，覆盖率模型置于何处等。

从功能描述到覆盖率

覆盖率实现之考量

- 考虑哪些数值是重要的?
- 哪些变量之间是有联系依赖的?
- 不同的功能有哪些边界情况需要检查?
- 有哪些非法的情况需要考虑?
- 是否可以区别哪些是重要的功能，哪些是次要的功能?
- 在什么条件时适合去采样?
  - 只有对某些功能检查时，才可以使能对应的覆盖率收集
  - 当变量有效时或者稳定时

从功能描述到覆盖率

MCDT功能测试清单

功能测试点	覆盖率形式	说明	测试用例
FIFO状态测试	Covergroup	1.每个channel FIFO空、满、非空非满标志 2.至少两个channel FIFO同时为满，满足各个组合情况	
FIFO满时数据写入ready应拉低	Property assertion	1.既满足FIFO写满，同时有效数据写入时，ready应该拉低	
Channel激励组合	Covergroup	1.每个channel有burst写入和非burst写入情况 2.在同一个周期，2个channel同时写入，满足各种组合 3.在同一个周期，3个channel同时写入	

从功能描述到覆盖率

MCDT功能测试清单（续）

功能测试点	覆盖率形式	说明	测试用例
FIFO读写端组合	Property coverage	1.在同一周期，只写入不读出 2.只读出不写入 3.既写入又读出	
Aribter仲裁	Covergroup	1.在同一个周期，只有一个channel请求 2.两个channel请求 3.三个channel请求	
Aribiter每次授权最多只有一个channel	Property assertion	1.在只有channel请求的情况下，每次授权只应有最多一个授权对象	
Ariber数据发送	Covergroup	1.同一个channel数据连续或者非连续发送 2.不同channel数据连续或者非连续发送	

Agenda

1. **UVM总览**
2. 基础结构
3. 工厂机制
4. 消息机制
5. TLM通信

UVM总览

从SV到UVM的测试平台升级

- UVM是开源库，包含了许多预定义好的验证环境组件
- 工厂机制的引入方便了组件创建、环境配置、类型覆盖
- UVM从开始到结束的整个过程引入了“阶段”（phase）概念
- 消息管理机制提高了信息打印的可控性
- 将验证环境和测试用例隔离，提高了环境的复用性
- …… ……

UVM总览

预定义好的验证环境组件(uvm_component)

- uvm_driver
- uvm_sequencer
- uvm_monitor
- uvm_agent
- uvm_scoreboard
- uvm_env
- uvm_test

UVM总览

工厂机制的引入

- uvm_factory使得任何UVM类型在定义时均需要注册
- 在例化UVM类型时必须使用工厂来创建对象
- 利用工厂机制创建的过程中，不但自然建立了软件对象之间的层次，也建立了通过"字符串"实现的层次，这为稍后通过字符串进行层次化的配置提供了便利
- 由于创建对象必须通过工厂，这也使得在创建对象之前还有机会可以通过工厂来替换创建类型，这使得类型或者对象覆盖成为了可能

UVM总览

阶段（phase）的引入

- 在学习SV和完成实验的过程中，我们经历了验证组件从创建到连接，最后再到运行和报告的阶段。
- 在复杂层次化的验证环境建立和运行过程中，有必要让所有的组件在各个阶段中按照层次逐个完成，UVM引入了“阶段”（phase），这使得verifier不需要考虑各个阶段的执行，而由UVM帮助完成。

UVM总览

消息管理机制

- 在没有引入消息管理机制之前，各个组件打印的消息格式、出处、等级、冗余度都无法得到很好的管理。
- UVM消息管理不断对消息验证严重等级、冗余度、格式等都做好了规范，而且还可以结合层次化配置，在不同的测试阶段或者测试用例中做出个性化的配置。

UVM总览

固定资产和动态资源的划分

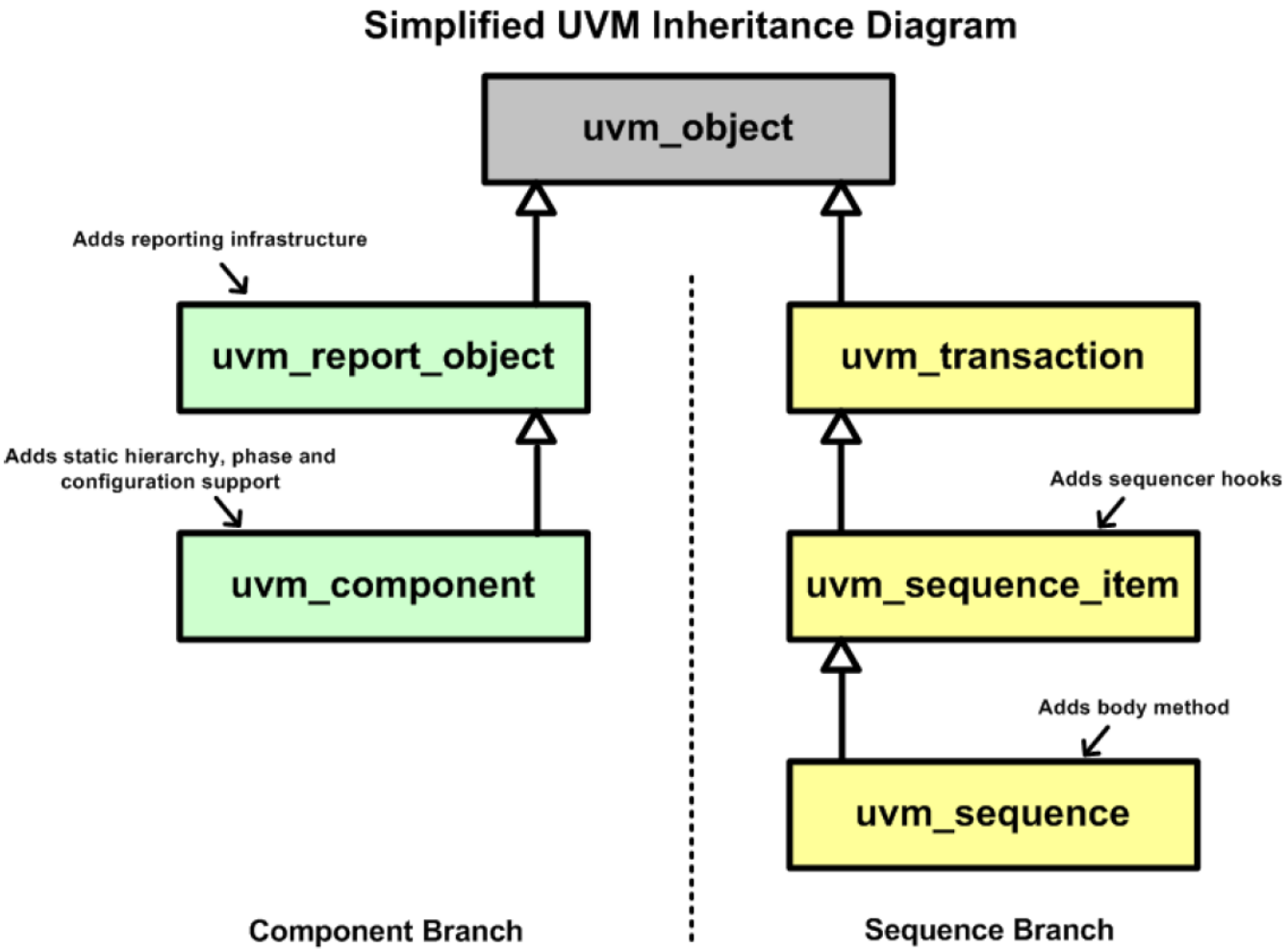
- 类似于静态的硬件结构，在UVM环境创建时，所有的组件会伴随着UVM顶层结构一同创建，这属于UVM测试平台的固定资产。
- 在接下来的测试过程中，对于发送的激励、监测到的总线事务等都属于动态资源，即它们会在一定的时刻（不一定是0时刻）创建，而又在一定的时刻被回收。
- 固定资产一般是uvm_component类型，动态资源一般是uvm_object类型。

Agenda

1. UVM总览
2. **基础结构**
3. 工厂机制
4. 消息机制
5. TLM通信

基础结构

核心基类uvm_object(红宝书10.4节)



基础结构

核心基类uvm_object

- uvm_object是UVM环境结构（静态资产+动态资源）的根类，或者也可称为核心基类。
- uvm_object内置了一些常用方法，包括复制、比较、打印等。
- 由uvm_object的继承，定义了两个子类，uvm_component为所有环境组件（静态资产）的父类，uvm_transaction为所有测试用例（动态资源）的父类。
- 在接下来学习uvm_component或者uvm_transaction之前，我们先掌握定义uvm_object的有关要素。

基础结构

定义uvm_object类

```
class box extends uvm_object;
   int volume = 120;
   color_t color = WHITE;
   string name = "box";
   `uvm_object_utils_begin(box)
     `uvm_field_int(volume, UVM_ALL_ON)
     `uvm_field_enum(color_t, color, UVM_ALL_ON)
     `uvm_field_string(name, UVM_ALL_ON)
   `uvm_object_utils_end
   function new(string name="box");
     super.new(name);
     this.name = name;
   endfunction
 endclass
```

基础结构

定义uvm_object类

- 声明类时需要继承于uvm_object
- 必须使用`uvm_object_utils来"注册"该类，即将该类的定义注册到稍后学习到的工厂之中
- 如果需要想使用object类（uvm_object类简称）的copy()、compare()、print()等方法，还需要通过`uvm_field_xxx等宏来对成员变量做"域的自动化"声明
- 需要明确定义new函数，并且new函数的形式参数格式无法变化，即只能带有一个参数string name

基础结构

域的自动化（红宝书10.4.1节）

- 参考红宝书10.4.1域的自动化和表10.2“域的自动化宏列表”
- 对于常见的SV标准声明，从表可以看到，需要注意几个参数的传递，ARG，T和FLAG。
- ARG表示成员变量，T表示成员变量的类型，而FLAG可以参照表10.3，或者简单起见，使用UVM_DEFAULT或者UVM_ALL_ON。

域 类 型	域自动化的宏声明	成 员 类 型
标量	`uvm_field_int(ARG, FLAG)	整形
	`uvm_field_object(ARG, FLAG)	继承与 uvm_object 的类型句柄
	`uvm_field_string(ARG, FLAG)	字符串类型
	`uvm_field_enum(T, ARG, FLAG)	枚举类型
	`uvm_field_event(ARG, FLAG)	事件类型
	`uvm_field_real(ARG, FLAG)	实数类型

基础结构

new函数

- 无论是object类，还是component类（uvm_component），都必须定义new函数，同时它们定义的new函数参数必须按照固定的范式要求。
- 之所以有这样的要求，是因为在稍后通过工厂创建对象（object或者component）时，工厂都会按照默认的参数格式来调用new函数。
- new函数也自然会参与到稍后工厂创建UVM组件的过程当中，只不过这仅仅还是一部分。

基础结构

component组件类（红宝书11.1节）

- 从uvm_component到uvm_object之间的继承关系来看，它们还有中间类uvm_report_object，该类即添加了有关消息报告管理的方法。
- uvm_component在uvm_report_object基础上添加了与层次构建有关的方法和phase机制。
- 与ovm_object::new()函数近似的是，uvm_component::new()函数也有固定的参数：
  ```
  function new(string name, uvm_component parent);
  ```
- 定义component类时，也需要同object类一样，注意类型的注册和new函数的定义。
- 具体可以参考红宝书11.1节"组件家族"。

基础结构

component组件类

- uvm_driver：从uvm_sequencer接收激励，并且将其发送至interface。
- uvm_sequencer：从uvm_sequence获取激励，并将其递交给uvm_driver。
- uvm_monitor：从接口监测到事务，做协议检查和覆盖率收集，并且将事务送至scoreboard。
- uvm_agent：环境中的标准单元，一般包括driver，sequencer和monitor。

基础结构

component组件类

- uvm_scoreboard：类似于SV中的checker，也可以内置reference model和存储monitor传输的缓存。
- uvm_env：比agent更高一级的环境组件，一个env可以容纳子的env组件、agent组件和其它组件，用来形成一个封闭的环境。
- uvm_test：一般只应该包括一个顶层的env环境组件和其它用来配置环境结构或者修改激励的配置对象或者覆盖方法等。

基础结构

component组件类

- 在学习uvm_driver与uvm_sequencer之前，可以将其它的组件类都视为固定的容器，即用来实现固定的功能，但组件之间不具备明显的差别。
- 即便这样，也应该注意test类必须是UVM环境的顶层类。

```
class comp1 extends uvm_component;
  `uvm_component_utils(comp1)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  ...
endclass
```

基础结构

phase机制（红宝书10.5节）

- component类之所以作为验证环境的固定资产，其与object类的一个重要差别就是引入了phase机制。
- phase机制伴随着UVM环境从开始建立到运行再到最后的结束，贯穿着UVM验证环境完整的生命周期。
- 常用的phase包括build、connect、run和report等。
- 所有的phase中尤其只有run phase是task（耗时的），其它phase均为function（不耗时，立即返回）。

基础结构

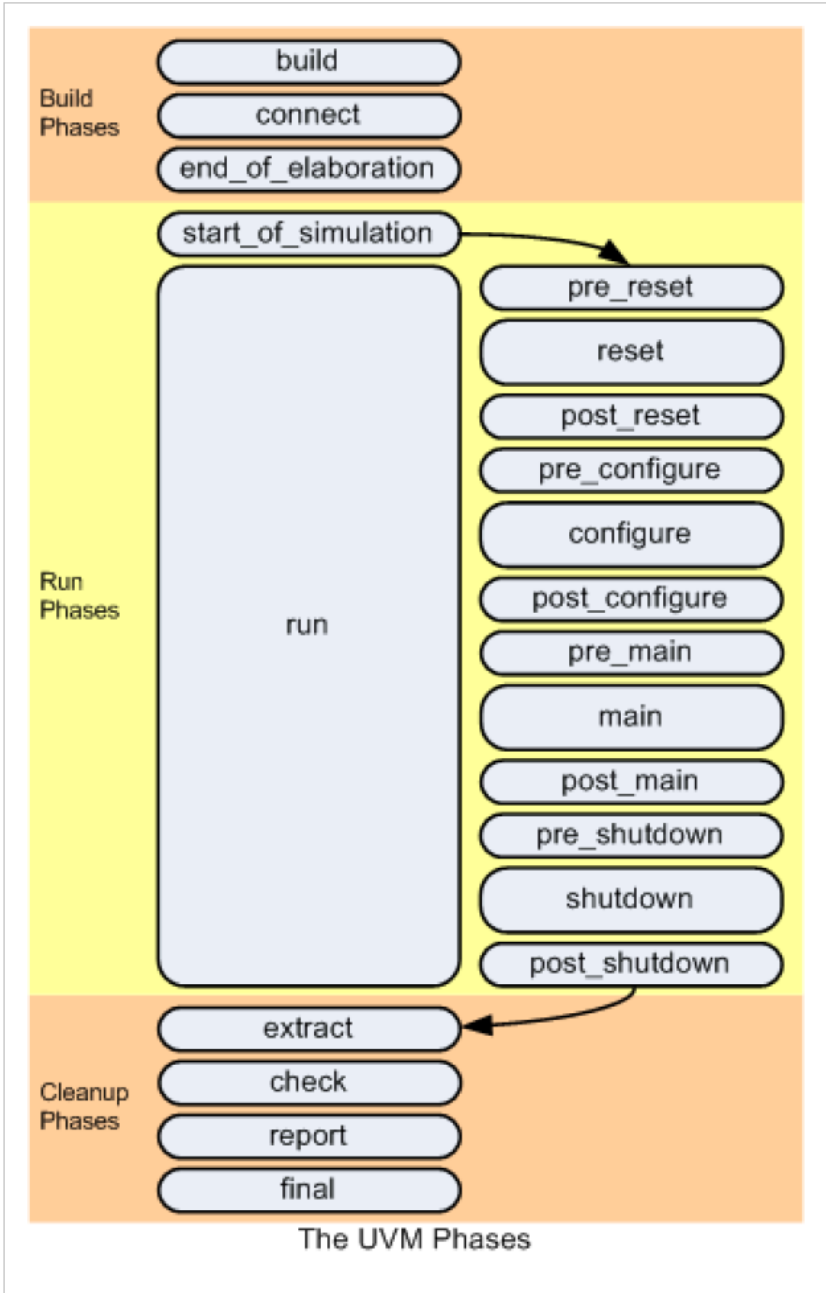
phase机制

表 10.4　UVM 的 phase 列表

phase	函数/任务	执 行 顺 序	功　　能	典型应用
build	函数	自顶向下	创建和配置测试平台的结构	创建组件和寄存器模型，设置或获取配置
connect	函数	自底向上	建立组件之间的连接	连接 TLM/TLM2 的端口，连接寄存器模型和 adapter
end_of_elaboration	函数	自底向上	测试环境的微调	显示环境结构，打开文件，为组件添加额外配置
start_of_simulation	函数	自底向上	准备测试环境的仿真	显示环境结构，设置断点，设置初始运行的配置值
run	任务	自底向上	激励设计	提供激励、采集数据和数据比较，与 OVM 兼容
extract	函数	自底向上	从测试环境中收集数据	从测试平台提取剩余数据，从设计观察最终状态
check	函数	自底向上	检查任何不期望的行为	检查不期望的数据
report	函数	自底向上	报告测试结果	报告测试结果，将结果写入到文件中
final	函数	自顶向下	完成测试活动结束仿真	关闭文件，结束联合仿真引擎

基础结构

phase机制



The UVM Phases

基础结构

phase机制

```
class subcomp extends uvm_component;
  `uvm_component_utils(subcomp)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    `uvm_info("build_phase", "", UVM_LOW)
  endfunction
  function void connect_phase(uvm_phase phase);
    `uvm_info("connect_phase", "", UVM_LOW)
  endfunction
  function void end_of_elaboration_phase(uvm_phase phase);
    `uvm_info("end_of_elaboration_phase", "", UVM_LOW)
  endfunction
  function void start_of_simulation_phase(uvm_phase phase);
    `uvm_info("start_of_simulation_phase", "", UVM_LOW)
  endfunction
  task run_phase(uvm_phase phase);
    `uvm_info("run_phase", "", UVM_LOW)
  endtask
  function void extract_phase(uvm_phase phase);
    `uvm_info("extract_phase", "", UVM_LOW)
  endfunction
  function void check_phase(uvm_phase phase);
    `uvm_info("check_phase", "", UVM_LOW)
  endfunction
  function void report_phase(uvm_phase phase);
    `uvm_info("report_phase", "", UVM_LOW)
  endfunction
  function void final_phase(uvm_phase phase);
    `uvm_info("final_phase", "", UVM_LOW)
  endfunction
endclass
```

基础结构

phase机制（红宝书）

```
class topcomp extends subcomp;
  subcomp c1, c2;
  `uvm_component_utils(topcomp)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    `uvm_info("build_phase", "", UVM_LOW)
    c1 = subcomp::type_id::create("c1", this);
    c2 = subcomp::type_id::create("c2", this);
  endfunction
endclass

class test1 extends uvm_test;
  topcomp t1;
  `uvm_component_utils(test1)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    t1 = topcomp::type_id::create("t1", this);
  endfunction
endclass

initial begin
  run_test("test1");
end
```

基础结构

phase机制

```
UVM_INFO @ 0: reporter [RNTST] Running test test1...
UVM_INFO common_phase_order.sv(48) @ 0: uvm_test_top.t1 [build_phase]
UVM_INFO common_phase_order.sv(13) @ 0: uvm_test_top.t1.c1 [build_phase]
UVM_INFO common_phase_order.sv(13) @ 0: uvm_test_top.t1.c2 [build_phase]
UVM_INFO common_phase_order.sv(16) @ 0: uvm_test_top.t1.c1 [connect_phase]
UVM_INFO common_phase_order.sv(16) @ 0: uvm_test_top.t1.c2 [connect_phase]
UVM_INFO common_phase_order.sv(16) @ 0: uvm_test_top.t1 [connect_phase]
UVM_INFO common_phase_order.sv(19) @ 0: uvm_test_top.t1.c1 [end_of_elaboration_phase]
UVM_INFO common_phase_order.sv(19) @ 0: uvm_test_top.t1.c2 [end_of_elaboration_phase]
UVM_INFO common_phase_order.sv(19) @ 0: uvm_test_top.t1 [end_of_elaboration_phase]
UVM_INFO common_phase_order.sv(22) @ 0: uvm_test_top.t1.c1 [start_of_simulation_phase]
UVM_INFO common_phase_order.sv(22) @ 0: uvm_test_top.t1.c2 [start_of_simulation_phase]
UVM_INFO common_phase_order.sv(22) @ 0: uvm_test_top.t1 [start_of_simulation_phase]
UVM_INFO common_phase_order.sv(25) @ 0: uvm_test_top.t1.c1 [run_phase]
UVM_INFO common_phase_order.sv(25) @ 0: uvm_test_top.t1.c2 [run_phase]
UVM_INFO common_phase_order.sv(25) @ 0: uvm_test_top.t1 [run_phase]
UVM_INFO common_phase_order.sv(28) @ 0: uvm_test_top.t1.c1 [extract_phase]
UVM_INFO common_phase_order.sv(28) @ 0: uvm_test_top.t1.c2 [extract_phase]
UVM_INFO common_phase_order.sv(28) @ 0: uvm_test_top.t1 [extract_phase]
UVM_INFO common_phase_order.sv(31) @ 0: uvm_test_top.t1.c1 [check_phase]
UVM_INFO common_phase_order.sv(31) @ 0: uvm_test_top.t1.c2 [check_phase]
UVM_INFO common_phase_order.sv(31) @ 0: uvm_test_top.t1 [check_phase]
UVM_INFO common_phase_order.sv(34) @ 0: uvm_test_top.t1.c1 [report_phase]
UVM_INFO common_phase_order.sv(34) @ 0: uvm_test_top.t1.c2 [report_phase]
UVM_INFO common_phase_order.sv(34) @ 0: uvm_test_top.t1 [report_phase]
UVM_INFO common_phase_order.sv(37) @ 0: uvm_test_top.t1 [final_phase]
UVM_INFO common_phase_order.sv(37) @ 0: uvm_test_top.t1.c1 [final_phase]
UVM_INFO common_phase_order.sv(37) @ 0: uvm_test_top.t1.c2 [final_phase]
```

Agenda

1. UVM总览
2. 基础结构
3. **工厂机制**
4. 消息机制
5. TLM通信

工厂机制（红宝书10.3节）

概述

- UVM工厂的存在就是为了更方便地替换验证环境中的实例或者注册了的类型，同时工厂的注册机制也带来了配置的灵活性。
- uvm_factory类尽管不会显式出现在验证环境中，但是它作为UVM环境建立的支持部件，起到了非常重要的作用。
- 从UVM基础角度出发，我们将学习利用uvm_factory做的对象创建和变量配置过程，类型覆盖则作为课后学习部分，请参考红宝书10.3.3节“覆盖方法”。

工厂机制

对象创建

```
class comp1 extends uvm_component;
  `uvm_component_utils(comp1)
  function new(string name="comp1", uvm_component parent=null);
    super.new(name, parent);
    $display($sformatf("%s is created", name));
  endfunction: new
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
  endfunction: build_phase
endclass
class obj1 extends uvm_object;
  `uvm_object_utils(obj1)
  function new(string name="obj1");
    super.new(name);
    $display($sformatf("%s is created", name));
  endfunction: new
endclass
```

工厂机制
对象创建

```
comp1 c1, c2;
obj1 o1, o2;
initial begin
  c1 = new("c1");
  o1 = new("o1");
  c2 = comp1::type_id::create("c2", null);
  o2 = obj1::type_id::create("o2");
end
```

输出结果:

```
c1 is created
o1 is created
c2 is created
obj1 is created
```

工厂机制
对象创建

- 在UVM代码中，我们需要遵循简单的原则即，所有的UVM对象创建都应该通过factory来创建。
- 有多种通过factory来创建对象的方法，我们可以先学习其中最常用的一种方法：

```
component_type::type_id::create(name, parent)
object_type::type_id::create(name);
```

- 在不考虑利用factory来做类型覆盖的高级用法时，通过factory创建对象也可以帮助建立验证环境的字符串结构，方便后期配置。
- 运用factory的步骤：
  - 将类注册到工厂
  - 利用工厂创建对象

工厂机制

变量配置

- 在SV环境中，如果要对某一个验证组件（软件对象）做变量配置，那么首先需要等待其组件被创建后才可进行。
- 这一要求使得一些可能会影响验证组件环境结构的配置需求无法得到满足，因此验证组件一旦被创建，那么结构变量即便完成修改，也无法影响验证组件的结构。
- 这就需要找到一个可以在验证组件被创建之前就能够存储这些配置变量，而在组件创建时可以从中获取配置信息的公共位置。
- SV的解决方案可能是package中的静态变量，但是这种方法由于不具备结构层次，不能很好地满足UVM结构中组件变量配置。
- 由于工厂的引入，使得UVM组件在建立时，不但建立了句柄之间的层次索引结构，也同时引入了字符串形式的层次结构。

工厂机制

变量配置

- 因此要通过配置机制完成变量配置，首先应该通过工厂来创建对象，才能够建立验证环境的层次（字符串路径）。
- 依靠UVM独立的层次存储、搜索和配置uvm_config_db类，可以完成变量的配置。
- 变量类型可以是标量、句柄或者是虚接口（接口指针，virtual interface）。

```
uvm_config_db#(T)::set(uvm_uvm_component cntxt, string
inst_name, string field_name, T value);
uvm_config_db#(T)::get(uvm_component cntxt, string
inst_name, string field_name, inout T value);
```

工厂机制

变量配置（标量）

```
class comp1 extends uvm_component;
  `uvm_component_utils(comp1)
  int val1 = 1;
  string str1 = "null";
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    `uvm_info("SETVAL", $sformatf("val1 is %d before get", val1), UVM_LOW)
    `uvm_info("SETVAL", $sformatf("str1 is %s before get", str1), UVM_LOW)
    uvm_config_db#(int)::get(this, "", "val1", val1);
    uvm_config_db#(string)::get(this, "", "str1", str1);
    `uvm_info("SETVAL", $sformatf("val1 is %d after get", val1), UVM_LOW)
    `uvm_info("SETVAL", $sformatf("str1 is %s after get", str1), UVM_LOW)
  endfunction
endclass
```

工厂机制

变量配置（标量）

```
class test1 extends uvm_test;
  `uvm_component_utils(test1)
  comp1 c1;
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    uvm_config_db#(int)::set(this, "c1", "val1", 100);
    uvm_config_db#(string)::set(this, "c1", "str1", "comp1");
    c1 = comp1::type_id::create("c1", this);
  endfunction
endclass
initial begin
  run_test("test1");
end
```

输出结果:

```
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] val1 is 1 before get
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] str1 is null before get
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] val1 is 100 after get
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] str1 is comp1 after get
```

工厂机制

变量配置（标量）

```
interface intf1;
  logic enable = 0;
endinterface
class comp1 extends uvm_component;
  `uvm_component_utils(comp1)
  virtual intf1 vif;
  ...
  function void build_phase(uvm_phase phase);
    if(!uvm_config_db#(virtual intf1)::get(this, "", "vif", vif)) begin
      `uvm_error("GETVIF", "no virtual interface is assigned")
    end
    `uvm_info("SETVAL", $sformatf("vif.enable is %b before set", vif.enable), UVM_LOW)
    vif.enable = 1;
    `uvm_info("SETVAL", $sformatf("vif.enable is %b after set", vif.enable), UVM_LOW)
  endfunction
endclass
```

工厂机制

变量配置（标量）

```
class test1 extends uvm_test;
  `uvm_component_utils(test1)
  comp1 c1;
  ...
endclass
intf1 intf();
initial begin
  uvm_config_db#(virtual intf1)::set(uvm_root::get(), "uvm_test_top.c1", "vif", intf);
  run_test("test1");
end
```

输出结果:

```
UVM_INFO @ 0: reporter [RNTST] Running test test1...
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] vif.enable is 0 before set
UVM_INFO @ 0: uvm_test_top.c1 [SETVAL] vif.enable is 1 after set
```

Agenda

1. UVM总览
2. 基础结构
3. 工厂机制
4. **消息机制**
5. TLM通信

消息机制（红宝书10.7节）

消息方法

- 在UVM环境中或者环境外，只要有引入uvm_pkg，均可以通过下面的方法来按照消息的严重级别和冗余度来打印消息。
  - `function void uvm_report_info(string id, string message, int verbosity = UVM_MEDIUM, string filename = "", int line = 0);`
  - `function void uvm_report_warning(string id, string message, int verbosity = UVM_MEDIUM, string filename = "", int line = 0);`
  - `function void uvm_report_error(string id, string message, int verbosity = UVM_LOW, string filename = "", int line = 0);`
  - `function void uvm_report_fatal(string id, string message, int verbosity = UVM_NONE, string filename = "", int line = 0);`

消息机制

消息方法

- 四个消息函数有若干共同的信息，它们是严重级别（severity）、冗余度（verbosity）、消息ID、消息、文件名和行号：
  - 严重级别：从函数名本身也可以得出，这四个严重级别分别是UVM_INFO、UVM_WARNING、UVM_ERROR、UVM_FATAL。不同的严重级别在打印的消息中也会有不同的指示来区别，同时仿真器对不同严重级别消息的处理方式也不一样。例如对于UVM_FATAL的消息，默认情况下仿真会停止。
  - 消息ID：该ID可以是任意的字符串，用来标记该消息。这个标记会同消息本身打印出来，同时不同的标记也可以用来进行消息处理。
  - 消息：即消息文本的主体。
  - 冗余度：冗余度与消息处理中的过滤直接相关。冗余度的设置如果低于过滤的开关，那么该消息会打印出来，否则不会被打印出来。但是无论信息是否会被打印出来，这都与对消息采取的其它措施没有关系，例如仿真停止。
  - 文件名和行号：这些信息用来提供消息发生时所在的文件和行号。用户可以使用默认值，而UVM后台会自动填补它们原本的文件名和行号，同时也在打印时将文件名和行号输出。

消息机制

消息处理

- 与每一条消息对应的是如何处理这些消息。通常情况下，消息处理的方式是同消息的严重级别对应的。如果用户有额外的需求，也可以修改对各个严重级别的消息处理方式。

处 理 方 式	说　　明
NO_ACTION	不做任何处理
UVM_DISPLAY	将消息输出到标准输出端口
UVM_LOG	将消息写入到文件中
UVM_COUNT	增加退出计数变量 quit_count。当 quit_count 达到一定数值时，停止仿真
UVM_EXIT	立刻退出仿真
UVM_CALL_HOOK	调用对应的回调函数
UVM_STOP	停止仿真

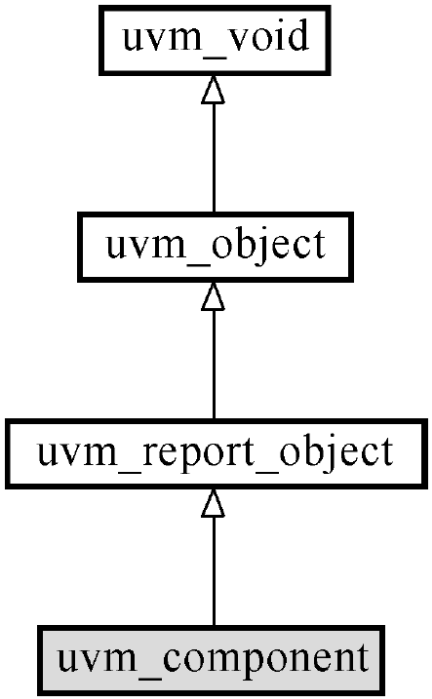
- 而不同的严重级别消息，用户可以使用默认的消息处理方式

严　重 级 别	默认处理方式
UVM_INFO	UVM_DISPLAY
UVM_WARNING	UVM_DISPLAY
UVM_ERROR	UVM_DISPLAY | UVM_COUNT
UVM_FATAL	UVM_DISPLAY | UVM_FATAL

消息管理

消息宏

- 如果要做自定义的消息处理方式，用户可以通过uvm_report_object类提供的方法进行配置。
- uvm_report_object类是间于uvm_object类与uvm_component类之间的中间类，它的主要功能是完成消息打印和管理。
- UVM也提供了一些宏来对应上面的消息方法，用户也可以使用这些宏来处理消息。



方 法 调 用	宏 调 用
uvm_report_info()	`uvm_info(ID, MESSAGE, VERBOSITY)
uvm_report_warning()	`uvm_warning(ID, MESSAGE)
uvm_report_error()	`uvm_error(ID, MESSAGE)
uvm_report_fatal()	`uvm_fatal(ID, MESSAGE)

Agenda

1. UVM总览
2. 基础结构
3. 工厂机制
4. 消息机制
5. **TLM通信**

TLM通信

概述

- TLM是一种基于事务（transaction）的通信方式，通常在高抽象级语言例如SystemC或者SV/UVM中作为模块之间的通讯方式。
- TLM并不是某一种语言的产物，而是作为一种提高数据传输抽象级的标准存在的。
- TLM成功地将模块内的计算和模块之间的通信从时间跨度方面剥离开了。
- 仿真速度是TLM对项目进度的最大贡献，同时TLM传输中的事务又可以保证足够大的信息量和准确性。
- 高抽象级的数据通信，可以用来表示宽松时间跨度内的硬件通信数据，而通过将低颗粒硬件周期内的数据打包为一个大数据，非常有利于整体环境的仿真速度。

TLM通信
基本概念

- TLM通信需要两个通信的对象，这两个对象分别称为initiator 和target 。区分它们的方法在于，谁先发起通信请求，谁就属于initiator，而谁作为发起通信的响应方，谁就属于target。
- 在初学过程中读者们还应该注意，通信发起方并不代表了transaction的流向起点，即不一定数据是从initiator流向target，也可能是从target流向了initiator。
- 因此按照transaction的流向，我们又可以将两个对象分为producer和consumer。区分它们的方法是，数据从哪里产生，它就属于producer，而数据流向了哪里，它就属于consumer。

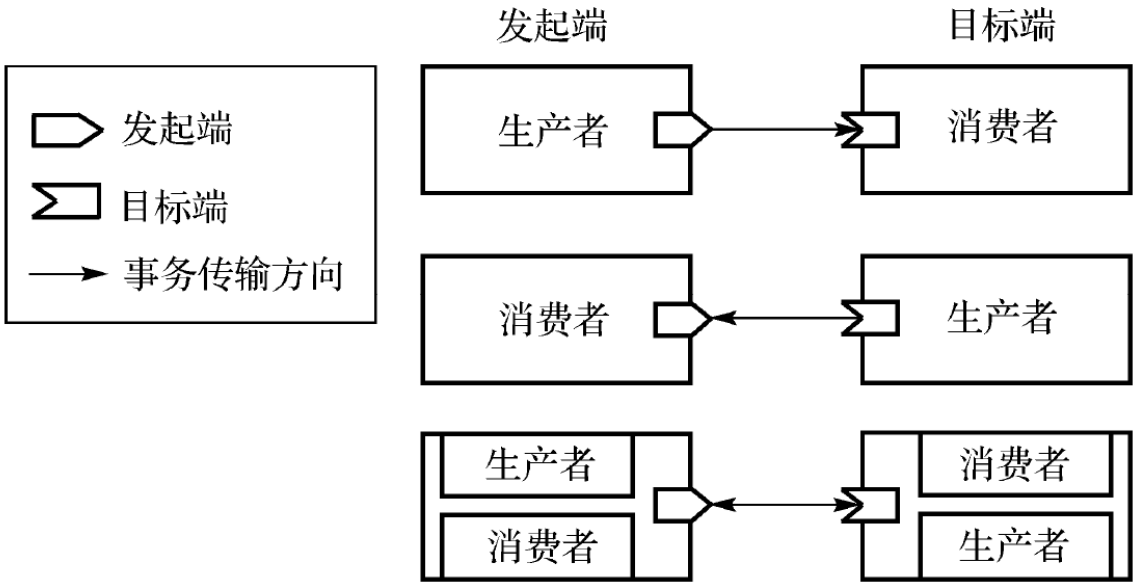
TLM通信

基本概念

•initiator与target的关系同producer与consumer的关系不是固定的。

•有了两个参与通信的对象之后，用户需要将TLM通信方法在target一端中实现，以便于initiator将来作为发起方可以调用target的通信方法，实现数据传输。

•在target实现了必要的通信方法之后，最后一步我们需要将两个对象进行连接，这需要在两个对象中创建TLM端口，继而在更高层次中将这两个对象进行连接。



TLM通信
分类

我们可以将TLM通信步骤分为：

- 分辨出initiator和target，producer和consumer。
- 在target中实现TLM通信方法。
- 在两个对象中创建TLM端口。
- 在更高层次中将两个对象的端口进行连接。

从数据流向来看，传输方向可以分为单向（unidirection）和双向（bidirection）。

- 单向传输：由initiator发起request transaction。
- 双向传输：由initiator发起request transaction，传送至target；而target在消化了request transaction后，会发起response transaction，继而返回给initiator。

TLM通信
分类

端口的按照类型可以划分为三种：

- port：经常作为initiator的发起端，initiator凭借port才可以访问target的TLM通信方法。
- export：作为initiator和target中间层次的端口。
- imp：只能作为target接收request的末端，它无法作为中间层次的端口，所以imp的连接无法再次延伸。

单向通信（红宝书12.2.1节）

概述

- 单向通信（unidirectional communication）指的是从initiator到target之间的数据流向是单一方向的，或者说initiator和target只能扮演producer和consumer中的一个角色。
- 在UVM中，单一数据流向的TLM端口有很多类型：
  - **uvm_blocking_put_PORT**
  - **uvm_nonblocking_put_PORT**
  - **uvm_put_PORT**
  - **uvm_blocking_get_PORT**
  - **uvm_nonblocking_get_PORT**
  - **uvm_get_PORT**
  - **uvm_blocking_peek_PORT**
  - **uvm_nonblocking_peek_PORT**
  - **uvm_peek_PORT**
  - **uvm_blocking_get_peek_PORT**
  - **uvm_nonblocking_get_peek_PORT**
  - **uvm_get_peek_PORT**

单向通信

类型

- 在PORT代表了三种端口名：port、export和imp。
- 按照UVM端口名的命名规则，它们指出了通信的两个要素：
  - 是否是阻塞的方式（即可以等待延时）
  - 何种通信方法

组合端口	单一端口	方法声明
uvm_put_PORT	uvm_blocking_put_PORT	task put(T t)
	uvm_nonblocking_put_PORT	function bit try_put(T t)
		function bit can_put()
uvm_get_PORT	uvm_blocking_get_PORT	task get(output T t)
	uvm_nonblocking_get_PORT	function bit try_get(output T t)
		function bit can_get()
uvm_peek_PORT	uvm_blocking_peek_PORT	task peek(output T t)
	uvm_nonblocking_peek_PORT	function bit try_peek(output T t)
		function bit can_peek()
uvm_get_peek_PORT	uvm_blocking_get_peek_PORT	task get(output T t)
		task peek(output T t)
	uvm_nonblocking_get_peek_PORT	function bit try_get(output T t)
		function bit can_get()
		function bit try_peek(output T t)
		function bit can_peek()

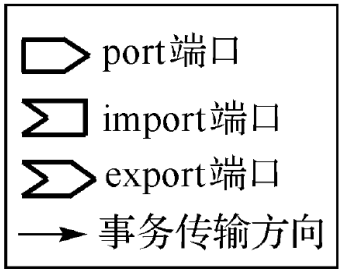
单向通信
方法

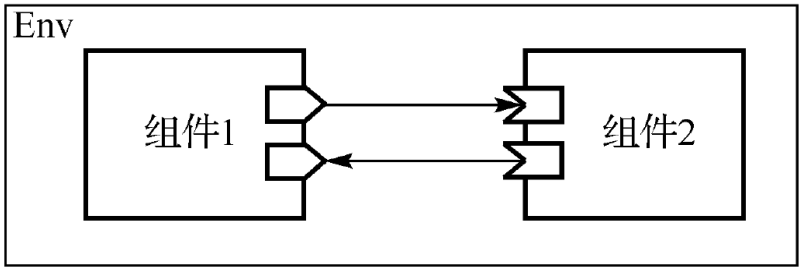
- 阻塞传输方式将blocking前缀作为函数名的一部分，而非阻塞方式则名为nonblocking。阻塞端口的方法类型为task，这保证了可以实现事件等待和延时；非阻塞端口的方式类型为function，这确保了方法调用可以立即返回。
- 我们从方法名也可以发现，例如uvm_blocking_put_PORT提供的方法task put()会在数据传送完后返回，uvm_nonblocking_put_PORT对应的两个函数try_put()和can_put()是立刻返回的。
- uvm_put_PORT则分别提供了blocking和nonblocking的方法，这为通讯方式提供了更多选择。blocking阻塞传输的方法包含：
  - Put()：initiator先生成数据T t，同时将该数据传送至target。
  - Get()：initiator从target获取数据T t，而target中的该数据T t则应消耗。
  - Peek()：initiator从target获取数据T t，而target中的该数据T t还应保留。

单向通信
方法

- 与上述三种任务对应的nonblocking非阻塞方法分别是：
  - try_put()
  - can_put()
  - try_get()
  - can_get()
  - try_peek()
  - can_peek()
- 这六个非阻塞函数与对应阻塞任务的区别在于，它们必须立即返回，如果try_xxx函数可以发送或者获取数据，那么函数应该返回1，如果执行失败则应返回0。
- 或者通过can_xxx函数先试探target是否可以接收数据，如果可以，再通过try_xxx函数发送，提高数据发送的成功率。

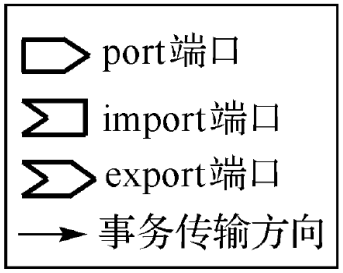
单向通信
示例

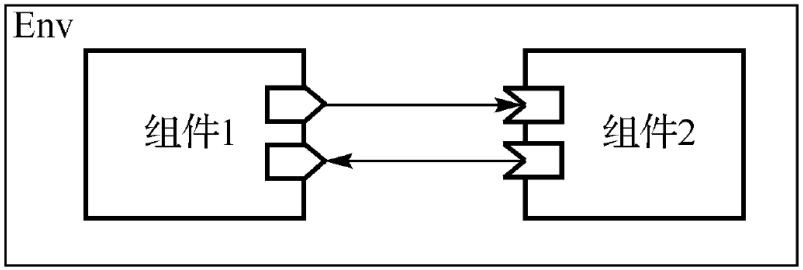





```
class itrans extends uvm_transaction;
  int id;
  int data;
  ...
endclass
class otrans extends uvm_transaction;
  int id;
  int data;
  ...
endclass
class comp1 extends uvm_component;
  uvm_blocking_put_port #(itrans) bp_port;
  uvm_nonblocking_get_port #(otrans) nbg_port;
  `uvm_component_utils(comp1)
```

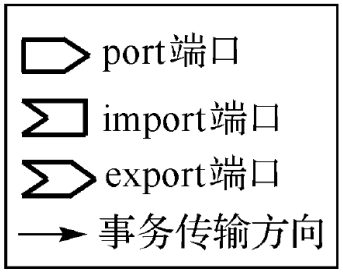
单向通信
示例

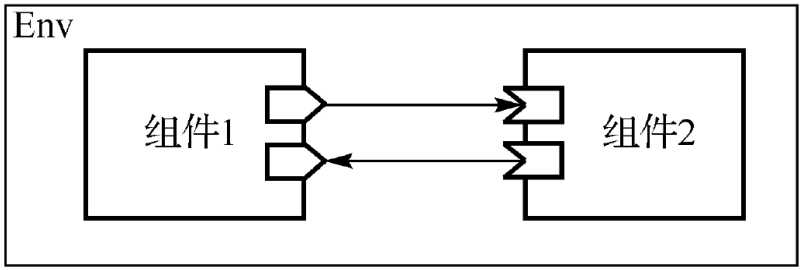





```
...
task run_phase(uvm_phase phase);
  itrans itr;
  otrans otr;
  int trans_num = 2;
  fork
    begin
      for(int i=0; i<trans_num; i++) begin
        itr = new("itr", this);
        itr.id = i;
        itr.data = 'h10 + i;
        this.bp_port.put(itr);
        `uvm_info("PUT", $sformatf("put itrans id: 'h%0x , data: 'h%0x",
             itr.id, itr.data), UVM_LOW)
      end
    end
    begin
       for(int j=0; j<trans_num; j++) begin
         forever begin
           if(this.nbg_port.try_get(otr) == 1) break;
           else #1ns;
         end
         `uvm_info("TRYGET", $sformatf("get otrans id: 'h%0x , data:
             'h%0x", otr.id, otr.data), UVM_LOW)
       end
    end
  join
 endtask
endclass
```

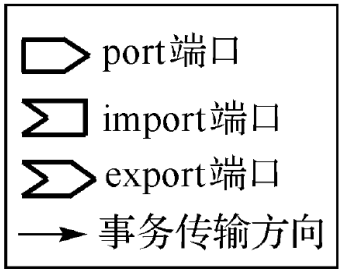
单向通信
示例

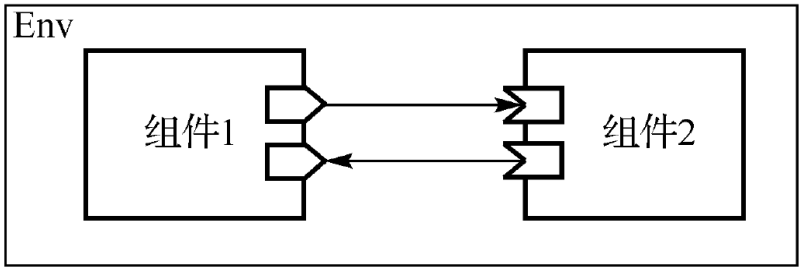





```
class comp2 extends uvm_component;
  uvm_blocking_put_imp #(itrans, comp2) bp_imp;
  uvm_nonblocking_get_imp #(otrans, comp2) nbg_imp;
  itrans itr_q[$];
  `uvm_component_utils(comp2)
  ...
  task put(itrans t);
    itr_q.push_back(t);
  endtask
  function bit try_get (output otrans t);
    itrans i;
    if(itr_q.size() != 0) begin
      i = itr_q.pop_front();
      t = new("t", this);
      t.id = i.id;
      t.data = i.data << 8;
      return 1;
    end
```

单向通信
示例



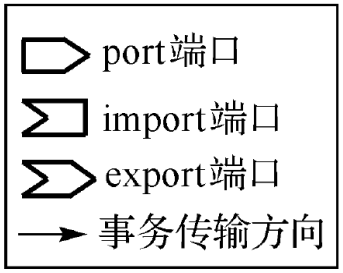


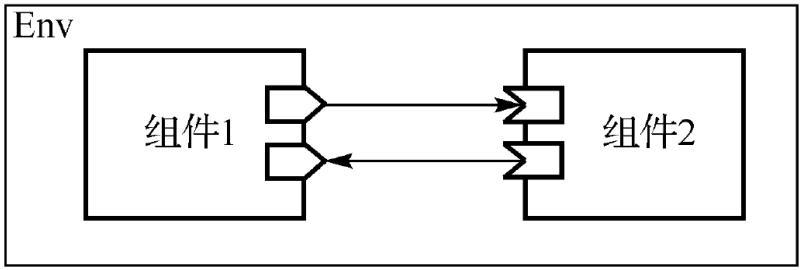

```
    else return 0;
  endfunction
  function bit can_get();
    if(itr_q.size() != 0) return 1;
    else return 0;
  endfunction
endclass
class env1 extends uvm_env;
  comp1 c1;
  comp2 c2;
  `uvm_component_utils(env1)
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    c1 = comp1::type_id::create("c1", this);
    c2 = comp2::type_id::create("c2", this);
  endfunction: build_phase
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    c1.bp_port.connect(c2.bp_imp);
    c1.nbg_port.connect(c2.nbg_imp);
  endfunction: connect_phase
endclass
```

输出结果：

```
uvm_test_top.env.c1 [PUT] put itrans id: 'h0 , data: 'h10
uvm_test_top.env.c1 [PUT] put itrans id: 'h1 , data: 'h11
uvm_test_top.env.c1 [TRYGET] get otrans id: 'h0 , data: 'h1000
uvm_test_top.env.c1 [TRYGET] get otrans id: 'h1 , data: 'h1100
```

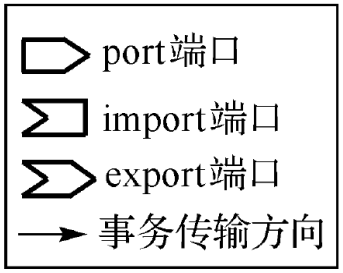
单向通信
示例

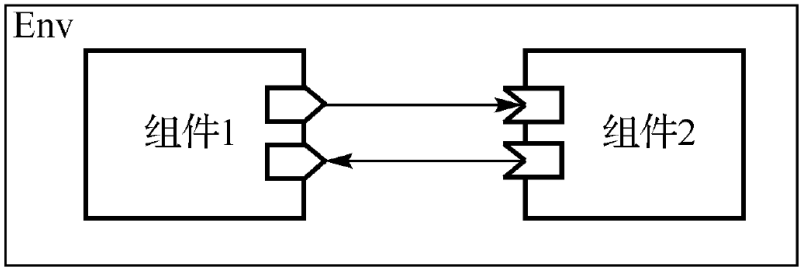




- 首先comp1例化了两个port端口：
  - uvm_blocking_put_port #(itrans) bp_port;
  - uvm_nonblocking_get_port #(otrans) nbg_port;
- comp2作为target则相应例化了两个对应的imp端口：
  - uvm_blocking_put_imp #(itrans, comp2) bp_imp;
  - uvm_nonblocking_get_imp #(otrans, comp2) nbg_imp;
- env1环境将comp1与comp2连接之前，需要在comp2中实现两个端口对应的方法：
  - task put(itrans t)
  - function bit try_get (output otrans t)
  - function bit can_get();

单向通信
示例





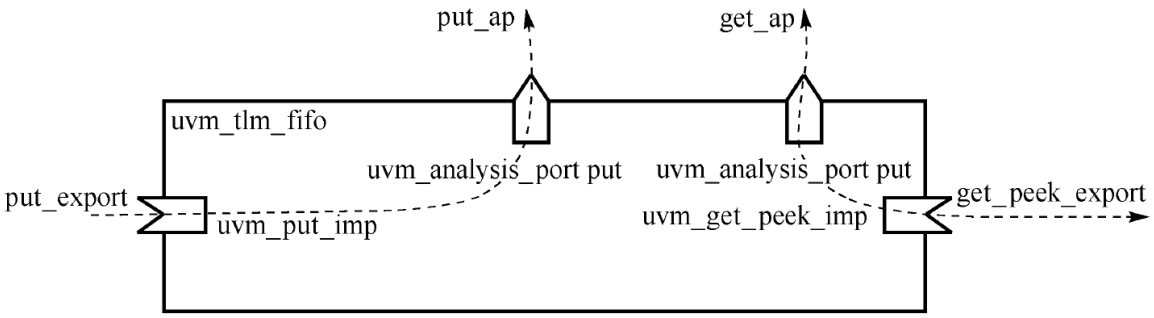
- 接下来env1对两个组件的端口进行了连接，这使得comp1在run phase可以通过自身端口间接调用comp2中定义的端口方法。
- 因此在调用端口方法之前的几个步骤是必不可少的：
  - 定义端口
  - 实现对应方法
  - 在上层将端口进行连接

通信管道

概述

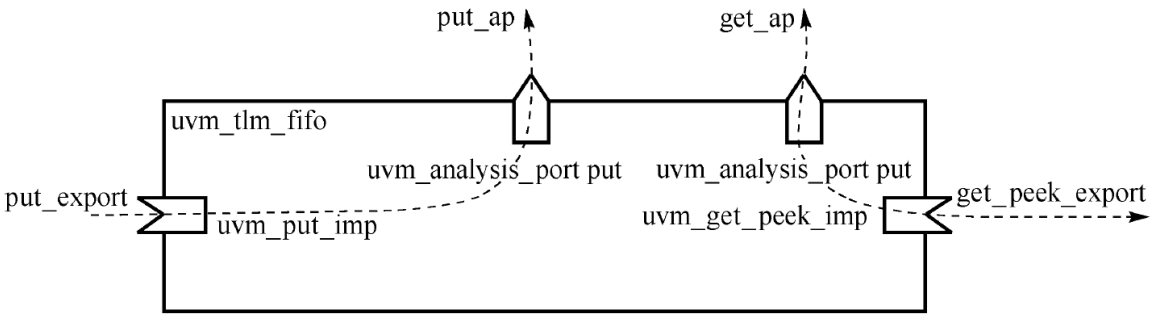
- TLM通信的实现方式，这些通信有一个共同的地方即都是端对端的，同时在target一端需要实现传输方法，例如put()或者get()。
- 这种方式在实际使用也可能会给用户带来一些烦恼，如何可以不自己实现这些传输方法，同时可以享受到TLM的好处？
- 对于monitor、coverage collector等组件在传输数据时，会存在一端到多端的传输，如何解决这一问题？
- 几个TLM组件和端口可以帮助用户免除这些烦恼：
  - TLM FIFO
  - analysis port
  - analysis TLM FIFO

通信管道
TLM_FIFO



- 在一般TLM传输过程中，无论是initiator给target发起一个transaction，还是initiator从target获取一个transaction，transaction最终都会流向consumer。
- 用户需要分别在两个组件中例化端口，同时在target中实现相应的传输方法，传输方法都是相似的，方法的主要内容即是为了实现一个数据缓存功能。
- uvm_tlm_fifo类是一个新组件，它继承于uvm_component类，而且已经预先内置了多个端口以及实现了多个对应方法供用户使用。

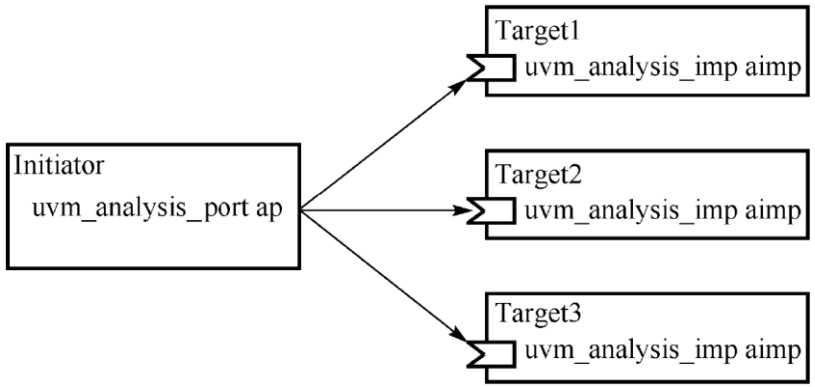
通信管道
TLM_FIFO



- uvm_tlm_fifo的功能类似于mailbox，不同的地方在于uvm_tlm_fifo提供了各种端口供用户使用。

```
• uvm_put_imp      #(T, this_type) blocking_put_export;
• uvm_put_imp      #(T, this_type) nonblocking_put_export;
• uvm_get_peek_imp #(T, this_type) blocking_get_export;
• uvm_get_peek_imp #(T, this_type) nonblocking_get_export;
• uvm_get_peek_imp #(T, this_type) get_export;
• uvm_get_peek_imp #(T, this_type) blocking_peek_export;
• uvm_get_peek_imp #(T, this_type) nonblocking_peek_export;
• uvm_get_peek_imp #(T, this_type) peek_export;
• uvm_get_peek_imp #(T, this_type) blocking_get_peek_export;
• uvm_get_peek_imp #(T, this_type) nonblocking_get_peek_export;
```

通信管道
Analysis Port

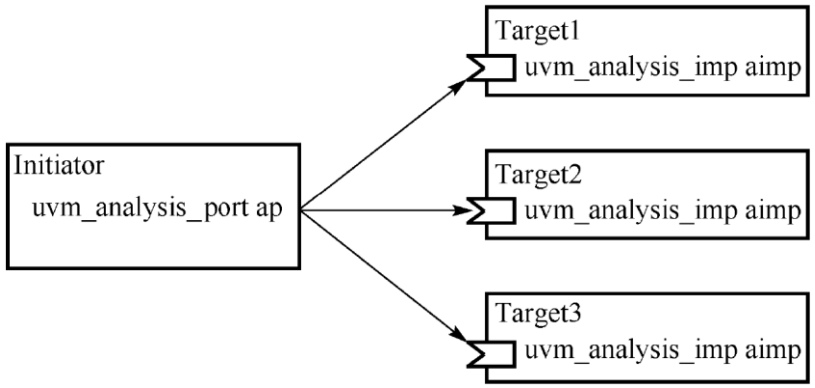


- 除了端对端的传输，在一些情况下还有多个组件会对同一个数据进行运算处理。
- 如果这个数据是从同一个源的TLM端口发出到达不同组件，这就要求该种端口可以满足从一端到多端的需求。
- 一个典型的analysis port类型端口的连接方式，类似于其它TLM端口的是，按照传输方法和端口方向组合可以将analysis port分为uvm_analysis_port、uvm_analysis_export以及uvm_analysis_imp。

通信管道

Analysis Port

```
initiator.ap.connect(target1.aimp);
initiator.ap.connect(target2.aimp);
initiator.ap.connect(target3.aimp);
```



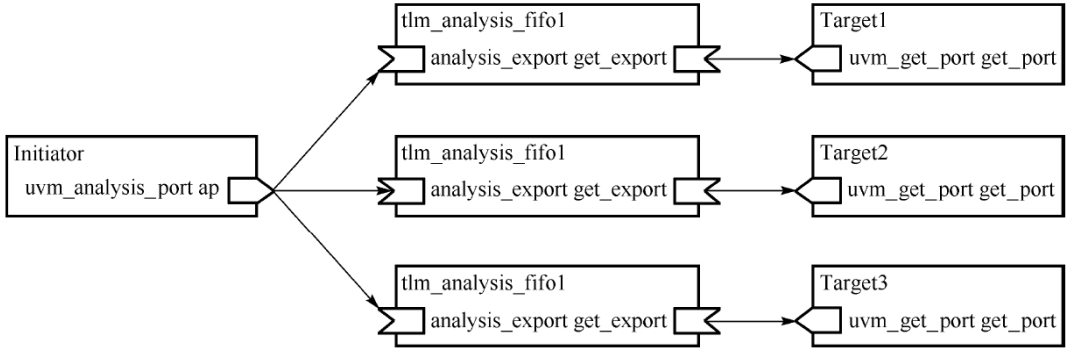
•target一侧例化了uvm_analysis_imp后还需要实现write()函数。

•在顶层可以将initiator端的uvm_analysis_port同多个target端的uvm_analysis_imp进行连接。

•在initiator端调用write()函数时，实际上它是通过循环的方式将所有连接的target端内置的write()函数进行了调用。

•由于函数立即返回的特点，无论连接多少个target端，initiator端调用write()函数总是可以立即返回的。不同于之前单一端口函数调用的是，即使没有target与之相连，调用write()函数时也不会发生错误。

通信管道
Analysis TLM FIFO

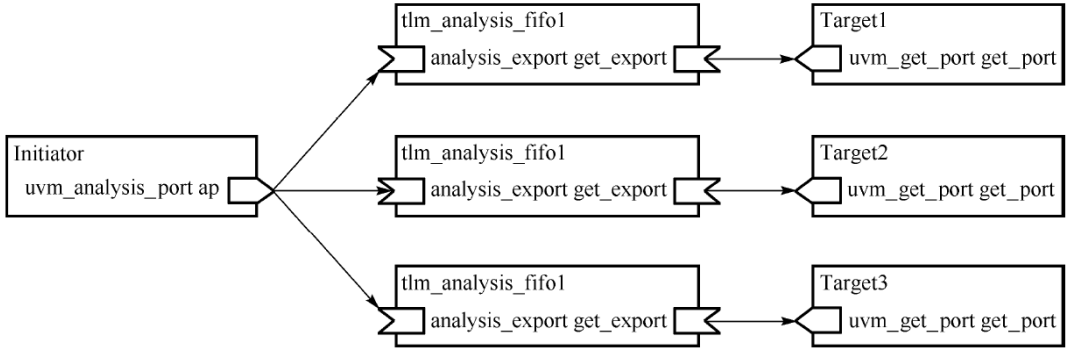


•由于analysis端口提出**实现**了一端到多端的TLM数据**传输**，而一个新的数据**缓**存**组**件类uvm_tlm_analysis_fifo**为**用**户们**提供了可以搭配uvm_analysis_port端口uvm_analysis_imp端口和write()函数。

•uvm_tlm_analysis_fifo类**继**承于uvm_tlm_fifo，**这**表明它本身具有面向**单**一TLM端口的数据**缓**存特性，而同**时该**类又有一个uvm_analysis_imp端口analysis_export并且**实现**了write()函数：

```
uvm_analysis_imp #(T, uvm_tlm_analysis_fifo #(T)) analysis_export;
```

通信管道
Analysis TLM FIFO



•基于initiator到多个target的连接方式，用户如果想轻松实现一端到多端的数据传输，可以插入多个uvm_tlm_analysis_fifo，我们这里给出连接方式：

- 将initiator的analysis port连接到tlm_analysis_fifo的analysis_export端口，这样数据可以从initiator发起，写入到各个tlm_analysis_fifo的缓存中。
- 将多个target的get_port连接到tlm_analysis_fifo的get_export，注意保持端口类型的匹配，这样从target一侧只需要调用get()方法就可以得到先前存储在tlm_analysis_fifo中的数据。

```
initiator.ap.connect(tlm_analysis_fifo1.analysis_export);
target1.get_port.connect(tlm_analysis_fifo1.get_export);

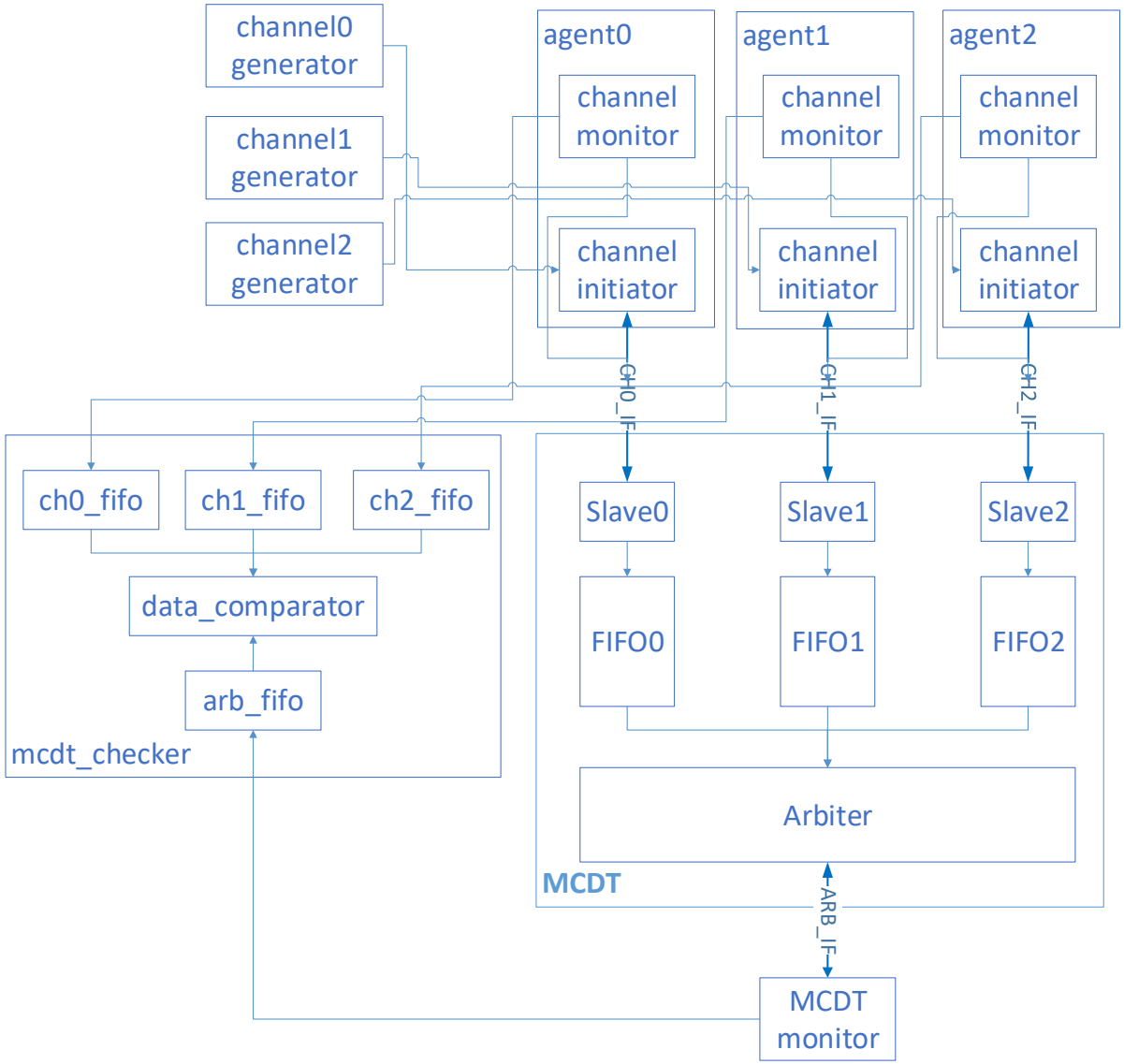
initiator.ap.connect(tlm_analysis_fifo2.analysis_export);
target2.get_port.connect(tlm_analysis_fifo2.get_export);

initiator.ap.connect(tlm_analysis_fifo3.analysis_export);
target3.get_port.connect(tlm_analysis_fifo3.get_export);
```

Agenda

1. **MCDT测试平台结构**
2. 迁移channel stimulator
3. 迁移channel monitor
4. 迁移mcdt monitor
5. 迁移mcdt checker
6. 迁移顶层环境
7. 迁移测试用例

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

MCDT测试平台结构

- channel initiator与channel monitor一一对应
- initiator与monitor被封装在了agent中
- 由于arbiter不需要握手需要，因此只外置了mcdt monitor而不需要mcdt responder
- 所有的monitor都将监测到的数据送至mcdf checker
- mcdf checker内置若干FIFO，接收从MCDT的输入端和输出端监测到的数据
- 由于checker输入和输出端FIFO中存储的数据类型一致，这为数据比较带来了方便

Agenda

1. MCDT测试平台结构
2. **迁移channel stimulator**
3. 迁移channel monitor
4. 迁移mcdt monitor
5. 迁移mcdt checker
6. 迁移顶层环境
7. 迁移测试用例

SV channel stimulator

```
class chnl_trans;
  rand bit[31:0] data[];
  rand int ch_id;
  rand int pkt_id;
  rand int data_nidles;
  rand int pkt_nidles;
  bit rsp;
  local static int obj_id = 0;
  constraint cstr{
    soft data.size inside {[4:8]};
    foreach(data[i])
      data[i] == 'hC000_0000 + (this.ch_id<<24) + (this.pkt_id<<8) + i;
    soft ch_id == 0;
    soft pkt_id == 0;
    data_nidles inside {[0:2]};
    pkt_nidles inside {[1:10]};
  };
```

SV channel stimulator

```
  function new();
    this.obj_id++;
  endfunction

  function chnl_trans clone();
    chnl_trans c = new();
    c.data = this.data;
    c.ch_id = this.ch_id;
    c.pkt_id = this.pkt_id;
    c.data_nidles = this.data_nidles;
    c.pkt_nidles = this.pkt_nidles;
    c.rsp = this.rsp;
    return c;
  endfunction
endclass: chnl_trans
```

UVM channel stimulator

```
class chnl_trans extends uvm_sequence_item;
  rand bit[31:0] data[];
  rand int ch_id;
  rand int pkt_id;
  rand int data_nidles;
  rand int pkt_nidles;
  bit rsp;

  constraint cstr{
    soft data.size inside {[4:32]};
    foreach(data[i]) data[i] == 'hC000_0000 + (this.ch_id<<24) +
                                (this.pkt_id<<8) + i;
    soft ch_id == 0;
    soft pkt_id == 0;
    soft data_nidles inside {[0:2]};
    soft pkt_nidles inside {[1:10]};
  };
```

UVM channel stimulator

```
  `uvm_object_utils_begin(chnl_trans)
    `uvm_field_array_int(data, UVM_ALL_ON)
    `uvm_field_int(ch_id, UVM_ALL_ON)
    `uvm_field_int(pkt_id, UVM_ALL_ON)
    `uvm_field_int(data_nidles, UVM_ALL_ON)
    `uvm_field_int(pkt_nidles, UVM_ALL_ON)
    `uvm_field_int(rsp, UVM_ALL_ON)
  `uvm_object_utils_end

  function new (string name = "chnl_trans");
    super.new(name);
  endfunction
endclass: chnl_trans
```

SV channel stimulator

```
class chnl_initiator;
  local string name;
  local virtual chnl_intf intf;
  mailbox #(chnl_trans) req_mb;
  mailbox #(chnl_trans) rsp_mb;

  function new(string name = "chnl_initiator");
    this.name = name;
  endfunction

  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

SV channel stimulator

```
task run();
  this.drive();
endtask

task drive();
  chnl_trans req, rsp;
  @(posedge intf.rstn);
  forever begin
    this.req_mb.get(req);
    this.chnl_write(req);
    rsp = req.clone();
    rsp.rsp = 1;
    this.rsp_mb.put(rsp);
  end
endtask
```

SV channel stimulator

```
  task chnl_write(input chnl_trans t);
    foreach(t.data[i]) begin
      @(posedge intf.clk);
      intf.ch_valid <= 1;
      intf.ch_data <= t.data[i];
      @(negedge intf.clk);
      wait(intf.ch_ready === 'b1);
      $display("%0t channel initiator [%s] sent data %x",
               $time, name, t.data[i]);
      repeat(t.data_nidles) chnl_idle();
    end
    repeat(t.pkt_nidles) chnl_idle();
  endtask

  task chnl_idle();
    @(posedge intf.clk);
    intf.ch_valid <= 0;
    intf.ch_data <= 0;
  endtask
endclass: chnl_initiator
```

UVM channel stimulator

```
class chnl_driver extends uvm_driver #(chnl_trans);
  virtual chnl_intf intf;
  mailbox #(chnl_trans) req_mb;
  mailbox #(chnl_trans) rsp_mb;

  `uvm_component_utils(chnl_driver)

  function new (string name = "chnl_driver", uvm_component parent);
    super.new(name, parent);
  endfunction

  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      `uvm_error("GETVIF","interface handle is NULL, please check if target
                  interface has been intantiated")
    else
      this.intf = intf;
  endfunction
```

UVM channel stimulator

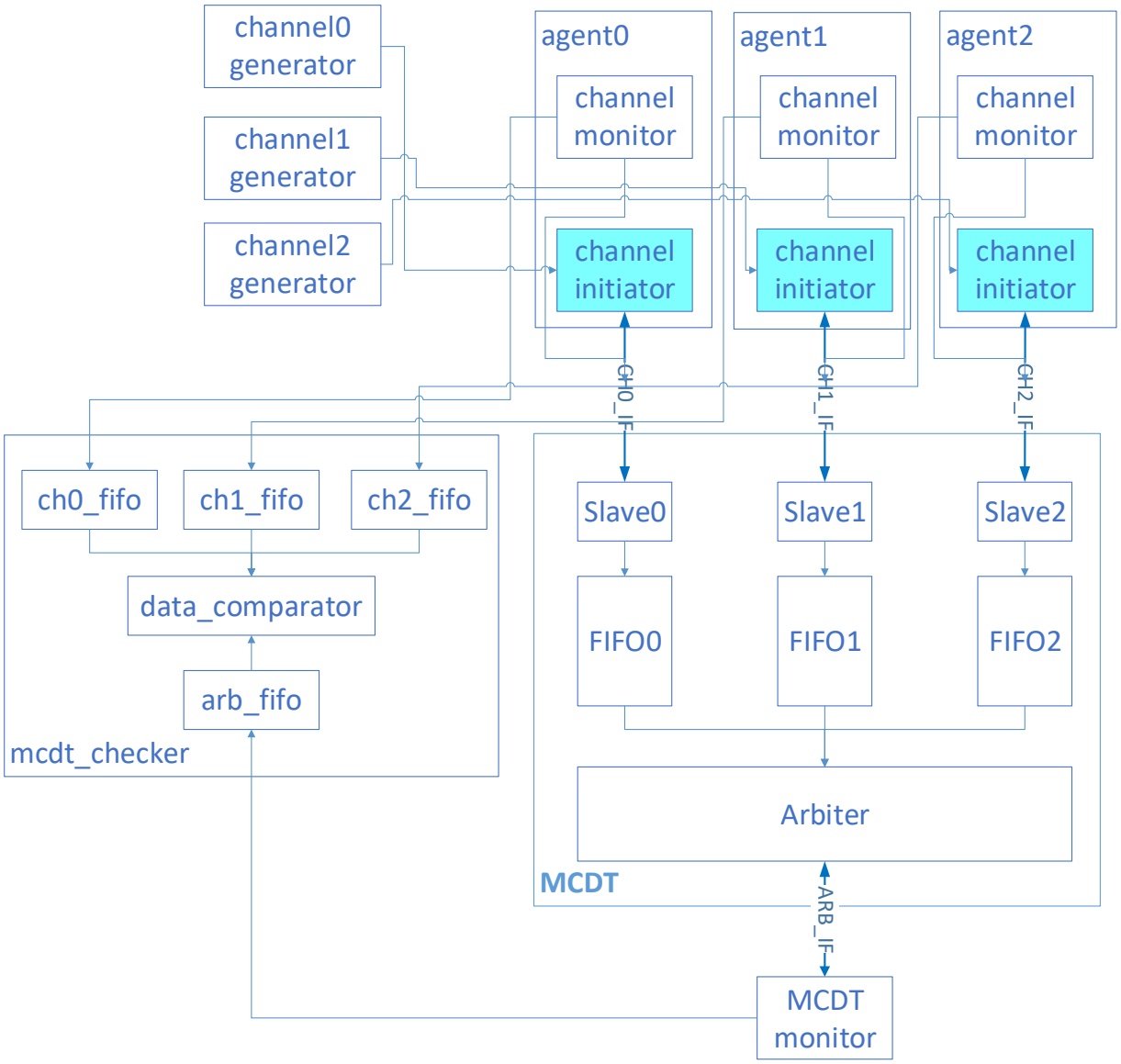
```
task run_phase(uvm_phase phase);
  fork
   this.do_drive();
   this.do_reset();
  join
endtask
task do_reset();
  forever begin
    @(negedge intf.rstn);
    intf.ch_valid <= 0;
    intf.ch_data <= 0;
  end
endtask
task do_drive();
  chnl_trans req, rsp;
  @(posedge intf.rstn);
  forever begin
    this.req_mb.get(req);
    this.chnl_write(req);
    void'($cast(rsp, req.clone()));
    rsp.rsp = 1;
    this.rsp_mb.put(rsp);
  end
endtask
```

UVM channel stimulator

```
  task chnl_write(input chnl_trans t);
    foreach(t.data[i]) begin
      @(posedge intf.clk);
      intf.drv_ck.ch_valid <= 1;
      intf.drv_ck.ch_data <= t.data[i];
      @(negedge intf.clk);
      wait(intf.ch_ready === 'b1);
      `uvm_info(get_type_name(), $sformatf("sent data 'h%8x", t.data[i]),
                UVM_HIGH)
      repeat(t.data_nidles) chnl_idle();
    end
    repeat(t.pkt_nidles) chnl_idle();
  endtask

  task chnl_idle();
    @(posedge intf.clk);
    intf.drv_ck.ch_valid <= 0;
    intf.drv_ck.ch_data <= 0;
  endtask
endclass: chnl_driver
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

SV channel stimulator

```
class chnl_generator;
   rand int pkt_id = -1;
   rand int ch_id = -1;
   rand int data_nidles = -1;
   rand int pkt_nidles = -1;
   rand int data_size = -1;
   rand int ntrans = 10;
   mailbox #(chnl_trans) req_mb;
   mailbox #(chnl_trans) rsp_mb;
   constraint cstr{
     soft ch_id == -1;
     soft pkt_id == -1;
     soft data_size == -1;
     soft data_nidles == -1;
     soft pkt_nidles == -1;
     soft ntrans == 10;
   }
   function new();
     this.req_mb = new();
     this.rsp_mb = new();
   endfunction
```

SV channel stimulator

```
  task run();
    repeat(ntrans) send_trans();
    run_stop_flags.put();
  endtask

  // generate transaction and put into local mailbox
  task send_trans();
    chnl_trans req, rsp;
    req = new();
    assert(req.randomize with {local::ch_id >= 0 -> ch_id == local::ch_id;
                               local::pkt_id >= 0 -> pkt_id == local::pkt_id;
                               local::data_nidles >= 0 -> data_nidles == local::data_nidles;
                               local::pkt_nidles >= 0 -> pkt_nidles == local::pkt_nidles;
                               local::data_size >0 -> data.size() == local::data_size;
                              })
      else $fatal("[RNDFAIL] channel packet randomization failure!");
    this.pkt_id++;
    this.req_mb.put(req);
    this.rsp_mb.get(rsp);
    assert(rsp.rsp)
      else $error("[RSPERR] %0t error response received!", $time);
  endtask
endclass: chnl_generator
```

UVM channel stimulator

```
class chnl_generator extends uvm_component;
  rand int pkt_id = 0;
  rand int ch_id = -1;
  rand int data_nidles = -1;
  rand int pkt_nidles = -1;
  rand int data_size = -1;
  rand int ntrans = 10;

  mailbox #(chnl_trans) req_mb;
  mailbox #(chnl_trans) rsp_mb;

  constraint cstr{
    soft ch_id == -1;
    soft pkt_id == 0;
    soft data_size == -1;
    soft data_nidles == -1;
    soft pkt_nidles == -1;
    soft ntrans == 10;
  }
```

UVM channel stimulator

```
`uvm_component_utils_begin(chnl_generator)
  `uvm_field_int(pkt_id, UVM_ALL_ON)
  `uvm_field_int(ch_id, UVM_ALL_ON)
  `uvm_field_int(data_nidles, UVM_ALL_ON)
  `uvm_field_int(pkt_nidles, UVM_ALL_ON)
  `uvm_field_int(data_size, UVM_ALL_ON)
  `uvm_field_int(ntrans, UVM_ALL_ON)
`uvm_component_utils_end

function new (string name = "chnl_generator", uvm_component parent);
  super.new(name, parent);
  this.req_mb = new();
  this.rsp_mb = new();
endfunction

task start();
  repeat(ntrans) send_trans();
  run_stop_flags.put();
endtask
```

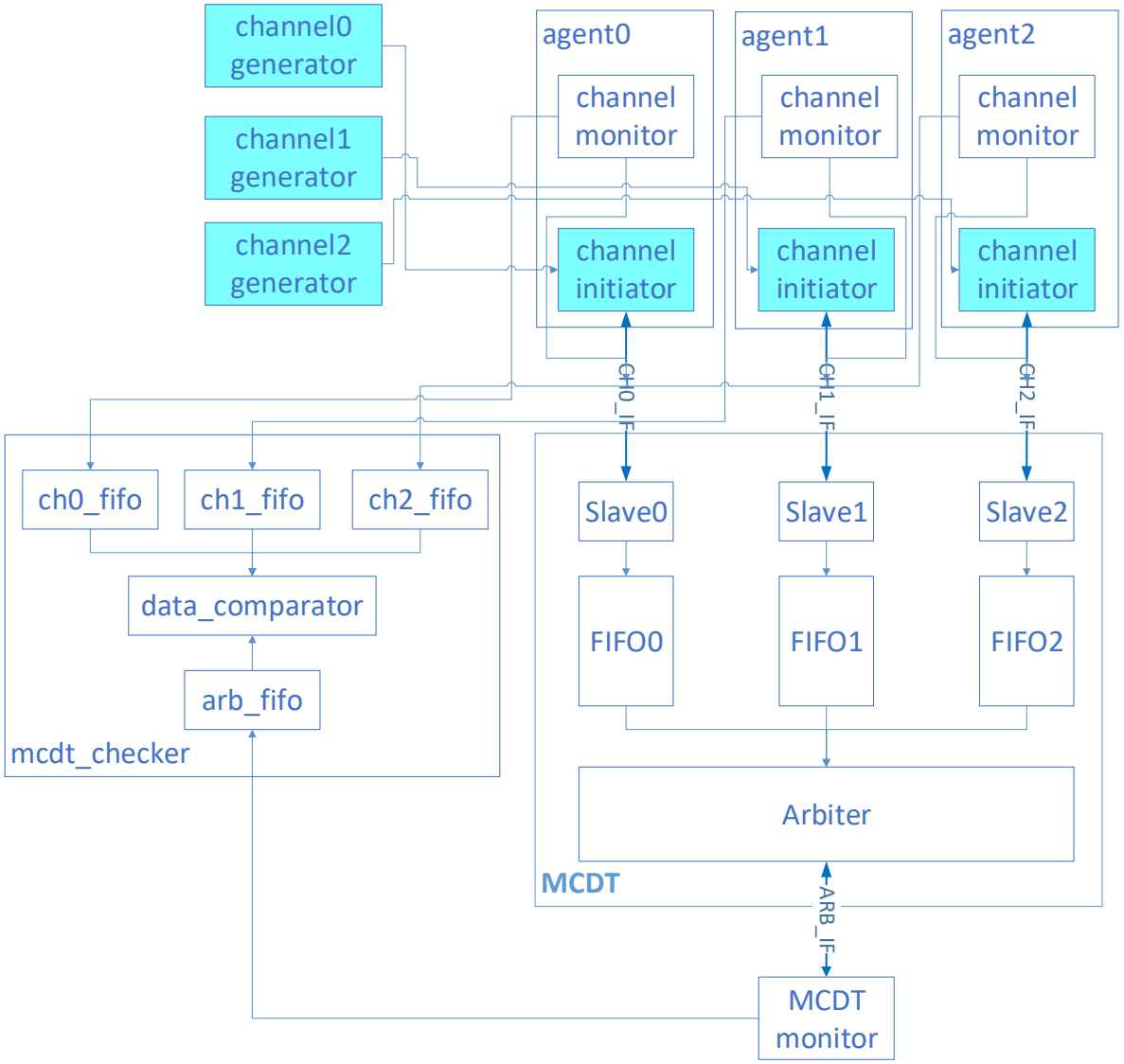
UVM channel stimulator

```
task send_trans();
  chnl_trans req, rsp;
  req = chnl_trans::type_id::create("req");;
  assert(req.randomize with {local::ch_id >= 0 -> ch_id
                               == local::ch_id;
                             local::pkt_id >= 0 -> pkt_id
                               == local::pkt_id;
                             local::data_nidles >= 0 -> data_nidles
                               == local::data_nidles;
                             local::pkt_nidles >= 0 -> pkt_nidles
                               == local::pkt_nidles;
                             local::data_size >0 -> data.size()
                               == local::data_size;
                           })
    else `uvm_fatal("RNDFAIL", "channel packet randomization failure!")
  this.pkt_id++;
  `uvm_info(get_type_name(), req.sprint(), UVM_HIGH)
  this.req_mb.put(req);
  this.rsp_mb.get(rsp);
  `uvm_info(get_type_name(), rsp.sprint(), UVM_HIGH)
  assert(rsp.rsp)
    else `uvm_error("RSPERR", "error response received!")
endtask
```

UVM channel stimulator

```
  function void post_randomize();
    string s;
    s = {s, "AFTER RANDOMIZATION \n"};
    s = {s, "=======================================\n"};
    s = {s, "chnl_generator object content is as below: \n"};
    s = {s, super.sprint()};
    s = {s, "=======================================\n"};
    `uvm_info(get_type_name(), s, UVM_HIGH)
  endfunction
endclass: chnl_generator
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

Agenda

1. MCDT测试平台结构
2. 迁移channel stimulator
3. **迁移channel monitor**
4. 迁移mcdt monitor
5. 迁移mcdt checker
6. 迁移顶层环境
7. 迁移测试用例

SV channel monitor

```
typedef struct packed {
  bit[31:0] data;
  bit[1:0] id;
} mon_data_t;

class chnl_monitor;
  local string name;
  local virtual chnl_intf intf;
  mailbox #(mon_data_t) mon_mb;
  function new(string name="chnl_monitor");
    this.name = name;
  endfunction
  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

SV channel monitor

```
  task run();
    this.mon_trans();
  endtask

  task mon_trans();
    mon_data_t m;
    forever begin
      @(posedge intf.clk iff (intf.ch_valid===1'b1 && intf.ch_ready===1'b1));
      m.data = intf.ch_data;
      mon_mb.put(m);
      $display("%0t %s monitored channle data %8x",
               $time, this.name, m.data);
    end
  endtask
endclass: chnl_monitor
```

UVM channel monitor

```
typedef struct packed {
  bit[31:0] data;
  bit[1:0] id;
} mon_data_t;

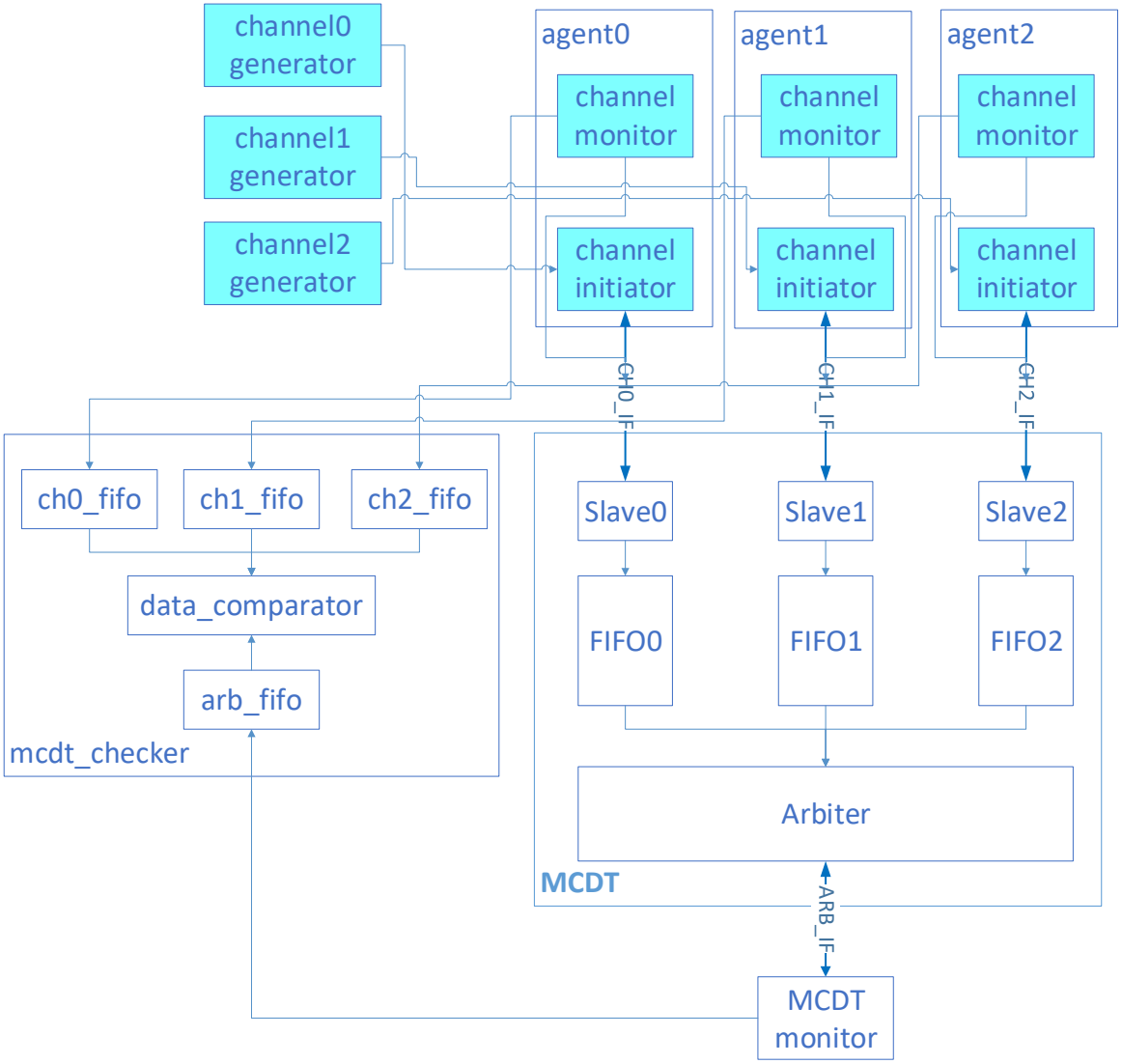
class chnl_monitor extends uvm_monitor;
  virtual chnl_intf intf;
  mailbox #(mon_data_t) mon_mb;
  `uvm_component_utils(chnl_monitor)
  function new(string name="chnl_monitor", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void set_interface(virtual chnl_intf intf);
    if(intf == null)
      `uvm_error("GETVIF", "interface handle is NULL, please check if
                            target interface has been intantiated")
    else
      this.intf = intf;
  endfunction
```

UVM channel monitor

```
  task run_phase(uvm_phase phase);
    this.mon_trans();
  endtask

  task mon_trans();
    mon_data_t m;
    forever begin
      @(posedge intf.clk iff (intf.ch_valid===1'b1 && intf.ch_ready===1'b1));
      m.data = intf.ch_data;
      mon_mb.put(m);
      `uvm_info(get_type_name(),
                $sformatf("monitored channel data 'h%8x", m.data),UVM_HIGH)
    end
  endtask
endclass: chnl_monitor
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt_checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
MCDT monitor

Agenda

1. MCDT测试平台结构
2. 迁移channel stimulator
3. 迁移channel monitor
4. **迁移mcdt monitor**
5. 迁移mcdt checker
6. 迁移顶层环境
7. 迁移测试用例

SV mcdt monitor

```
class mcdt_monitor;
  local string name;
  local virtual mcdt_intf intf;
  mailbox #(mon_data_t) mon_mb;
  function new(string name="mcdt_monitor");
    this.name = name;
  endfunction
  task run();
    this.mon_trans();
  endtask

  function void set_interface(virtual mcdt_intf intf);
    if(intf == null)
      $error("interface handle is NULL,
              please check if target interface has been intantiated");
    else
      this.intf = intf;
  endfunction
```

SV mcdt monitor

```
  task mon_trans();
     mon_data_t m;
     forever begin
       @(posedge intf.clk iff intf.mcdt_val===1'b1);
       m.data = intf.mcdt_data;
       m.id = intf.mcdt_id;
       mon_mb.put(m);
       $display("%0t %s monitored mcdt data %8x and id %0d",
                $time, this.name, m.data, m.id);
     end
   endtask
endclass: mcdt_monitor
```

UVM mcdt monitor

```
class mcdt_monitor extends uvm_monitor;
  virtual mcdt_intf intf;
  mailbox #(mon_data_t) mon_mb;

  `uvm_component_utils(mcdt_monitor)

  function new(string name="mcdt_monitor", uvm_component parent);
    super.new(name, parent);
  endfunction

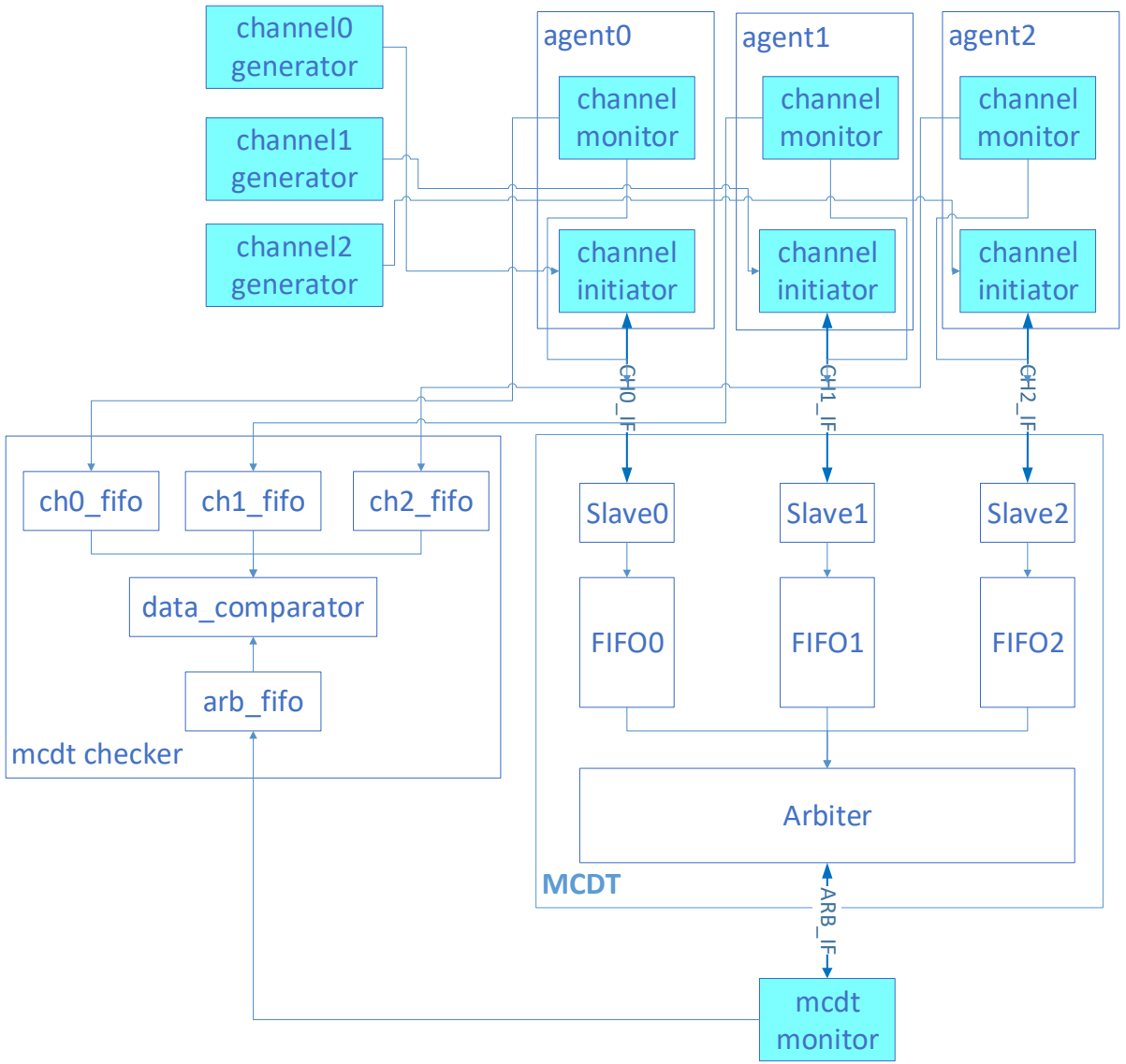
  task run_phase(uvm_phase phase);
    this.mon_trans();
  endtask

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // get virtual interface
    if(!uvm_config_db#(virtual mcdt_intf)::get(this,"","intf", intf)) begin
      `uvm_fatal("GETVIF","cannot get vif handle from config DB")
    end
  endfunction
```

UVM mcdt monitor

```
  task mon_trans();
    mon_data_t m;
    forever begin
      @(posedge intf.clk iff intf.mcdt_val===1'b1);
      m.data = intf.mcdt_data;
      m.id = intf.mcdt_id;
      mon_mb.put(m);
      `uvm_info(get_type_name(),
                $sformatf("monitored channel data 'h%8x", m.data),UVM_HIGH)
    end
  endtask
endclass: mcdt_monitor
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

Agenda

1. MCDT测试平台结构
2. 迁移channel stimulator
3. 迁移channel monitor
4. 迁移mcdt monitor
5. **迁移mcdt checker**
6. 迁移顶层环境
7. 迁移测试用例

SV mcdt checker

```
class mcdt_checker;
  local string name;
  local int error_count;
  local int cmp_count;
  mailbox #(mon_data_t) in_mbs[3];
  mailbox #(mon_data_t) out_mb;

  function new(string name="mcdt_checker");
    this.name = name;
    foreach(this.in_mbs[i]) this.in_mbs[i] = new();
    this.out_mb = new();
    this.error_count = 0;
    this.cmp_count = 0;
  endfunction

  task run();
    this.do_compare();
  endtask
```

SV mcdt checker

```
  task do_compare();
    mon_data_t im, om;
    forever begin
      out_mb.get(om);
      case(om.id)
        0: in_mbs[0].get(im);
        1: in_mbs[1].get(im);
        2: in_mbs[2].get(im);
        default: $fatal("id %0d is not available", om.id);
      endcase
      if(om.data != im.data) begin
        this.error_count++;
        $error("[CMPFAIL] Compared failed! mcdt out data %8x ch_id %0d is
               not equal with channel in data %8x",
               om.data, om.id, im.data);
      end
      else begin
        $display("[CMPSUCD] Compared succeeded! mcdt out data %8x
                  ch_id %0d is equal with channel in data %8x",
                  om.data, om.id, im.data);
      end
      this.cmp_count++;
    end
  endtask
endclass: mcdt_checker
```

UVM mcdt checker

```
class mcdt_checker extends uvm_scoreboard;
  int error_count;
  int cmp_count;
  mailbox #(mon_data_t) in_mbs[3];
  mailbox #(mon_data_t) out_mb;

  `uvm_component_utils(mcdt_checker)

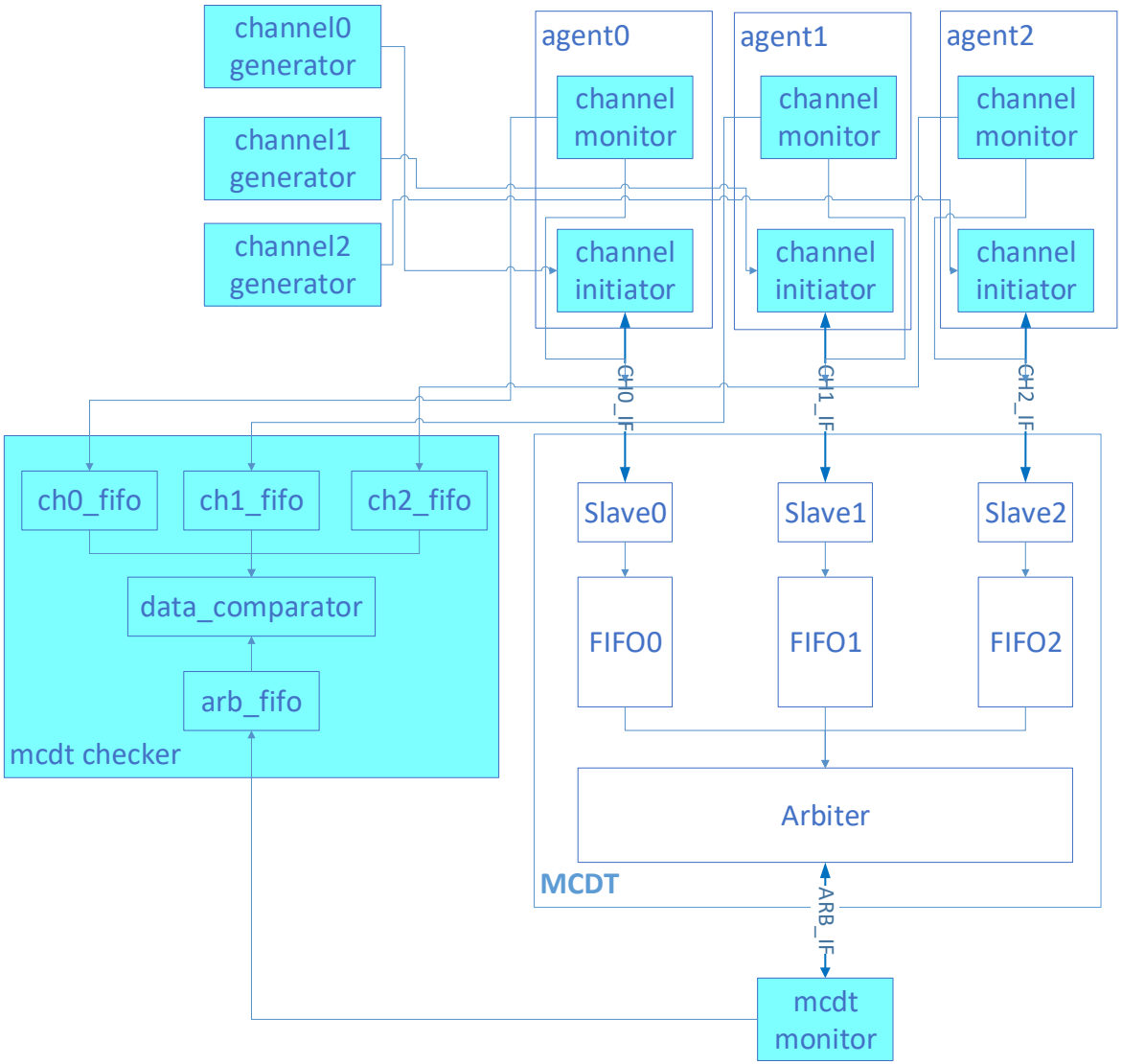
  function new (string name = "mcdt_checker", uvm_component parent);
    super.new(name, parent);
    foreach(this.in_mbs[i]) this.in_mbs[i] = new();
    this.out_mb = new();
    this.error_count = 0;
    this.cmp_count = 0;
  endfunction

  task run_phase(uvm_phase phase);
    this.do_data_compare();
  endtask
```

UVM mcdt checker

```
  task do_data_compare();
    mon_data_t im, om;
    forever begin
      out_mb.get(om);
      case(om.id)
        0: in_mbs[0].get(im);
        1: in_mbs[1].get(im);
        2: in_mbs[2].get(im);
        default: `uvm_fatal(get_type_name(),
                            $sformatf("id %0d is not available", om.id))
      endcase
      if(om.data != im.data) begin
        this.error_count++;
        `uvm_error("CMPFAIL", $sformatf("Compared failed! mcdt out data
                   %8x ch_id %0d is not equal with channel in data %8x",
                   om.data, om.id, im.data))
      end
      else begin
        `uvm_info("CMPSUCD", $sformatf("Compared succeeded! mcdt out data
                   %8x ch_id %0d is equal with channel in data %8x",
                   om.data, om.id, im.data), UVM_HIGH)
      end
      this.cmp_count++;
    end
  endtask
endclass: mcdt_checker
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

Agenda

1. MCDT测试平台结构
2. 迁移channel stimulator
3. 迁移channel monitor
4. 迁移mcdt monitor
5. 迁移mcdt checker
6. **迁移顶层环境**
7. 迁移测试用例

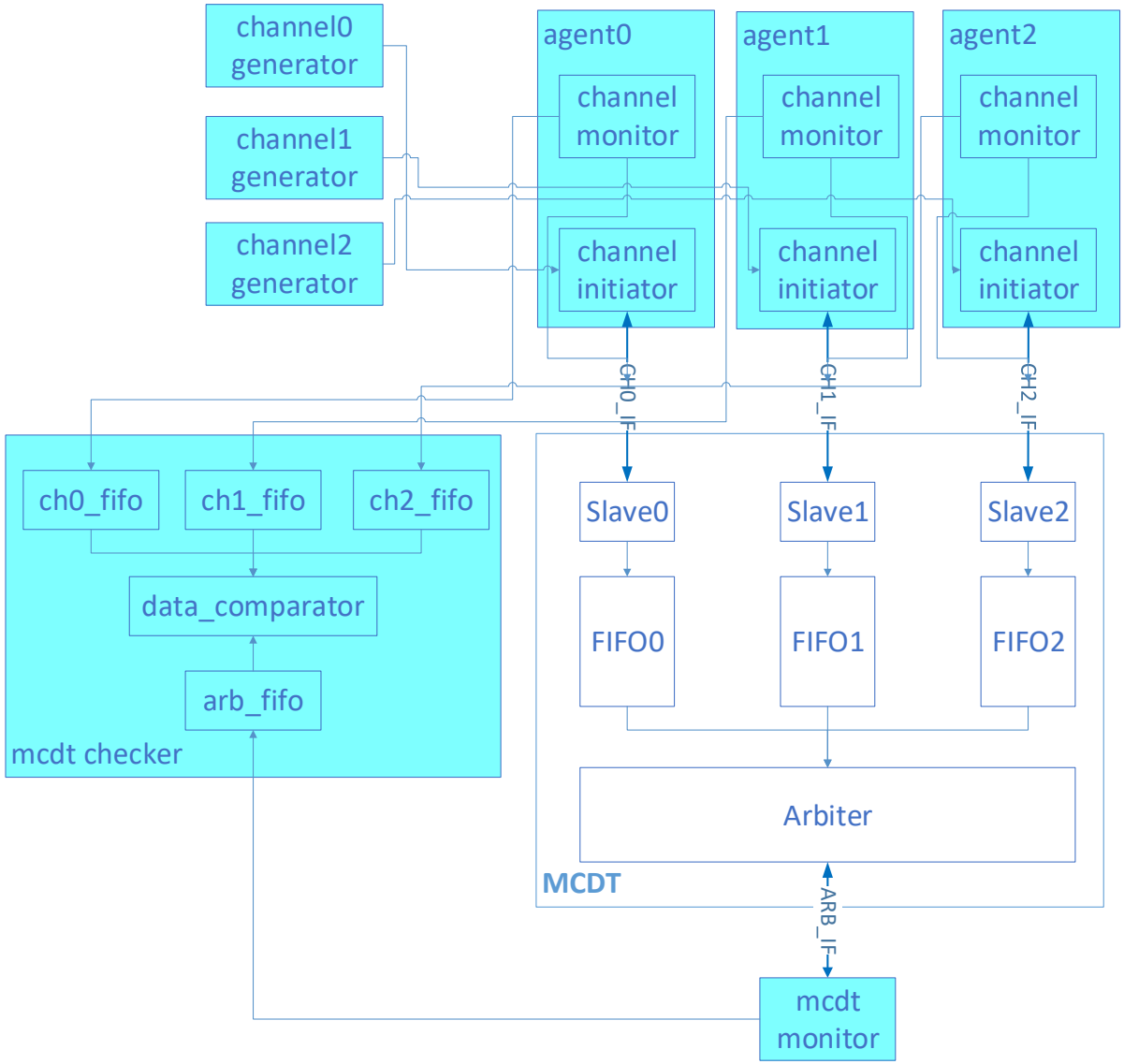
SV 顶层环境

```
class chnl_agent;
  local string name;
  chnl_initiator init;
  chnl_monitor mon;
  virtual chnl_intf vif;
  function new(string name = "chnl_agent");
    this.name = name;
    this.init = new({name, ".init"});
    this.mon = new({name, ".mon"});
  endfunction
  function void set_interface(virtual chnl_intf vif);
    this.vif = vif;
    init.set_interface(vif);
    mon.set_interface(vif);
  endfunction
  task run();
    fork
      init.run();
      mon.run();
    join
  endtask
endclass: chnl_agent
```

UVM 顶层环境

```
class chnl_agent extends uvm_agent;
  chnl_driver driver;
  chnl_monitor monitor;
  virtual chnl_intf vif;
  `uvm_component_utils(chnl_agent)
  function new(string name = "chnl_agent", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db#(virtual chnl_intf)::get(this,"","vif", vif)) begin
      `uvm_fatal("GETVIF","cannot get vif handle from config DB")
    end
    driver = chnl_driver::type_id::create("driver", this);
    monitor = chnl_monitor::type_id::create("monitor", this);
  endfunction
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    driver.set_interface(vif);
    monitor.set_interface(vif);
  endfunction
endclass: chnl_agent
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
channel monitor
channel initiator
agent1
channel monitor
channel initiator
agent2
channel monitor
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor

SV 顶层环境

```
class mcdt_root_test;
  chnl_generator gen[3];
  chnl_agent agents[3];
  mcdt_monitor mcdt_mon;
  mcdt_checker chker;
  protected string name;
  event gen_stop_e;
  function new(string name = "mcdt_root_test");
    this.name = name;
    this.chker = new();
    foreach(agents[i]) begin
      this.agents[i] = new($sformatf("agents[%0d]",i));
      this.gen[i] = new();
      this.agents[i].init.req_mb = this.gen[i].req_mb;
      this.agents[i].init.rsp_mb = this.gen[i].rsp_mb;
      this.agents[i].mon.mon_mb = this.chker.in_mbs[i];
    end
    this.mcdt_mon = new();
    this.mcdt_mon.mon_mb = this.chker.out_mb;
    $display("%s instantiated and connected objects", this.name);
  endfunction
```

SV 顶层环境

```
virtual task gen_stop_callback();
  // empty
endtask

virtual task run_stop_callback();
  $display("run_stop_callback enterred");
  // by default, run would be finished once generators raised 'finish'
  $display("%s: wait for all generators have generated
            and tranferred transcations", this.name);
  run_stop_flags.get(3);
  $finish();
endtask
```

SV 顶层环境

```
virtual task run();
  this.do_config();
  fork
    agents[0].run();
    agents[1].run();
    agents[2].run();
    mcdt_mon.run();
    chker.run();
  join_none
  // run first the callback thread to conditionally disable gen_threads
  fork
    this.gen_stop_callback();
    @(this.gen_stop_e) disable gen_threads;
  join_none
  fork : gen_threads
    gen[0].run();
    gen[1].run();
    gen[2].run();
  join
  run_stop_callback(); // wait until run stop control task finished
endtask
```

SV 顶层环境

```
  virtual function void set_interface(virtual chnl_intf ch0_vif
                                      ,virtual chnl_intf ch1_vif
                                      ,virtual chnl_intf ch2_vif
                                      ,virtual mcdt_intf mcdt_vif
                                     );
    agents[0].set_interface(ch0_vif);
    agents[1].set_interface(ch1_vif);
    agents[2].set_interface(ch2_vif);
    mcdt_mon.set_interface(mcdt_vif);
  endfunction

  virtual function void do_config();
  endfunction
endclass: mcdt_root_test
```

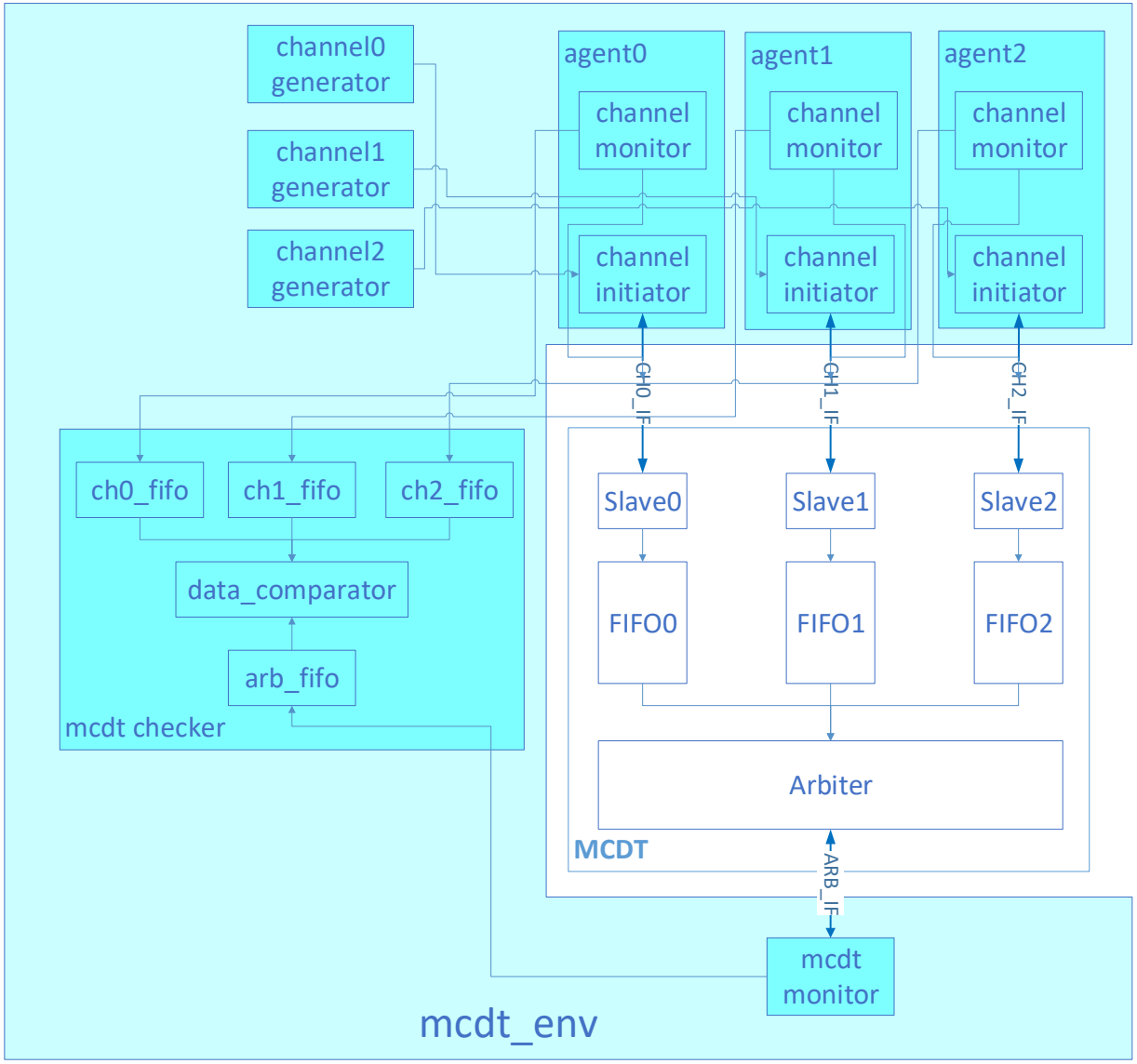
UVM 顶层环境

```
class mcdt_env extends uvm_env;
  chnl_agent agents[3];
  mcdt_monitor mcdt_mon;
  mcdt_checker chker;
  mcdt_coverage cvrg;
  `uvm_component_utils(mcdt_env)
  function new (string name = "mcdt_env", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    this.chker = mcdt_checker::type_id::create("chker", this);
    foreach(agents[i]) begin
      this.agents[i] = chnl_agent::type_id::create(
                       $sformatf("agents[%0d]",i), this);
    end
    this.mcdt_mon = mcdt_monitor::type_id::create("mcdt_mon", this);
    this.cvrg = mcdt_coverage::type_id::create("cvrg", this);
  endfunction
```

UVM 顶层环境

```
 function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    foreach(agents[i]) begin
      this.agents[i].monitor.mon_mb = this.chker.in_mbs[i];
    end
    this.mcdt_mon.mon_mb = this.chker.out_mb;
  endfunction
endclass: mcdt_env
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor
mcdt_env

UVM 顶层环境

```
class mcdt_root_test extends uvm_test;
  mcdt_env env;
  chnl_generator gens[3];
  event gen_stop_e;
  `uvm_component_utils(mcdt_root_test)
  function new(string name = "mcdt_root_test", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = mcdt_env::type_id::create("env", this);
    foreach(gens[i]) begin
      gens[i] = chnl_generator::type_id::create(
                $sformatf("gens[%0d]", i),this);
    end
  endfunction
```

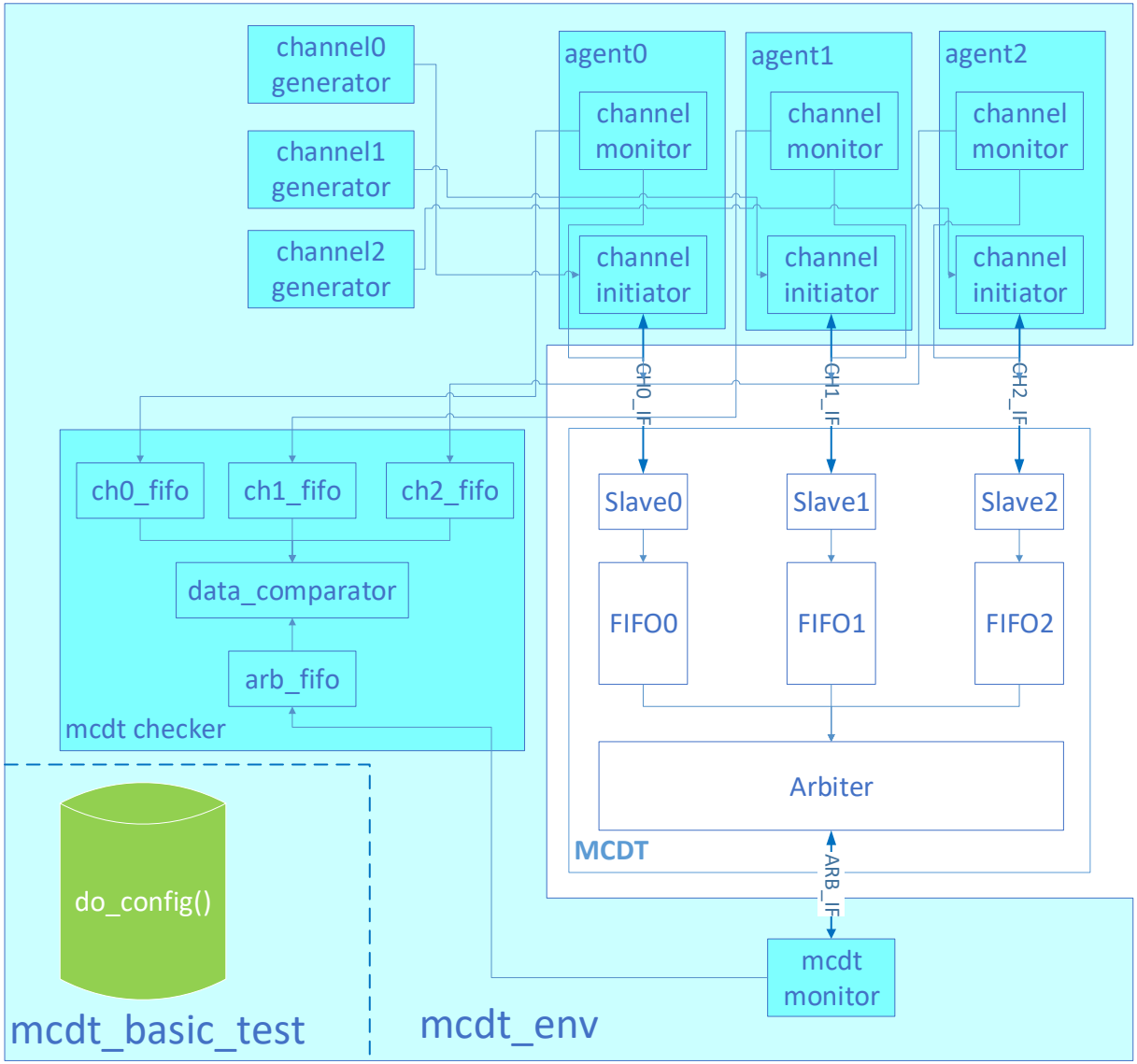
UVM 顶层环境

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  foreach(gens[i]) begin
    env.agents[i].driver.req_mb = gens[i].req_mb;
    env.agents[i].driver.rsp_mb = gens[i].rsp_mb;
  end
endfunction
function void end_of_elaboration_phase(uvm_phase phase);
  super.end_of_elaboration_phase(phase);
  uvm_root::get().set_report_verbosity_level_hier(UVM_HIGH);
  uvm_root::get().set_report_max_quit_count(1);
  uvm_root::get().set_timeout(10ms);
endfunction
virtual function void do_config();
endfunction
virtual task gen_stop_callback();
endtask
virtual task run_stop_callback();
  `uvm_info(get_type_name(), "run_stop_callback enterred", UVM_HIGH)
  `uvm_info(get_type_name(), "wait for all generators have generated and
                              tranferred transcations", UVM_HIGH)
  run_stop_flags.get(3);
endtask
```

UVM 顶层环境

```
  task run_phase(uvm_phase phase);
    // NOTE:: raise objection to prevent simulation stopping
    phase.raise_objection(this);
    `uvm_info(get_type_name(), "=====STARTED=====", UVM_LOW)
    this.do_config();
    // run first the callback thread to conditionally disable gen_threads
    fork
      this.gen_stop_callback();
      @(this.gen_stop_e) disable gen_threads;
    join_none
    fork : gen_threads
      gens[0].start();
      gens[1].start();
      gens[2].start();
    join
    run_stop_callback(); // wait until run stop control task finished
    `uvm_info(get_type_name(), "=====FINISHED=====", UVM_LOW)
    // NOTE:: drop objection to request simulation stopping
    phase.drop_objection(this);
  endtask
endclass: mcdt_root_test
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor
do_config()
mcdt_basic_test
mcdt_env

Agenda

1. MCDT测试平台结构
2. 迁移channel stimulator
3. 迁移channel monitor
4. 迁移mcdt monitor
5. 迁移mcdt checker
6. 迁移顶层环境
7. **迁移测试用例**

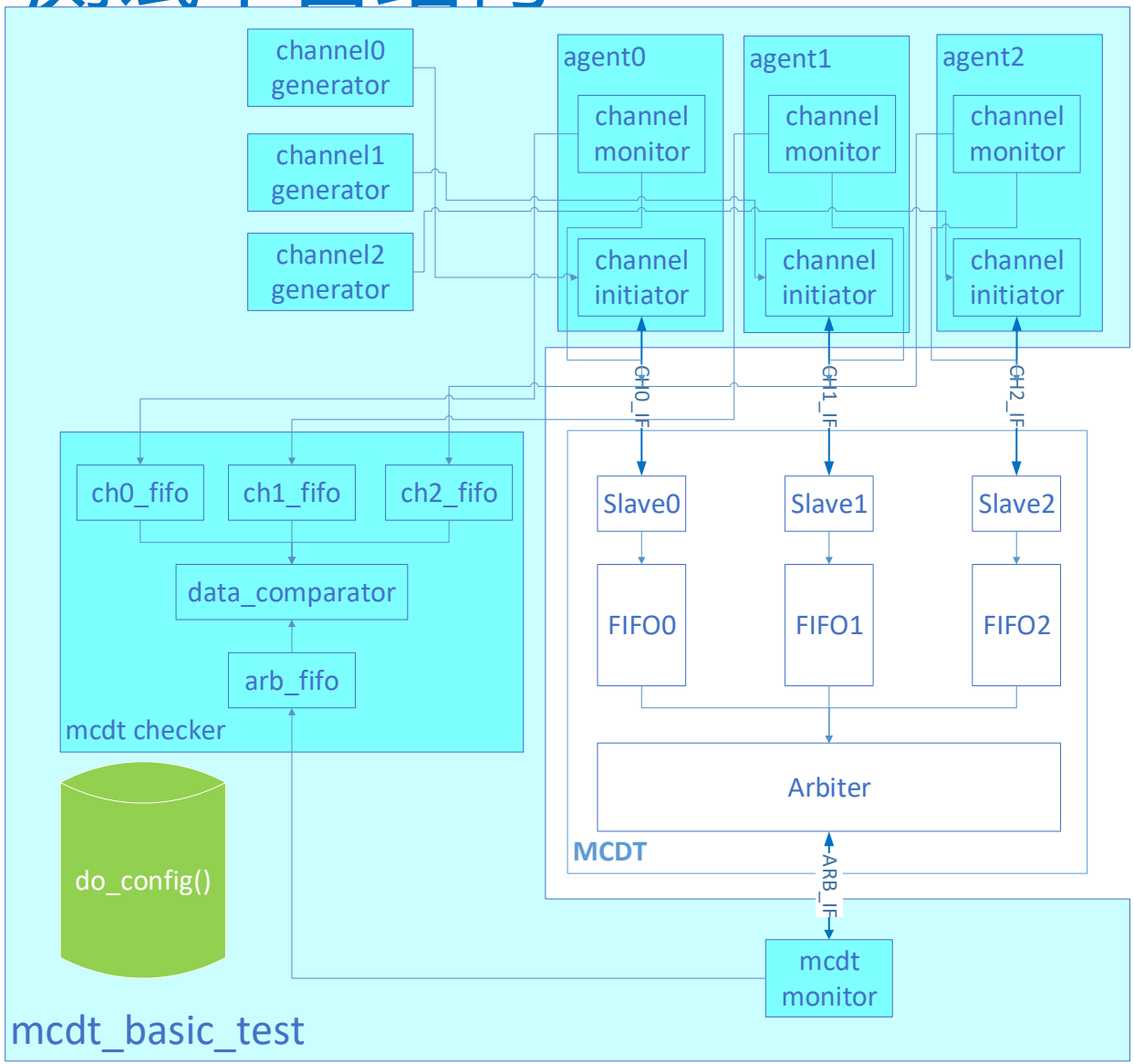
SV 测试用例

```
class mcdt_basic_test extends mcdt_root_test;
  function new(string name = "mcdt_basic_test");
    super.new(name);
  endfunction
  virtual function void do_config();
    super.do_config();
    assert(gen[0].randomize() with
           {ntrans==100; data_nidles==0; pkt_nidles==1; data_size==8;})
      else $fatal("[RNDFAIL] gen[0] randomization failure!");
    assert(gen[1].randomize() with
          {ntrans==50; data_nidles inside {[1:2]};
           pkt_nidles inside {[3:5]}; data_size==6;})
      else $fatal("[RNDFAIL] gen[1] randomization failure!");
    assert(gen[2].randomize() with
           {ntrans==80; data_nidles inside {[0:1]};
            pkt_nidles inside {[1:2]}; data_size==32;})
      else $fatal("[RNDFAIL] gen[2] randomization failure!");
  endfunction
endclass: mcdt_basic_test
```

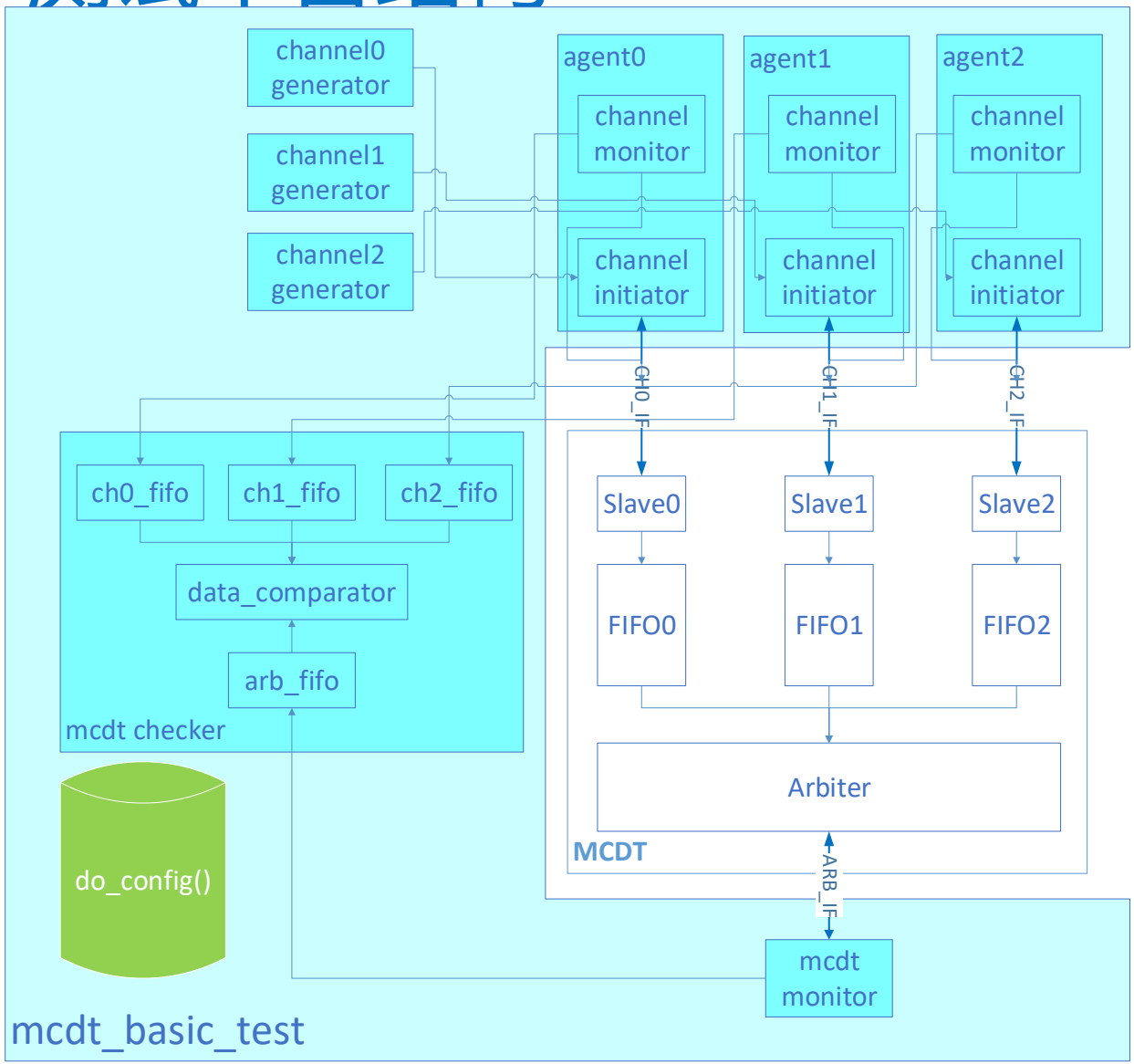
UVM 测试用例

```
class mcdt_basic_test extends mcdt_root_test;
  `uvm_component_utils(mcdt_basic_test)
  function new(string name = "mcdt_basic_test", uvm_component parent);
    super.new(name, parent);
  endfunction
  virtual function void do_config();
    super.do_config();
    assert(gens[0].randomize() with {ntrans==100; data_nidles==0;
          pkt_nidles==1; data_size==8;})
      else `uvm_fatal("RNDFAIL", "gen[0] randomization failure!")
    assert(gens[1].randomize() with {ntrans==50; data_size==6;
          data_nidles inside {[1:2]}; pkt_nidles inside {[3:5]};})
      else `uvm_fatal("RNDFAIL", "gen[1] randomization failure!")
    assert(gens[2].randomize() with {ntrans==80; data_size==32;
          data_nidles inside {[0:1]}; pkt_nidles inside {[1:2]};})
      else `uvm_fatal("RNDFAIL", "gen[2] randomization failure!")
  endfunction
endclass: mcdt_basic_test
```

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
do_config()
mcdt monitor
mcdt_basic_test

MCDT测试平台结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel monitor
channel monitor
channel initiator
channel initiator
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
do_config()
mcdt monitor
mcdt_basic_test

Agenda

1. 序列发送

2. 序列组织

序列发送
概述

- 在UVM世界，利用其核心特性，在创建了组件和顶层环境，并且完成组件之间的TLM端口连接以后，接下来就可以使得整个环境开始运转了。
- 运转的必要条件是组件之间需要有事务（transaction）传送，这就同管道连接好需要引入水流一样。
- 在本章我们将主要围绕下面几个核心词来阐述它们的作用、分类以及之间的互动关系：
  - sequence item
  - sequence
  - sequencer
  - driver

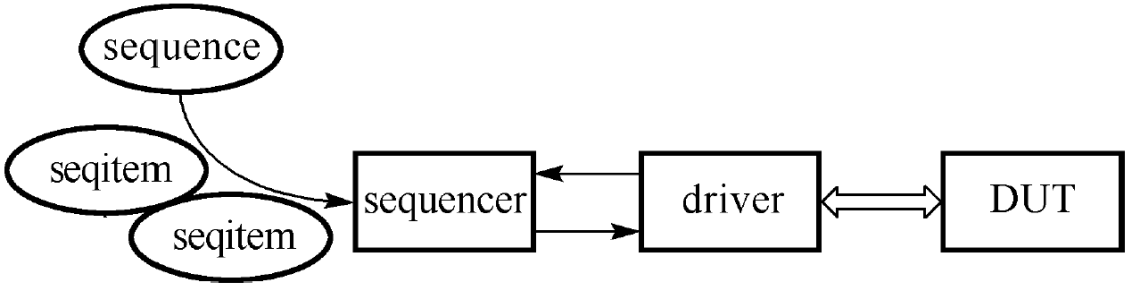
序列发送

概述

- 如果按交通道路的车流来打比方，sequence就是道路，sequence item是道路上行驶的货车，sequencer是目的地的关卡，而driver便是最终卸货的地方。
- 从软件实施层面来讲，这里的货车是从sequence一端出发的，经过了sequencer，最终抵达driver。
- 经过driver的卸货，每一辆货车也就完成了它的使命。
- driver对每一件到站的货物，经过扫描处理，将它们分解为更小的信息量，提供给DUT。

序列发送

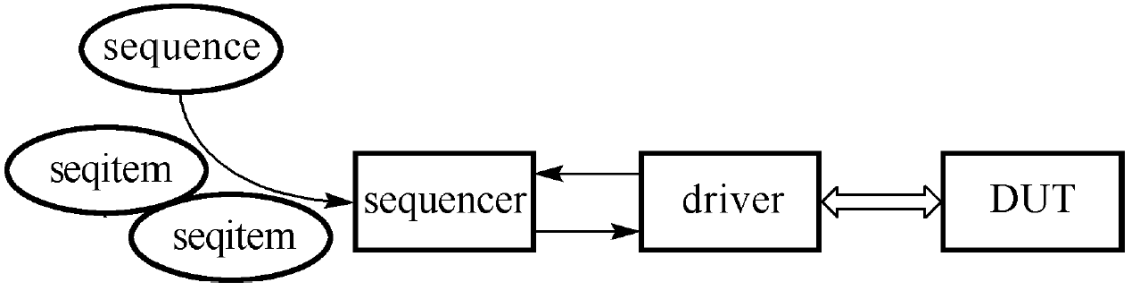
序列组件的互动



- sequence对象会产生目标数量的sequence item对象。借助于SV的随机化和sequence item对随机化的支持，使得产生的每个sequence item对象中的数据内容都不相同。
- sequence item是driver与DUT每一次互动的最小粒度内容。
- 产生的sequence item会经过sequencer再流向driver。
- driver陆续得到每一个sequence item，经过数据解析，将数据按照与DUT的物理接口协议写入到接口上，对DUT形成有效激励。
- 当有必要时，driver在每解析并消化完一个sequence item后，它会将最后的状态信息写回sequence item对象再返回给sequencer，最终抵达sequence对象一侧。

序列发送

序列组件的互动

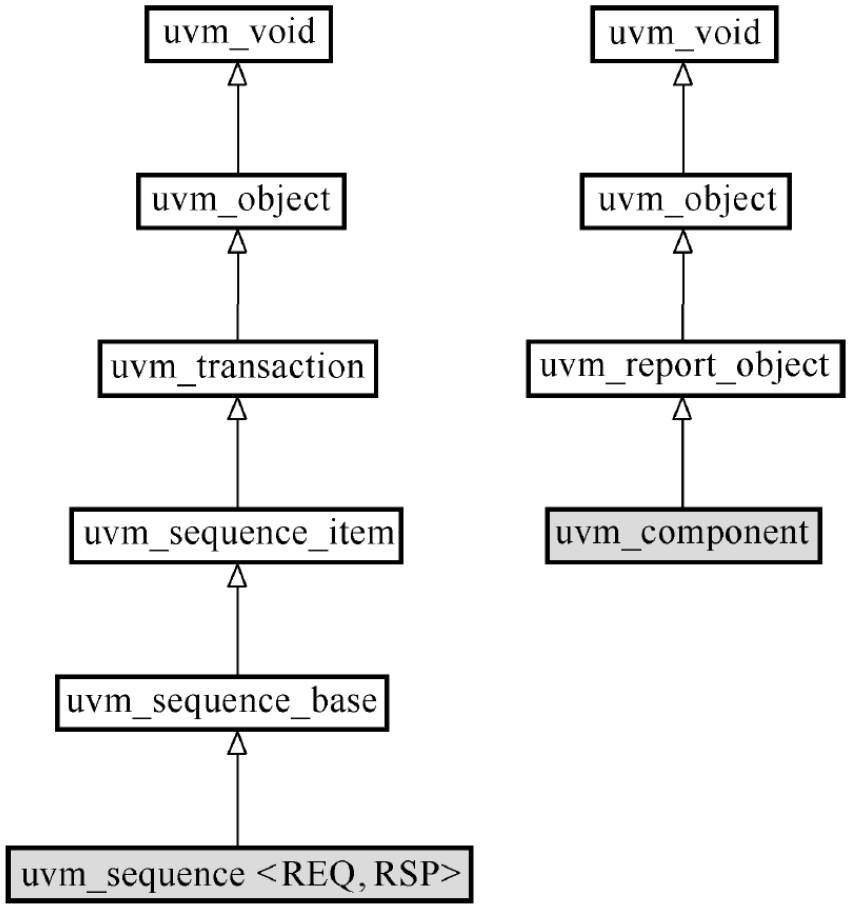


- 对于一个sequence而言，它会产生多个sequence item，也可以产生多个sequence。
- 在sequence与driver之间起到桥梁作用的是sequencer。由于sequencer与driver均是component组件，它们之间的通信也是通过TLM端口实现的。
- driver与sequencer之间的TLM通信参数即sequence item类。由于这一限制，使得sequencer到driver的传输数据类型不能改变，同时与sequencer挂接的sequence创建的sequence item类型也应为指定类型。

序列发送

继承关系

- uvm_sequence_item 和 uvm_sequence 都是基于 uvm_object，它们不同于 uvm_component只应当在build阶段作为UVM环境的“固定资产”进行创建和配置，而是可以在任何阶段创建。



序列发送

继承关系

- 由于无法判定环境在run阶段什么时间点会创建sequence和将其挂载（attach）到sequencer上面，所以无法通过UVM环境结构或者phase机制来识别sequence的运行阶段。
- 由于uvm_object独立于build阶段之外，这使得用户可以有选择地、动态地在合适时间点挂载所需要的sequence和item。
- 考虑到uvm_sequence和uvm_sequence_item并不处于UVM结构当中，所以顶层在做配置时，无法按照层次关系直接配置到sequence中。
- 如果sequence一旦活动起来，它必须挂载到一个sequencer上，这样sequence可以依赖于sequencer的结构关系，间接通过sequencer来获取顶层的配置和更多信息。

序列发送

Sequence和Item

- 在介绍item和sequence如何有序穿过sequencer抵达driver之前，有必要首先认识sequence与item之间的关系。
- 这里的sequence指的是uvm_sequence类，而item指的是uvm_sequence_item类，我们简称其为sequence和item。
- 对于激励生成和场景控制，是由sequence来编织的，而对于激励所需要的具体数据和控制要求，则是从item的成员数据得来的。
- item是基于uvm_object类，这表明了它具备UVM核心基类所必要的数据操作方法，例如copy()、clone()、compare()、record()等。

序列发送

Sequence和Item

```
class bus_trans extends uvm_sequence_item;
  rand bit write;
  rand int data;
  rand int addr;
  rand int delay;
  static int id_num;
  `uvm_object_utils_begin(bus_trans)
    `uvm_field_int...
  `uvm_object_utils_end
  ...
endclass
class test1 extends uvm_test;
  `uvm_component_utils(test1)
  ...
  task run_phase(uvm_phase phase);
    bus_trans t1, t2;
    phase.raise_objection(phase);
    #100ns;
    t1 = new("t1");
    t1.print();
    #200ns;
    t2 = new("t2");
    void'(t2.randomize());
    t2.print();
    phase.drop_objection(phase);
  endtask
endclass
```

序列发送

Sequence和Item

```
--------------------------------
Name       Type        Size  Value
--------------------------------
t1         bus_trans   -     @370
  write    integral    1     'h0
  data     integral    32    'h0
  addr     integral    32    'h0
  delay    integral    32    'h0
  id_num   integral    32    'h1
t2         bus_trans   -     @374
  write    integral    1     'h0
  data     integral    32    'h633b2c71
  addr     integral    32    'he309631a
  delay    integral    32    'hab31e005
  id_num   integral    32    'h2
```

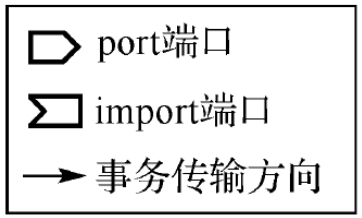
序列发送

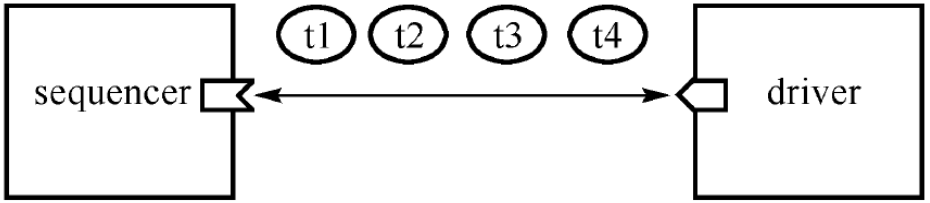
Sequence Item示例

- item使用时的特点：
  - 如果数据域属于需要用来做驱动，那么用户应考虑定义为rand类型，同时按照驱动协议给出合适的constraint。
  - 由于item本身的数据属性，为了充分利用UVM域声明的特性，我们建议将必要的数据成员都通过`uvm_field_xxx宏来声明，以便日后uvm_object的基本数据方法自动实现，例如上面的print()函数。
  - item对象的生命应该开始于sequence的body()方法，而后经历了随机化并穿越sequencer最终到达driver，直到被driver消化之后，它的生命一般来讲才会结束。

Sequencer和Driver

概述

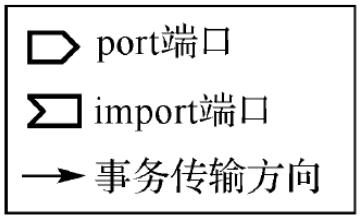


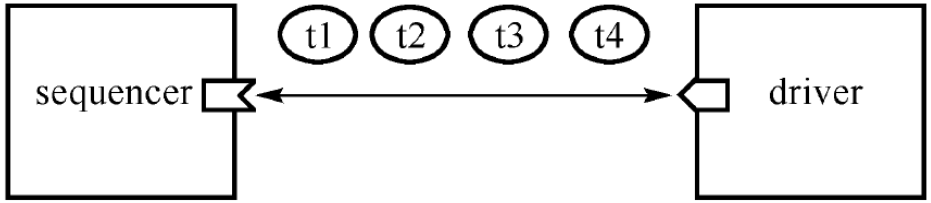


- driver同sequencer之间的TLM通信采取了get模式，即由driver发起请求，从sequencer一端获得item，再由sequencer将其传递至driver。
- sequencer和item只应该在合适的时间点产生需要的数据，而至于怎么处理数据，则会由driver来实现。
- 为了便于item传输，UVM专门定义了匹配的TLM端口供sequencer和driver使用。

Sequencer和Driver

端口和方法

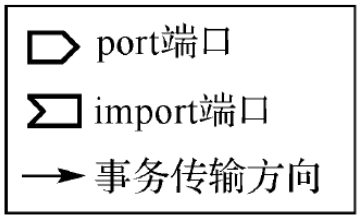


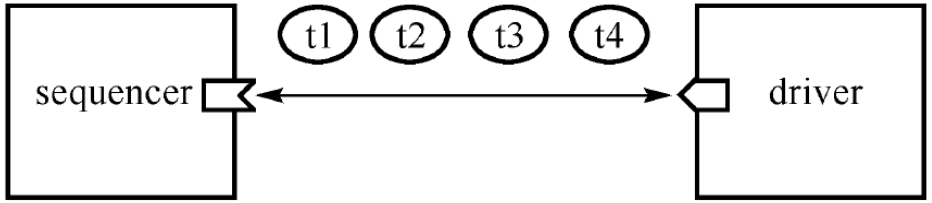


- 通常情况下，用户可以通过匹配driver与sequencer的TLM端口完成item的完整传送，即driver::seq_item_port和sequencer::seq_item_export。
- 这一对端口在连接时同其它端口连接方式一样，即通过driver::seq_item_port.connect(sequencer::seq_item_export)完成。
- 这一类端口功能主要用来实现driver与sequencer的request获取和response返回。

Sequencer和Driver

端口和方法

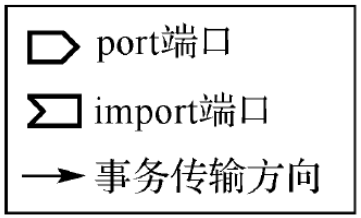


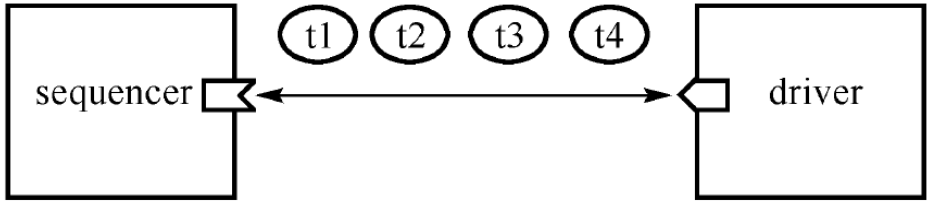


- 该类型的TLM端口支持如下常用方法：
  - task get_next_item(output REQ req_arg)：采取blocking的方式等待从sequence获取下一个item。
  - function void item_done(input RSP rsp_arg=null)：用来通知sequence当前的sequence item已经消化完毕，可以选择性地传递RSP参数，返回状态值。
  - function void put_response(input RSP rsp_arg)：采取nonblocking方式发送response，如果成功返回1，否则返回0。

Sequencer和Driver

端口和方法





- 通常情况下RSP类型与REQ类型保持一致，这么做的好处是为了便于统一处理，方便item对象的拷贝、修改等操作。
- driver消化完当前的request后，可以通过item_done(input RSP rsp_arg=null)方法来告知sequence此次传输已经结束，参数中的RSP可以选择填入，返回相应的状态值。
- driver也可以通过put_response()或者put()方法来单独发送response。

Sequencer和Driver

事务传输实例

```
class bus_trans extends uvm_sequence_item;
   rand int data;
   `uvm_object_utils_begin(bus_trans)
     `uvm_field_int(data, UVM_ALL_ON)
   `uvm_object_utils_end
   ...
endclass
class flat_seq extends uvm_sequence;
   `uvm_object_utils(flat_seq)
   ...
   task body();
     uvm_sequence_item tmp;
     bus_trans req, rsp;
     tmp = create_item(bus_trans::get_type(), m_sequencer, "req");
     void'($cast(req, tmp));
     start_item(req);
     req.randomize with {data == 10;};
     `uvm_info("SEQ", $sformatf("sent a item \n %s", req.sprint()), UVM_LOW)
     finish_item(req);
     get_response(tmp);
     void'($cast(rsp, tmp));
     `uvm_info("SEQ", $sformatf("got a item \n %s", rsp.sprint()), UVM_LOW)
   endtask
endclass
```

Sequencer和Driver

事务传输实例

```
class sequencer extends uvm_sequencer;
  `uvm_component_utils(sequencer)
  ...
endclass
class driver extends uvm_driver;
  `uvm_component_utils(driver)
  ...
  task run_phase(uvm_phase phase);
    REQ tmp;
    bus_trans req, rsp;
    seq_item_port.get_next_item(tmp);
    void'($cast(req, tmp));
    `uvm_info("DRV", $sformatf("got a item \n %s", req.sprint()), UVM_LOW)
    void'($cast(rsp, req.clone()));
    rsp.set_sequence_id(req.get_sequence_id());
    rsp.data += 100;
    seq_item_port.item_done(rsp);
    `uvm_info("DRV", $sformatf("sent a item \n %s", rsp.sprint()), UVM_LOW)
  endtask
endclass
```

Sequencer和Driver

事务传输实例

```
class env extends uvm_env;
  sequencer sqr;
  driver drv;
  `uvm_component_utils(env)
  ...
  function void build_phase(uvm_phase phase);
    sqr = sequencer::type_id::create("sqr", this);
    drv = driver::type_id::create("drv", this);
  endfunction
  function void connect_phase(uvm_phase phase);
    drv.seq_item_port.connect(sqr.seq_item_export);
  endfunction
endclass
```

Sequencer和Driver

事务传输实例

```
class test1 extends uvm_test;
  env e;
  `uvm_component_utils(test1)
  ...
  function void build_phase(uvm_phase phase);
    e = env::type_id::create("e", this);
  endfunction
  task run_phase(uvm_phase phase);
    flat_seq seq;
    phase.raise_objection(phase);
    seq = new();
    seq.start(e.sqr);
    phase.drop_objection(phase);
  endtask
endclass
```

Sequencer和Driver

事务传输实例

输出结果：

```
UVM_INFO @ 0: uvm_test_top.e.sqr@@flat_seq [SEQ] sent a item
...
UVM_INFO @ 0: uvm_test_top.e.drv [DRV] got a item
...
UVM_INFO @ 0: uvm_test_top.e.drv [DRV] sent a item
...
UVM_INFO @ 0: uvm_test_top.e.sqr@@flat_seq [SEQ] got a item
...
```

- 展示了从item定义，到sequence定义，最后到sequencer与driver的连接，即sequencer和driver之间的item传输过程。

- 对于理解driver从sequencer获取item，经过时序处理再返回给sequence的握手过程很有帮助。

Sequencer和Driver
事务传输过程分析

- flat_seq作为动态创建的数据生成载体，它的主任务flat_seq::body()做了如下的几件事情：
  - 通过方法create_item()创建request item对象。
  - 调用start_item()准备发送item。
  - 在完成发送item之前对item进行随机处理。
  - 调用finish_item()完成item发送。
  - 有必要的情况下可以从driver那里获取response item。

Sequencer和Driver
事务传输过程分析

- 在定义driver时，它的主任务driver::run_phase()也应通常做出如下处理：
  - 通过seq_item_port.get_next_item(REQ)从sequencer获取request item。
  - 从request item中获取数据，进而产生数据激励。
  - 对request item进行克隆生成新的对象response item。
  - 修改response item中的数据成员，最终通过seq_item_port.item_done(RSP)将response item对象返回给sequence。
- 对于uvm_sequence::get_response(RSP)和uvm_driver::item_done(RSP)这种成对的操作，是可选的而不是必须的，即用户可以选择uvm_driver不返回response item，同时sequence也无需获取response item。

Sequencer和Driver

事务传输过程分析

- 在高层环境中，应该在connect phase中完成driver到sequencer的TLM端口连接，比如例码在env::connect_phase()中通过drv.seq_item_port.connect(sqr.seq_item_export)完成了driver与sequencer之间的连接。
- 在完成了flat_seq、sequencer、driver和env的定义之后，到了test1层，除了需要考虑挂起objection防止提前退出，便可以利用uvm_sequence类的方法uvm_sequence::start(SEQUENCER)来实现sequence到sequencer的挂载。

Sequencer和Sequence

发送sequence/item方法

- 第一个方法是针对将sequence挂载到sequencer上的应用。
  uvm_sequence::start(uvm_sequencer_base sequencer,
  uvm_sequence_base parent_sequence = null)
- 在使用该方法的过程中，用户首先应该指明sequencer的句柄。
- 如果该sequence是顶部的sequence，即没有更上层的sequence嵌套它，则它可以省略对第二个参数parent_sequence的指定。

Sequencer和Sequence

发送sequence/item方法

- 第二种发送方法是针对将item挂载到sequencer上的应用
  uvm_sequence::start_item (uvm_sequence_item item);
  uvm_sequence::finish_item (uvm_sequence_item item);
- 如果对比start()方法和start_item()/finish_item()，读者首先要分清它们面向的挂载对象是不同的。
- 在执行start()过程中，默认情况下会执行sequence的pre_body()和post_body() 。

Sequencer和Sequence

发送序列的相关宏

宏		执行顺序								
		create	sync	pre_do ( )	randomize( )	constraint randomization	mid_do( )	post-sync	body ( )	post_do ( )
Item	uvm_do(item)	x	x	x	x		x	x		x
	uvm_do_with(item, {constraints})	x	x	x		x	x	x		x
	uvm_create(item)	x								
	uvm_send(item)		x	x			x	x		x
	uvm_rand_send(item)		x	x	x		x	x		x
	uvm_rand_send_with(item, {constraints})		x	x		x	x	x		x
Sequence	uvm_do(seq)	x		x	x		x		x	x
	uvm_do_with{seq, {constraint}}	x		x		x	x		x	x
	uvm_create(seq)	x								
	uvm_send(seq)			x			x		x	x
	uvm_rand_send(seq)			x	x		x		x	x
	uvm_rand_send_with(item, {constraints})			x		x	x		x	x

Sequencer和Sequence

发送序列的相关宏

- 正是通过几个sequence/item宏来打天下的方式，用户可以通过`uvm_do/`uvm_do_with来发送sequence和item。
- 在使用它们之前，需要先了解它们背后的sequence和item各自发送的方法。
- 不同的宏，可能会包含创建对象的过程也可能不会创建对象。例如`uvm_do/`uvm_do_with会创建对象，而`uvm_send则不会创建对象，也不会将对象做随机处理。

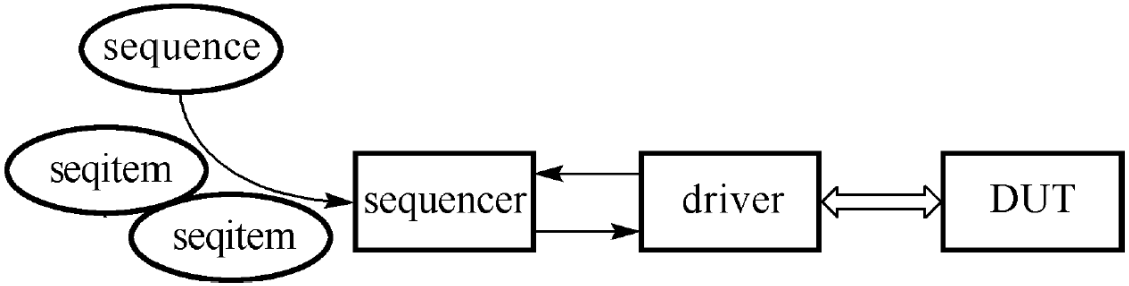
Sequencer和Sequence

序列宏的示例

```
class child_seq extends uvm_sequence;
  ...
  task body();
    bus_trans req;
    `uvm_create(req)
    `uvm_rand_send_with(req, {data == 10;})
  endtask
endclass

class top_seq extends uvm_sequence;
  ...
  task body();
    child_seq cseq;
    bus_trans req;
    // send child sequence via start()
    `uvm_do(cseq)
    // send sequence item
    `uvm_do_with(req, {data == 20;})
  endtask
endclass
```

Agenda



1. 序列发送

2. 序列组织

序列组织

Item与Sequence的关系

- 一个sequence可以包含一些有序组织起来的item实例，sequence在声明时也需要预留一些可供外部随机化的变量。
- 为了区分几种常见的sequence定义方式，我们在介绍sequence之前首先将其分类为：
  - 扁平类（flat sequence）。这一类往往只用来组织更细小的粒度，即item实例构成的组织。
  - 层次类（hierarchical sequence）。这一类是由更高层的sequence用来组织底层的sequence，进而让这些sequence或者按照顺序方式，或者按照并行方式，挂载到同一个sequencer上。
  - 虚拟类（virtual sequence）。鉴于整个环境中往往存在不同种类的sequencer和其对应的sequence，我们需要一个虚拟的sequence来协调顶层的测试场景。该序列本身并不会固定挂载于某一种sequencer类型上，而是将其内部不同类型sequence最终挂载到不同的目标sequencer上面。

序列组织

Flat Sequence介绍

- 一个flat sequence往往由细小的sequence item群落构成，在此之上sequence还有更多的信息来完备它需要实现的激励场景。

```
class bus_trans extends uvm_sequence_item;
  rand bit write;
  rand int data;
  rand int addr;
  rand int delay;
  static int id_num;
  `uvm_object_utils_begin(bus_trans)
    `uvm_field_int...
  `uvm_object_utils_end
  ...
endclass
```

序列组织

Flat Sequence示例

```
class flat_seq extends uvm_sequence;
  rand int length;
  rand int addr;
  rand int data[];
  rand bit write;
  rand int delay;
  constraint cstr {
    data.size() == length;
    foreach(data[i]) soft data[i] == i;
    soft addr == 'h100;
    soft write == 1;
    delay inside {[1:5]};
  };
  `uvm_object_utils(flat_seq)
  ...
  task body();
    bus_trans tmp;
    foreach(data[i]) begin
      tmp = new();
      tmp.randomize() with {data == local::data[i];
                            addr == local::addr + i<<2;
                            write == local::write;
                            delay == local::delay;};
      tmp.print();
    end
  endtask
endclass
```

序列组织

Flat Sequence示例

```
class test1 extends uvm_test;
  `uvm_component_utils(test1)
  ...
  task run_phase(uvm_phase phase);
    flat_seq seq;
    phase.raise_objection(phase);
    seq = new();
    seq.randomize() with {addr == 'h200; leng
    seq.body();
    phase.drop_objection(phase);
  endtask
endclass
```

```
--------------------------------
Name       Type       Size  Value
--------------------------------
bus_trans  bus_trans  -     @363
  write    integral   1     'h1
  data     integral   32    'h0
  addr     integral   32    'h800
  delay    integral   32    'h1
  id_num   integral   32    'h1
bus_trans  bus_trans  -     @380
  write    integral   1     'h1
  data     integral   32    'h1
  addr     integral   32    'h804
  delay    integral   32    'h1
  id_num   integral   32    'h2
```

序列组织
Flat Sequence示例

- 我们暂时没有使用sequence的宏或者其它发送item的宏来实现sequence/item与sequencer之间的传送，而是用更直白的方式来描述这种层次关系。
- flat_seq类可以看作是一个更长的数据包，数据包的具体内容、长度、地址等信息都包含在flat_seq中。
- 初学者需要首先认清sequence与item之间的关系。该例只给出在flat_seq::body()任务中创建和随机化item，而省略了发送item。

序列组织

Hierarchical Sequence介绍

- Hierarchical sequence区别于flat sequence的地方在于，它可以使用其他sequence，当然还有item，这么做是为了创建更丰富的激励场景。
- 通过层次嵌套关系，可以让hierarchical sequence使用其它hierarchical sequence、flat sequence和sequence item。
- 接下来就之前定义的bus_trans和flat_seq给出一个简单的hier_seq类，帮助理解这些sequence/item类之间的联系。

序列组织

Hierarchical Sequence示例

```
class hier_seq extends uvm_sequence;
  `uvm_object_utils(flat_seq)
  function new(string name = "hier_seq");
    super.new(name);
  endfunction
  task body();
    bus_trans t1, t2;
    flat_seq s1, s2;
    `uvm_do_with(t1, {length == 2;})
    fork
      `uvm_do_with(s1, {length == 5;})
      `uvm_do_with(s2, {length == 8;})
    join
    `uvm_do_with(t2, {length == 3;})
  endtask
endclass
```

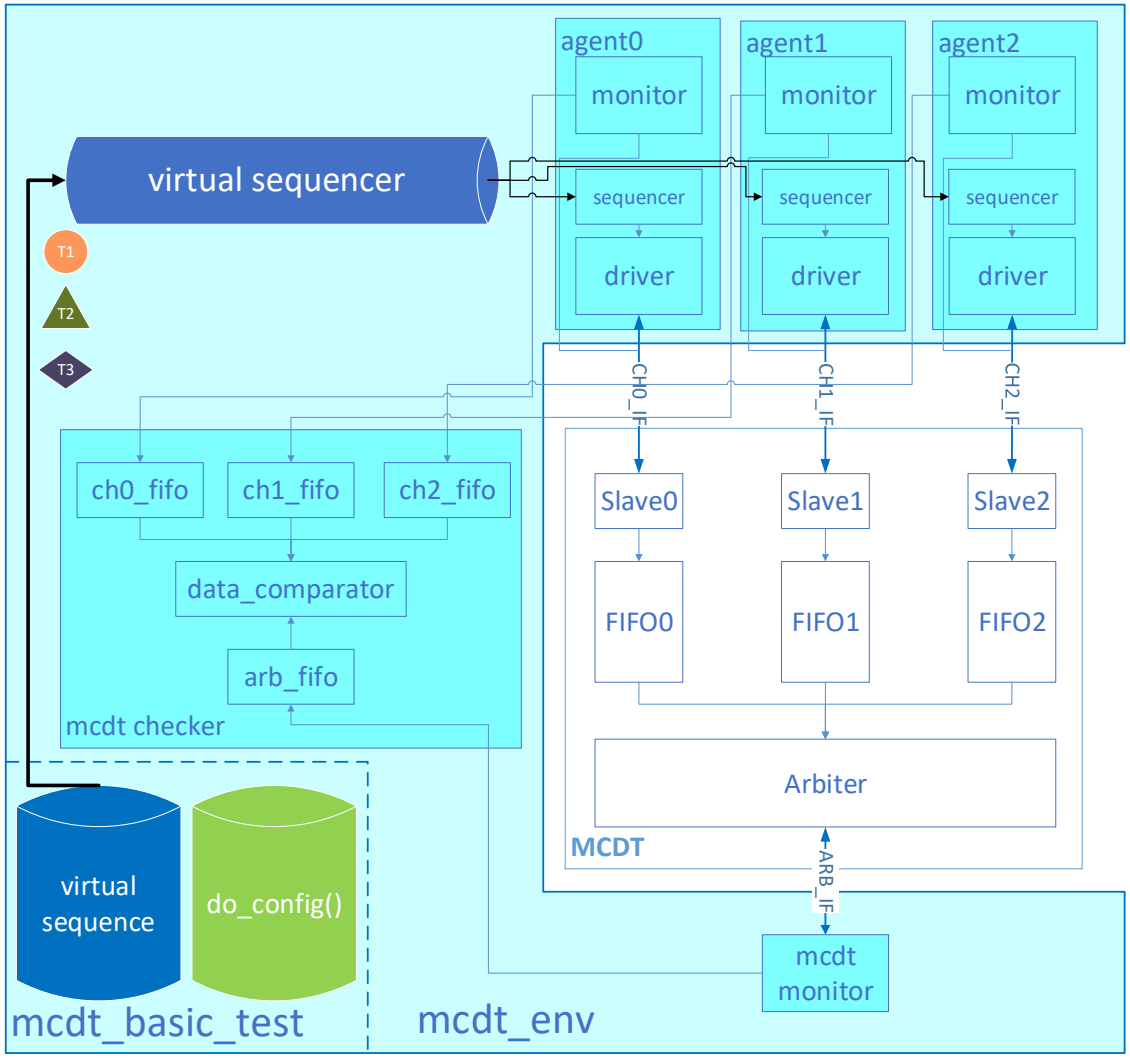
序列组织

Hierarchical Sequence示例

- 从hier_seq::body()来看，它包含有bus_trans t1，t2和flat_seq s1，s2，而它的层次关系就体现在了对于各个sequence/item的协调上面。`uvm_do_with宏完成了三个步骤：
  - sequence或者item的创建
  - sequence或者item的随机化
  - sequence或者item的传送
- 这个例码通过`uvm_do_with宏帮助理解，所谓的sequence复用就是通过高层的sequence来嵌套底层的sequence/item。
- 在示例中既有串行的激励关系，也有并行的激励关系。

序列组织

Virtual Sequence介绍



序列组织

Virtual Sequence介绍

- 在MCDT子系统验证环境集成过程中，需要考虑如何复用各个模块的element sequence和hierarchical sequence。
- MCDT的测试序列要协调的不再只是面向一个sequencer的sequences，而是要面向多个sequencer的sequences。
- 那么面向不同sequencer的sequences在组织以后，如何分别挂载到不同的sequencer上呢？

序列组织

Virtual Sequence介绍

- 我们在之前介绍的sequence，都是面向单一的sequencer，因此挂载也很简单，即通过uvm_sequence::start()来挂载root sequence，而在内部的child sequence则可以通过宏`uvm_do来实现。
- 如果将各个模块环境的element sequence和hierarchical sequence都作为可以复用的sequence资源，那么就需要一个可以容纳各个sequence的容器来承载它们。
- 同时也需要一个合适的routing sequencer来组织不同结构中的sequencer，这样的sequence和sequencer分别称之为virtual sequence和virtual sequencer。

序列组织

Virtual Sequence介绍

- 就之前的sequence和sequencer而言，它们之间的差别在于：
  - virtual sequence可以承载不同目标sequencer的sequence群落，而组织协调这些sequence的方式则类似于高层次的hierarchical sequence。virtual sequence一般只会挂载到virtual sequencer上面。
  - virtual sequencer与普通的sequencer相比有着很大的不同，它们起到了桥接其它sequencer的作用，即virtual sequencer是一个链接所有底层sequencer句柄的地方，它是一个中心化的路由器。
  - 同时virtual sequencer本身并不会传送item数据对象，因此virtual sequencer不需要与任何的driver进行TLM连接。所以UVM用户需要在顶层的connect阶段，做好virtual sequencer中各个sequencer句柄与底层sequencer实体对象的一一对接，避免句柄悬空。
- 接下来的示例用来表示element sequence/hierarchical sequence与virtual sequence的关系，以及底层sequencer与virtual sequencer的联系，同时也说明virtual sequence与virtual sequencer的挂载方法。

序列组织

Virtual Sequence示例

```
class mcdt_root_virtual_sequence extends uvm_sequence #(uvm_sequence_item);
  chnl_data_sequence chnl_data_seq;
  `uvm_object_utils(mcdt_root_virtual_sequence)
  `uvm_declare_p_sequencer(mcdt_virtual_sequencer)
  function new (string name = "mcdt_root_virtual_sequence");
    super.new(name);
  endfunction

  virtual task body();
    `uvm_info(get_type_name(), "============STARTED============", UVM_LOW)
    this.do_data();
    `uvm_info(get_type_name(), "============FINISHED===========", UVM_LOW)
  endtask

  // do data transition from 3 channel slaves
  virtual task do_data();
    //User to implment the task in the child virtual sequence
  endtask

  task wait_cycles(int n);
    repeat(n) @(posedge p_sequencer.mcdt_vif.clk);
  endtask
endclass
```

序列组织

Virtual Sequence示例

```
class mcdt_basic_virtual_sequence extends mcdt_root_virtual_sequence;
  `uvm_object_utils(mcdt_basic_virtual_sequence)
  function new (string name = "mcdt_basic_virtual_sequence");
    super.new(name);
  endfunction

  task do_data();
    @(posedge p_sequencer.mcdt_vif.rstn); this.wait_cycles(10);
    fork
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[0],
                      {ntrans==100; ch_id==0; data_nidles ==0;
                       pkt_nidles==1; data_size==8;})
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[1],
                      {ntrans== 50; ch_id==1; data_nidles inside {[1:2]};
                       pkt_nidles inside {[3:5]}; data_size==6;})
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[2],
                      {ntrans== 80; ch_id==2; data_nidles inside {[0:1]};
                       pkt_nidles inside {[1:2]}; data_size==32;})
    join
    #10us; // wait until all data haven been transfered through MCDT
endtask
  endclass
```

序列组织

Virtual Sequence示例

```
class mcdt_virtual_sequencer extends uvm_sequencer
                                     #(uvm_sequence_item);
 chnl_sequencer chnl_sqrs[3];
 virtual mcdt_intf mcdt_vif;
 `uvm_component_utils(mcdt_virtual_sequencer)
 function new (string name = "mcdt_virtual_sequencer",
               uvm_component parent);
   super.new(name, parent);
 endfunction
 function void build_phase(uvm_phase phase);
   super.build_phase(phase);
   if(!uvm_config_db#(virtual mcdt_intf)::get
      (this,"","mcdt_vif", mcdt_vif)) begin
     `uvm_fatal("GETVIF","cannot get vif handle from config DB")
   end
 endfunction
endclass
```

序列组织

Virtual Sequence示例

```
class mcdt_env extends uvm_env;
   chnl_agent agents[3];
   mcdt_monitor mcdt_mon;
   mcdt_checker chker;
   mcdt_coverage cvrg;
   mcdt_virtual_sequencer virt_sqr;
   `uvm_component_utils(mcdt_env)
   function new (string name = "mcdt_env", uvm_component parent);
     super.new(name, parent);
   endfunction
   function void build_phase(uvm_phase phase);
     super.build_phase(phase);
     this.chker = mcdt_checker::type_id::create("chker", this);
     foreach(agents[i]) begin
       this.agents[i] =
         chnl_agent::type_id::create($sformatf("agents[%0d]",i), this);
     end
     this.mcdt_mon = mcdt_monitor::type_id::create("mcdt_mon", this);
     this.cvrg = mcdt_coverage::type_id::create("cvrg", this);
     this.virt_sqr = mcdt_virtual_sequencer::type_id::create
                     ("virt_sqr", this);
   endfunction
```

序列组织

Virtual Sequence示例

```
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    foreach(agents[i]) begin
      agents[i].monitor.mon_ana_port.connect(
        chker.in_tlm_fifos[i].analysis_export);
      virt_sqr.chnl_sqrs[i] = agents[i].sequencer;
    end
    mcdt_mon.mon_ana_port.connect(chker.out_tlm_fifo.analysis_export);
  endfunction
endclass
```

序列组织

Virtual Sequence示例

```
class mcdt_root_test extends uvm_test;
  mcdt_env env;
  `uvm_component_utils(mcdt_root_test)
  function new(string name = "mcdt_root_test", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = mcdt_env::type_id::create("env", this);
  endfunction
  function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);
    uvm_root::get().set_report_verbosity_level_hier(UVM_HIGH);
    uvm_root::get().set_report_max_quit_count(1);
    uvm_root::get().set_timeout(10ms);
  endfunction
  task run_phase(uvm_phase phase);
    // NOTE:: raise objection to prevent simulation stopping
    phase.raise_objection(this);
    this.run_top_virtual_sequence();
    // NOTE:: drop objection to request simulation stopping
    phase.drop_objection(this);
  endtask
  virtual task run_top_virtual_sequence();
    // User
  endtask
endclass: mcdt_root_test
```

序列组织

Virtual Sequence示例

```
class mcdt_basic_test extends mcdt_root_test;
   `uvm_component_utils(mcdt_basic_test)
   function new(string name = "mcdt_basic_test", uvm_component parent);
     super.new(name, parent);
   endfunction
   task run_top_virtual_sequence();
     mcdt_basic_virtual_sequence top_seq = new();
     top_seq.start(env.virt_sqr);
   endtask
 endclass: mcdt_basic_test
```

序列组织

Virtual Sequence示例

- 对于virtual sequence mcdf_normal_seq而言，它可以承载各个子模块环境的element sequence，而通过最后挂载的virtual sequencer mcdf_virtual_sequencer中的各个底层sequencer句柄，各个element sequence可以分别挂载到对应的底层sequencer上。
- 将virtual sequence挂载到了virtual sequencer上面，这种挂载的目的是为了提供给virtual sequence一个中心化的sequencer路由。
- 借助在virtual sequence mcdf_normal_seq中使用了宏 `uvm_declare_p_sequencer，使得virtual sequence可以使用声明后的成员变量p_sequencer，来进一步回溯到virtual sequencer内部的各个sequencer句柄。

序列组织

Virtual Sequence示例

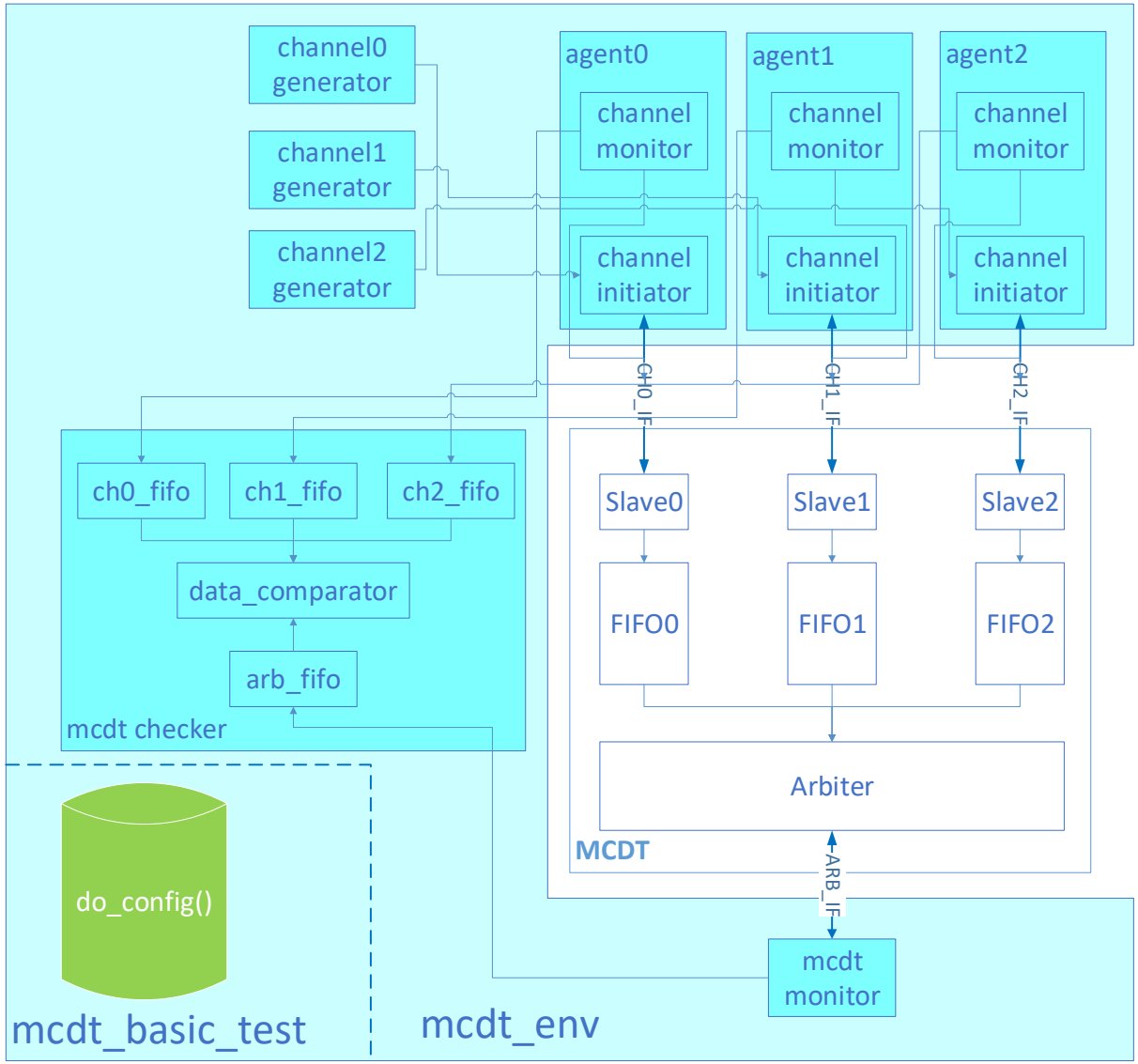
- 只要声明的挂载sequencer类型正确，用户可以通过这个宏，完成方便的类型转换，因此才可以通过p_sequencer索引到在mcdt_virtual_sequencer中声明的各个sequencer句柄。
- 初学者需要理解virtual sequence的协调作用，virtual sequencer的路由作用，以及在顶层中需要完成virtual sequencer同底层sequencer的连接，并最终在test层实现virtual sequence挂载到virtual sequencer上。
- 这种中心化的协调方式，使得顶层环境在场景创建和激励控制方面更加得心应手，而且在代码后期维护中，测试场景的可读性也得到了提高。

Agenda

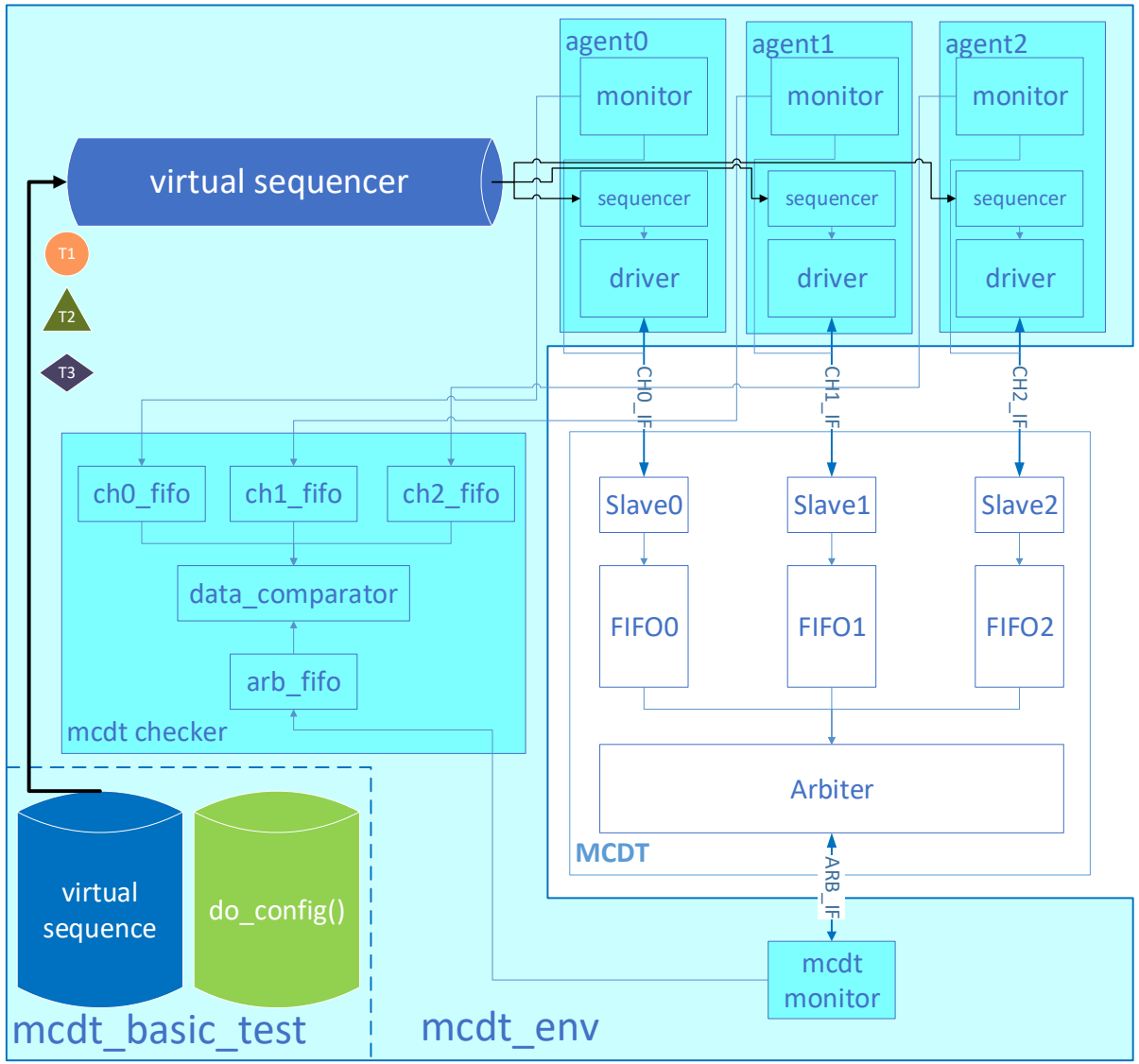
1. 最终的测试结构

2. 验证环境的更新

最终的测试结构


channel0 generator
channel1 generator
channel2 generator
agent0
agent1
agent2
channel monitor
channel initiator
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor
do_config()
mcdt_basic_test
mcdt_env

最终的测试结构


agent0
monitor
sequencer
driver
agent1
monitor
sequencer
driver
agent2
monitor
sequencer
driver
virtual sequencer
T1
T2
T3
CH0_IF
CH1_IF
CH2_IF
ch0_fifo
ch1_fifo
ch2_fifo
data_comparator
arb_fifo
mcdt checker
Slave0
Slave1
Slave2
FIFO0
FIFO1
FIFO2
Arbiter
MCDT
ARB_IF
mcdt monitor
virtual sequence
do_config()
mcdt_basic_test
mcdt_env

Agenda

1. 最终的测试结构

2. 验证环境的更新

验证环境的更新

1. 将generator生成并发送transaction的任务拆分为sequence与sequencer。
2. 在agent中集成driver，sequencer和monitor。
3. 创建virtual sequence与virtual sequencer。
4. 调整env的结构，添加virtual sequencer，移除原有的generator。

验证环境的更新

5. 将原有的test中用来配置channel generator发送数据的do_config()方法剥离出来，交由virtual sequence来完成。
6. 分别建立virtual sequence与test的继承层次关系。
7. 最终在test中创建virtual sequence，并挂载到virtual sequencer，即完成了整个验证环境的更新。

channel generator

```
class chnl_generator extends uvm_component;
  rand int pkt_id = 0;
  rand int ch_id = -1;
  rand int data_nidles = -1;
  rand int pkt_nidles = -1;
  rand int data_size = -1;
  rand int ntrans = 10;

  mailbox #(chnl_trans) req_mb;
  mailbox #(chnl_trans) rsp_mb;

  constraint cstr{
    soft ch_id == -1;
    soft pkt_id == 0;
    soft data_size == -1;
    soft data_nidles == -1;
    soft pkt_nidles == -1;
    soft ntrans == 10;
  }
```

channel generator

```
`uvm_component_utils_begin(chnl_generator)
  `uvm_field_int(pkt_id, UVM_ALL_ON)
  `uvm_field_int(ch_id, UVM_ALL_ON)
  `uvm_field_int(data_nidles, UVM_ALL_ON)
  `uvm_field_int(pkt_nidles, UVM_ALL_ON)
  `uvm_field_int(data_size, UVM_ALL_ON)
  `uvm_field_int(ntrans, UVM_ALL_ON)
`uvm_component_utils_end

function new (string name = "chnl_generator", uvm_component parent);
  super.new(name, parent);
  this.req_mb = new();
  this.rsp_mb = new();
endfunction

task start();
  repeat(ntrans) send_trans();
  run_stop_flags.put();
endtask
```

channel generator

```
task send_trans();
  chnl_trans req, rsp;
  req = chnl_trans::type_id::create("req");;
  assert(req.randomize with {local::ch_id >= 0 -> ch_id
                               == local::ch_id;
                             local::pkt_id >= 0 -> pkt_id
                               == local::pkt_id;
                             local::data_nidles >= 0 -> data_nidles
                               == local::data_nidles;
                             local::pkt_nidles >= 0 -> pkt_nidles
                               == local::pkt_nidles;
                             local::data_size >0 -> data.size()
                               == local::data_size;
                           })
    else `uvm_fatal("RNDFAIL", "channel packet randomization failure!")
  this.pkt_id++;
  `uvm_info(get_type_name(), req.sprint(), UVM_HIGH)
  this.req_mb.put(req);
  this.rsp_mb.get(rsp);
  `uvm_info(get_type_name(), rsp.sprint(), UVM_HIGH)
  assert(rsp.rsp)
    else `uvm_error("RSPERR", "error response received!")
endtask
```

channel generator

```
  function void post_randomize();
    string s;
    s = {s, "AFTER RANDOMIZATION \n"};
    s = {s, "=======================================\n"};
    s = {s, "chnl_generator object content is as below: \n"};
    s = {s, super.sprint()};
    s = {s, "=======================================\n"};
    `uvm_info(get_type_name(), s, UVM_HIGH)
  endfunction
endclass: chnl_generator
```

验证环境的更新

拆分generator

```
class chnl_data_sequence extends uvm_sequence #(chnl_trans);
  rand int pkt_id = 0;
  rand int ch_id = -1;
  rand int data_nidles = -1;
  rand int pkt_nidles = -1;
  rand int data_size = -1;
  rand int ntrans = 10;
  rand int data[];
  constraint cstr{
    soft pkt_id == 0;
    soft ch_id == -1;
    soft data_nidles == -1;
    soft pkt_nidles == -1;
    soft data_size == -1;
    soft ntrans == 10;
    soft data.size() == data_size;
    foreach(data[i]) soft data[i] == -1;
  };
  `uvm_object_utils_begin(chnl_data_sequence)
    `uvm_field_int(pkt_id, UVM_ALL_ON)
    `uvm_field_int(ch_id, UVM_ALL_ON)
    `uvm_field_int(data_nidles, UVM_ALL_ON)
    `uvm_field_int(pkt_nidles, UVM_ALL_ON)
    `uvm_field_int(data_size, UVM_ALL_ON)
    `uvm_field_int(ntrans, UVM_ALL_ON)
  `uvm_object_utils_end
```

验证环境的更新

拆分generator

```
function new (string name = "chnl_data_sequence");
  super.new(name);
endfunction
task body();
  repeat(ntrans) send_trans();
endtask
task send_trans();
  chnl_trans req, rsp;
  `uvm_do_with(req, {local::ch_id >= 0 -> ch_id == local::ch_id;
                     local::pkt_id >= 0 -> pkt_id == local::pkt_id;
                     local::data_nidles >= 0 -> data_nidles == local::data_nidles;
                     local::pkt_nidles >= 0 -> pkt_nidles == local::pkt_nidles;
                     local::data_size >0 -> data.size() == local::data_size;
                     foreach(local::data[i]) local::data[i] >= 0 -> data[i]
                       == local::data[i];
                     })
  this.pkt_id++;
  `uvm_info(get_type_name(), req.sprint(), UVM_HIGH)
  get_response(rsp);
  `uvm_info(get_type_name(), rsp.sprint(), UVM_HIGH)
  assert(rsp.rsp)
    else $error("[RSPERR] %0t error response received!", $time);
endtask
```

验证环境的更新
拆分generator

```
  function void post_randomize();
    string s;
    s = {s, "AFTER RANDOMIZATION \n"};
    s = {s, "========================================\n"};
    s = {s, "chnl_data_sequence object content is as below: \n"};
    s = {s, super.sprint()};
    s = {s, "========================================\n"};
    `uvm_info(get_type_name(), s, UVM_HIGH)
  endfunction
endclass: chnl_data_sequence

class chnl_sequencer extends uvm_sequencer #(chnl_trans);
  `uvm_component_utils(chnl_sequencer)
  function new (string name = "chnl_sequencer", uvm_component parent);
    super.new(name, parent);
  endfunction
endclass: chnl_sequencer
```

验证环境的更新

agent集成driver/sequencer/monitor

```
class chnl_agent extends uvm_agent;
  chnl_driver driver;
  chnl_monitor monitor;
  chnl_sequencer sequencer;
  virtual chnl_intf vif;
  `uvm_component_utils(chnl_agent)
  function new(string name = "chnl_agent", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // get virtual interface
    if(!uvm_config_db#(virtual chnl_intf)::get(this,"","vif", vif)) begin
      `uvm_fatal("GETVIF","cannot get vif handle from config DB")
    end
    driver = chnl_driver::type_id::create("driver", this);
    monitor = chnl_monitor::type_id::create("monitor", this);
    sequencer = chnl_sequencer::type_id::create("sequencer", this);
  endfunction
```

验证环境的更新

agent集成driver/sequencer/monitor

```
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    driver.seq_item_port.connect(sequencer.seq_item_export);
    driver.set_interface(vif);
    monitor.set_interface(vif);
  endfunction
endclass: chnl_agent
```

验证环境的更新

创建virtual sequence与virtual sequencer

```
class mcdt_root_virtual_sequence extends uvm_sequence #(uvm_sequence_item);
  chnl_data_sequence chnl_data_seq;
  `uvm_object_utils(mcdt_root_virtual_sequence)
  `uvm_declare_p_sequencer(mcdt_virtual_sequencer)
  function new (string name = "mcdt_root_virtual_sequence");
    super.new(name);
  endfunction

  virtual task body();
    `uvm_info(get_type_name(), "============STARTED============", UVM_LOW)
    this.do_data();
    `uvm_info(get_type_name(), "============FINISHED===========", UVM_LOW)
  endtask

  // do data transition from 3 channel slaves
  virtual task do_data();
    //User to implment the task in the child virtual sequence
  endtask

  task wait_cycles(int n);
    repeat(n) @(posedge p_sequencer.mcdt_vif.clk);
  endtask
endclass: mcdt_root_virtual_sequence
```

验证环境的更新

创建virtual sequence与virtual sequencer

```
class mcdt_virtual_sequencer extends uvm_sequencer
                                        #(uvm_sequence_item);
  chnl_sequencer chnl_sqrs[3];
  virtual mcdt_intf mcdt_vif;
  `uvm_component_utils(mcdt_virtual_sequencer)
  function new (string name = "mcdt_virtual_sequencer",
                uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db#(virtual mcdt_intf)::get
       (this,"","mcdt_vif", mcdt_vif)) begin
      `uvm_fatal("GETVIF","cannot get vif handle from config DB")
    end
  endfunction
endclass: mcdt_root_virtual_sequencer
```

验证环境的更新

调整mcdt_env结构

```
class mcdt_env extends uvm_env;
  chnl_agent agents[3];
  mcdt_monitor mcdt_mon;
  mcdt_checker chker;
  mcdt_coverage cvrg;
  mcdt_virtual_sequencer virt_sqr;
  `uvm_component_utils(mcdt_env)
  function new (string name = "mcdt_env", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    this.chker = mcdt_checker::type_id::create("chker", this);
    foreach(agents[i]) begin
      this.agents[i] =
        chnl_agent::type_id::create($sformatf("agents[%0d]",i), this);
    end
    this.mcdt_mon = mcdt_monitor::type_id::create("mcdt_mon", this);
    this.cvrg = mcdt_coverage::type_id::create("cvrg", this);
    this.virt_sqr = mcdt_virtual_sequencer::type_id::create
                    ("virt_sqr", this);
  endfunction
```

验证环境的更新

调整mcdt_env结构

```
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    foreach(agents[i]) begin
      agents[i].monitor.mon_ana_port.connect(
        chker.in_tlm_fifos[i].analysis_export);
      virt_sqr.chnl_sqrs[i] = agents[i].sequencer;
    end
    mcdt_mon.mon_ana_port.connect(chker.out_tlm_fifo.analysis_export);
  endfunction
endclass: mcdt_env
```

验证环境的更新

剥离test，将数据发送迁移到virtual sequence

```
class mcdt_root_test extends uvm_test;
  mcdt_env env;
  `uvm_component_utils(mcdt_root_test)
  function new(string name = "mcdt_root_test", uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = mcdt_env::type_id::create("env", this);
  endfunction
  function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);
    uvm_root::get().set_report_verbosity_level_hier(UVM_HIGH);
    uvm_root::get().set_report_max_quit_count(1);
    uvm_root::get().set_timeout(10ms);
  endfunction
  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    this.run_top_virtual_sequence();
    phase.drop_objection(this);
  endtask
  virtual task run_top_virtual_sequence();
    // User
  endtask
endclass: mcdt_root_test
```

验证环境的更新

建立virtual sequence与test的继承和对应关系

```
class mcdt_basic_virtual_sequence extends mcdt_root_virtual_sequence;
  `uvm_object_utils(mcdt_basic_virtual_sequence)
  function new (string name = "mcdt_basic_virtual_sequence");
    super.new(name);
  endfunction

  task do_data();
    @(posedge p_sequencer.mcdt_vif.rstn); this.wait_cycles(10);
    fork
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[0],
                      {ntrans==100; ch_id==0; data_nidles ==0;
                       pkt_nidles==1; data_size==8;})
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[1],
                      {ntrans== 50; ch_id==1; data_nidles inside {[1:2]};
                       pkt_nidles inside {[3:5]}; data_size==6;})
      `uvm_do_on_with(chnl_data_seq, p_sequencer.chnl_sqrs[2],
                      {ntrans== 80; ch_id==2; data_nidles inside {[0:1]};
                       pkt_nidles inside {[1:2]}; data_size==32;})
    join
    #10us; // wait until all data haven been transfered through MCDT
  endtask
endclass: mcdt_basic_virtual_sequence
```

验证环境的更新

建立virtual sequence与test的继承和对应关系

```
class mcdt_basic_test extends mcdt_root_test;
  `uvm_component_utils(mcdt_basic_test)
  function new(string name = "mcdt_basic_test", uvm_component parent);
    super.new(name, parent);
  endfunction
  task run_top_virtual_sequence();
    mcdt_basic_virtual_sequence top_seq = new();
    top_seq.start(env.virt_sqr);
  endtask
endclass: mcdt_basic_test
```

路科验证

芯片验证技术 咨询&培训

专注于系统验证思想和前沿验证资讯，
为 IC 从业人员提供技术食粮。



长按关注 与路科验证一起进步！

RKV
路科验证

赢得信任 互相成就